



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Bevezető, alapfogalmak
2023. február 27.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



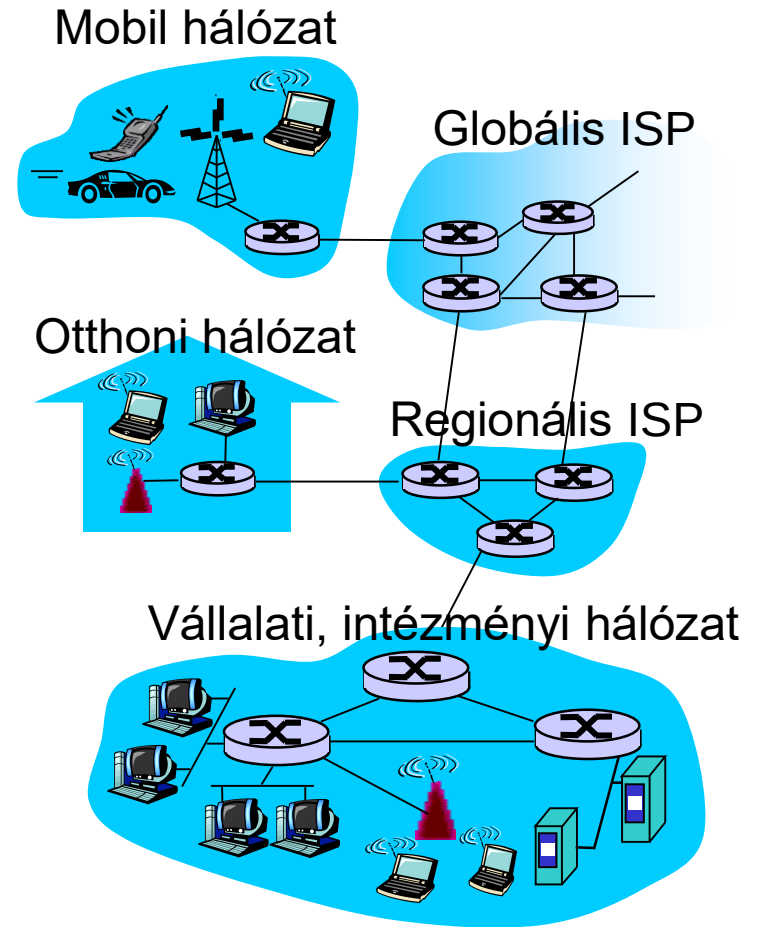
- Hogy képes működni az Internet?
- Méretek
 - 30 Mrd (?) eszköz
 - 150.000 GB adat másodpercenként
- Elvárások
 - Rengetegféle eszköz és alkalmazás
 - Bedugom/bekapcsolom és működik
 - Szolgáltatás minőség (sebesség, megbízhatóság, késleltetés, stb.)
- Egy egyszerű weboldal betöltése sem triviális...

- Számítógép(?) hálózatok alapfogalmai, felépítése és alapvető építőelemei
 - Hálózati rétegek
 - Protokollok
 - Eszközök
 - Napjainkban elterjedt megoldások
- Hálózatok üzemeltetése
 - Eszközök konfigurációja
 - Felügyelet, menedzsment
 - Hibakeresés, hibaelhárítás

1. Hálózatok alapelemei, az Internet alapjai
2. Protokoll, szerkezet és szolgáltatás
3. Az Internet szerkezeti részei
4. Áramkörkapcsolás és csomagkapcsolás
5. Csomagvesztés, késleltetés, átbocsátóképesség

MI AZ INTERNET?

- Összekapcsolt intelligens eszközök milliói: **hosztok (végponti rendszerek)**
 - **hálózati alkalmazásokat** futtatnak
- **Kommunikációs összeköttetések (adat-kapcsolatok, linkek)**
 - fényvezető, réz, rádiós és műholdas
 - adatátviteli sebesség, elterjedt, de nem teljesen korrekt nevén **sávszélesség**
- **Útvonalválasztók (routerek)**
 - csomagokat (adatarabkák) továbbítanak

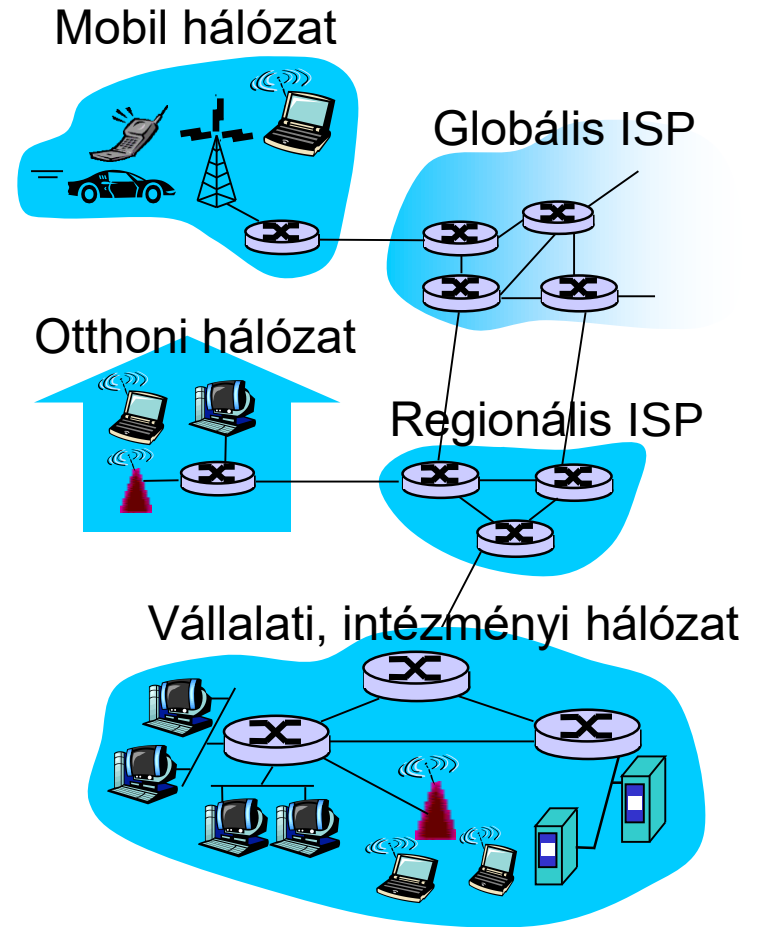


- Számítógép?
- Mobil kommunikációs eszközök
- IoT eszközök
- Számítógépek!



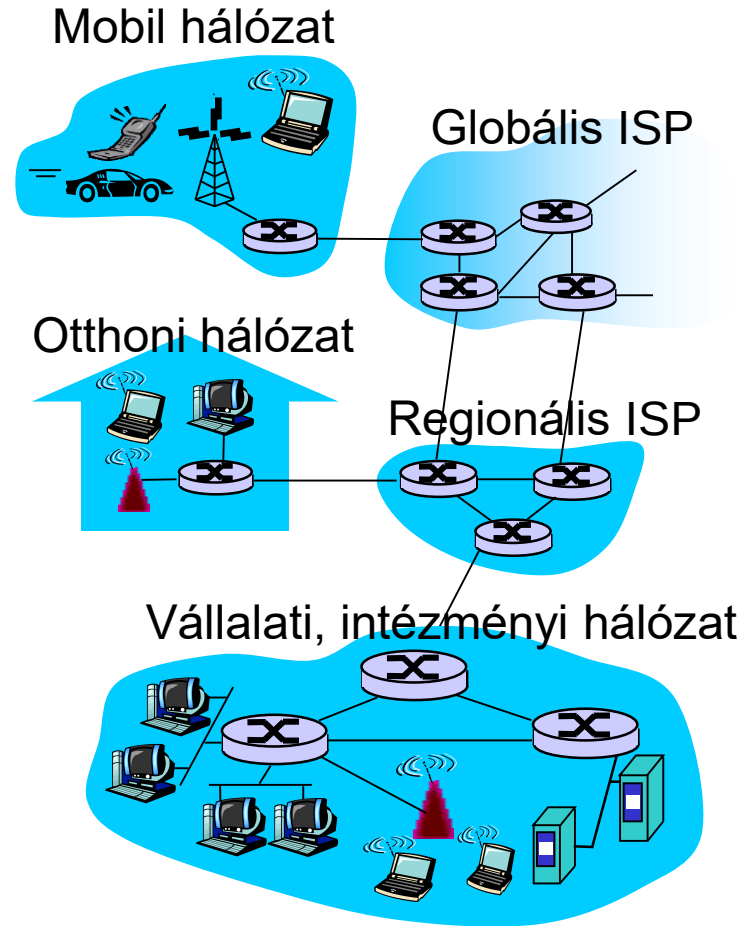
ÉS MÉG MI AZ INTERNET?

- **Kommunikációs infrastruktúra**, ami elosztott alkalmazások futtatását teszi lehetővé, pl:
 - web, VoIP, email
 - e-kereskedelem, fájlmegosztás, játékok
- **Az alkalmazásoknak nyújtott kommunikációs szolgáltatások**
 - Adatszállítás forrás és nyelő között
 - Sokféle igény



HOGYAN MŰKÖDHEK AZ INTERNET?

- **Hálózatok hálózata**
 - lazán hierarchikus
 - nyilvános Internet, privát intranetek
- **Protokollok** az üzenetek küldésének és fogadásának vezérléséhez
- Internet **szabványok**
 - RFC: Request for comments
 - IETF: Internet Engineering Task Force



1. Hálózatok alapelemei, az Internet alapjai
2. **Protokoll, szerkezet és szolgáltatás**
3. Az Internet szerkezeti részei
4. Áramkörkapcsolás és csomagkapcsolás
5. Csomagvesztés, késleltetés, átbocsátóképesség

- Emberi kommunikáció
 - Kapcsolatfelvétel
 - Információcsere
 - Elkészítés
 - A céltól, helyzettől függ

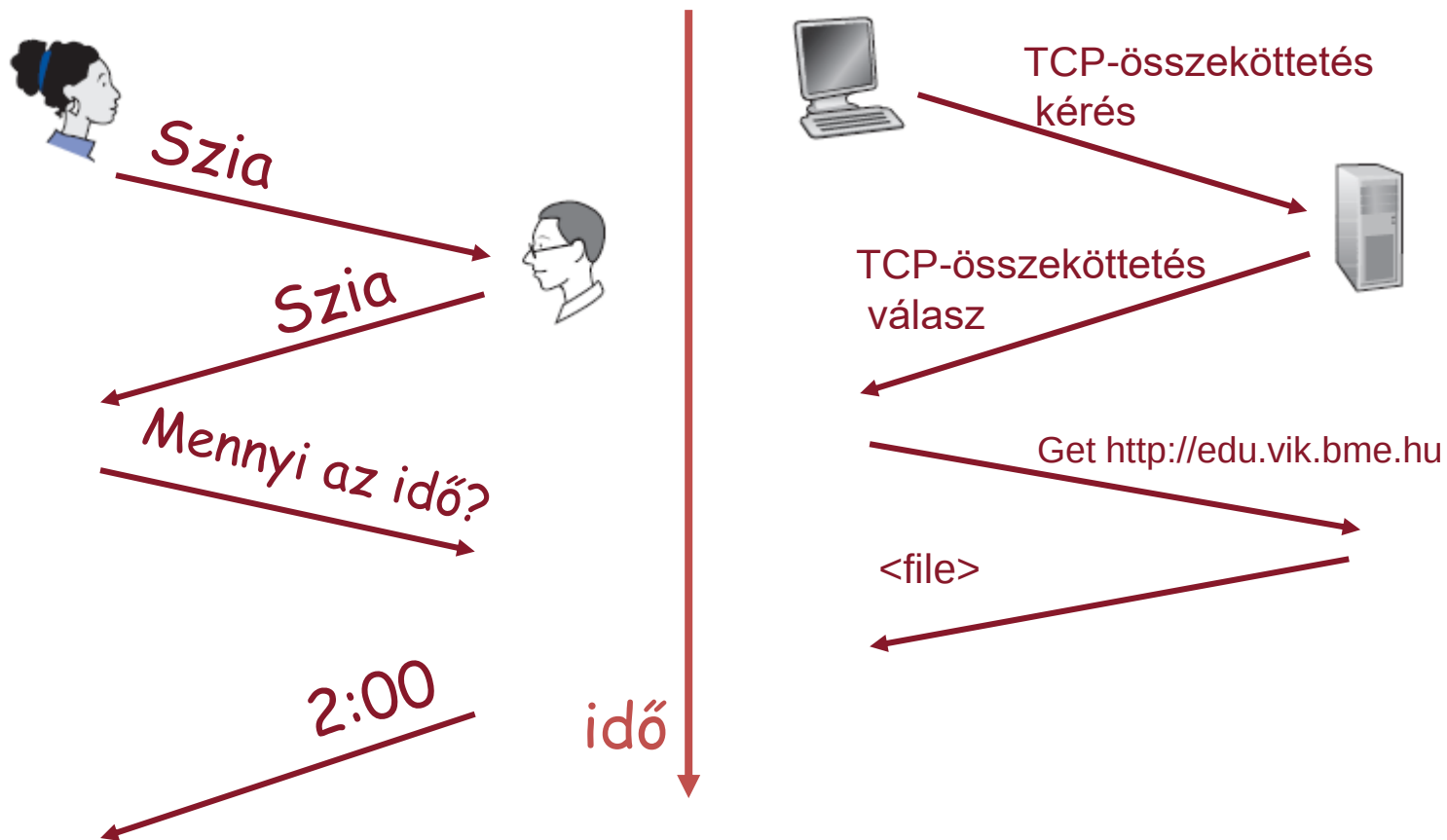
Protokoll

Üzenetváltást leíró szabályok

- Sorrend
- Formátum
- Reakciók eseményekre
 - Küldés/fogadás
 - Egyéb

- Gépi kommunikáció
 - Hasonlít az emberire
 - Pontosabb szabályok kellenek
 - Gépek generálják és dolgozzák fel az üzeneteket
 - Formátumok
 - Állapotok

Időbeli lefolyás emberi és gépi kommunikáció esetén

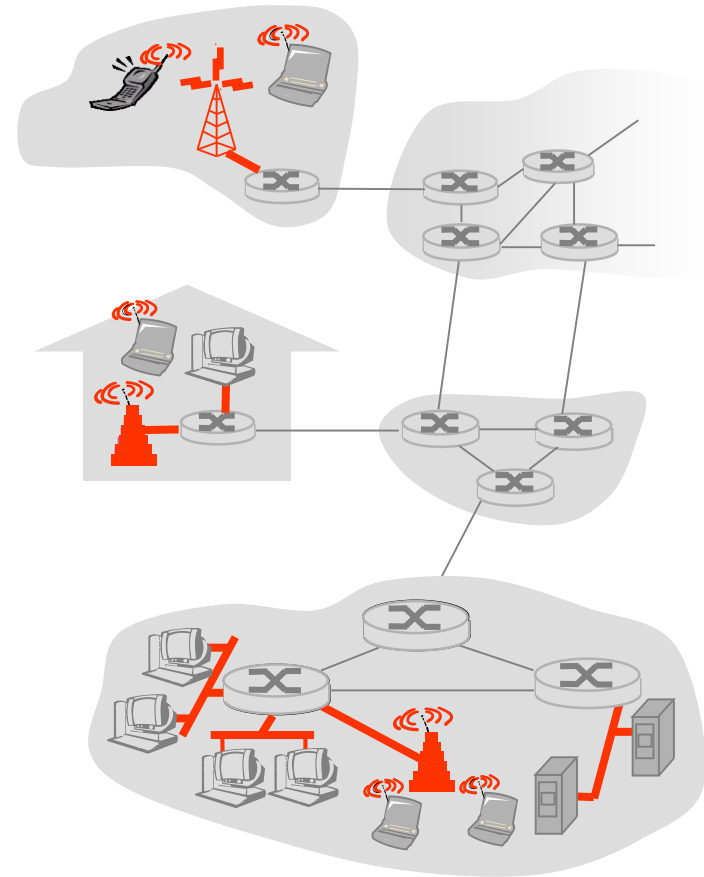


- A hosztok alkalmazásokat futtatnak
 - Kliens szerep: adatot igényel
 - Szerver szerep: adatot szolgáltat
- **Kliens-szerver architektúra**
 - Egyértelmű, dedikált szerep
 - Például web böngészés
- **Peer-to-peer architektúra**
 - Minden elem mindkét szerepben (lehet)
 - Például skype, torrent

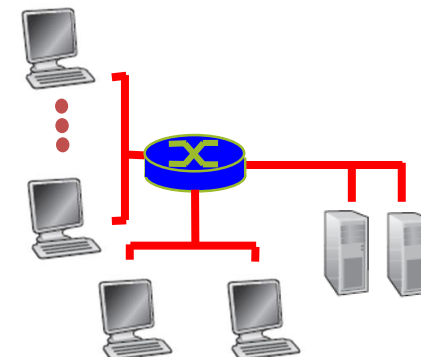
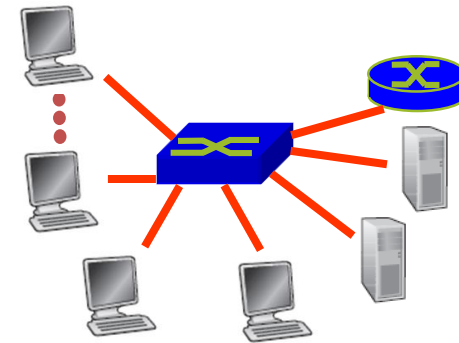
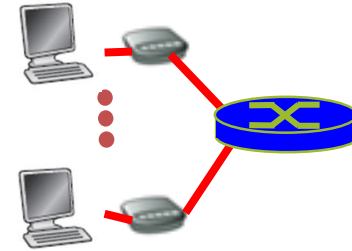
1. Hálózatok alapelemei, az Internet alapjai
2. Protokoll, szerkezet és szolgáltatás
3. Az Internet szerkezeti részei
4. Áramkörkapcsolás és csomagkapcsolás
5. Csomagvesztés, késleltetés, átbocsátóképesség

- A **hálózat széle, pereme (edge)**: alkalmazások és hosztok
- A **hozzáférési hálózat (access)**: fizikai közeg, vezetékes, vezeték nélküli kommunikációs linkek
- A **hálózat magja (core)**: hálózatba kötött útvonalválasztók

- Csomópontok
 - A hosztok hálózati funkciójú elemei, pl. hálózati kártya
 - Hálózati eszközök
 - Összekapcsolva egymással és a hálózat magjával
- Hordozó közeg
 - Fizikai közeg

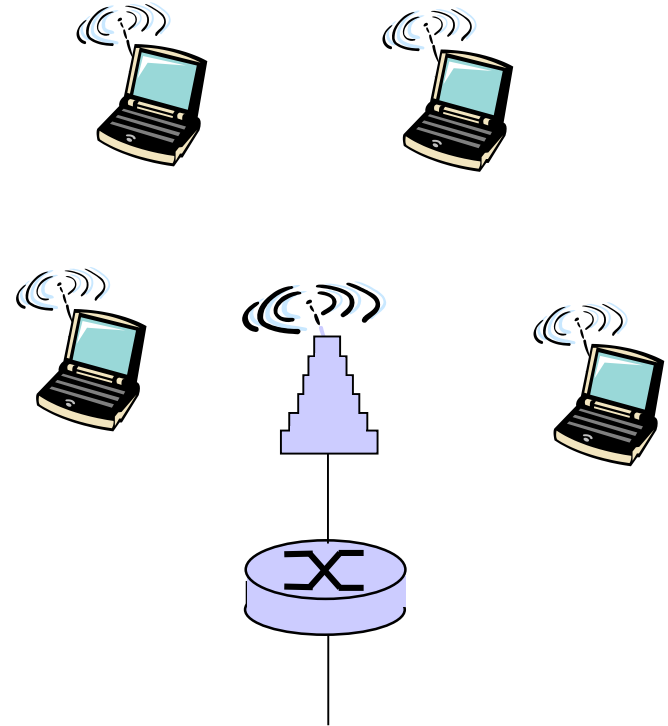


- Otthonról
 - **Digitális előfizető vonal** (DSL) változatai – dedikált, aszimmetrikus
 - **Kábel** (Coax vagy HFC) – dedikált vagy osztott aszimmetrikus
 - **Optikai kábel** (FTTx) – dedikált aszimmetrikus
- Vállalati vagy egyetemi környezetben
 - tipikusan **Ethernet** – osztott szimmetrikus
- Vezeték nélküli

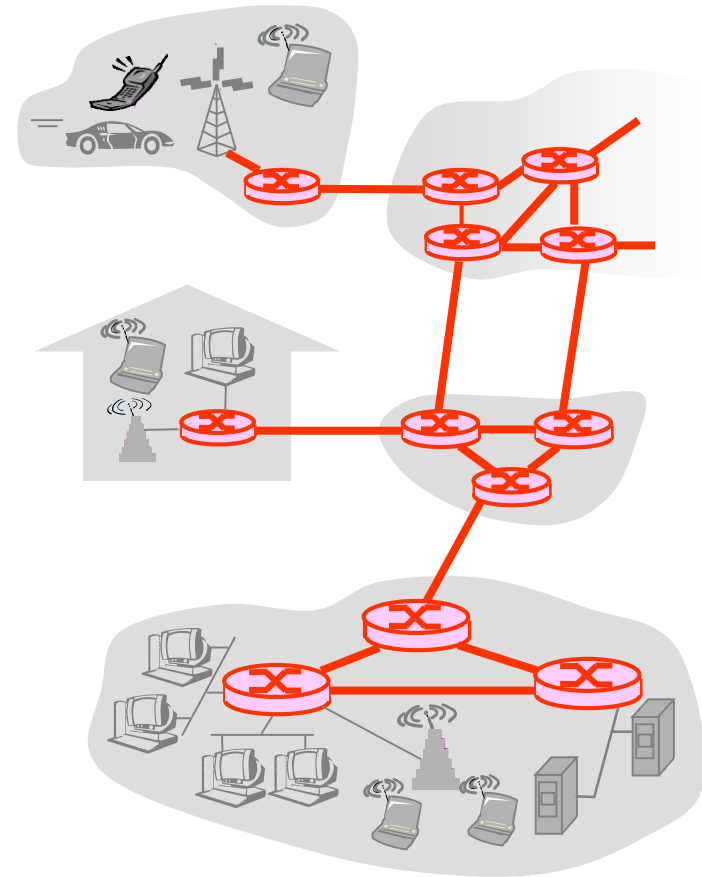


VEZETÉK NÉLKÜLI ÉS MOBIL CSATLAKOZÁS

- Osztott hozzáférésű vezeték nélküli / rádiós hálózatok csatlakoztatják a hosztokat
 - **Hozzáférési pontokon** (access point), vagy **bázisállomásokon** (base station) keresztül
- **Wireless LAN**
 - Legelterjedtebb a 802.11b/g (WiFi)
- **Nagytávolságú rádiós link**
 - Mobilszolgáltató hálózatán keresztül
 - Cella alapú hálózatok, 3G, 4G, 5G ...
 - WiMAX



- Összekapcsolt útvonalválasztók szövevényes hálózata
- Kérdés, hogy hogyan továbbítódnak az adatok a hálózaton?
 - **Áramkörkapcsolás:** kapcsolatonként dedikált csatorna, régi példa: vezetékes távbeszélő
 - **Csomagkapcsolás:** az adatok diszkrét "darabkákban" továbbítódnak



1. Hálózatok alapelemei, az Internet alapjai
2. Protokoll, szerkezet és szolgáltatás
3. Az Internet szerkezeti részei
4. **Áramkörkapcsolás és csomagkapcsolás**
5. Csomagvesztés, késleltetés, átbocsátóképesség

- Egy-egy kapcsolat (hívás) végponttól végpontig foglal le erőforrásokat
 - Dedikált erőforrások, a lefoglalt részen nincs erőforrás-megosztás más kapcsolatokkal
 - Áramkörszerű (garantált) teljesítőképesség
 - Hívásfelépítés és lebontás szükséges
- A hálózati erőforrásokat (pl., a sávszélesség) “darabokra” osztjuk
 - Az erőforrás darabokat kapcsolatokhoz rendeljük
 - Az éppen egy kapcsolathoz sem rendelt erőforrás darabok nincsenek használatban (nincs erőforrás-megosztás)

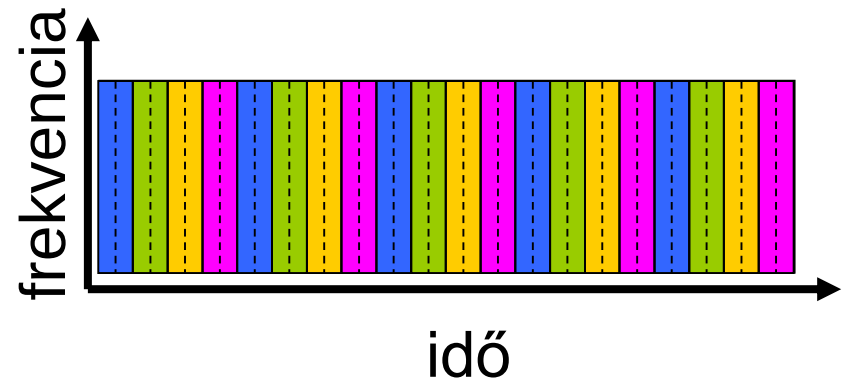
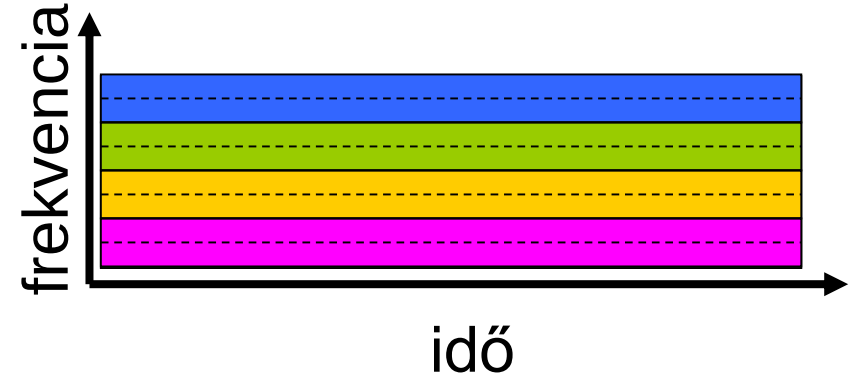
Pl: Telefonálás

Garantált sebesség

Nincs késleltetés ingadozás

A és B kívül senki más nem használhatja ezt a sávot.

- Link sávszélességének “feldarabolása”
 - **Frekvenciaosztás** – FDM (Frequency Division Multiplexing)
 - **Időosztás** – TDM (Time Division Multiplexing)
 - További felosztási módszerek
 - Kódosztás – CDM
 - Hullámhosszosztás – WDM
- Kapcsolatok multiplexálása



- Minden adatfolyam csomagokra van bontva
 - A és B felhasználó csomagjai **osztóznak** a hálózati erőforrásokon
 - Minden csomag a teljes link-sávszélességet használja
 - Az erőforrások felhasználása **igény szerint**
 - Az erőforrás „darabjai” nincsenek lefoglalva és egyes adatfolyamokhoz rendelve
- Verseny az erőforrásokért
 - Az aggregált erőforrásigény akár túl is lépheti a rendelkezésre álló erőforrások mennyiségét
 - **Torlódás (congestion):** a csomagok sorba állnak, várják, hogy használhassák a linket

PI: Internet

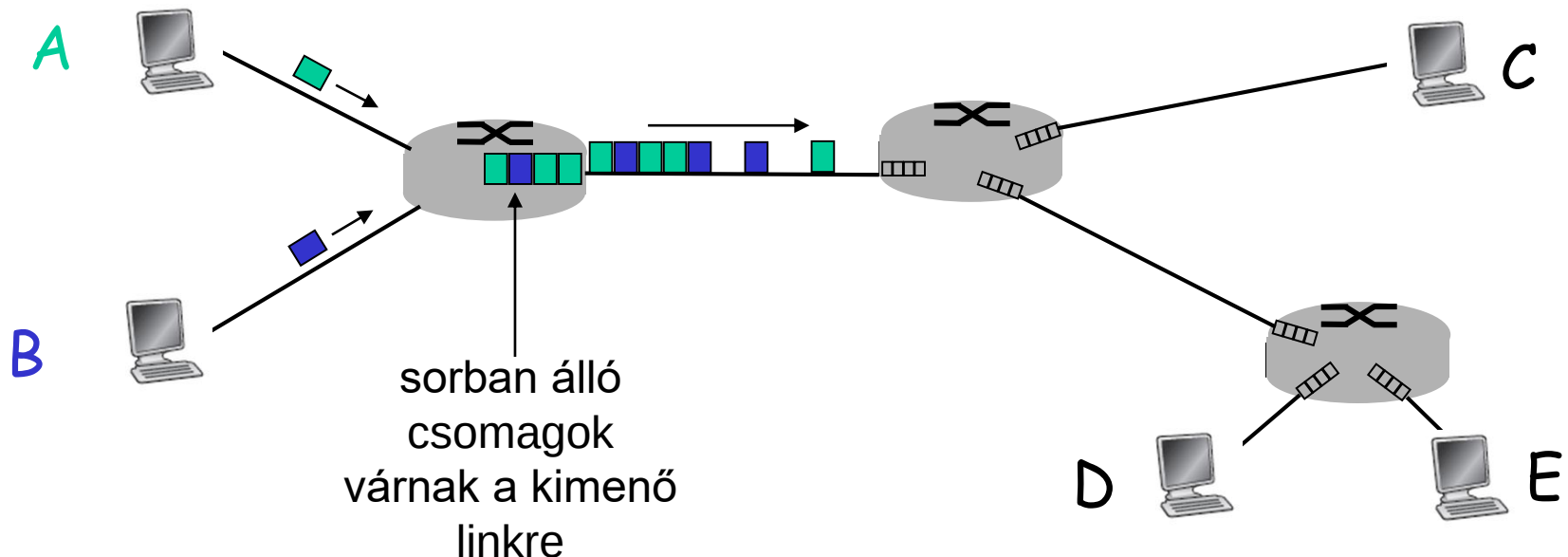
Rugalmas és hatékony

Csomagok különböző úton mehetnek

Nincs előzetes foglalás

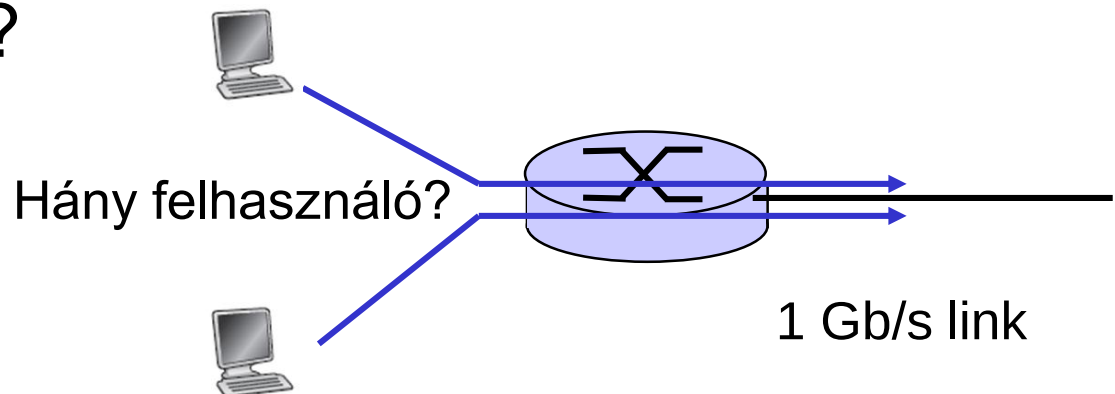
Egy üzenet kis csomagokra vágódik és különböző utakon keresztül jut el a célhoz.

- A és B csomópont csomagjainak sorozata
 - Nincs fix sorrend
 - Nem biztos, hogy fix a csomagok mérete
 - Az adatfolyamok a sávszélességen igény szerint osztoznak
 - Emlékezzünk: időosztás esetén egy adott adatfolyam mindig ugyanazt az időrést kapja a TDM-keretekben



- Áramkörkapcsolás
 - Hosszú kapcsolat (kapcsolat felépítése/lebontása)
 - Egyenletes erőforrásigény
 - Megbízható minőség (pl. késleltetés)
 - Pl. telefonhívás
- Csomagkapcsolás
 - Rövid kapcsolat
 - Dinamikus erőforrásigény
 - Minőségi garancia másodlagos
 - Pl. **adatátvitel**

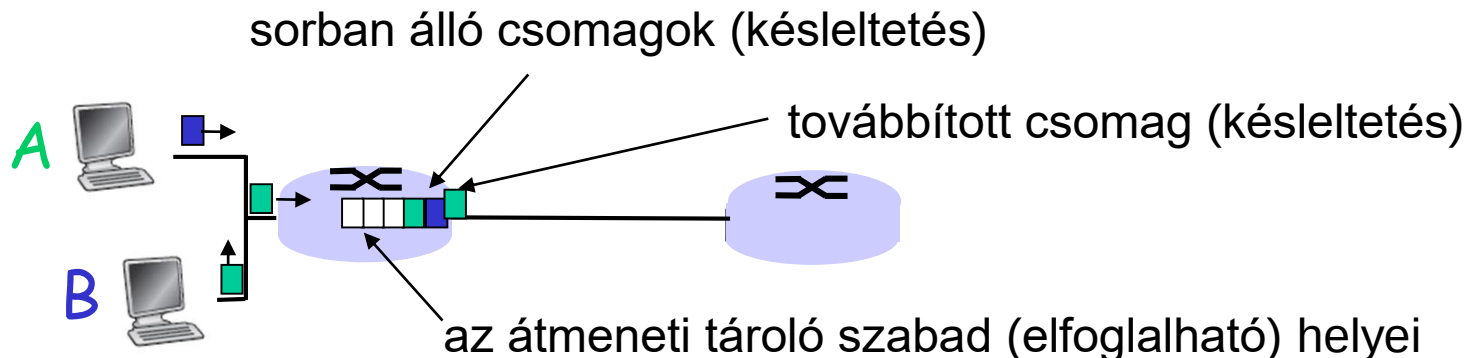
- Hányan használhatják egyszerre a hálózatot?
 - Példa
 - 1 Gb/s sávszélességű link
 - Az idő 10%-ában aktív, 100 Mb/s igényű felhasználó
 - Áramkörkapcsolás
 - Összesen 10 felhasználó egyidejű kiszolgálása sikeres
 - Csomagkapcsolás
 - Összesen 35 felhasználó egyidejű kiszolgálása esetén annak a valószínűsége, hogy egyszerre több mint 10-en aktívak, kevesebb, mint 0,004 (vagyis 0,4%)
- Tehát melyik jobb?



- Nagyszerűen megfelel **löketszerűen érkező (bursty) adatok továbbítására**
 - **Statisztikusan megosztott** erőforrások
 - Egyszerű működés, nincs hívásfelépítés
- Nagyobb torlódás esetén mi lesz a csomagokkal?
 - Késleltetés a **sorbanállás** miatt
 - Csomagvesztés
 - Protokollok szükségesek a megbízható adatátvitelhez és torlódásvezérléshez
- Hogyan biztosítható az áramkörkapcsolásra hasonlító működés?
 - garantált sávszélesség kell az audio/video alkalmazások számára
- Fejlécek kellenek minden csomagra

1. Hálózatok alapelemei, az Internet alapjai
2. Protokoll, szerkezet és szolgáltatás
3. Az Internet szerkezeti részei
4. Áramkörkapcsolás és csomagkapcsolás
5. Csomagvesztés, késleltetés, átbocsátóképesség

- A csomagok **sorban állnak** az útvonalválasztók átmeneti tárolóiban (buffer, puffer)
 - A csomagok érkezési intenzitása eléri a kimenő link kapacitását (csomag/másodperc)
 - Várnak a továbbításukra
 - A kiszolgálás legegyszerűbb esetben FIFO
- A pufferek mérete véges
 - Teli puffer esetén érkező csomaggal mi legyen?



A KÉSLELTETÉS ÖSSZETEVŐI

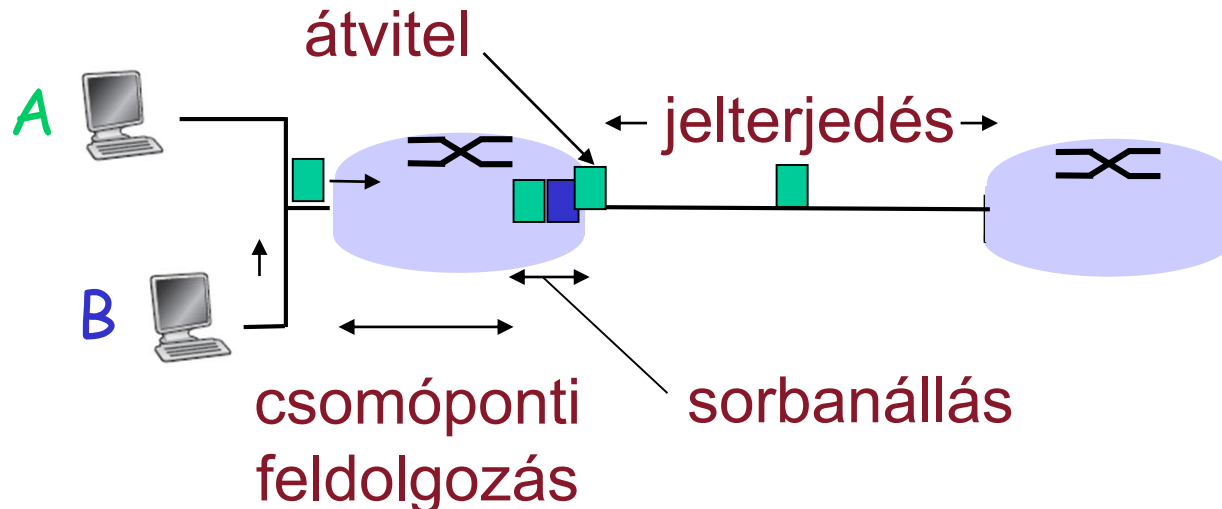
1. Csomóponti feldolgozás

- Bithibák ellenőrzése
- Kimenő link meghatározása
- Tipikusan néhány mikroszekundum, vagy kevesebb

2. Sorbanállás

- Az átvitelre várakozás a kimenő linknél
- Függ az adott útvonalválasztónál fennálló torlódás mértékétől

Változó komponensek!!

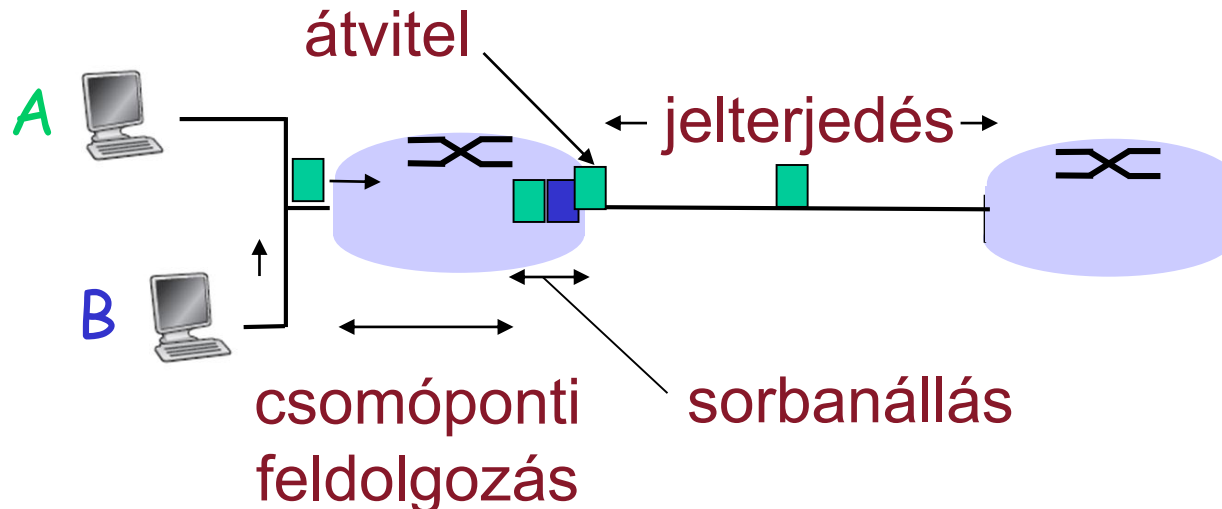


3. Átviteli késleltetés

- R = link sávszélesség (bps)
- L = csomagméret (bit)
- A csomag bitjeinek a linkre kiküldéséhez szükséges idő: L/R

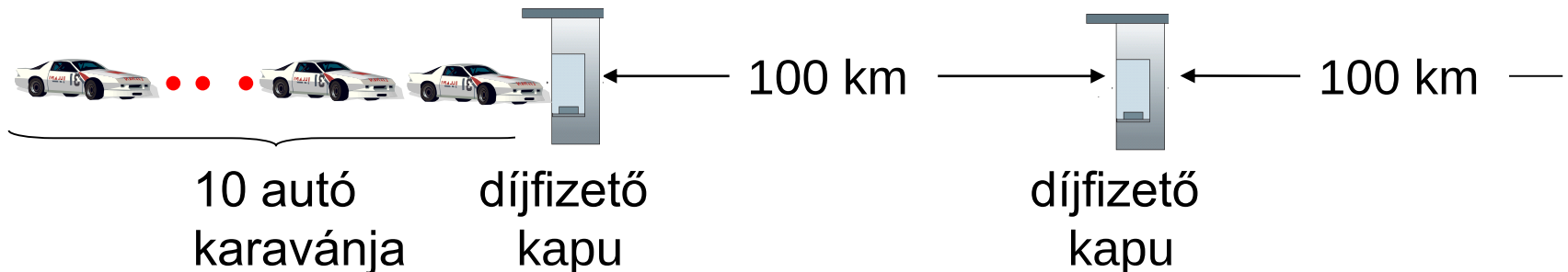
4. Jelterjedési késleltetés

- d = a fizikai link hossza (m)
- s = a jel terjedési sebesség a közegben ($\sim 2 \times 10^8$ m/sec)
- Jelterjedési késleltetés: d/s
- Néhány mikroszekundumtól néhány milliszekundumig



ANALÓGIA: AUTÓKARAVÁN AZ AUTÓPÁLYÁN

- Az autók “terjedési” sebessége 100 km/h
- A kapu 12 másodperc alatt szolgál ki egy autót (átviteli idő)
- Megfeleltetések
 - autó ~ bit
 - karaván ~ csomag
- Mennyi időbe telik, mire a karaván autói felsorakoznak a második kapunál?
- A teljes karaván (10 autó) következő útszakaszra való átengedéséhez szükséges idő: $12 \cdot 10 = 120$ mp
- Egy autó (az utolsó) az első kaputól a második kapuig : $100\text{km}/(100\text{km/h}) = 1$ óra alatt jut el
- Késeltetés: 62 perc

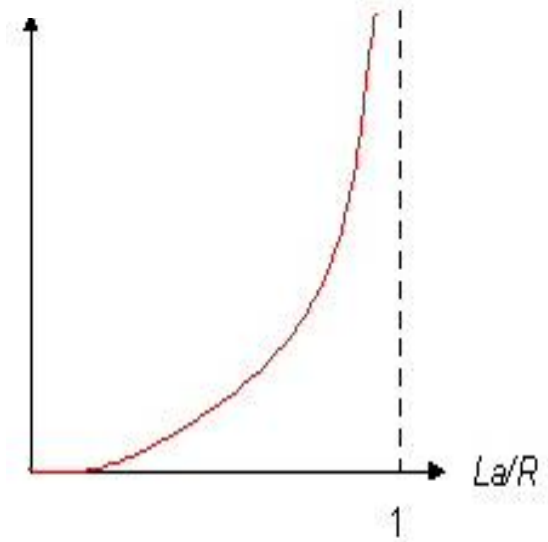


SORBANÁLLÁSI KÉSLELTETÉS

- A forgalom intenzitása = λa

- R =link sávszélesség (bps)
- L =csomagméret (bit)
- a =átlagos **csomagérkezési intenzitás** (1/sec)

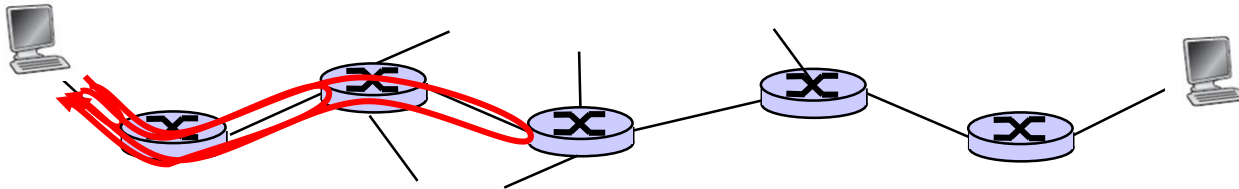
átlagos sorbanállási késleltetés



- $\lambda a \sim 0$: az átlagos sorbanállási késleltetés kicsi
- $\lambda a \rightarrow 1$: a késleltetés nő
- $\lambda a > 1$: több “tennivaló” érkezik, mint amennyi elvégezhető, az átlagos késleltetés végtelen nagy lesz!

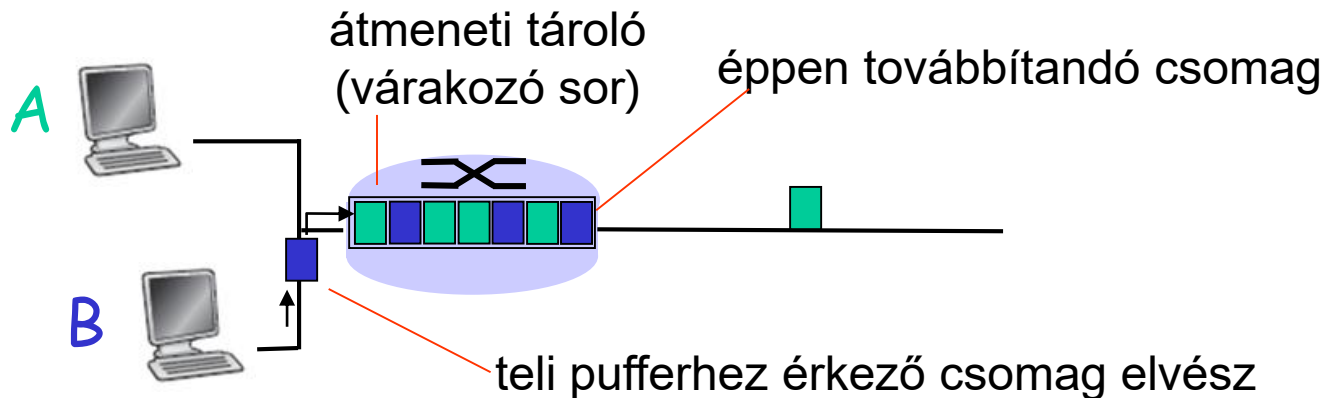
KÉSZLELTETÉS A TELJES ÚTVONALON

- Hogyan néz ki a késleltetés és a csomagvesztés a “valóságos” Interneten?

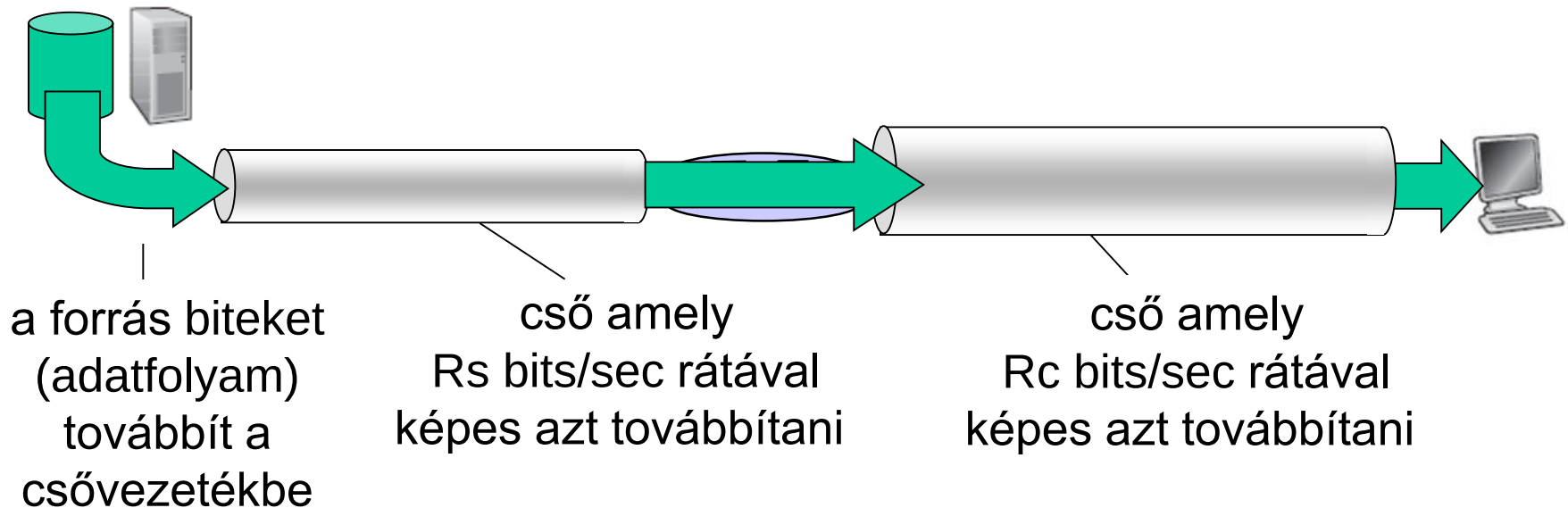


- Több útvonalválasztóban is sorban áll a csomag
- A korrekt kiszámítása összetett feladat
- Hogyan lehetne lemérni?
- **Traceroute** program: késleltetések mérése a forrástól a nyelőig minden útvonalválasztóra az útvonal mentén
 - A forrás csomagot küld minden egyes routerhez ami a nyelőhöz vezető úton van
 - Minden router visszaküld egy választ a forrásnak
 - A forrás ennek alapján kiszámolja a küldés és válasz közt eltelt időt: **RTT** (roundtrip time, körbejárási idő)

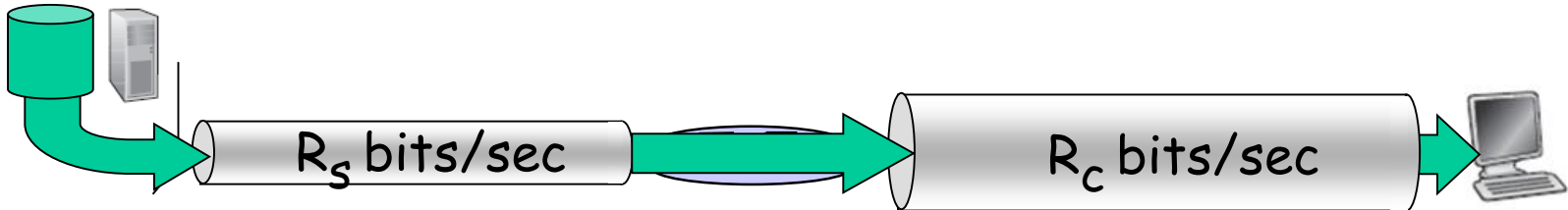
- A link előtti tároló kapacitása véges
- Ha akkor érkezik csomag, amikor tele a sor a csomagot el kell dobni: **csomagvesztés (packet loss)**
- Az elveszett csomagot újraküldheti az előző csomópont, a forrás hoszt, de az is lehet, hogy nincs újraküldés
- Más okból is veszhetnek el csomagok, például bithibák miatt



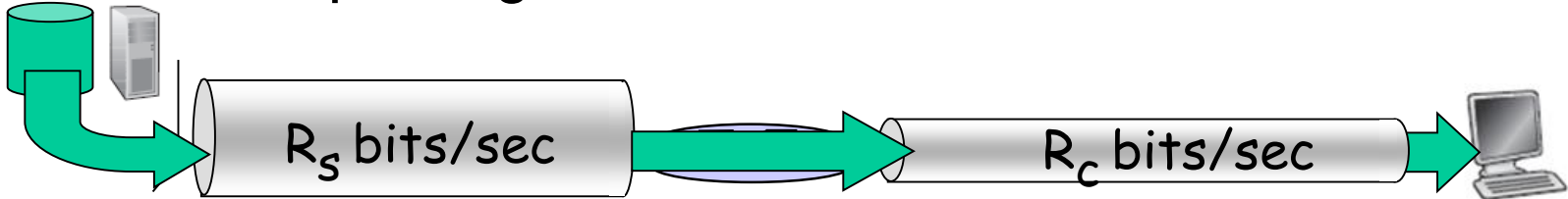
- Ráta (sávszélesség) amivel a bitek továbbítása
ténylegesen történik a forrás és a nyelő között (bps)
 - **Pillanatnyi:** egy adott helyen és időpontban
 - **Átlagos:** egy hosszabb időtartamra vonatkoztatva



- $R_s < R_c$ Mekkora a végpontok közötti átlagos átbocsátóképeség ?



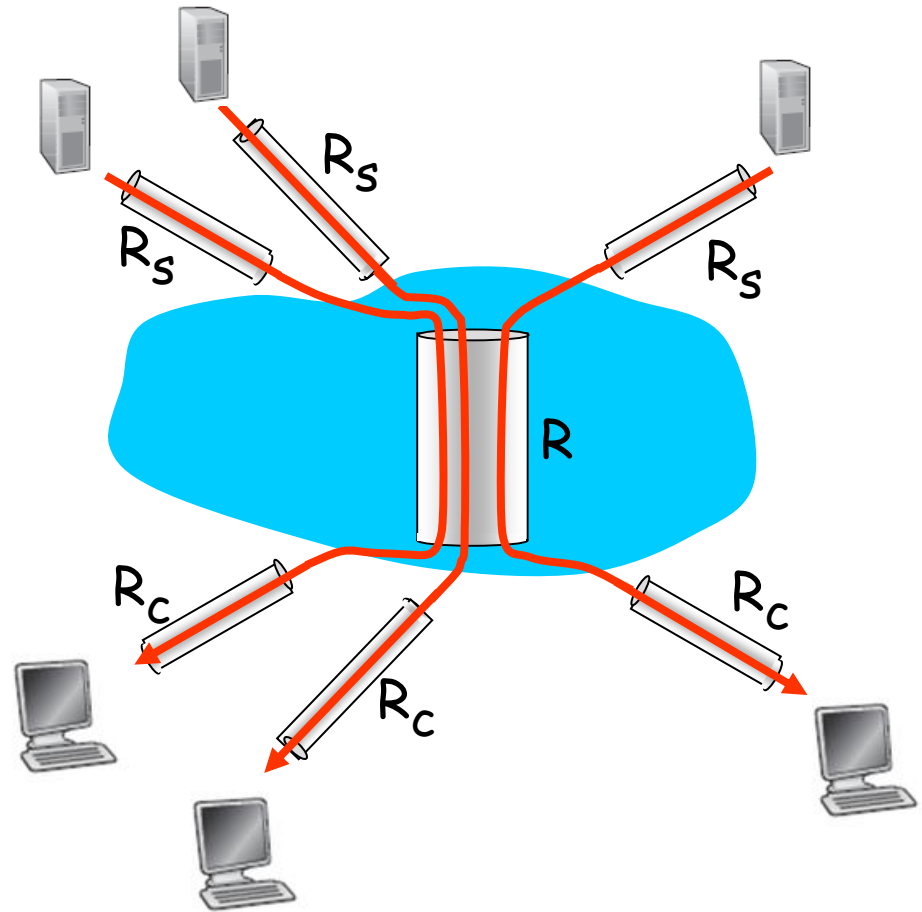
- $R_s > R_c$ Mekkora a végpontok közötti átlagos átbocsátóképeség ?



- **Szűk keresztmetszetű link (bottleneck)**
 - A végpontok közti átbocsátóképeséget korlátozó link
 - Több forrás-nyelő pár esetén azért nem ennyire egyszerű...

ÁTBOCSÁTÓKÉPESSÉG ÁLTALÁNOSABB ESETBEN

- Példa: 10 kapcsolat osztozik a gerinchálózati (szűk keresztmetszetű) R sáv szélességű linken
- Egyenlő (igazságos) osztozkodás esetén minden kapcsolatra a végpontok közti átbocsátóképesség:
 $\min(R_c, R_s, R/10)$
- A gyakorlatban: általában R_c vagy R_s a szűk keresztmetszet
- Bonyolultabb esetek
 - Több link hálózata
 - Nem igazságos osztozkodás



- Az eddig látott jellemzők az **adatátvitel minőségét** írják le
 - Késleltetés
 - Csomagvesztés
 - Átbocsátás (sávszélesség)
 - A késleltetés ingadozása
- Összefoglaló néven ezeket szolgáltatásminőségnek (**Quality of Service, QoS**) hívjuk
- Ezt az adatátvitelnél
 - Tapasztaljuk
 - Előírhatjuk?
- A **QoS követelmények** biztosítása a csomagkapcsolás miatt nem kézenfekvő
 - Erőforráslefoglalás
 - Prioritások

- Ebben a tárgyban nem részletezzük, de fontos
 - **Hálózatbiztonság**
 - Adataink és erőforrásaink védelme
 - **Megbízhatóság**
 - Ritkán szeretnénk meghibásodások következményeit tapasztalni



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Hálózati protokoll rétegek
2023. március 3.

Mészáros András

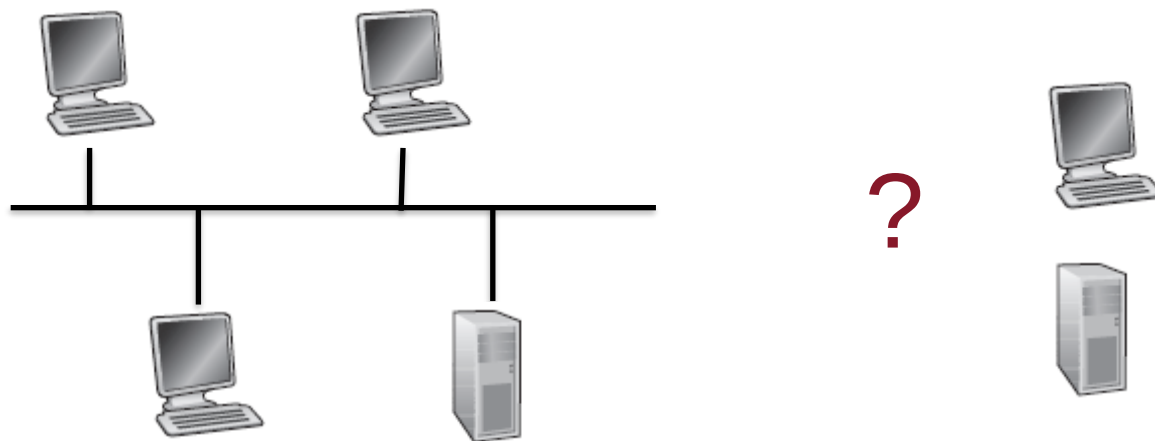
BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



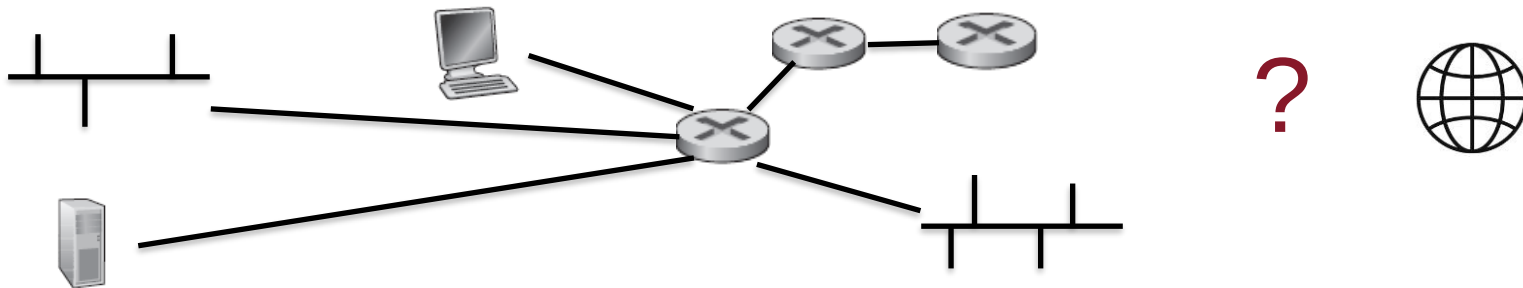
1. Az Internet felépítése
2. Protokollrétegek
3. Hálózati alkalmazások alapjai

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

- **Lokális hálózatok (Local Area Network, LAN)**
 - **Egymáshoz fizikailag közeli hosztok** Pl: Irodában 1 szoba/csoport
 - Vagy logikailag összetartozó hosztok
 - Egymást szomszédosnak látják
 - Nem kell útvonalválasztással foglalkozni a másik eléréséhez
 - Sokszor csak logikai a szomszédság
- A távol eső LAN-okat össze kellene kötni



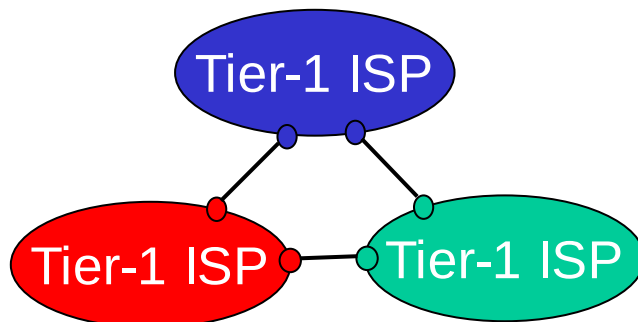
- **Nagyobb, városi kiterjedésű hálózat (Metropolitan Area Network, MAN)**
 - Egyik célja a nem túl távoli LAN-ok összekötése
 - Másik célja a LAN-ok csatlakoztatása az Internethez
 - Forgalom aggregációs (összegyűjtő) szerepe is van
- Nagyobb kapacitású hálózati linkek
 - Néhány 10 kilométeres hossz
- Útvonalválasztó csomópontok is megjelennek



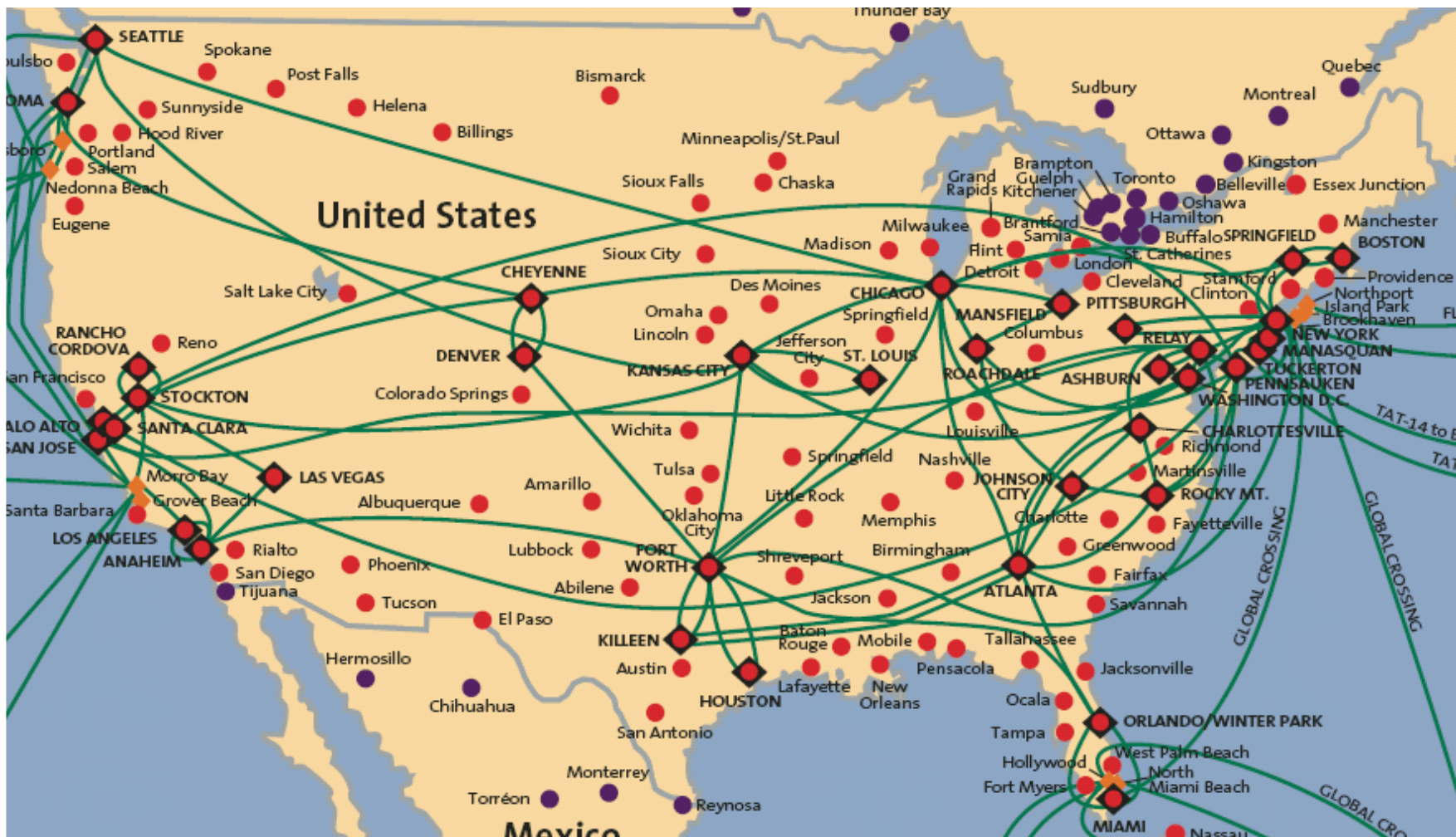
- **Wide Area Network (WAN)**
 - Fő célja, hogy távoli hálózatokat kössön össze
 - Tipikusan nincsenek benne hosztok
- Nagykapacitású linkek
 - Több 100 kilométeres hossz (logikailag)
- Nagysebességű útvonalválasztók
- Korábban eltérő technológia
 - Ma már szinte mindenütt IP és Ethernet



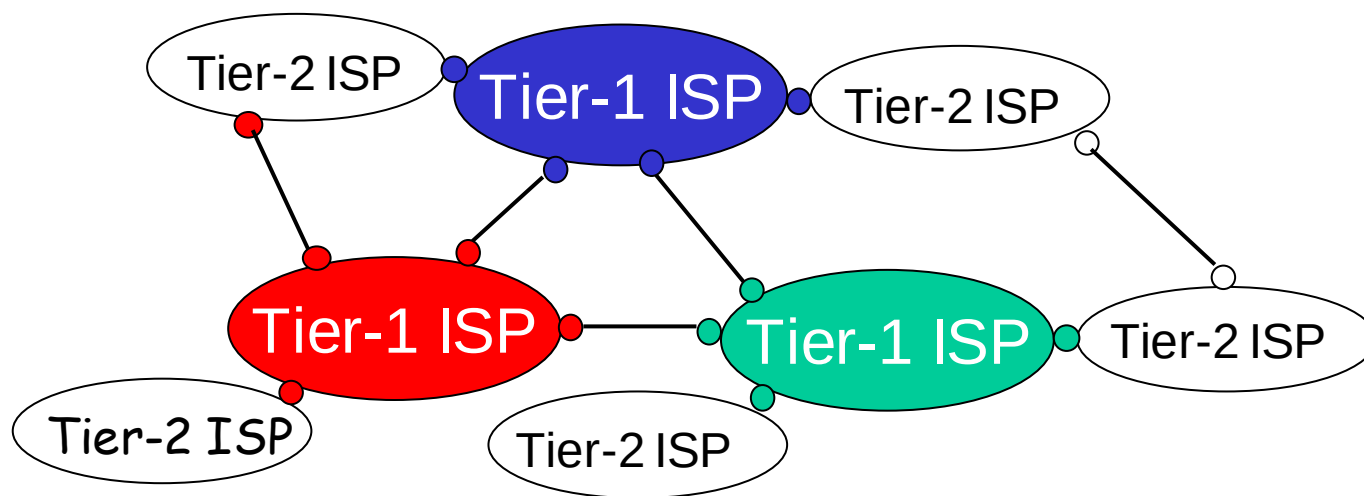
- Az Internet struktúrája nagyjából hierarchikus
 - Sok résztvevő van
 - Nem mindenki egyenlő
- A csúcson a **Tier-1** Internetszolgáltatók (ISP) hálózatai vannak, országos vagy nemzetközi lefedést biztosítva
 - Egymást egyenlő félként kezelik
 - Közvetlenül kapcsolódnak egymáshoz (peering)
 - Akár több link is lehet közöttük
 - Például Verizon, T-Mobile US (Sprint), AT&T, Deutsche Telekom



- A Sprint hálózata (néhány éve...)



- **Tier-2 ISP** Pl: Magyar Telekom, Yettel, One
 - Kisebb (gyakran regionális) ISP-k
 - Egy vagy több Tier-1 ISP-hez, és néhány Tier-2 ISP-hez is kapcsolódnak közvetlenül
- Kapcsolatok
 - A Tier-2 ISP fizet a Tier-1 ISP-nek azért, hogy az Internethez kapcsolja
 - A Tier-2 ISP felhasználója a Tier-1 szolgáltatónak, akár többnek is
 - Tier-2 ISP-k közvetlenül kapcsolódhatnak egymáshoz

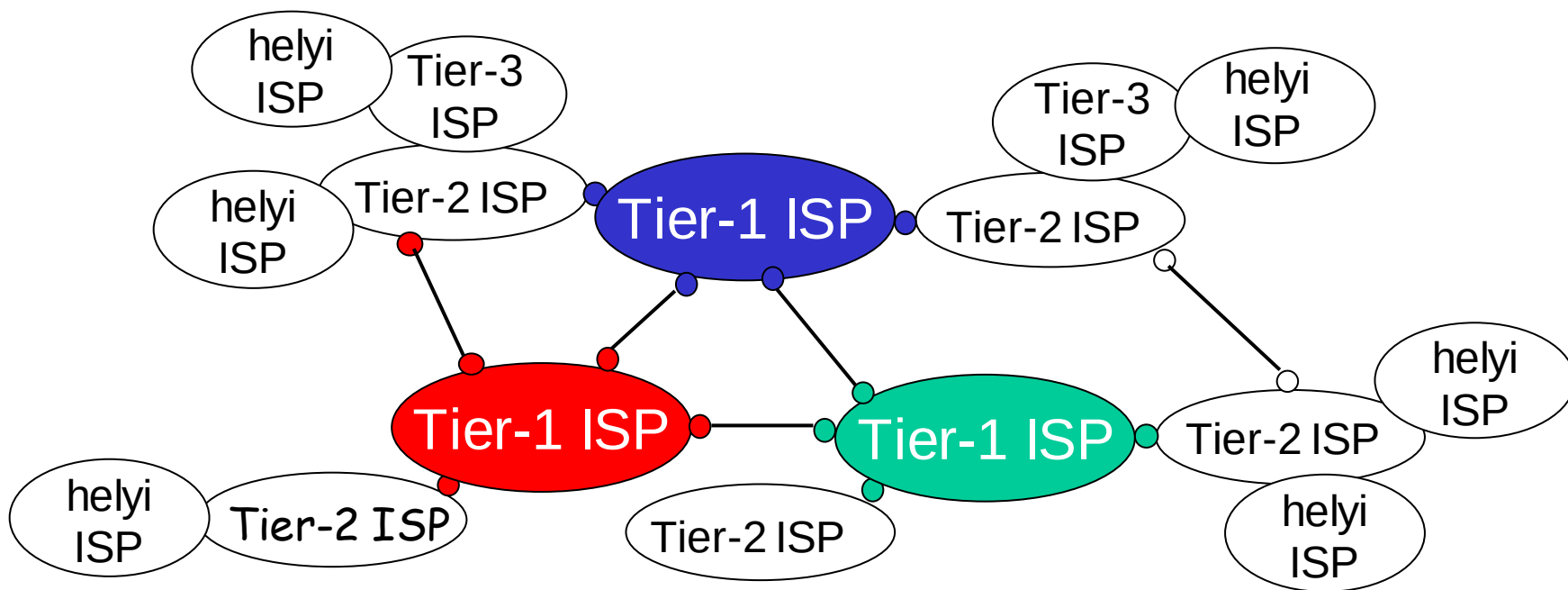


- **Tier-3 ISP és helyi ISP**

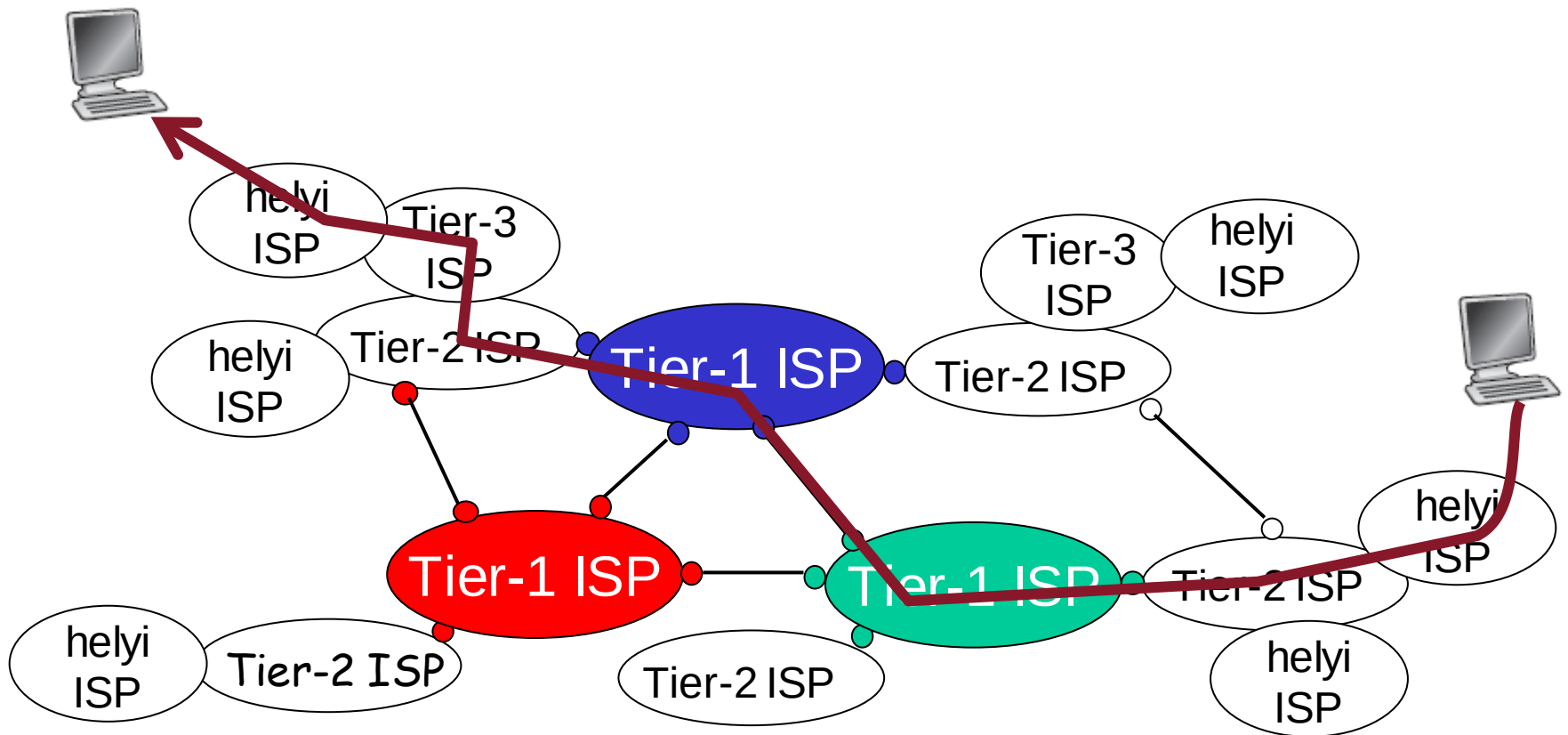
- Az utolsó szakasz a hosztok felé
- A végfelhasználóknak hozzáférési hálózatot nyújt

- Kapcsolatok

- A helyi és Tier-3 ISP-k felhasználói a magasabb szintű ISP-knek, melyek az Internethez kapcsolják őket



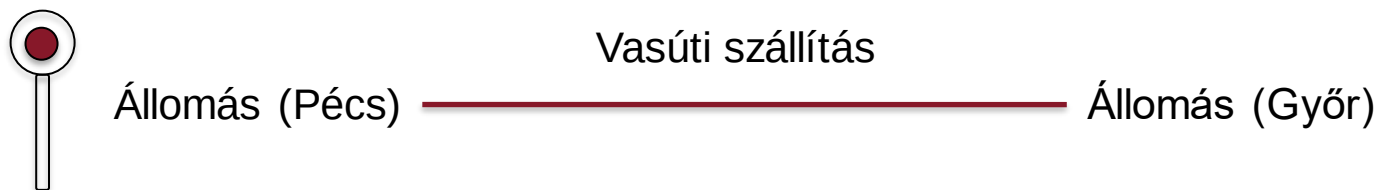
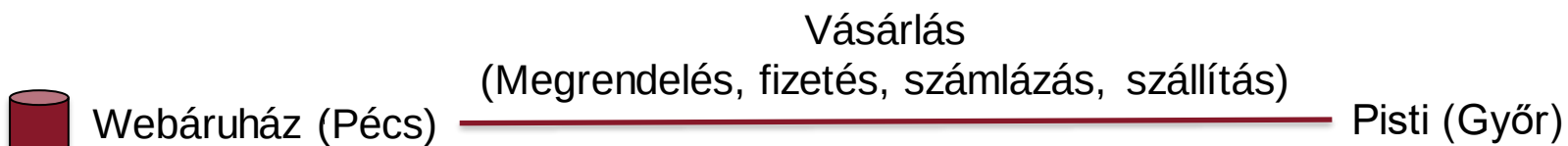
- Egy csomag számos hálózaton halad keresztül a célig



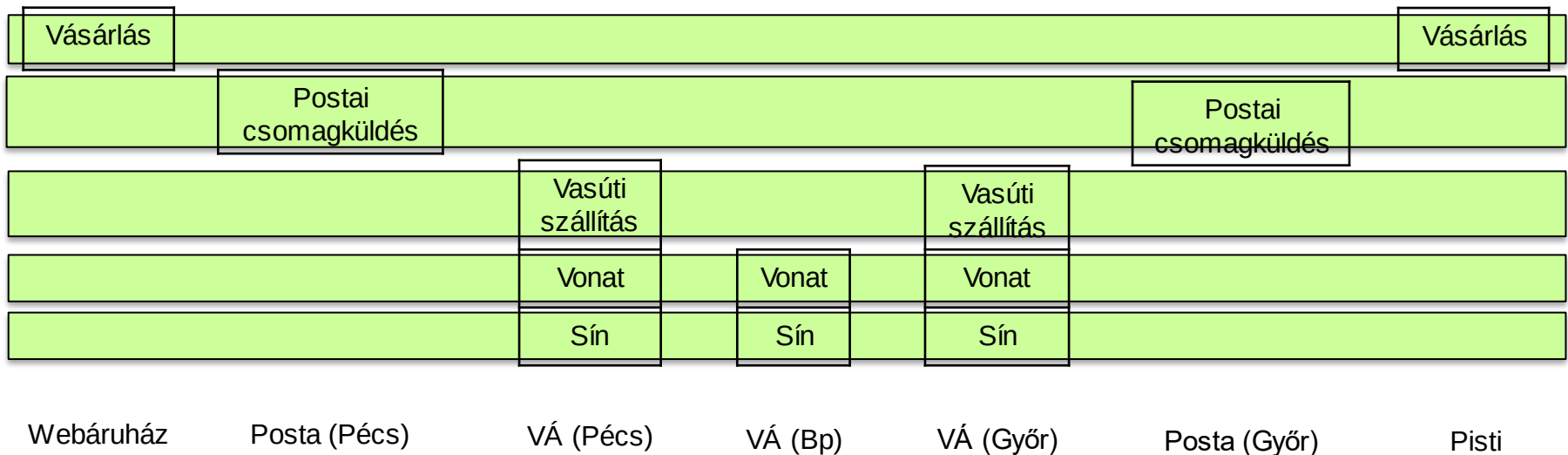
1. Az Internet felépítése
2. Protokollrétegek
3. Hálózati alkalmazások alapjai

- A hálózat bonyolult, összetett szerkezetű rendszer
 - Sok és sokféle elem
 - hosztok
 - útvonalválasztók
 - linkek
 - alkalmazások
 - protokollok
 - hardver, szoftver
- Van remény rendszerbe foglalni a hálózat szerkezetét (architektúráját)?
- Vagy legalább a hálózati architektúra tárgyalásának módját?

Egy webáruházban vásárolt termék házhozszállításának elemei



- Minden réteg egy szolgáltatást valósít meg
 - a rétegen belüli tevékenységekkel
 - az alacsonyabb rétegek szolgáltatásaira is támaszkodva



A RÉTEGSZERKEZET ELŐNYEI

- Komplex rendszereket átláthatóbbá tesz
- A világosan meghatározott szerkezet alapján a rendszer alkotórészeinek és azok viszonyainak azonosítását teszi lehetővé
 - Rétegelt **referenciamodell** a tárgyalás alapja
- A modularitás megkönnyíti a rendszer összetevőinek korszerűsítését, cseréjét
 - Egy rétegszolgáltatás megvalósításának módosítása transzparens a rendszer más elemei számára
 - Példánkban a postai csomagfeladás rendjének változása vagy akár a postai szolgáltató lecserélése közvetlenül nem befolyásolja a rendszer többi elemét
- Lehet ártalmas is a rétegezés?
 - Funkcióismétlés
 - Réteghatárok megsértése

A TCP/IP protocol stack rétegei:

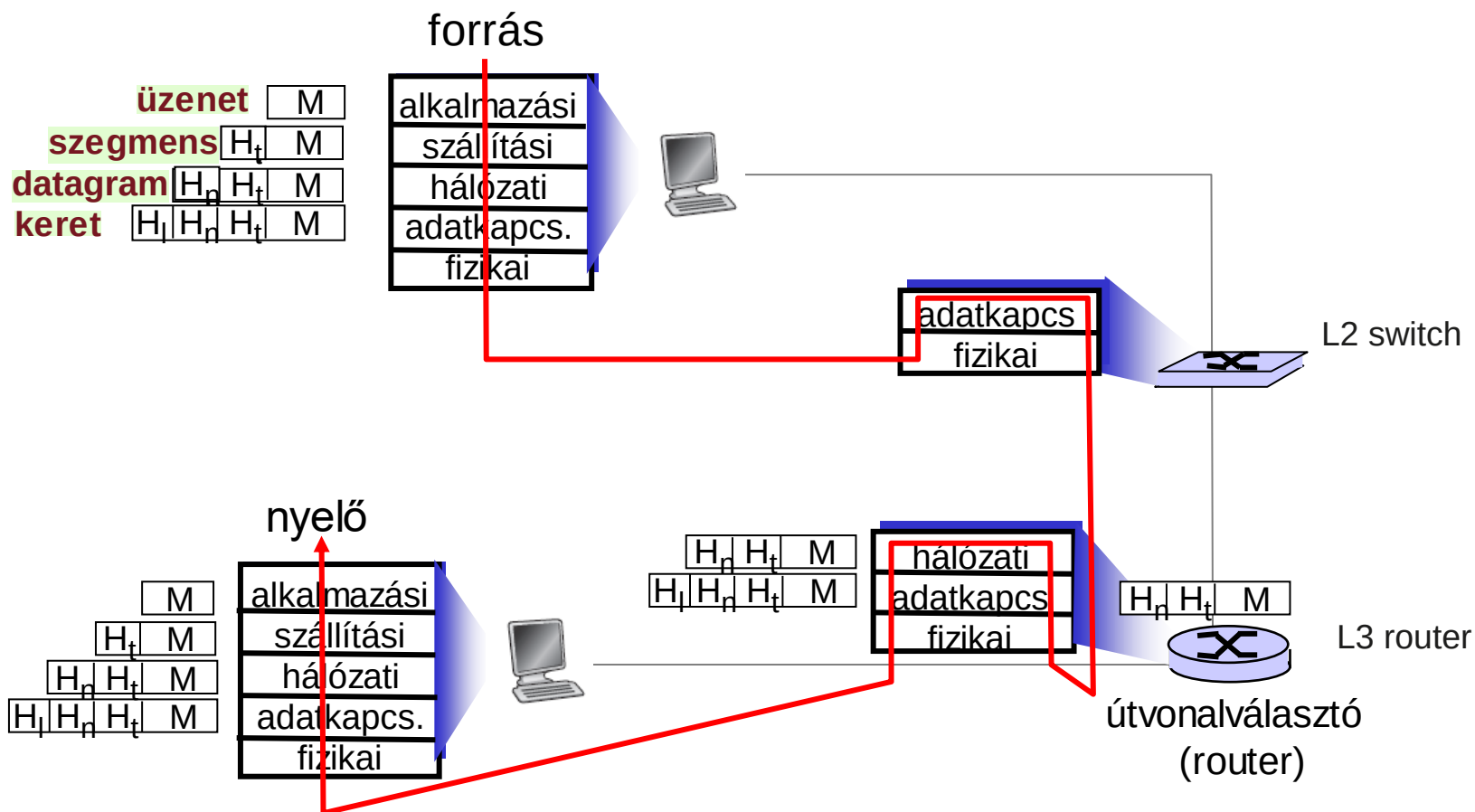
- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, SMTP, HTTP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, útvonalválasztó protokollok
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - PPP, Ethernet
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti összeköttetéseken

- ISO/OSI rétegezés
 - Két további réteg az alkalmazási és a szállítási réteg között
 - Megjelenítési (presentation)
 - Viszony (session)
 - Szükség esetén megvalósíthatók (valójában nem nagyon van rá szükség)
- Más hálózatokkal (nem Internet) való együttműködéshez
 - Valójában nem nagyon van már más

1. Az L. rétegben az L+1. rétegből kapott adatot feldaraboljuk
2. A darabokat az L. rétegbeli protokollnak megfelelően becsomagoljuk
 - Fejléccel látjuk el, hogy tudja kezelni a protokoll
 - Ez az L. réteg **Protocol Data Unit (PDU)**-ja
3. Továbbadjuk az L-1 rétegnek, ami elvégzi a szolgálatát
 - Pl. átviszi a PDU-nkat a hálózat egy másik pontjára
4. A túloldalon az ellenkezője történik
 1. Visszkapjuk a PDU-t
 2. Levesszük a fejlécet
 3. Összeillesztjük a darabokat

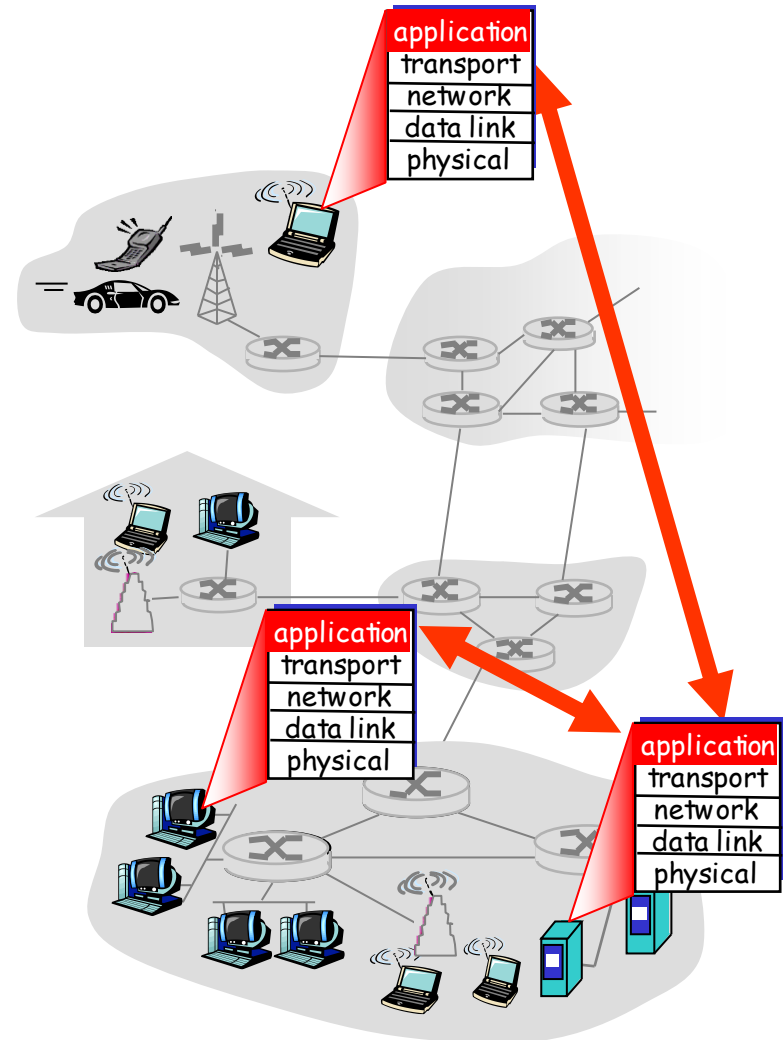
A BEÁGYAZÁS ILLUSZTRÁCIÓJA

- Adat eljutása a forrástól a nyelőig



1. Az Internet felépítése
2. Protokollrétegek
3. Hálózati alkalmazások alapjai

- Program, ami
 - (Különböző) hosztokon fut
 - Kommunikál hálózaton keresztül
 - Pl., a webszerver szoftvere kommunikál a böngésző (browser) szoftverével
- Az alkalmazások hosztokon futtatása az alkalmazások gyors fejlesztését és elterjesztését teszi lehetővé
- A hálózat belsejében (core) lévő eszközök nem futtatnak felhasználói alkalmazásokat
 - Más célú szoftvereket igen



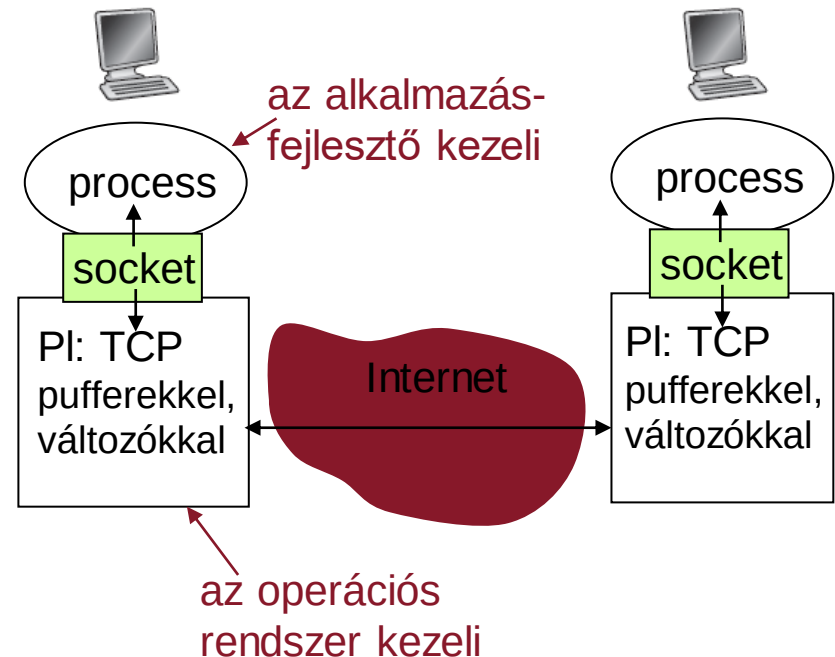
- **Processz** (folyamat): egy hoszton futó program vagy annak része 
- Azonos hoszton futó programok között inter-process kommunikáció lehetséges (az operációs rendszer által meghatározott módon)
- Különböző hosztokon futó processzek üzeneteket küldve kommunikálnak
 - **Kliens processz**: a kommunikációt kezdeményező program
 - **Szerver processz**: várja, hogy (a kliensek) kapcsolatba lépjenek vele („hallgatózik”)
- A peer-to-peer (P2P) architektúrájú alkalmazásoknak van kliens és szerver processzük is

- Logikai kommunikációs végpont

Socket = IP cím + portszám kombinációja

Egy kapcsolatot pedig KÉT socket határoz meg pontosan!

- A processz üzeneteket küld a hozzá kapcsolt socketnek és fogad a sockettől
- A socket az ajtóhoz hasonlítható
 - A küldő processz kiteszi az üzenetet az ajtó elé
 - A küldő processz az ajtó túloldalán lévő infrastruktúrára támaszkodik
 - A szállítási réteg szolgáltatja az átvitelt
 - Továbbítja az üzenetet a fogadó processz ajtajához



PROCESSZEK MEGCÍMZÉSE

- Ahhoz, hogy üzenetet kaphasson, a processznek rendelkeznie kell azonosítóval
- A hosztnak egyedi (például 32 bites IPv4) címe van
- Kérdés: a futtató hoszt IP címe megfelelően képes a processz azonosítására?
- Válasz: Nem, hiszen sok processz futhat egy adott hoszton.
- Az azonosító tartalmazza a hoszt (IP) **címét**, és azt a **portszámot** ami a processzhez van rendelve az adott hoszton
- Példák portszámokra:
 - HTTP szerver: 80
 - Mail szerver: 25
- Például: kommunikáció a moodle.hit.bme.hu biztonságos-webes alapú szerverrel
 - IP cím: 152.66.248.112
 - portszám: 443

- **A protokoll meghatározza**
 - **Az alkalmazott üzenetek típusát,**
 - pl., kérés, válasz
 - **Az üzenet szintakszísát (formátumát):**
 - Milyen mezők szerepelnek, és hogyan vannak elrendezve az üzenetben
 - **Az üzenet szemantikáját**
 - A mezőkben lévő információk jelentését
 - **Az üzenetek és a válaszok küldésének sorrendjét és módját**
- **Nyilvános (open) protokollok**
 - RFC-kben definiáltak
 - az együttműködést biztosítják
 - Például HTTP, SMTP
- **Szabadalommal védett (proprietary) protokollok**
 - Például Skype

- Az alkalmazásoknak eltérő (QoS) igényük lehet a szállítási szolgáltatásokkal szemben
- **Adatvesztés**
 - Egyes alkalmazások (pl. Internet telefon) elviselnek bizonyos mértékű adatvesztést
 - Másoknak (pl., fájlátvitel, telnet) 100%-ban megbízható adatátvitelre van szükségük
- **Időzítés, késleltetés**
 - Egyes alkalmazásoknak (pl. Internet telefon, interaktív játékok) alacsony késleltetésű adatátvitelre van szükségük
- **Sávszélesség**
 - Egyes alkalmazásoknak (pl. multimédia) szükségük van egy adott minimális sávszélességre
 - Mások (“elasztikus”, alkalmazások) bármilyen kis sávszélesség mellett képesek működni

Néhány alkalmazástípus szállítási szolgáltatási követelményei

Alkalmazás	Adatvesztés	Sávszélesség	Időzítésérzékeny
Fájlátvitel	veszteségmentes	rugalmas	nem
E-mail	veszteségmentes	rugalmas	nem
Webes dokumentumok	veszteségmentes	rugalmas (néhány kbps)	nem
Internetes telefonálás/ videokonferencia	veszteségtűrő	hang: néhány kbps- 1Mbps video: 10kbps- 10Mbps	igen, néhány 100 ms
Tárolt audio/video	veszteségtűrő	hang: néhány kbps- 1Mbps video: 10kbps- 25Mbps	igen, néhány másodperc
Interaktív játékok	veszteségtűrő	néhány kbps- 10Mbps	igen, néhány 100 ms
Azonnali üzenetküldés	veszteségmentes	rugalmas	nem

TCP szolgáltatás

- **Kapcsolatorientált:** összeköttetés felépítése (set up) szükséges a kliens és szerver process között
- **Megbízható szállítás:** a küldőtől minden megérkezik sorrendezve fogadóhoz
- **Forgalomszabályozás:** a küldő nem terheli túl a fogadót
- **Torlódáskezelés:** a küldő lefojtása ha hálózat túlterhelt
- Nem biztosít sem időzítést, sem minimális garantált sávszélességet

UDP szolgáltatás

- **Megbízhatatlan szállítás:** nem biztos, hogy minden és hogy helyes sorrendben érkezik meg
- Nem biztosít összeköttetés felépítést, forgalomszabályozást, torlódáskezelést, időzítést, vagy garantált sávszélességet

Alapvető hálózati alkalmazások

Alkalmazás	Alkalmazás rétegbeli protokoll	A használt szállítási protokoll
Email	SMTP [RFC 2821]	TCP
Távoli terminál elérés	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
Fájltávitel	FTP [RFC 959]	TCP
Multimédia adatfolyamok (streaming)	HTTP [RFC 1945 és 2616] (pl. YouTube), RTP	TCP vagy UDP
Internetes telefonálás	SIP, RTP, egyedi (pl. Skype)	tipikusan UDP



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Alkalmazási réteg
2023. március 10.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



A TCP/IP protocol stack rétegei:

- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, SMTP, HTTP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, útvonalválasztó protokollok
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - PPP, Ethernet
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti összeköttetéseken

1. A Web és a HTTP
2. Fájlátvitel
3. Az elektronikus levelezés protokolljai
4. Doménnév-szolgáltatás az Interneten

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

- **Weboldal**

- HTML nyelven írt oldal, ami további, hivatkozott objektumokat is tartalmaz
- Egy objektum lehet például kép, hang, videó, futtatható program vagy szkript

- **HTTP – HyperText Transfer Protocol**

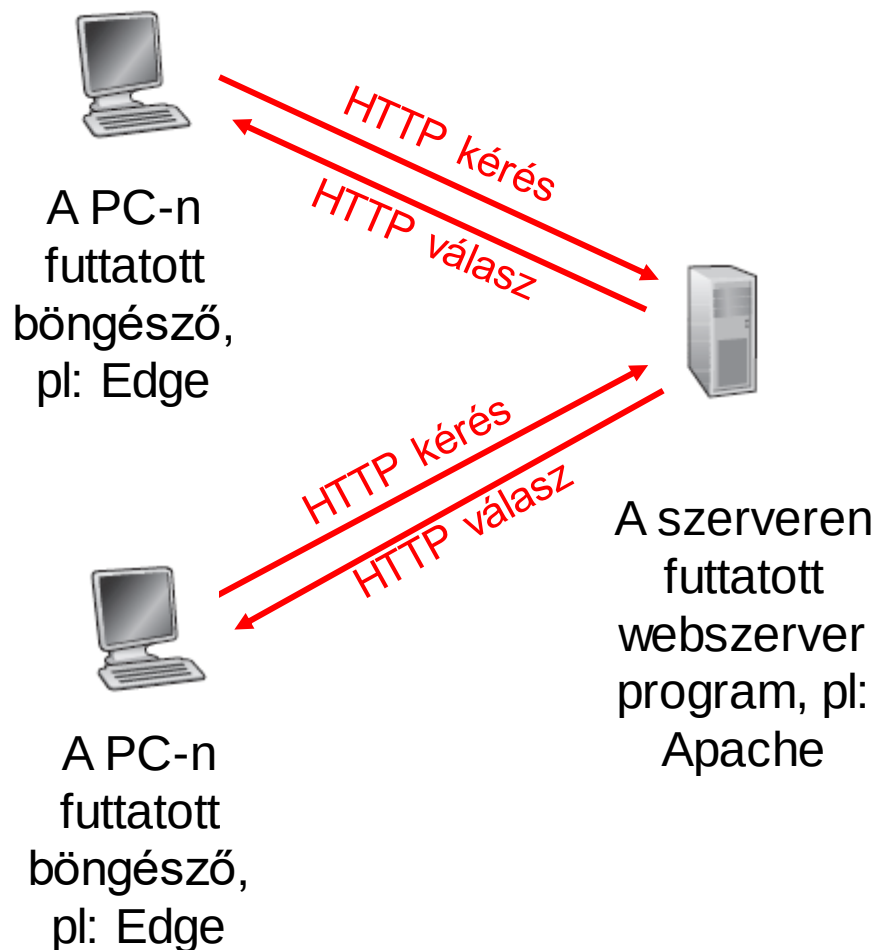
- **Objektum**

- Az oldalt alkotó objektumok nem feltétlenül ugyanazon a szerveren vannak
- Objektum hivatkozása **URL**-lel (Uniform Resource Locator) történik
- Például: `www.someschool.edu/someDept/pic.gif`

Szerver hosztneve

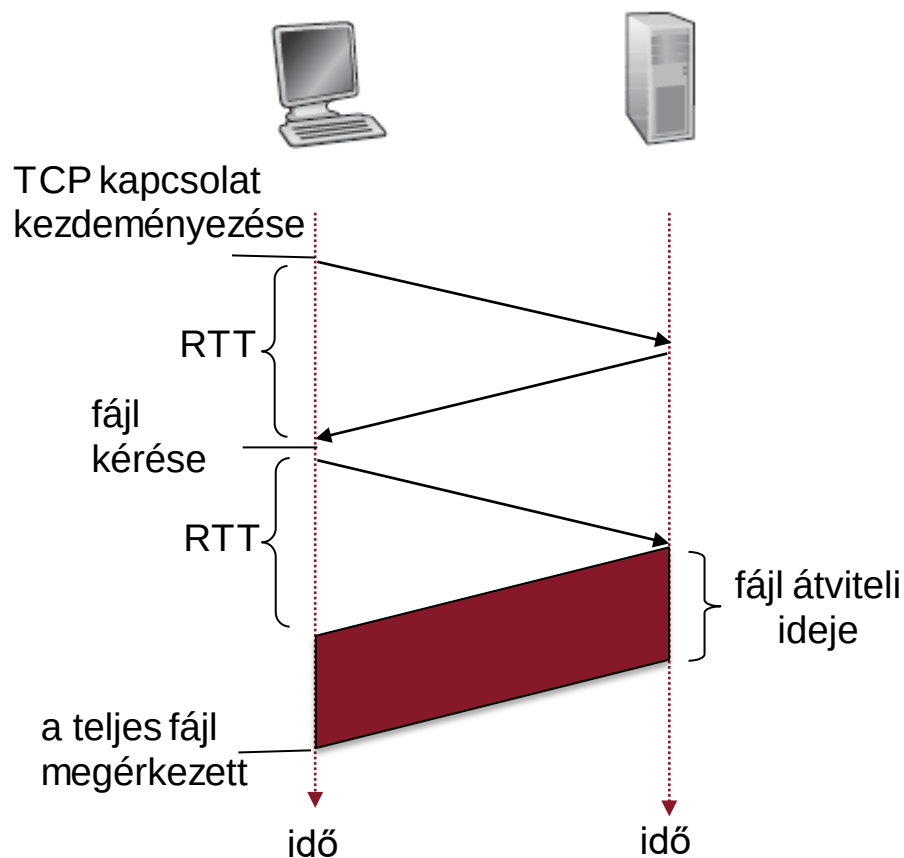
Elérési út a szerveren

- A Web alkalmazási réteg protokollja
 - **kliens**: böngésző, ami objektumokat kér, fogad és “megjelenít”
 - **szerver**: a webservert, ami objektumokat küld a kérésekre válaszul
- Verziók
 - HTTP/1.0: RFC 1945 (1996)
 - HTTP/1.1: RFC 2068 (1997)
 - HTTP/2: RFC 7540 (2015)
 - HTTP/3: RFC 9114 (2022)
 - A korábbi verziókhoz újabb RFC-k is íródtak



- **TCP-t használ**
 - A kliens TCP kapcsolatot kezdeményez a **szerverrel**, annak 80-as portján
 - a szerver elfogadja a TCP kapcsolatot a klienssel
 - HTTP üzeneteket váltanak
 - Végül lezárják a TCP kapcsolatot
 - Kivétel a HTTP/3
- Két üzenettípus
 - **Kérés** (request)
 - **Válasz** (response)
- **Állapotmentes** protokoll
 - A szerver nem tartja nyilván a kliens(ek) korábbi kéréseit
- Egy kapcsolaton belül küldött adatok
 - **Nem perzisztens** (időleges) eset: egyetlen objektumot/kapcsolat
 - HTTP/1.0
 - **Perzisztens** (állandó) eset: több (az összes) objektum/kapcsolat
 - HTTP/1.1 -től

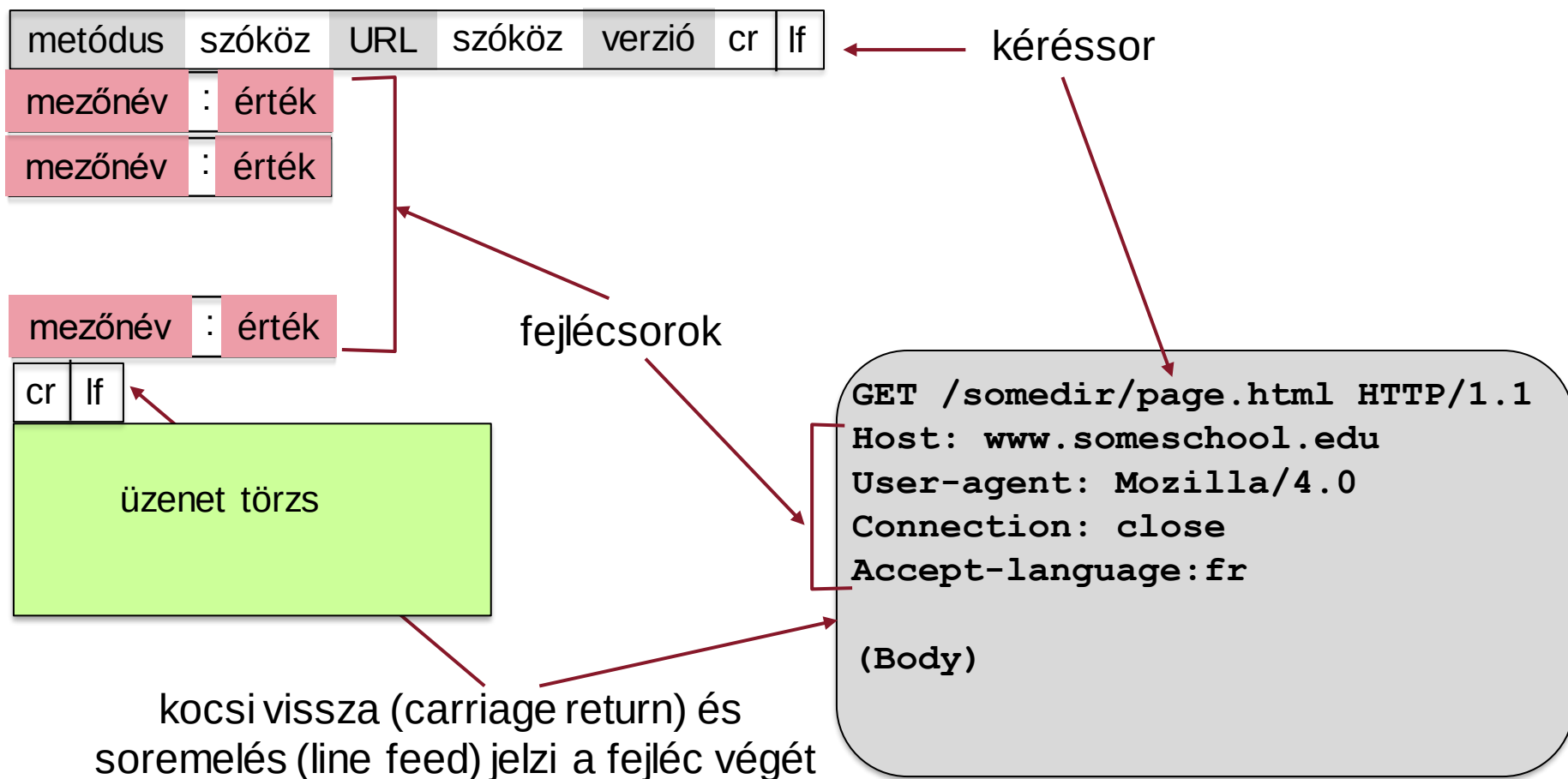
- Minden egyes objektumra
3·RTT+átviteli idő
 - TCP kapcsolat kezdeményezése (és elfogadása)
 - HTTP kérés és a HTTP válasz első byte-jának megérkezése
 - A fájl átviteli ideje
 - Kapcsolatbontás
- A böngésző esetleg nyithat párhuzamos TCP kapcsolatokat az egyes objektumokhoz
- Az operációs rendszernek kezelnie kell minden egyes TCP kapcsolatot



RTT: Round Trip Time

- A szerver **nyitva hagyja a TCP kapcsolatot** a válasz elküldése után
- Nem kell újra nyitni objektumonként
- Egymást követő HTTP üzenetváltások egy adott kliens-szerver viszonylatban ugyanazon a nyitott TCP kapcsolaton
- Pipelining (csővezetékezés, átlapolás) nélkül
 - A kliens csak akkor küld új kérést, ha az előző kérésre megérkezett a válasz
 - Kérés+válasz időigénye: $1 \cdot \text{RTT}$ **minden egyes hivatkozott objektumra**
 - N objektum időigénye: $(1+N+1) \cdot \text{RTT} + \text{átviteli idő}$
- Pipelining esetén
 - A kliens azonnal kérést küld amint hivatkozott objektumra bukkan
 - Kérés+válasz időigénye: $1 \cdot \text{RTT}$ **az hivatkozott objektumokra összesen**
 - N objektum időigénye: $(1+1+1) \cdot \text{RTT} + \text{átviteli idő}$
 - Alapértelmezett a HTTP/1.1-ben és újabbakban

ASCII (ember számára olvasható formátum)



HTTP/1.0

FONTOS TUDNI!!!

- **GET**
 - Objektum letöltése az URL alapján
 - Űrlap-adat feltöltése az URL-ben lévő információként

`www.hit.bme.hu/asearch?monkey&banana`

- **POST**
 - Űrlap-adat feltöltése az üzenet törzsében
- **HEAD** CSAK Header lekérése!
 - Csak fejléc (pl. hibakeresésre)

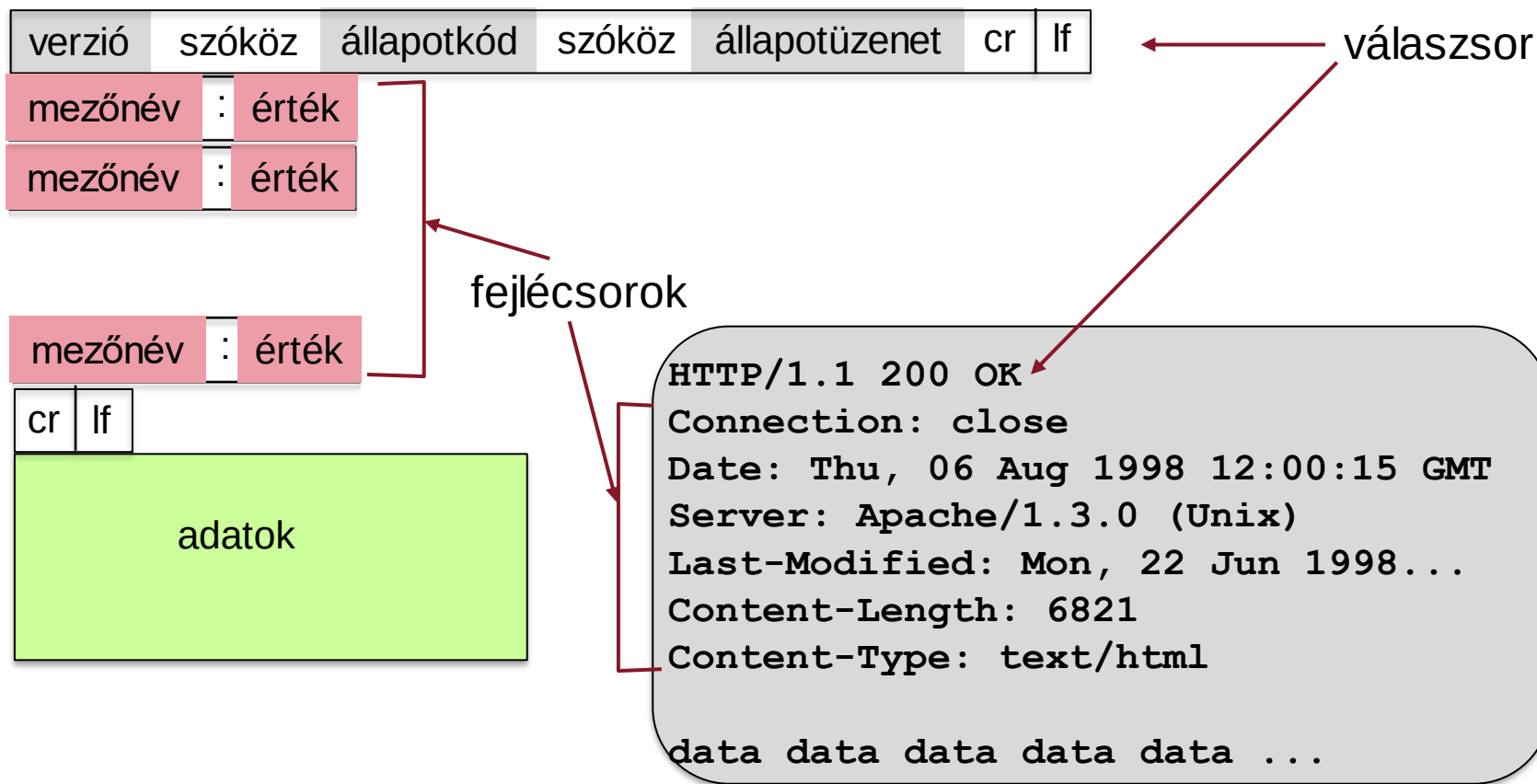
HTTP/1.1

- GET, POST, HEAD
- **PUT**
 - Fájl feltöltése adott könyvtárba
- **DELETE**
 - Fájl törlése könyvtárból
- Továbbiak: CONNECT, OPTIONS, TRACE

PUT -> Teljes objektum felülírása

PATCH -> csak megadott mezők módosítása

Felépítésében nagyon hasonló a kérésüzenethez



- A kérés teljesüléséről visszajelzést ad a szerver
- A legjellemzőbb állapot kódok és a kapcsolódó szöveges állapotleírások

200 OK

- sikeres kérés, a kért objektum az üzenetben

301 Moved Permanently

- a kért objektum elmozgatva, új helye megadva az üzenetben

400 Bad Request

- a szerver nem érti a kérésüzenetet

403 Forbidden

- az objektum hozzáférése megtagadva

404 Not Found

- a kért objektum nincs ezen a szerveren

505 HTTP Version Not Supported

- a küldéskor jelzett HTTP protokollváltozatot a szerver nem támogatja

- HTTP/2
 - A HTTP/1.x kiterjesztése
 - Új mechanizmusok
 - **Egyetlen TCP kapcsolat** épül fel
 - Több azonosított adatfolyam (stream) küldözget üzeneteket (message)
 - A folyamatokat **priorizálhatjuk** és **multiplexáljuk** (interleaving)
 - Adatátvitel **binárisan kódolt elemekben** (frame)
 - **Szerver oldali PUSH**: kérés nélkül küldhet objektumot
 - Előnyök
 - Gyorsabb
 - Head Of Line blokkolás elkerülhető* (ha az egyik folyam lassabb)
 - Prioritások állíthatók be
- HTTP/3
 - Nem kiterjesztés
 - **TCP helyett QUIC** transzport protokoll
 - Gyorsabb kapcsolat-felépítés tanúsítványkezeléssel

- A legtöbb weboldal használ sütiket (**cookie**)
- A mechanizmus alkotórészei
 - Süti fejlécsor a **HTTP válaszüzenetben**
 - Süti fejlécsor a **HTTP kérésüzenetben**
 - A kliens hoszt tárolja a sütit, amit a felhasználó böngészője kezel
 - **Háttéradatbázis a webszerveren**
- Mire használható?
 - **Felhasználó azonosítása**
 - **Felhasználói munkamenet vagy folyamat állapotának tárolása**, pl. kosár tartalma, webes e-mail
 - Ajánlások
- Személyes adatok védelme
 - A süti alapján egy sor dolgot megtudnak a szerverek felhasználókról
 - Érzékeny adatokat is tárolhatnak bennük
 - Az EU-ban jóvá kell hagynunk a használatát

SÜTIKEZELÉS ILLUSZTRÁCIÓJA

kliens sütifájl



Korábban eltárolt süti

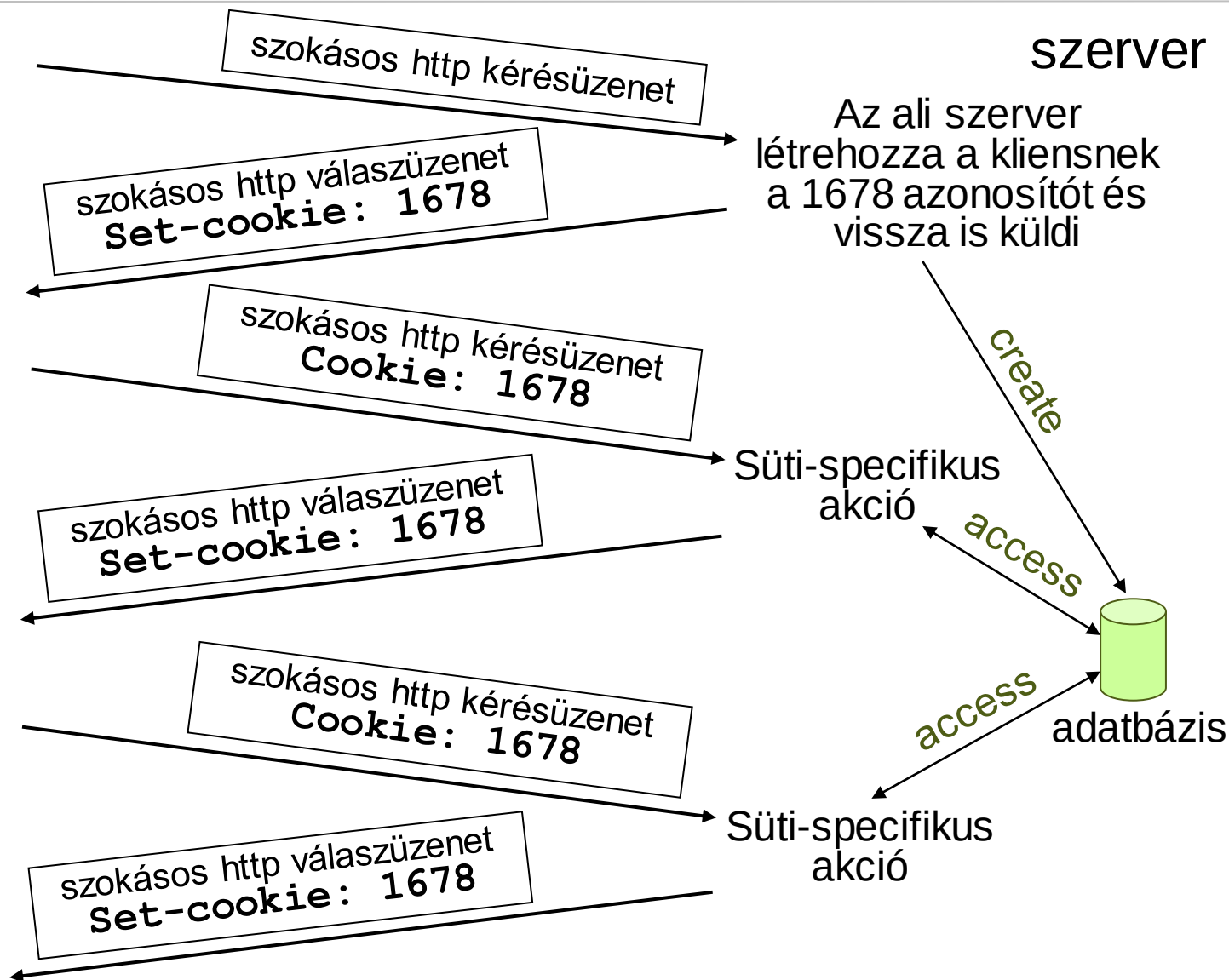


egy héttel később



szerver

Az ali szerver létrehozza a kliensnek a 1678 azonosítót és vissza is küldi



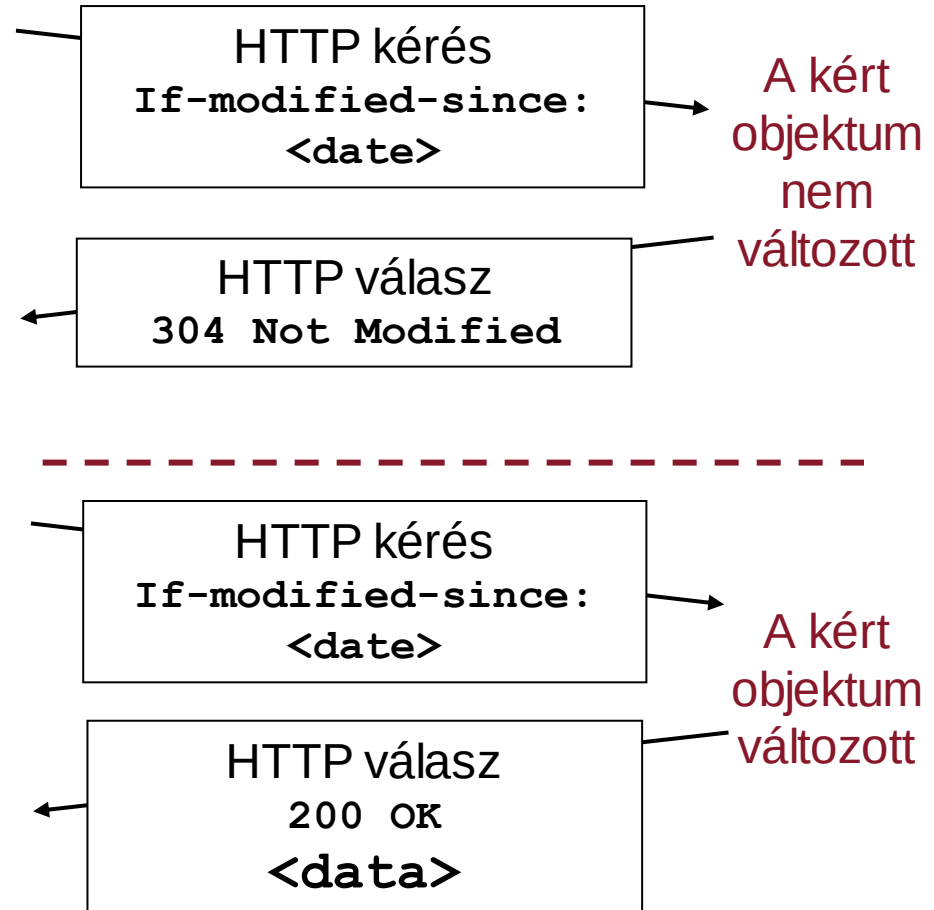
- A gyorsítótár (web-cache) segítségével a kliens kérését a tényleges szerver közreműködése nélkül szolgáljuk ki
- A böngésző minden HTTP kérést a gyorsítótárnak küld
 - Ha megvan a kért objektum a cache-ben: a cache elküldi a kliensnek
 - Ha nincs meg, akkor a cache kéri az objektumot a forrás szervertől, és miután megkapta, elküldi a kliensnek
- A cache kliensként és szerverként is funkcionál
- A gyorsítótárat tipikusan az ISP telepíti (egyetem, vállalat, lakossági szolgáltatást nyújtó ISP)

- Csökkentheti a kliens kérés kiszolgálásának válaszidejét
- Csökkenti az intézmény nyilvános csatlakozási linkjének forgalmát és kihasználtságát
- A sűrűn telepített cache-ek esetén a “szegényebb” tartalomszolgáltatók is hatékonyan tudják terjeszteni tartalmaikat az Interneten

- Cél, hogy ne küldjön el a szerver egy objektumot, ha annak legfrissebb változata megvan a cache-ben
- A cache a kérés fejlécében megadja a tárolt változat dátumát
- Ha a tárolt változat a legfrissebb, a szerver válasza nem tartalmazza az objektumot, és ezt a státuszban jelzi

cache

server



• HTTPS

- Biztonságos webelérés
- Hitelesítés – az oldal tényleg az, aminek mondja magát
- Titkosítás – egyszerű szöveg helyett titkosított adatátvitel
- A szerver a 443-as TCP porton hallgatózik

• REST

- Representational State Transfer
- Nem protokoll, hanem koncepció hálózati applikációkhoz, API-khoz
 - Állapotot csak a kliens tárol
 - Erőforrások azonosítása URI (Uniform Resource Identifier) segítségével
- A jellegzetes funkciók HTTP kérési metódusokkal és a rájuk kapott válaszokkal megoldhatók
 - CRUD (create, retrieve, update, delete)

1. A Web és a HTTP
2. Fájlátvitel
3. Az elektronikus levelezés protokolljai
4. Doménnév-szolgáltatás az Interneten

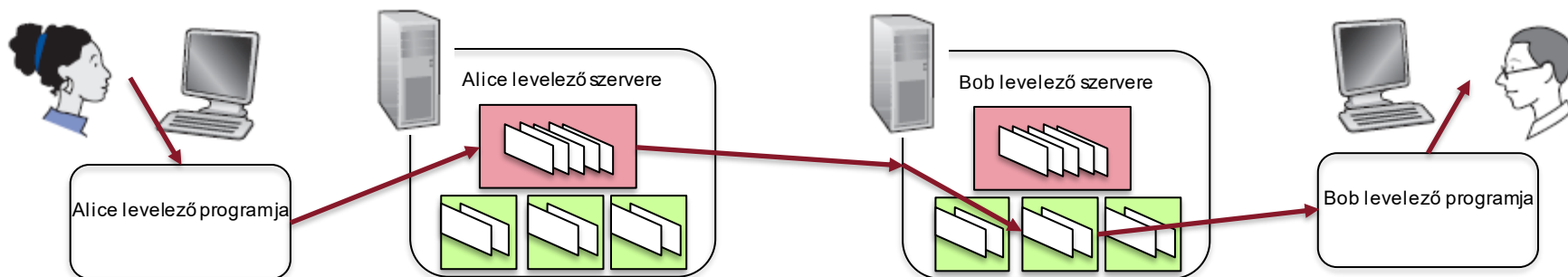
- File Transfer Protocol
- Kliens/szerver modell
 - Kliens: az átvitelt kezdeményező fél, aki távolra, vagy távolról szeretne **fájlokat másolni**
 - Szerver: a távoli hoszt
- **Sávon kívüli (Out-of-band) vezérlés**
 - Külön vezérlési és adatátviteli összeköttetés
- Nem állapotmentes

- Lépések
 1. Az FTP kliens csatlakozik az FTP szerverhez a 21-es TCP porton
 2. A kliens azonosítása a vezérlési összeköttetésen
 3. A kliens áttekintheti a távoli könyvtárakat megfelelő parancsokat küldve a vezérlési összeköttetésen
 4. A mikor fájlátvitel parancs érkezik a szerver nyit egy **újabb TCP összeköttetést** (a 20-as porton) a kliens felé a fájl átviteléhez
 5. A fájl átvitele után a szerver lezárja ezt a TCP összeköttetést, de a vezérlési kapcsolat tovább él, és ugrunk a 3. lépésre
- Parancsok
 - ASCII kódolású **szöveggént** küldve a vezérlési összeköttetésen
- Válaszok
 - **Státuszkódok** és szöveges leírások, hasonlóan mint a HTTP-nél

- FTP problémák
 - Nem biztonságos
 - Mindent külön paranccsal kell letölteni
- Secure CoPy (SCP)
 - Titkosított, ssh alapú átvitel: a 22-es TCP porton keresztül
 - Csak fájlátvitel
- SSH File Transfer Protocol (SFTP)
 - Titkosított, ssh alapú átvitel: a 22-es TCP porton keresztül
 - Könyvtárkezelés is elérhető (az ftp-hez hasonlóan)
- Server Message Block (SMB)
 - Fájlmegosztás Microsoft hálózatokban
 - Kliens-szerver architektúra
 - A 445-ös TCP porton keresztül
 - Szabad hozzáférésű implementáció: Samba

1. A Web és a HTTP
2. Fájltávitel
3. Az elektronikus levelezés protokolljai
4. Doménnév-szolgáltatás az Interneten

- Levelezőszerver
 - A **postafiók** (mailbox) a felhasználó bejövő üzeneteit (leveleit) tárolja
 - A **kimenő üzenetsorban** (message queue) az elküldendő levelek vannak
- Felhasználói ügynök program
 - Levél megírása, szerkesztése, olvasása
 - Levél küldése és lekérése a szervernek, ill. szervertől
 - pl. Outlook, Opera Mail, Thunderbird
- Azonosított szerepek:
 - Mail User Agent (MUA)
 - Mail Submission Agent (MSA)
 - Mail Transfer Agent (MTA)
 - Mail Delivery Agent (MDA)
- Az egyes hosztokon több szerep megvalósítása is szükséges lehet
- A levélküldési protokoll szempontjából egy levelezőszerver lehet:
 - Kliens – kezdeményező, küldő
 - Szerver – fogadó



1. Alice felhasználói ügynökével (MUA-Alice) levelet ír és megcímzi `bob@someschool.edu`
2. Alice felhasználói ügynöke elküldi a levelet Alice levelezőszerverének (MSA-Alice), ami a szerveren a kimenő üzenetsorba teszi
3. A levelezőszerver (MTA-Alice) kapcsolatot létesít Bob levelezőszerverével (MTA-Bob)
4. Az egyik szerver (MTA-Alice) elküldi a levelet a másiknak (MTA-Bob)
5. Bob levelezőszervere elhelyezi a megkapott levelet Bob postafiókjába (MDA-Bob)
6. Bob a felhasználói ügynöke (MUA-Bob) segítségével elolvassa a levelet, levél-hozzáférés

- TCP-t használ a levelek megbízható továbbítására a kliens és a szerver között, a 25-ös porton
- közvetlen átvitel a küldő és fogadó levelezőszerver között
- három szakaszból áll
 - kézfogás, üdvözlés (greeting)
 - üzenet (levél) továbbítása
 - lezárás
- parancs/válasz protokollüzenetek
 - ASCII szöveges parancsok
 - állapotkód, szöveges állapotleírás a válaszban
- az üzenetnek (levél) 7-bites ASCII kódolásúnak kell lennie

SMTP KAPCSOLAT ILLUSZTRÁCIÓ

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: The Internet?
C: We are not interested in it.
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

- Eredetileg csak **szöveges** üzenet formátum
 - Fejlécsorok, pl:
 - To:
 - From:
 - Subject:
 - Nem tévesztendő össze az SMTP parancsokkal!
 - Törzs
 - a “levélüzenet”: csak ASCII karakterek
- SMTP problémák
 - Csak szöveg?
 - Azonosítás?
 - Adatbiztonság?



- Kiterjesztés **bináris** adatokra
 - Multimedia Mail Extension
 - További fejlécek jelzik, pl:
 - MIME-Version
 - Content-Transfer-Encoding
 - Content-Type

- **Post Office Protocol (POP3)**

- Szöveges parancsok és válaszok
- Azonosítás
- Tranzakciók
 - **letöltés és törlés mód:** a MUA lokálisan tárolja a leveleket
 - letöltés és megtartás mód: később a felhasználó másik MUA-ja is letöltheti
- A POP3 nem tárol állapotinformációkat az egymást követő munkamenetek (session) között

- **Internet Mail Access Protocol (IMAP)**

- Minden levélüzenetet egy helyen tárol: a levelezőszerveren
- Támogatja, hogy a felhasználó mappákba (folder) rendezze tárolt üzeneteit
- A felhasználói állapotokat is tárolja az egymást követő munkamenetek (session) között:
- A mappák neveit, valamint a mappák és levélüzenet azonosítók összerendelését
- HTTP: **Gmail**, Hotmail, Yahoo! Mail, stb.

1. A Web és a HTTP
2. Fájltávitel
3. Az elektronikus levelezés protokolljai
4. Doménnév-szolgáltatás az Interneten

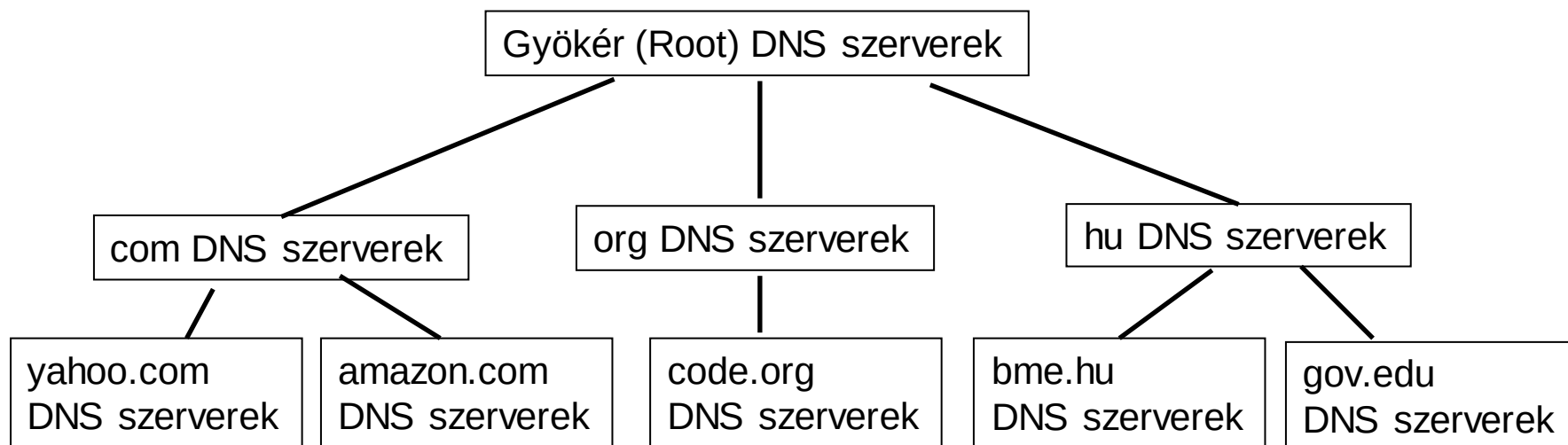
- Internet hosztok, routerek azonosítása
 - IP cím
 - Emberi használatra is alkalmas hosztnév, pl. www.hit.bme.hu
 - Hogyan párosíthatók az IP címek és a nevek?
- **Domain Name System (DNS)**
 - Elosztott adatbázis hierarchiába szervezett, nagyszámú szerver
 - Alkalmazás rétegbeli protokoll
- **DNS szolgáltatások**
 - Hosztnév IP címre „fordítása” www.google.com -> 142.250.74.14
 - Hoszt álnevek (alias) valódi (kanonikus) nevekre fordítása
 - Levelezőszerverek neve
 - Terheléselosztás: több IP cím tartozik egy kanonikus névhez, pl. meg többszörözött web szerverek

- Elosztott információ-tárolás

- Elosztott felelősség
- Robusztusság
- Karbantartás
- Skálázhatóság

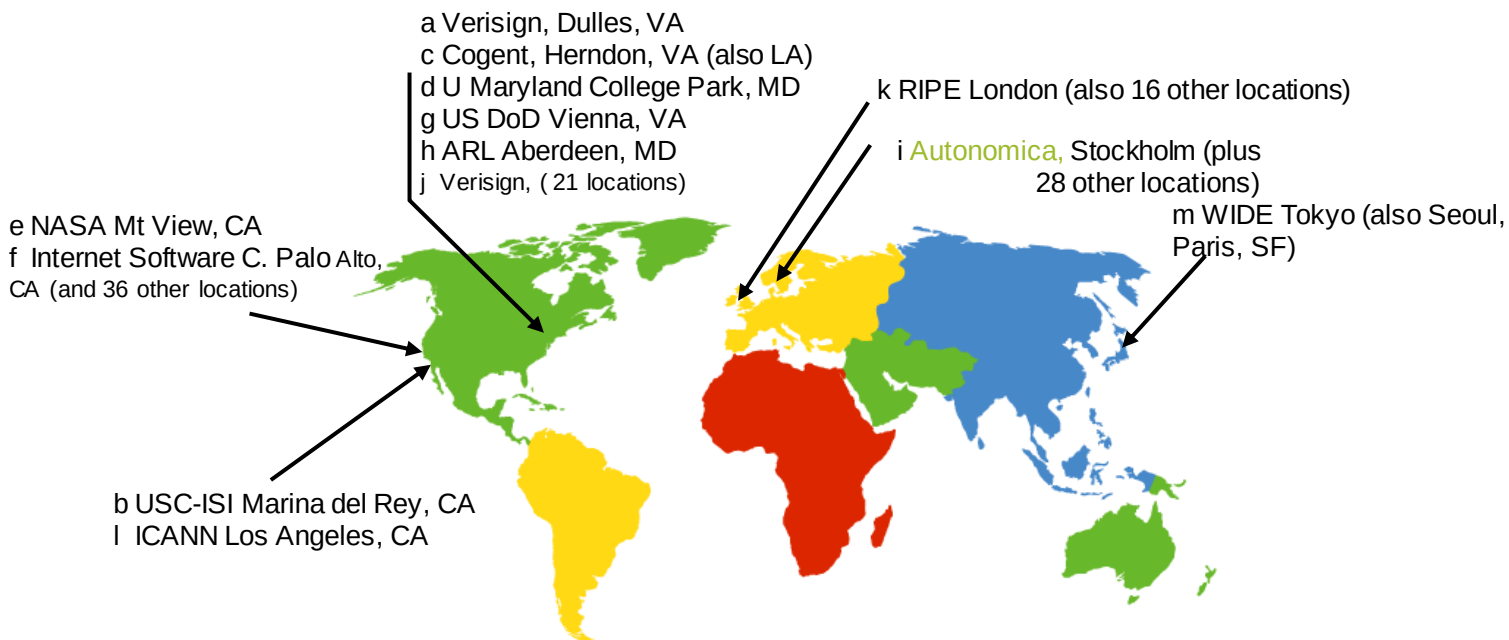
- **Hierarchia**

- Strukturált keresés
- Egy szervernek elég az alatta lévő szint szervereit ismerni
- Skálázhatóság



- **Gyökér (Root) szint** Tudja hol vannak a TLD szerverek
 - Világszerte 13 szerver
 - Ismeri a TLD szerverek címét
 - Hozzájuk fordulnak a helyi szerverek

- **Legfelső szint (Top Level Domain, TLD)** .com, .hu végződések
 - A com, org, net, edu **domének** (körzetek)
 - Országokhoz tartozó domének, pl: hu, it, de



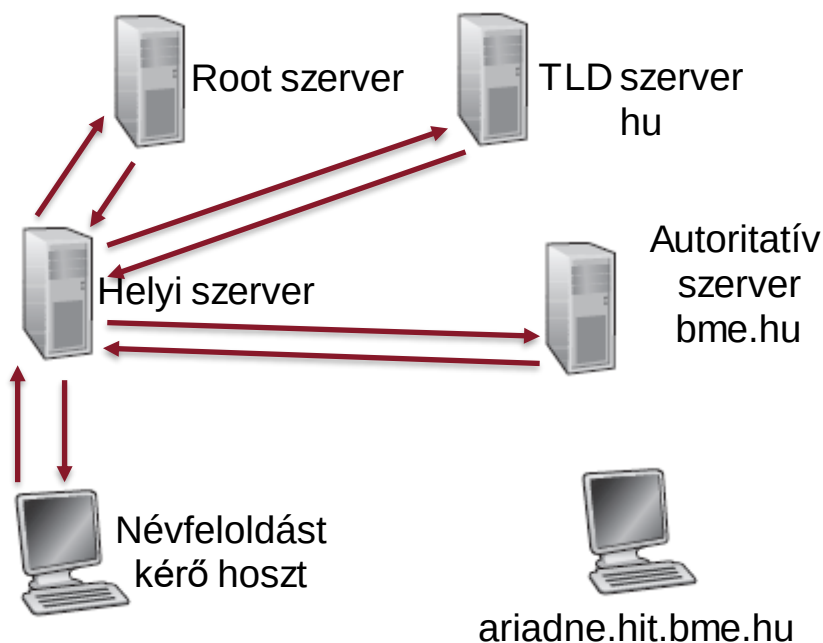
- **Hiteles (Authoritative) szerverek**
 - Internetes hosztokat üzemeltető szervezetek szerverei
 - Gyakorlatilag minden céghez, egyetemhez, stb. tartozik egy
 - A szervezet hosztjainak, szervereinek nevét tudja feloldani
 - Nem biztos, hogy a szervezet üzemelteti

- **Helyi (Local, Default) szerver**
 - Nem tartozik a hierarchiába
 - Minden ISP működtet egyet
 - A hosztok ettől kérik egy ismeretlen doménnév vagy hosztnév feloldását
- **Gyorsítótár (cache)**
 - Egy megismert összerendelést a szerver bizonyos ideig tárol
 - Minden szerver alkalmazhatja
- TLD címek helyi tárolása

• Iteratív lekérdezés

- a megszólított névszerver megadja, hogy helyette melyik névszerverhez kell fordulni a kéréssel

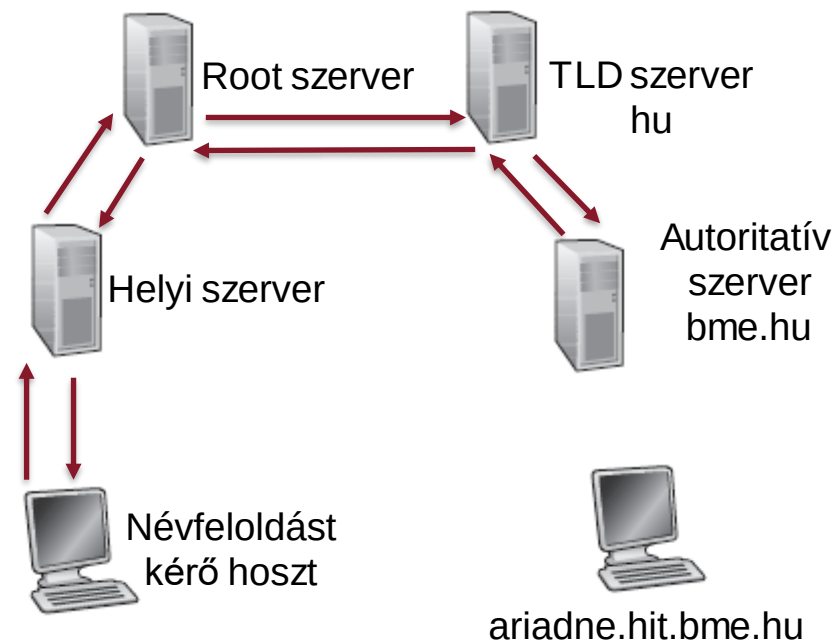
Rekurzívvál ellentétben a resolver minden lépésnél visszaküldi a következő szerver címét. Többkörös párbeszéd.



• Rekurzív lekérdezés:

- A névfeloldás feladatát a megszólított névszerverre bízza

Te megkérdezed a DNS resolvert és ő elvégz mindent!
Csak egy kérdést tesztel fel, egy választ kapsz vissza



- **Resource Record (RR)**

- Típus (type)
- Név (name)
- Érték (value)
- Élettartam (Time To Live)

- **A** típusú RR

- név: **hosztnév**
- érték: a névhez tartozó IP cím

- **NS** típusú RR

- név: egy **domén** neve (pl. foo.com)
- érték: a domén autoritativ szerverének a hosztneve

- **AAAA** típusú RR

- név: hosztnév
- érték: a névhez tartozó IPv6 cím

- **CNAME** típusú RR

- név: egy valódi (**kanonikus**) névhez tartozó álnév (pl. www.polito.it)
- érték: a kanonikus név

- **MX** típusú RR

- név: a levelező szerver álneve
- érték: levelező szerver kanonikus neve

- **Lekérdezés (query) és válasz (reply)**
 - Azonos üzenetformátum
 - A fejléc jelzőbitjeiből derül ki, hogy melyik
- **Azonosítás**
 - 16 bites szám a lekérdezéshez
 - A válaszban ugyanez az azonosítószám
- A kérdések név- és típusmezőkkel adottak

azonosítás	jelzőbitek
kérdések száma	válasz RR-ek száma
hiteles RR-ek száma	kiegészítő RR-ek száma
Kérdések (változó számú kérdés)	
Válaszok (változó számú RR erőforrás-bejegyzés)	
Autoritatív szerver adatok (változó számú RR erőforrás-bejegyzés)	
Kiegészítő információk (változó számú RR erőforrás-bejegyzés)	



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Socketek, szállítási réteg
2023. március 13.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



A TCP/IP protocol stack rétegei:

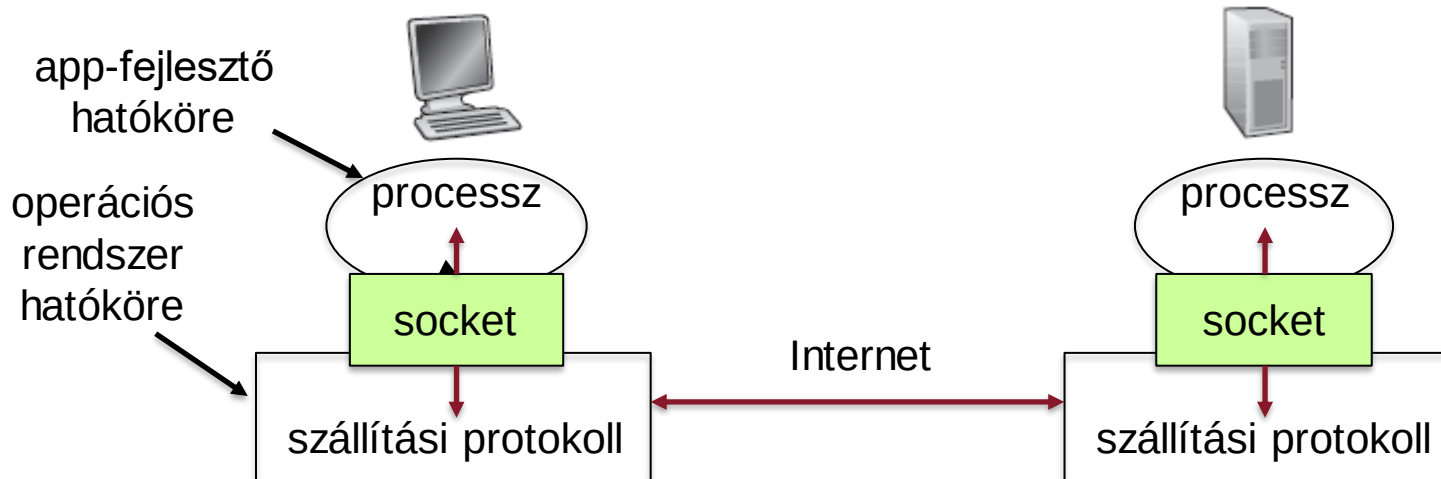
- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, SMTP, HTTP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, útvonalválasztó protokollok
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - PPP, Ethernet
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti összeköttetéseken

1. Socketek programozása
2. A szállítási réteg szolgáltatásai
3. Nyalábolás és nyalábbontás
4. Összeköttetés nélküli szállítás: UDP

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

- **Socket** (ismétlés)
 - A **hálózati applikáció processze** nyitja meg
 - A hoszt operációs rendszere kezeli
 - Egyfajta ajtószerű interfész a processz számára
 - **Üzenetek küldésére és fogadására is képes**
 - Szállítási réteg szintű kapcsolatot teremt távoli processzek között
- **Socket API**
 - 1981-ben vezetik be a BSD4.1 UNIX-ban
 - Az applikáció felelőssége
 - Létrehozás
 - Használat
 - Megszüntetés
 - Kliens-szerver kommunikációra
 - Alapvető típusok
 - Megbízható, **kapcsolat-orientált**, **adatfolyam (stream)** átvitelére
 - Megbízhatatlan, **adatcsomag (datagram)** orientált

- Megbízható szállítás TCP-vel (ismétlés)
 - Kapcsolatorientált
 - Nyugtázott (minden megérkezik)
 - Sorrendhelyes
- Átvitt adatok
 - Összetartozó adatrészek, szegmensek
 - Nem az applikáció feladata az újraküldetés vagy átszortrendezés



- **Szerver**

- **Folyamatosan futó** processz
- **Várakozik a kliensre**
- Hallgatózik egy adott porton:
Egy nyitott szerver-socketet
tart fenn a kliensek fogadására

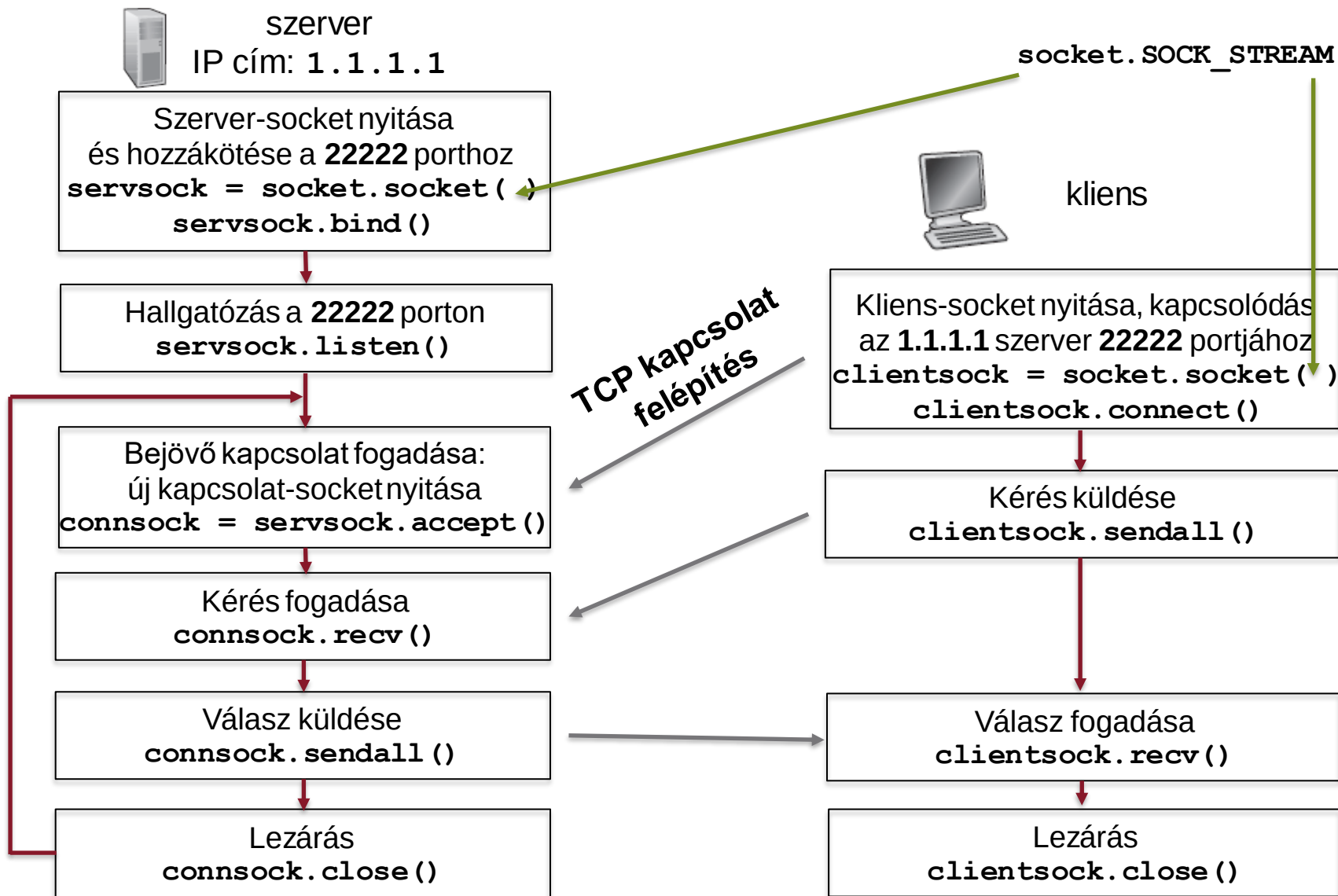
- **Kliens**

- **Kezdeményezi a csatlakozást**
a szerverhez
- **Egy kliens-socketet** nyit
- A szerver-socket adatait
használja: **IP cím, portszám**
- A kliens-sockethez az
operációs rendszer rendel
portszámot általában

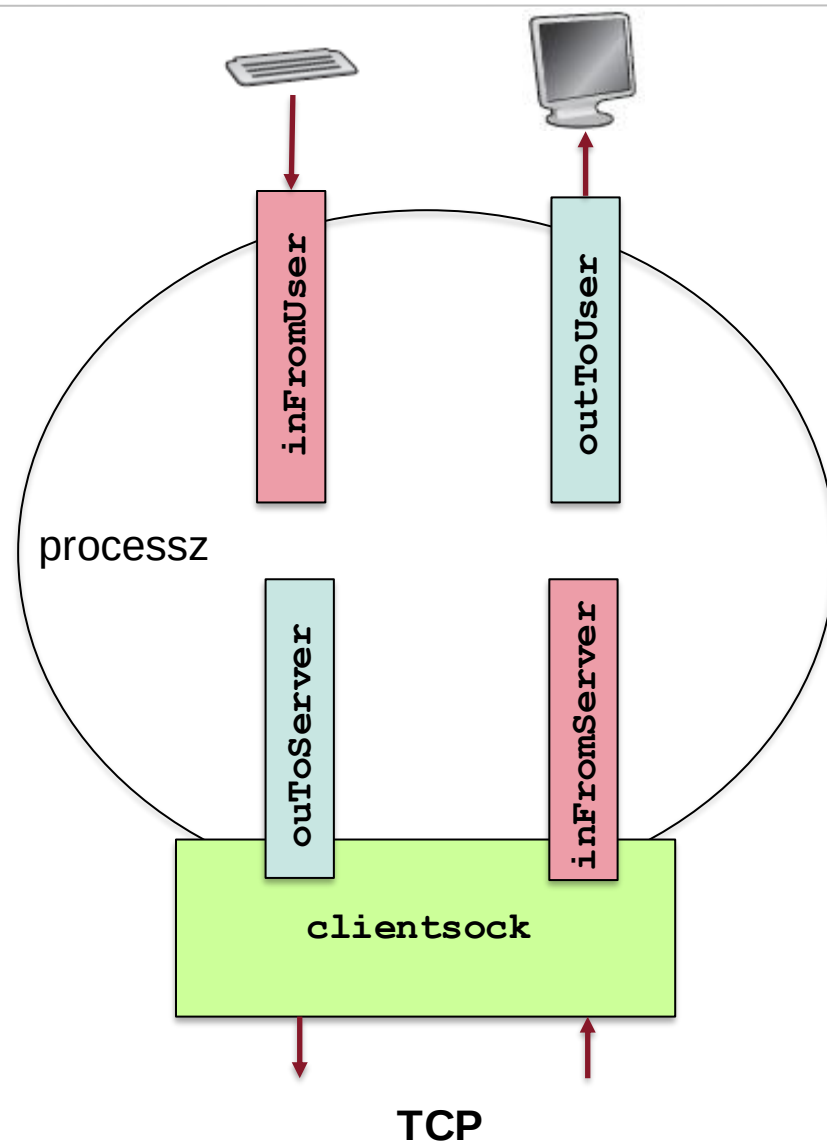
- **Adatátvitel**

- A kliens-socket felől kapcsolat
épül fel a szerver-socket felé
- A **szerver fogadja a klienst**,
de...
- **Az egyes kliensekhez újabb és
újabb kapcsolat-socketeket**
nyit meg
- A kliens oldali **IP cím és
portszám** alapján tudja a
szerver a megfelelő kapcsolat-
sockethez irányítani az
adatokat
- Így kommunikálhat **több
kliens** is a szervernek
ugyanazt a portját használva

TCP ALAPÚ SOCKET-KOMMUNIKÁCIÓ LÉPÉSEI



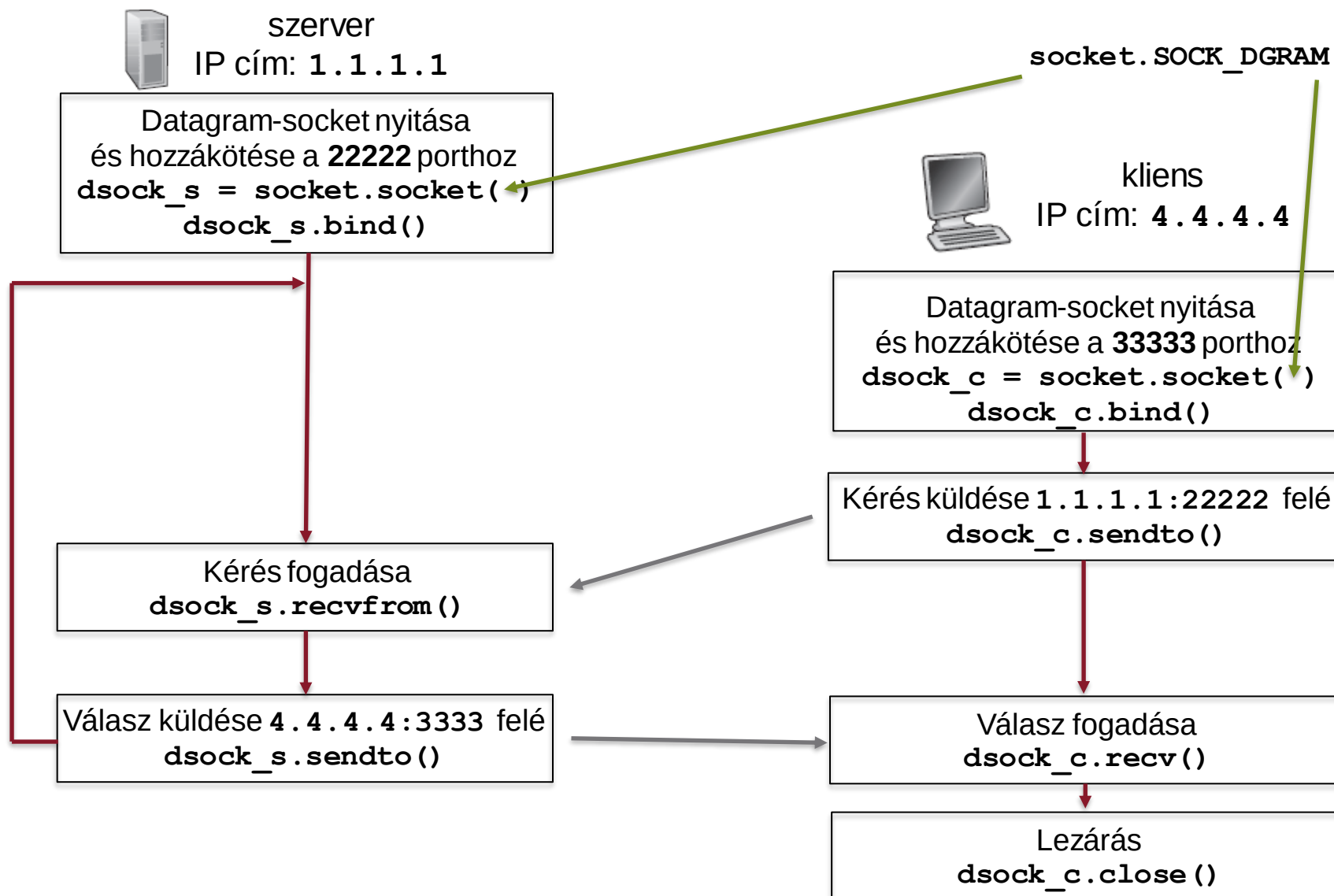
- Egy **folyam (stream)** egy processzből kijövő, vagy abba bemenő **bájt sorozat**
- Egy bemeneti (input) stream **egy forráshoz** van kötve, pl. billentyűzet, vagy socket
- Egy kimeneti (output) stream **egy nyelőhöz** van kötve, pl. képernyő, vagy socket



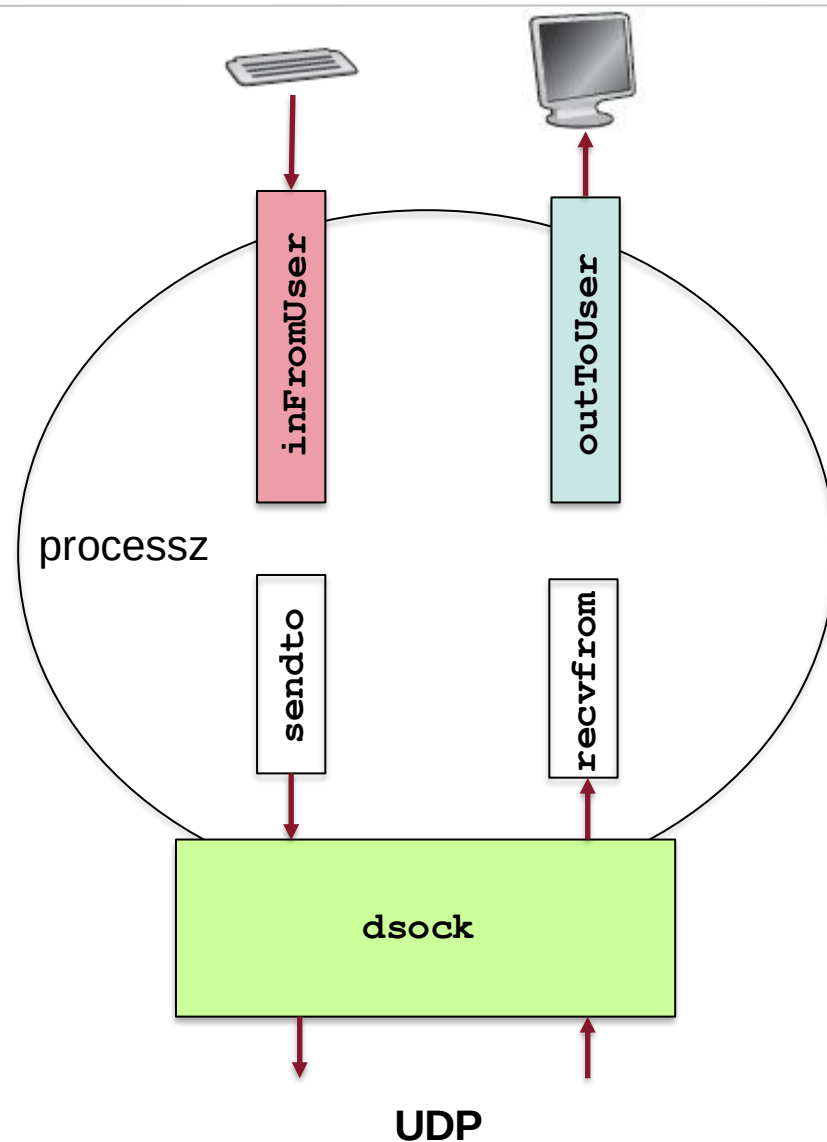
- Egyszerű hálózati alkalmazás
- Kliens
 - Beolvas egy sort és elküldi a szervernek
 - Kiírja amit a szervertől kapott
- Szerver
 - Kiolvassa a socketből amit a klienstől kapott
 - Konvertálja csupa nagybetűre
 - Az eredményt visszaküldi a kliensnek

- UDP
 - Nincs felépített kapcsolat a kliens és a szerver között
 - Nincs nyugtázás
 - Nincs sorrendezés
- Datagramok küldése (kliens)
 - A socket a processztől kapott datagramhoz hozzáteszi a portszámot és az IP címet
- Datagramok fogadása (szerver)
 - A kliens azonosítása csak közvetlenül a küldési IP cím és portszám alapján történhet
- Következmény
 - Egy processz egy socketen keresztül több kliens-socket felől kaphat datagramokat
 - A különböző kliensek datagramjai keverve érkeznek a processzhez

UDP ALAPÚ SOCKET-KOMMUNIKÁCIÓ LÉPÉSEI



- A socketbe nem stream jelleggel megy az adat, hanem egyesével címezve
- A socketből nem stream jelleggel jön az adat, hanem egyesével
- A példa ugyanaz



- TCP és UDP socketek
- Akkor az alkalmazás-fejlesztőnek kell minden magasabb szintű protokollt implementálni?
 - Nyilván nem.
 - Internet protokollok implementációi szabványos könyvtárakban, pl:
 - http, webbrowser
 - smtpplib, smtpd, imaplib, poplib
 - ftplib
 - ssl (biztonság)
- A python beépített modulokkal támogat
- Csak python szkriptekben írhatunk alkalmazást?
 - Nyilván nem.
 - Más nyelvek is támogatják a socketeket
 - Java, C++, C#, Ruby, stb.
 - És sok közülük a protokollokat is

1. Socketek programozása
2. A szállítási réteg szolgáltatásai
3. Nyalábolás és nyalábbontás
4. Összeköttetés nélküli szállítás: UDP

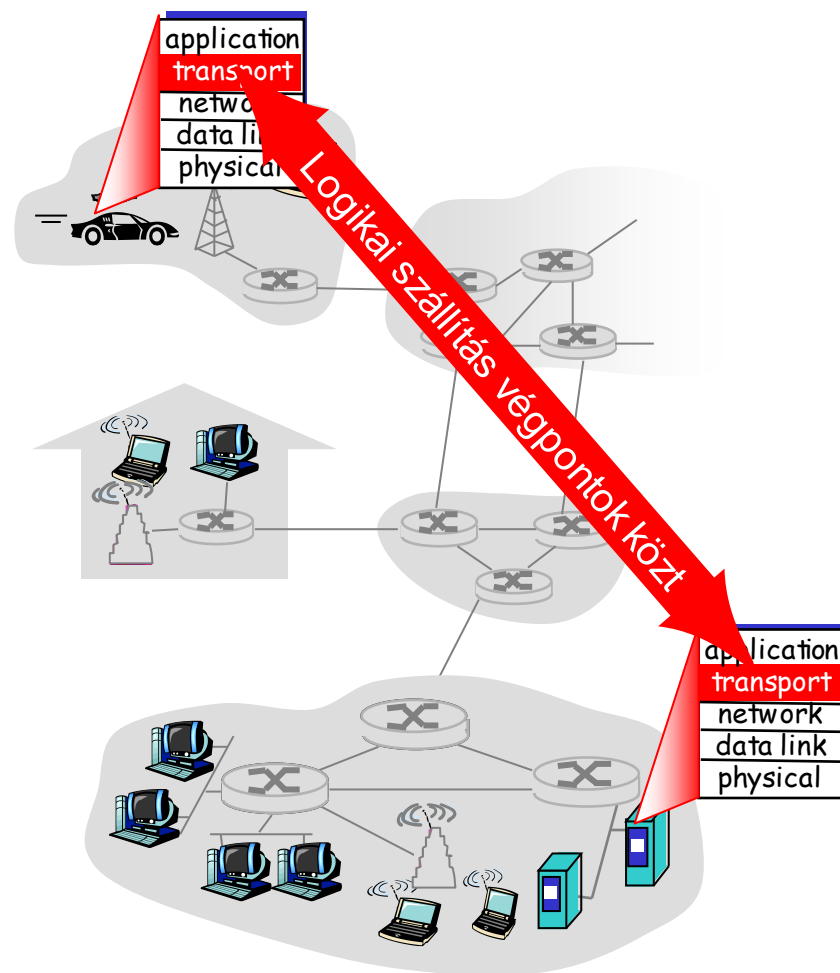
SZÁLLÍTÁSI SZOLGÁLTATÁSOK ÉS PROTOKOLLOK

- A különböző hosztokon futó alkalmazások processzei közötti **logikai kommunikációt** biztosítják
- A szállítási protokollok a végponti rendszereken, hosztokon futnak
 - Küldő oldal: az alkalmazásprotokoll üzeneteit **szegmensekre darabolva** adja át a hálózati rétegnek
 - Fogadó oldal: **összeállítja az eredeti üzenetet a szegmensekből**, és átadja az alkalmazási rétegnek
- Szállítási protokollok
 - Internet: TCP és UDP (és QUIC)

Processz: Zoom videóhívás

TE gépeden is van egy "Zoom processz"

A FOGADÓ gépén is van egy "Zoom processz"



- Szállítási réteg: logikai kommunikáció a (hosztokon futó) **processzek között**
- Hálózati réteg: logikai kommunikáció a **hosztok között**
IP cím alapján megtalálja a célgépet.
- Minden réteg az alatta lévő szolgáltatásait használja:
 - Alkalmazási réteg a szállítási réteget
 - Szállítási réteg a hálózati réteget

TCP szolgáltatás

- **Kapcsolatorientált:** összeköttetés felépítése (set up) szükséges a kliens és szerver processz között
- **Megbízható szállítás:** a küldőtől minden megérkezik sorrendezve fogadóhoz
- **Forgalomszabályozás:** a küldő nem terheli túl a fogadót
- **Torlódáskezelés:** a küldő lefojtása ha hálózat túlterhelt
- Nem biztosít sem időzítést, sem minimális garantált sávszélességet

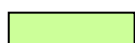
UDP szolgáltatás

- **Megbízhatatlan szállítás:** nem biztos, hogy minden és hogy helyes sorrendben érkezik meg
- **Nem biztosít összeköttetés felépítést, forgalomszabályozást, torlódáskezelést, időzítést, vagy garantált sávszélességet**

1. Socketek programozása
2. A szállítási réteg szolgáltatásai
3. Nyalábolás és nyalábbontás
4. Összeköttetés nélküli szállítás: UDP

NYALÁBOLÁS/NYALÁBBONTÁS

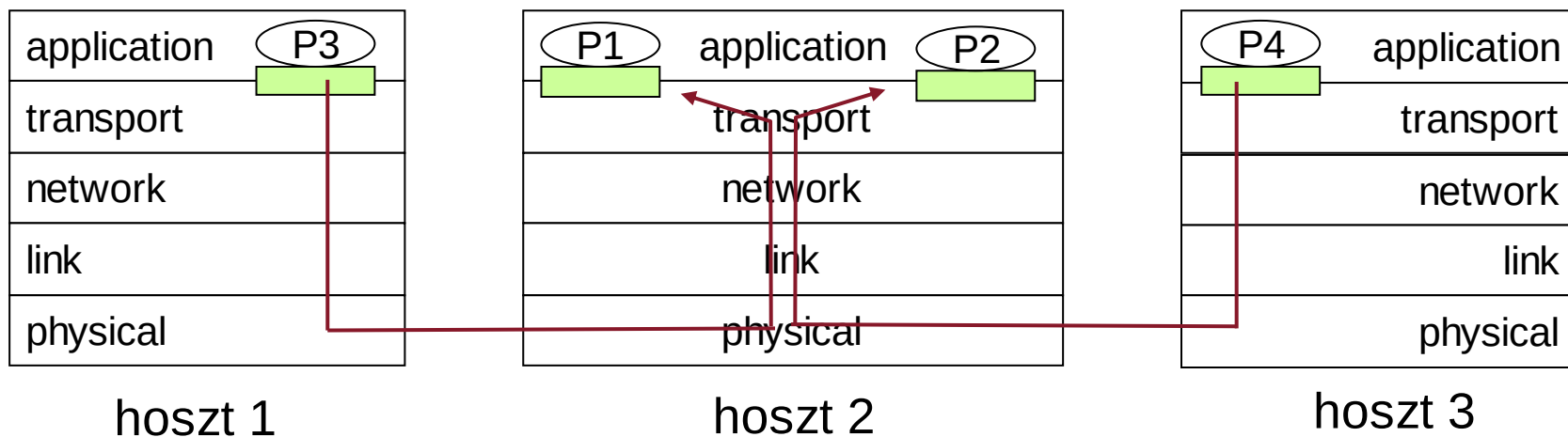
- Nyalábolás (**multiplexálás**) a küldő hosztnál
 - Különböző socketek adatainak összegyűjtése,
 - Fejlécezése (a demultiplexáláshoz)
- Nyalábbontás (**demultiplexálás**) a fogadó hosztnál
 - A vett szegmensek továbbítása a megfelelő socketnek



= socket



= processz



A DEMULTIPLEXÁLÁS MŰKÖDÉSE

- A hoszt megkapja az **IP datagramot (csomag, packet)**
 - Minden datagramnak van forrás és cél IP-címe
 - Minden egyes datagram pontosan egy darab szállítási rétegbeli **szegmenst** hordoz
 - Minden szegmensnek van forrás és cél portszáma
- A hoszt a szegmenst az IP-cím és a portszám alapján irányítja a megfelelő sockethez



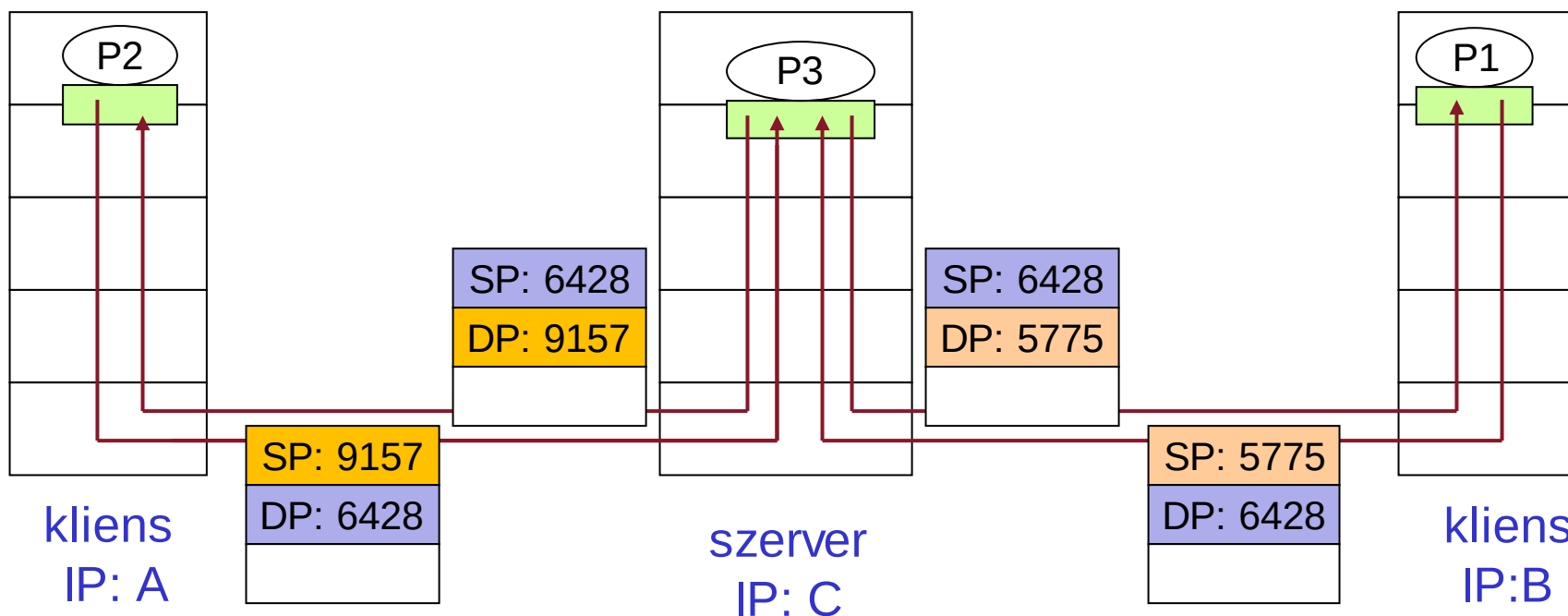
a TCP/UDP szegmens szerkezete

- Socket létrehozása és portszám-hozzárendelés

```
dsock_s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
dsock_s.bind(('1.1.1.1', 2222))
```

- Az UDP socketet egy értékpár azonosítja
 - (cél IP cím, cél portszám)
- Amikor a hoszt kap egy UDP szegmenst:
 - megvizsgálja a szegmensben szereplő cél portszámot
 - az ennek megfelelő portszámú socketnek adja át a szegmenst
- A különböző forrás IP-című és/vagy forrás portszámot tartalmazó szegmensek a cél portszámuk alapján ugyanahhoz a sockethez kerülhetnek

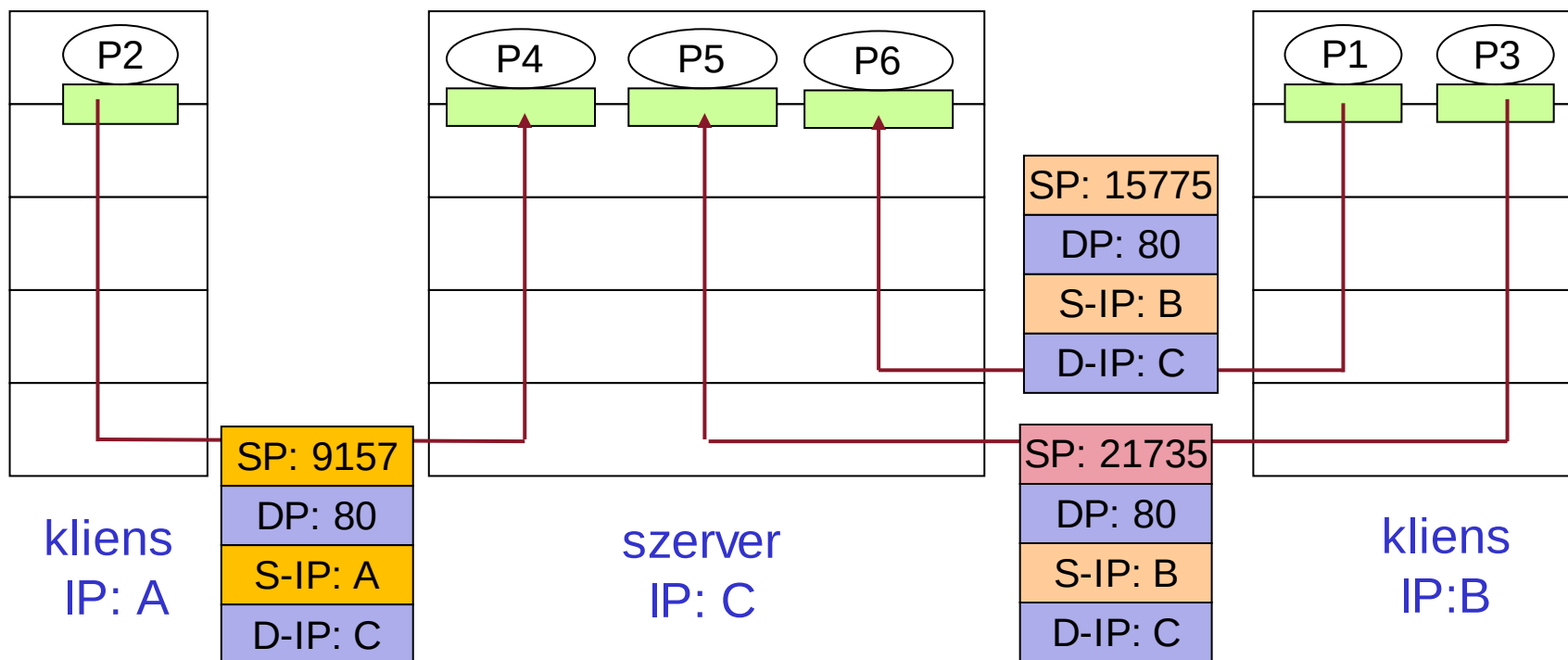
- A kérdésben lévő forrás portszám (SP) lesz majd a válasz célportja (DP)
 - A szervernél vett SP, DP fordítva lesz a válaszban



- A TCP kapcsolat-socketet egy értéknégyes azonosítja:
 - Forrás IP cím
 - Forrás portszám
 - Cél IP cím
 - Cél portszám
- A vevő oldali hoszt mind a négy értéket felhasználja a szegmens megfelelő sockethez továbbítására
- Egy szerver szerepű host egyszerre számos TCP socketet kezelhet:
 - Minden egyes socketet saját értéknégyese azonosít
- Egy web szerver más-más socketen keresztül kezeli az egyes kapcsolódott klienseket
 - A nem perzisztens HTTP külön socketen keresztül kezel minden egyes kérést

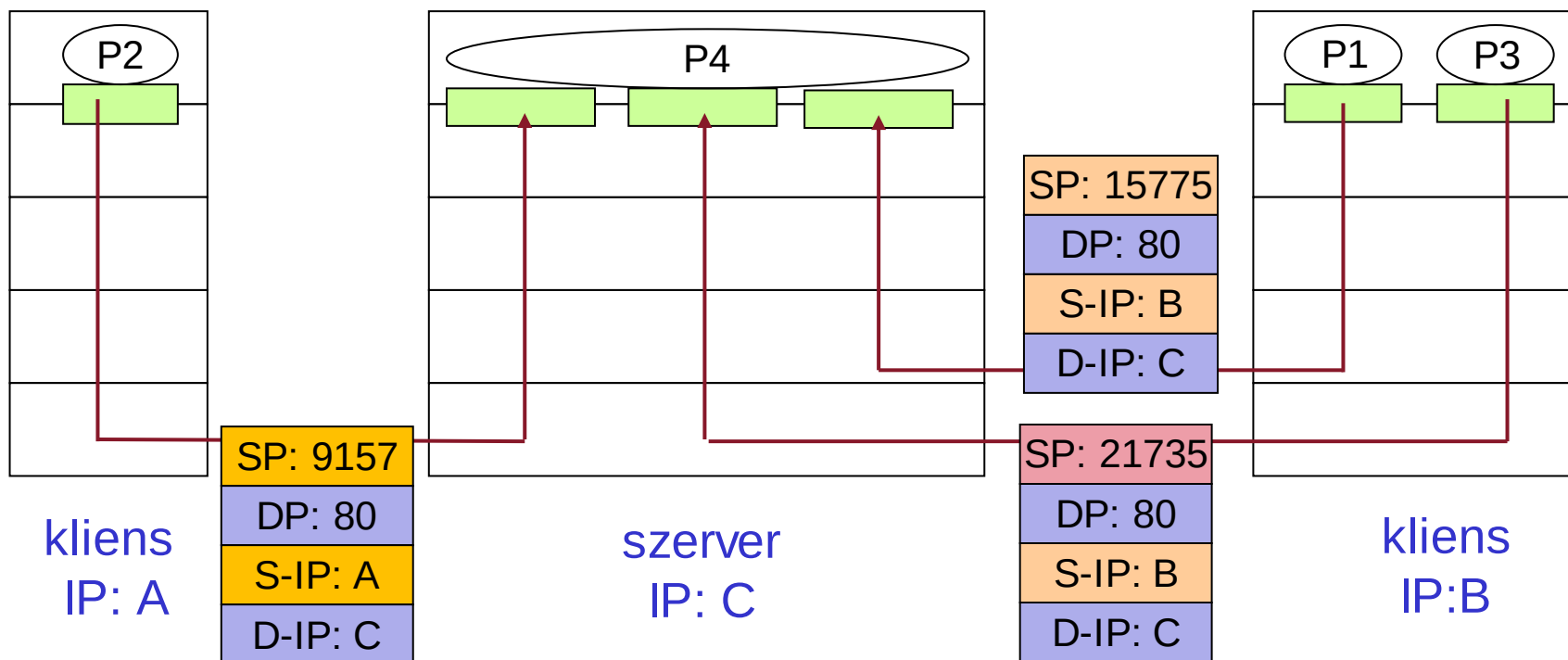
TCP DEMULTIPLÉXÁLÁS ILLUSZTRÁCIÓ – EGY SOCKET

- Külön processz minden sockethez



TCP DEMULTIPLEXÁLÁS ILLUSZTRÁCIÓ – TÖBB SOCKET

- Egy processzhez több socket is tartozik

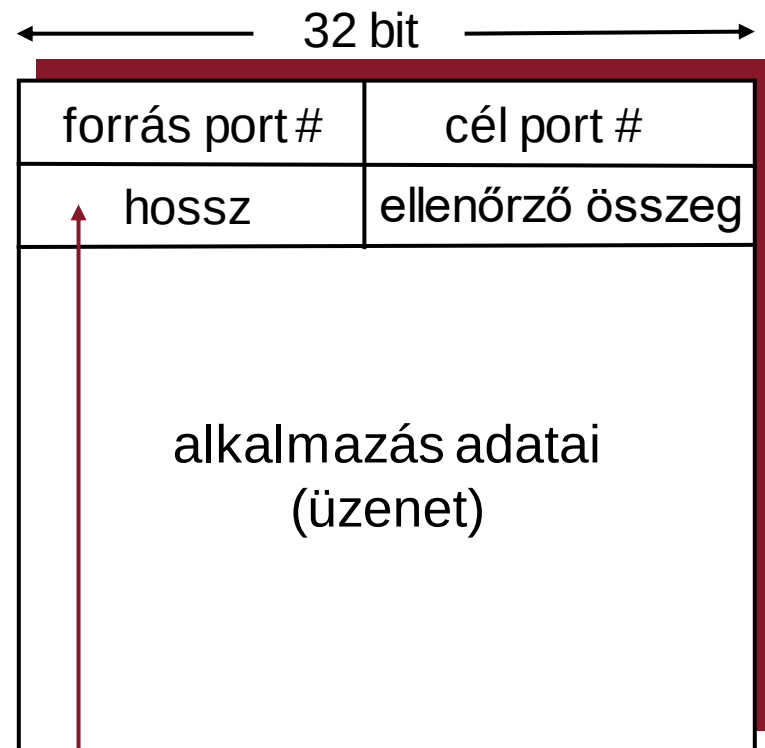


1. Socketek programozása
2. A szállítási réteg szolgáltatásai
3. Nyalábolás és nyalábbontás
4. Összeköttetés nélküli szállítás: UDP

- “Lecsupaszított” Internet transzport protokoll
- “Best effort” szolgáltatás, a szegmensek
 - Elveszhetnek
 - Összekeveredhetnek
- Összeköttetés nélküli
 - Nincs „kézfogás” az UDP küldő és fogadó oldala között
 - Minden egyes UDP szegmens kezelése független a többitől
- Miért kell az UDP (is)?
 - Nem szükséges összeköttetés felépítés: **nincs kezdeti késleltetés**
 - **Egyszerű:** nincs kapcsolatállapot-kezelés sem a küldő, sem a fogadó oldalon
 - **Kis méretű** szegmensfejléc
 - Nincs kézfogás és torlódáskezelés: az UDP a lehető **leggyorsabban továbbítja** az adatokat

- Gyakori alkalmazása a **multimédia folyam**
szállítás (streaming)
 - **Veszteségtűrő**
 - **Sávszélesség-érzékeny**
- További UDP felhasználás
 - DNS
 - SNMP (később)
- Az UDP feletti megbízható átvitelről az alkalmazási rétegben kell gondoskodni
 - Pl. alkalmazás specifikus hibakezelés

UDP szegmens formátuma



Az UDP szegmens hossza
(byte-ok száma) a fejléccel együtt

- **Cél a hibák detektálása**
(pl. „átbillent” bitek) az átvitt szegmensben
- Küldő oldal:
 - A szegmenst 16 bites egész számok sorozataként kezeli
 - **Ellenőrzőösszeg**
 - A számok összege
 - Invertálva
 - Beleteszi a fejlécbe
- Fogadó oldal:
 - **Azonos módszerrel kiszámítja az összeget**
 - **Hozzáadja az ellenőrzőösszeg mezőt**
 - Az eredmény csak egyeseket tartalmazhat
 - **Ha nem, akkor hiba volt, eldobható a szegmens**
 - Egyébként elfogadja (Biztos nem volt hiba?)

ELLENŐRZŐÖSSZEG SZÁMÍTÁSA

- Binárisan összeadjuk a számokat, és a legmagasabb helyi értékű bitek összeadásakor keletkező átvitelt a végén még hozzá kell adni az eredményhez
- Példa: két 16-bites egész összeadása

		1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	
	+	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	
visszacsatolás: átvitel jobbról beléptetése		1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
összeg		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0	
ellenőrző összeg		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1	



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

TCP

2023. március 17.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



1. A TCP protokoll
2. Szegmensek nyugtázása
3. Forgalomszabályozás
4. Kapcsolatkezelés

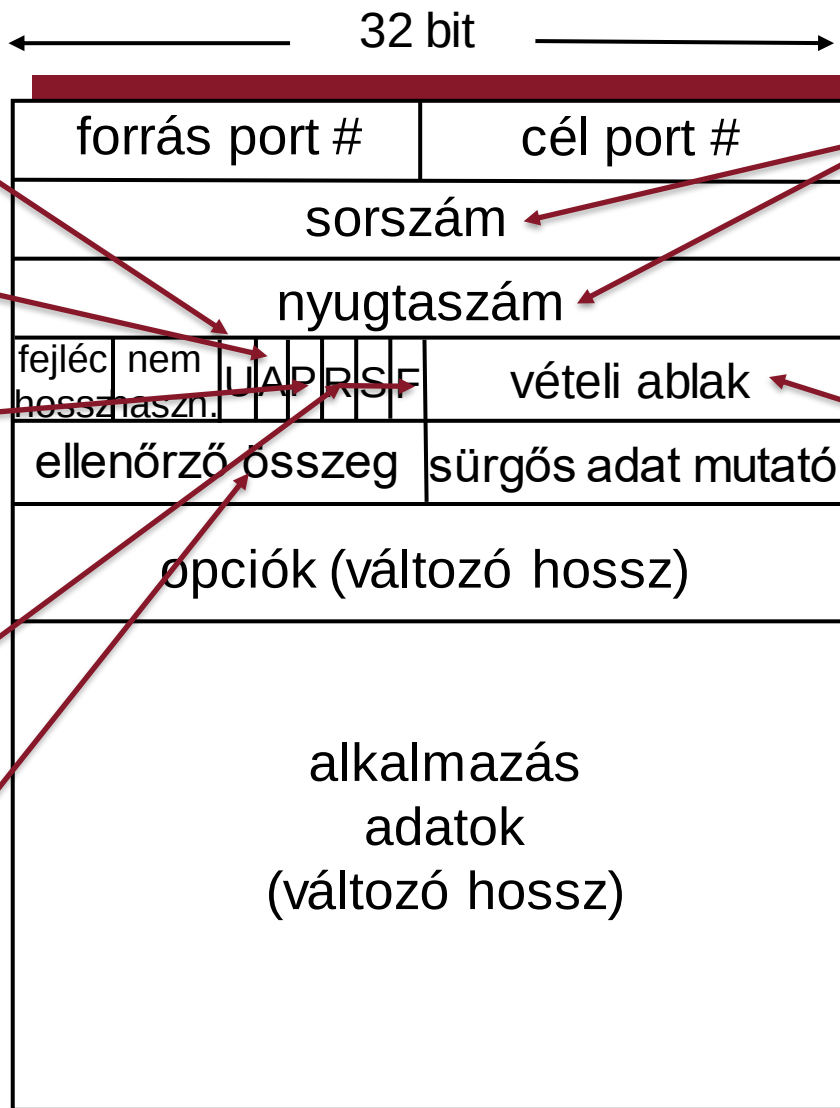
A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

A TCP ÁTTEKINTÉSE

- Több változat, különböző RFC-kben
 - 793, 1122, 1323, 2018, 2581
- **Pont-pont** elrendezés
 - Egy küldő
 - Egy fogadó
- **Megbízható, sorrendtartó adatfolyam**
 - Nincs korlátozva a tartalom, sima bájtok
- Csővezetékezett (pipelined)
 - Aktuálisan küldhető adatok „ablaka”
 - Torlódáskezelésben és forgalomszabályozásban használjuk
- **Ablakozás** a küldő és a fogadó oldalon
- **Full-duplex** adatátvitel
 - Kétirányú adatáramlás ugyanazon a összeköttetésen
- **Maximális szegmensméret**
 - MSS (maximum segment size)
- **Összeköttetés** alapú (connection-oriented)
 - Kézfogás: vezérlő üzenetek cseréje, a küldő és a fogadó megfelelő állapotba hozására
 - A kliens (küldő) kezdeményezi
 - Az adatcsere megkezdése előtt
 - Lezárás
- **Forgalomszabályozás**
 - A küldő ne terhelje túl a fogadót
- **Torlódáskezelés**
 - Igazodás a hálózat aktuális állapotához

TCP SZEGMENS-SZERKEZET

- **URG**: sürgős adat (általában nem használt)
- **ACK**: nyugtamező érvényes
- **PSH**: (push data now) azonnal továbbítandó (általában nem használt)
- **RST, SYN, FIN**: összeköttetést vezérlő flagek (felépítés, lebontás parancsok)
- Internet **ellenőrző összeg** (mint UDP-nél)

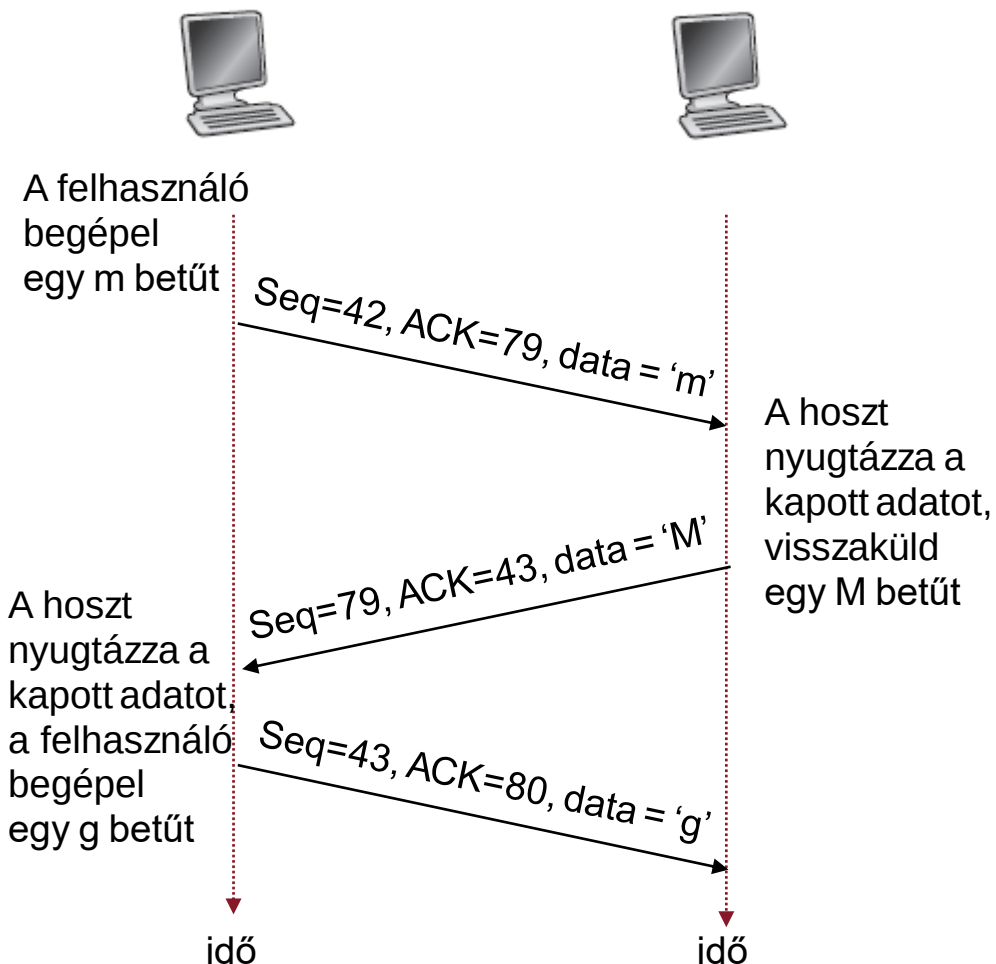


- Alkalmazás adat bájtsorszám (nem a szegmens sorszáma)
- A vevőnél fogadható bájtszám

1. A TCP protokoll
2. Szegmensek nyugtázása
3. Forgalomszabályozás
4. Kapcsolatkezelés

SORSZÁM ÉS NYUGTASZÁM

- **Sorszám (SEQ, sequence number)**
 - A teljes adat hányadik bájttal kezdődik a szegmens adatrésze
- **Nyugta (ACK, acknowledgement)**
 - Vételi információ
 - Melyik bájjal kezdődjön a következő küldése
 - Nyugta arról, hogy az addigiak megjöttek
 - **Összegző (kumulatív) nyugta:** minden addigi megjött
- Mi lesz a nem sorrendben érkező szegmensekkel?
 - Nincs előírva a specifikációban
 - Megvalósításfüggő



- Meddig várjunk a nyugtára?
- Ha túl sokat késik, akkor időtúllépésnek (timeout) veszi a küldő
- A **Timeout** idő értéke
 - Túl kicsi: felesleges időtúllépés
 - Felesleges újraküldések
 - Túl nagy: lassú reagálás egy szegmens elvesztésére
 - Nagyobb legyen, mint az RTT
 - Az RTT nem állandó
- Az RTT **becslése** szükséges
- Mérés alapú minta: **SampleRTT**
 - Egy szegmens elküldési és az arra kapott nyugta érkezési idejének különbsége
 - Az ismételten átvitt szegmenseket figyelmen kívül hagyva
- A **SampleRTT** nem állandó
 - Elég változékony
 - Simítani kell
 - Több minta alapján készült, mozgó átlag kell a becsléshez

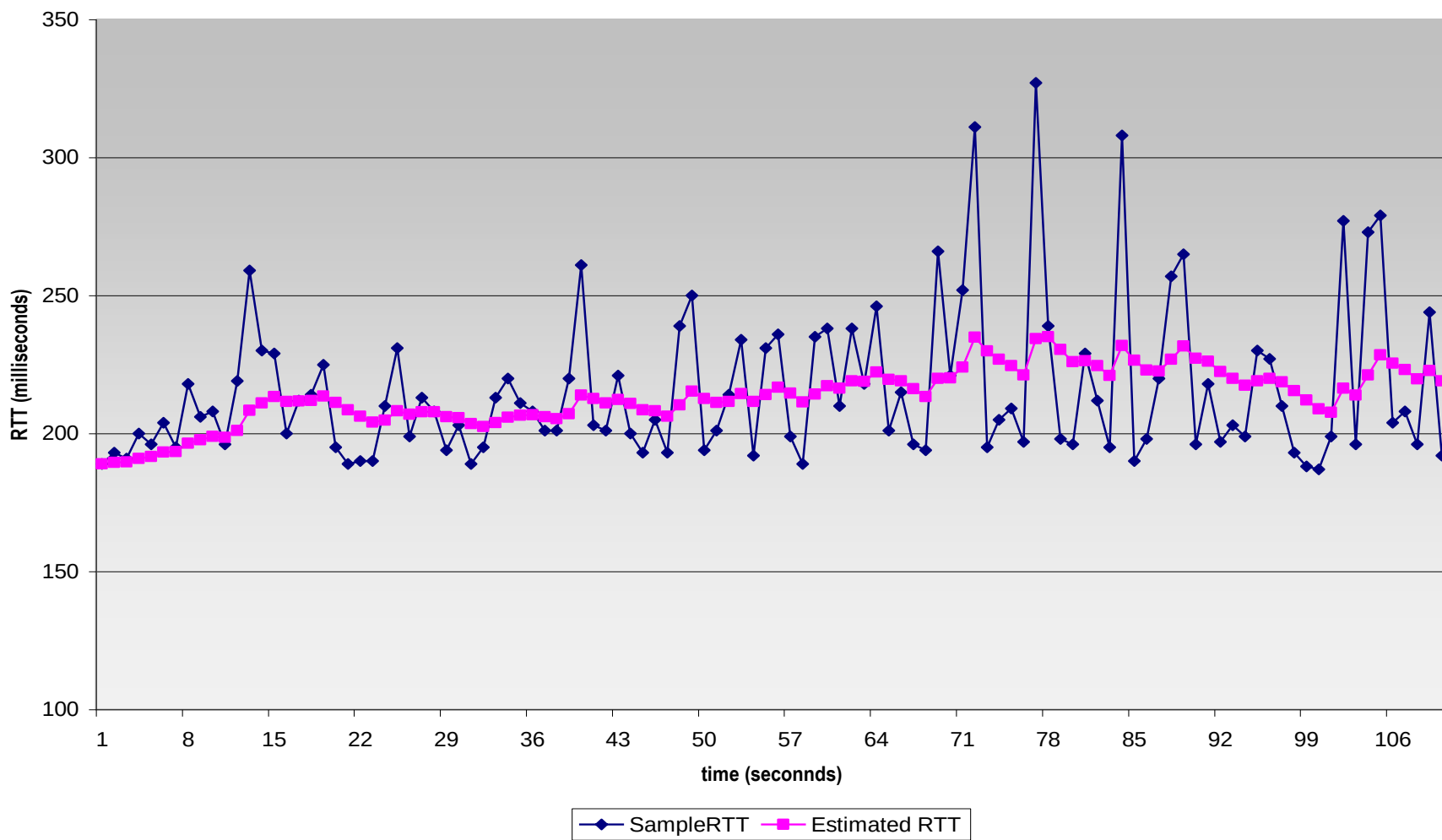
- Mozgóátlag **exponenciális súlyozással**
 - Exponential weighted moving average
 - A korábbi minták hatása exponenciálisan (hatványozottan) csökken

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- Javasolt érték: $\alpha = 0.125$

PÉLDA AZ RTT BECSLÉSÉRE

RTT: gaia.cs.umass.edu to fantasia.eurecom.fr



- A becsült RTT némi biztonsági ráhagyással megfelel
 - Ha nagyobb a minták dinamikája, akkor nagyobb ráhagyás kell
 - Mozgóátlag exponenciális súlyozással
 - Figyelembe véve, hogy mennyire tér el az aktuálisan mért RTT a becsléstől

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

- Javasolt érték $\beta = 0.25$

- Timeout értéke

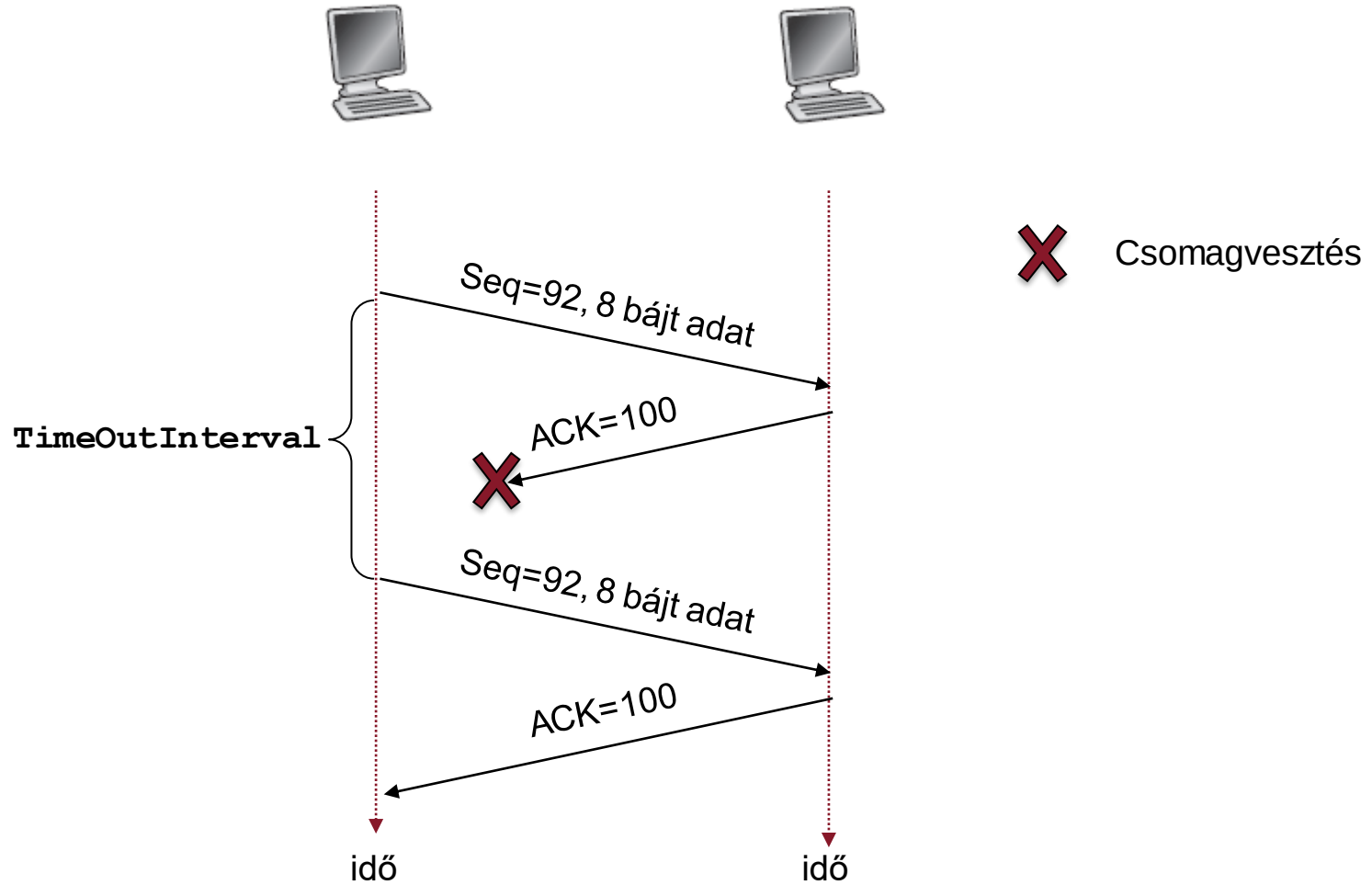
$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

MEGBÍZHATÓ ADATSZÁLLÍTÁS

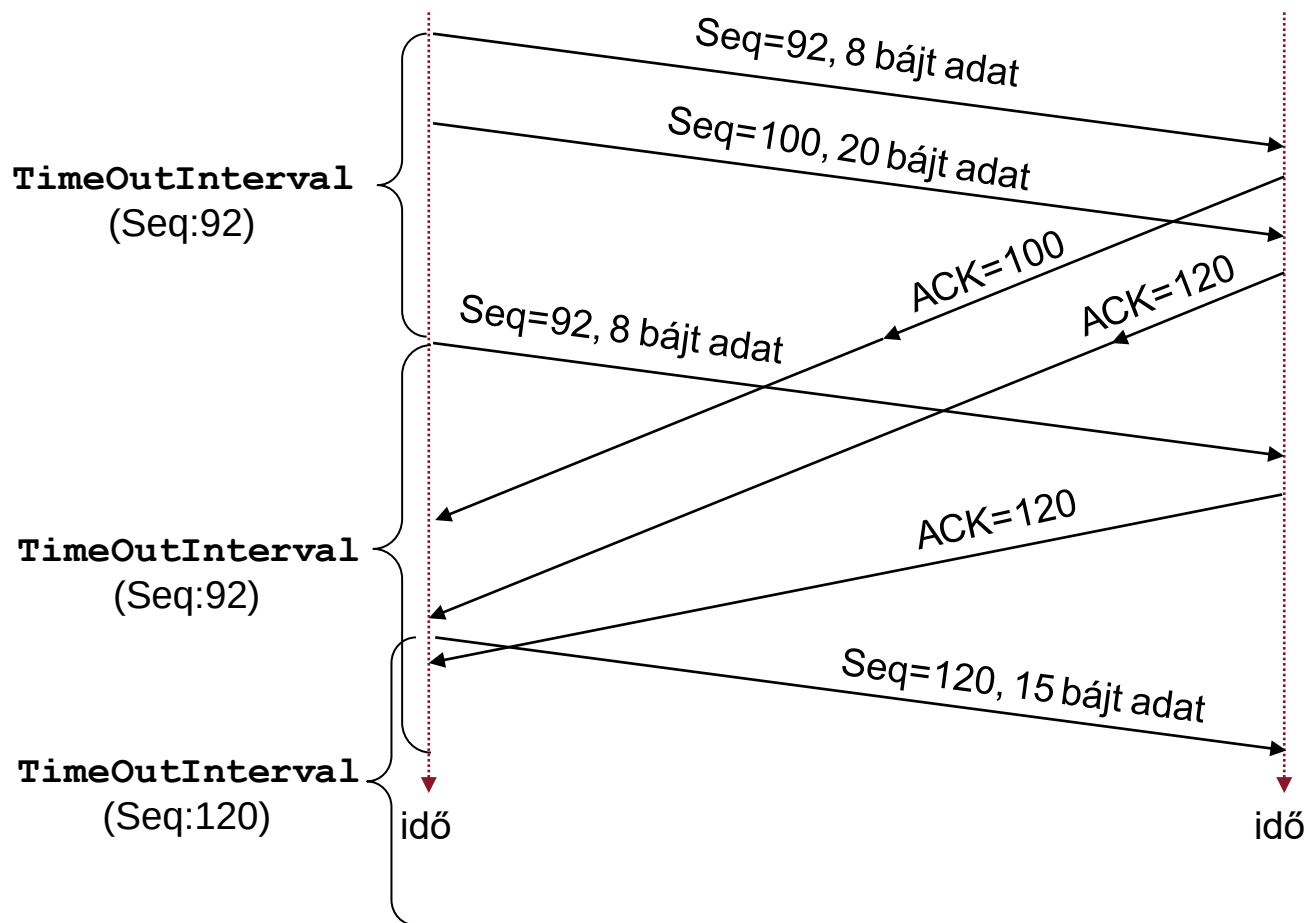
- A TCP a megbízhatatlan IP szolgáltatása felett biztosít megbízható átvitelt
- Szegmensek küldése pipeline módban
 - Nem kell bevárni az előző küldött szegmens nyugtáját
 - A vevő úgysem tudja mikor küldtük el, csak minden megérkezzen (jó esetben sorrendben)
- Összegző nyugta
- Egyetlen időmérési mechanizmus az újraküldéshez
- Újraküldést kiváltó okok
 - Időtúllépés
 - **Kettőzött nyugta (duplicate ack)**
- Vizsgáljunk meg először egy egyszerűsített esetet
 - Nincs kettőzött nyugta
 - Nincs forgalomvezérlés
 - Nincs torlódáskezelés

- Megkapja az alkalmazás adatait
 - Szegmens képzése MSS-nek megfelelően
 - Az első bájt sorszáma kerül a Seq mezőbe
 - Elindítja az időzítőt, ha még nem fut
 - Legjobb lenne a legrégebb, még nem nyugtázott szegmens idejét mérni
 - Az időzítő lejárt: **TimeoutInterval**
- Időtúllépés
 - Az időtúllépést okozó szegmens újraküldése
 - Időzítő újraindítása
- Nyugta érkezik
 - Ha még nem nyugtázott szegmensre vonatkozik
 - A már nyugtázott szegmensek halmazának frissítése
 - Időzítő újraindítása, ha van még nyugtázatlan szegmens

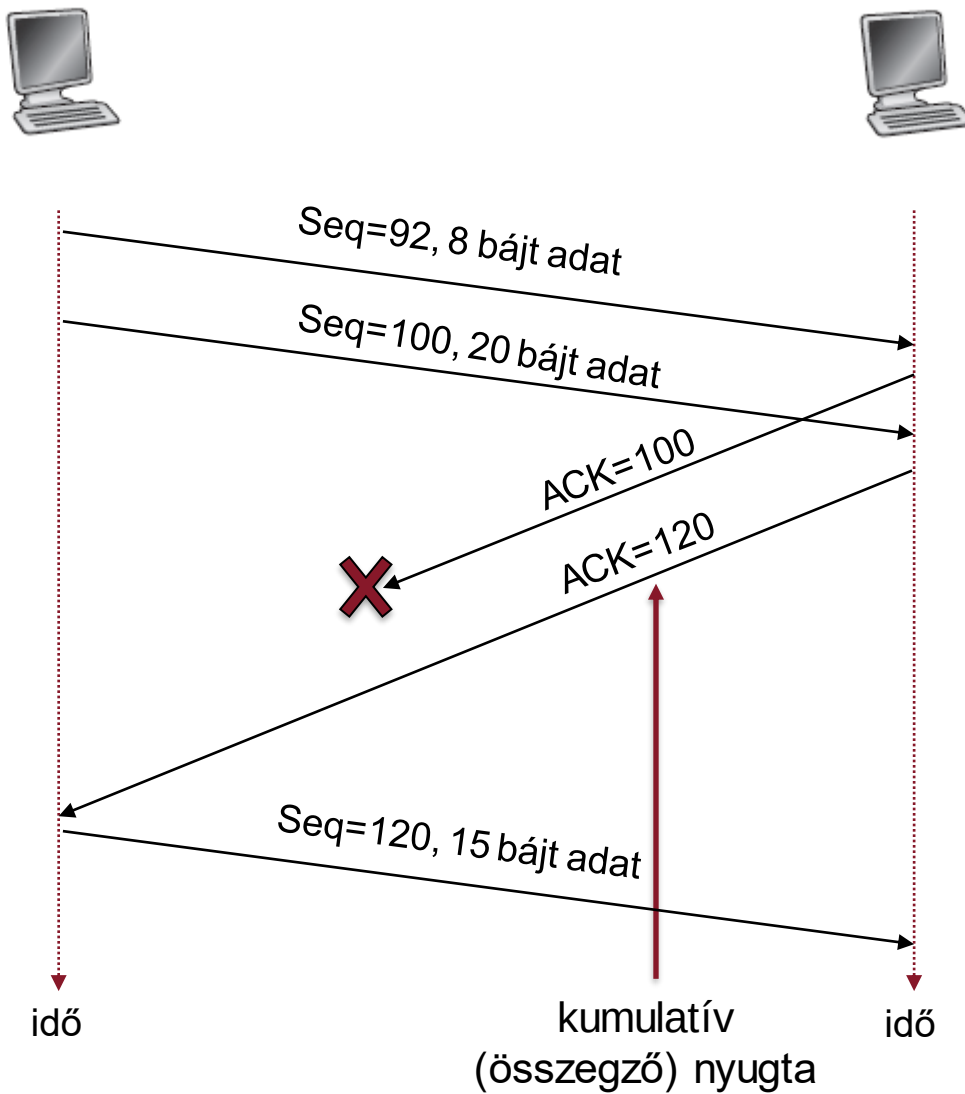
ÚJRAKÜLDÉS CSOMAGVESZTÉS ESETÉN



ÚJRAKÜLDÉS KORAI IDŐTÜLLÉPÉS ESETÉN



KUMULATÍV NYUGTA CSOMAGVESZTÉS MIATT



• Esemény a vevőnél

- Helyes sorrendben érkező, elvárt (nem túl nagy) sorszámú szegmens érkezik és minden addigi szegmens nyugtázva van

- Helyes sorrendben érkező, elvárt (nem túl nagy) sorszámú szegmens érkezik és van korábbi nem nyugtázott szegmens

- A sorrendből kilógó, túl nagy sorszámú szegmens érkezik

- Lyukat betömő szegmens érkezik

• Akció

- Késleltetett nyugtázás (delayed ACK), legfeljebb 500ms-t vár a következő szegmensre és utána nyugtáz

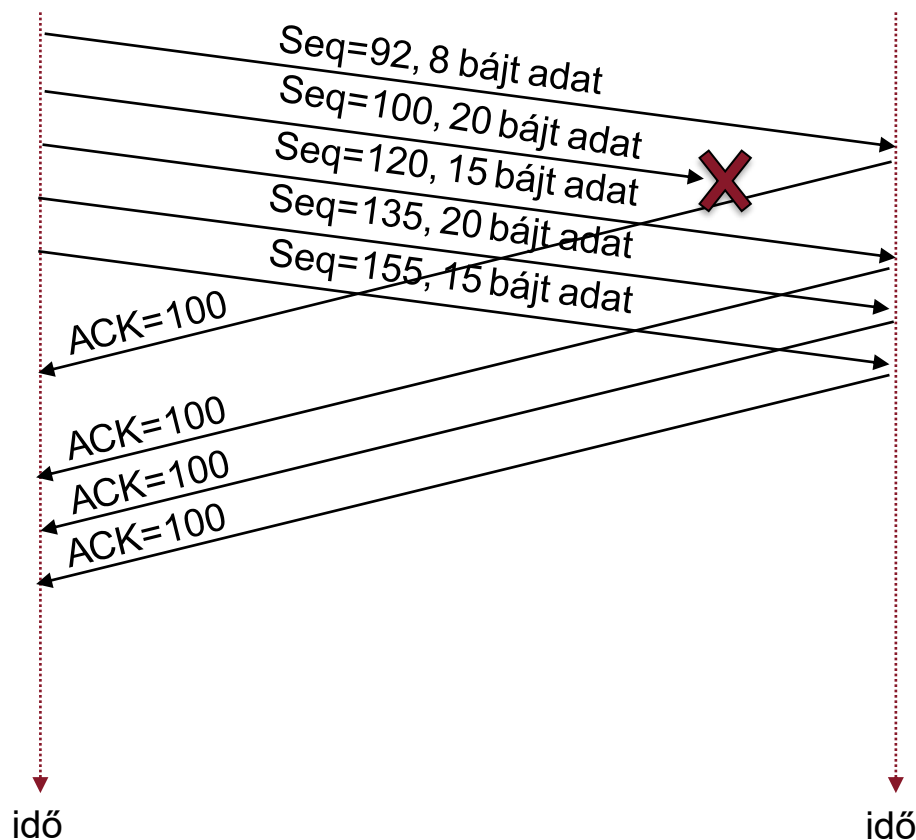
- Azonnal kumulatív nyugtát küld a szegmensekre

- Azonnal kettőzött (ismételt) nyugtát küld a hiányzó szegmenst megadva az ACK-ban

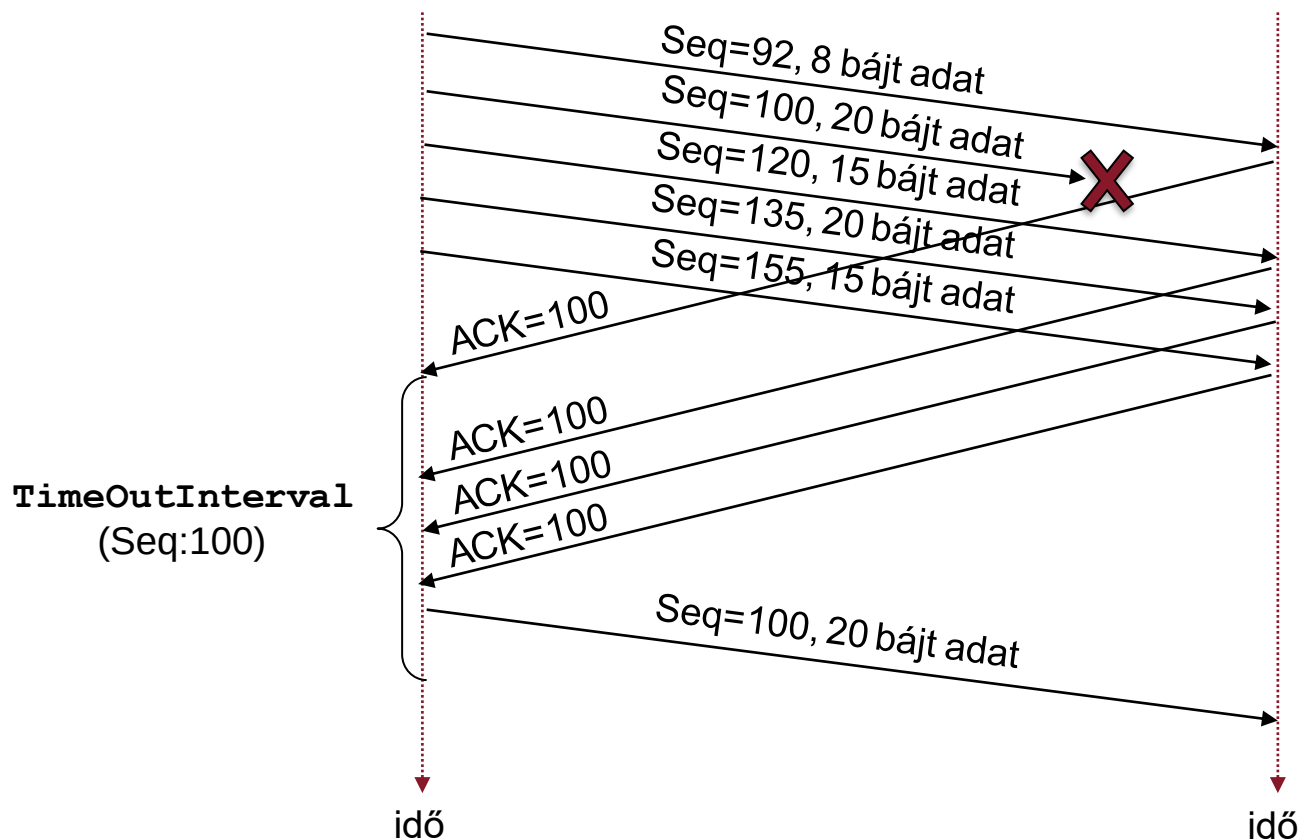
- Azonnal nyugtát küld a legrégebb megmaradt lyuk elejét megadva az ACK-ban



SORRENDHELYESSÉG!

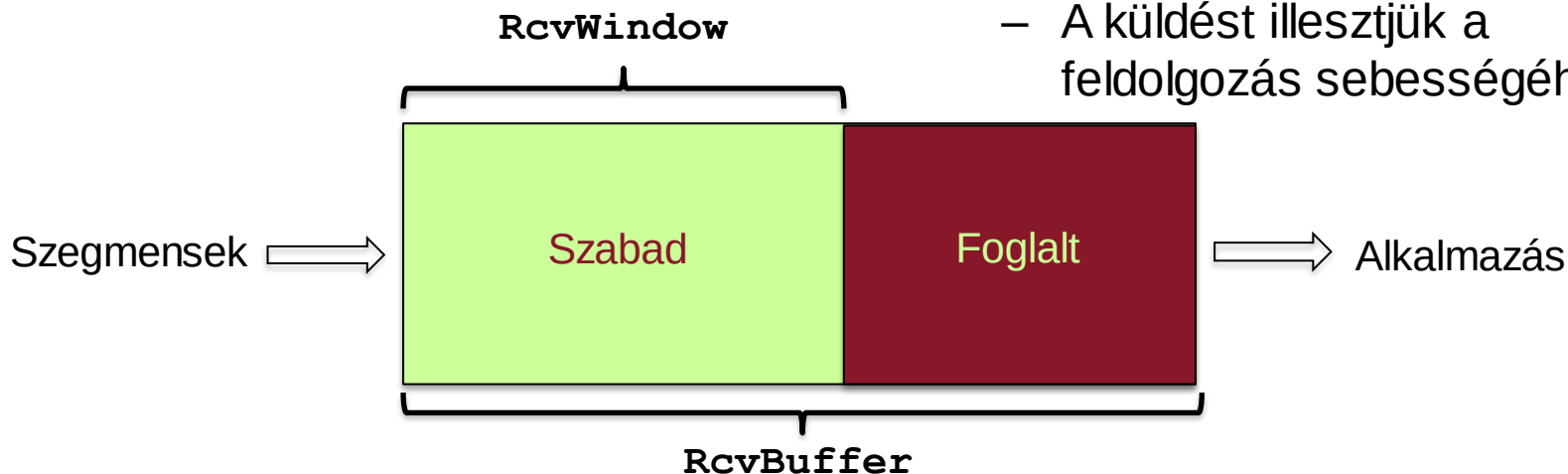


- A Timeout gyakran indokolatlanul nagy
 - Túl sokat várunk mielőtt újraküldenénk
- Csomagvesztés felismerhető duplikált (kettőzött, többszörös) nyugták alapján is
 - A küldő sok szegmenst küld el egymás után
 - Egy szegmens elveszése sok duplikált nyugtát indukál
- A harmadik duplikált nyugta után feltehetjük, hogy csomagvesztés volt
 - Gyors újraküldés (Fast Retransmit)
 - Szegmens újraküldése időtűllépés nélkül
 - (Nem biztos, hogy tényleg volt rá szükség)

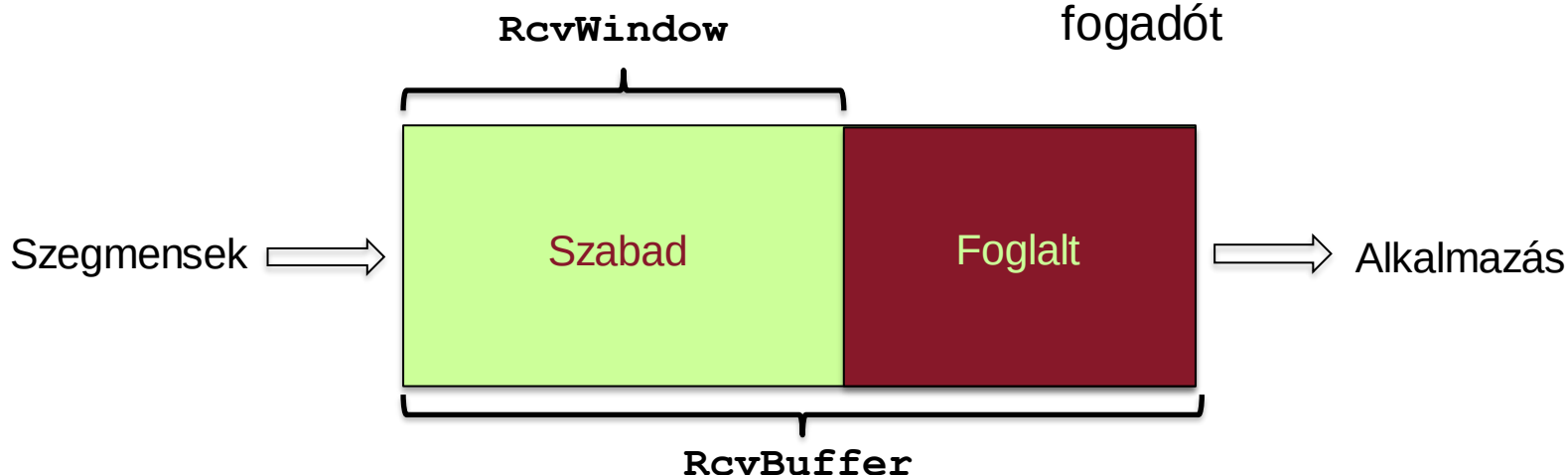


1. A TCP protokoll
2. Szegmensek nyugtázása
3. Forgalomszabályozás
4. Kapcsolatkezelés

- A vevő oldalra érkező adatok egy véges méretű pufferbe kerülnek
- Az alkalmazás estenként lassan üríti a puffert
- Forgalomszabályozás (**flow control**)
 - Megakadályozza, hogy a küldő **túltöltse** a puffert
 - Túl gyorsan küld
 - Túl sokat küld
- **Sebességillesztési szolgáltatás**
 - A küldést illesztjük a feldolgozás sebességéhez



- Egyszerűsítés: tekintsünk el a nem sorrendben érkező szegmensektől
- Szabad hely a pufferben
= **RcvWindow**
= **RcvBuffer** -
[**LastByteRcvd** -
LastByteRead]
- A fogadó jelzi a szabad hely mennyiségét a (nyugta)fejléc **RcvWindow** mezőjében
- A küldőnek legfeljebb **RcvWindow** mennyiségű elküldött, de nyugtázatlan adata lehet
 - Így biztosan nem tölti túl a fogadót



1. A TCP protokoll
2. Szegmensek nyugtázása
3. Forgalomszabályozás
4. **Kapcsolatkezelés**

- **A TCP kapcsolatorientált**

- Kapcsolatfelépítés az adatok küldése előtt
- TCP-változók beállítása
 - Sorszám inicializálás
 - Forgalm szabályozás inicializálása
- **Kliens kezdeményez**

```
clientsock = socket.socket()
clientsock.connect()
```

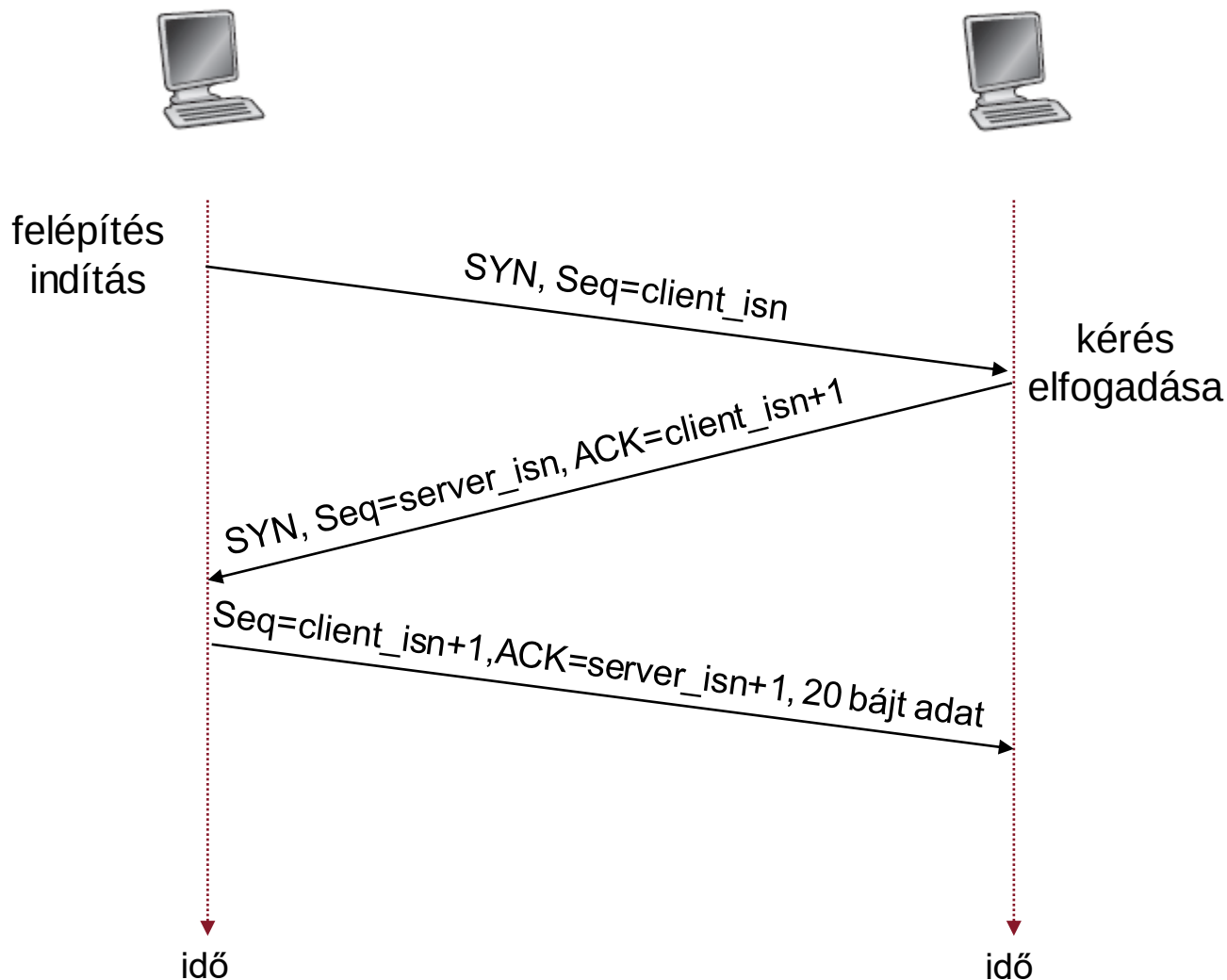
- **Szerver fogadja a megkeresést**

```
connsock = servsock.accept()
```

- Kapcsolatfelépítés: háromutas kézfogás (**three-way-handshake**)

1. A kliens **SYN** típusú vezérlő szegmenst küld a szervernek
 - Kezdő sorszám küldése
 - **Adat nélkül**
2. A szerver válaszol egy **SYNACK** típusú vezérlő szegmenssel
 - Inicializáló lépések ezen az oldalon is
3. A kliens válaszol egy **ACK** szegmenssel, **ami már adatot is tartalmaz**

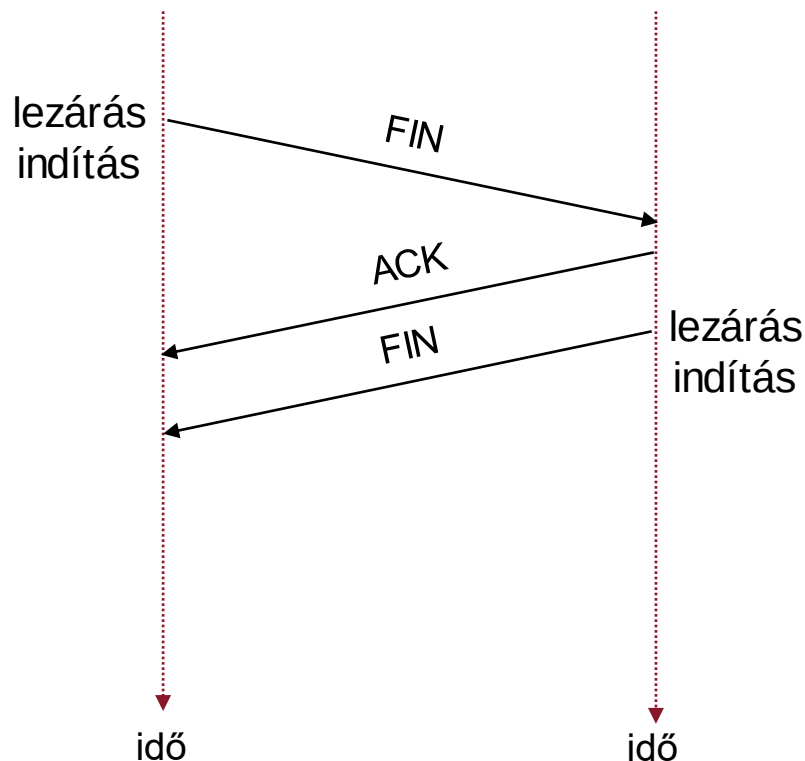
KAPCSOLAT FELÉPÍTÉSE



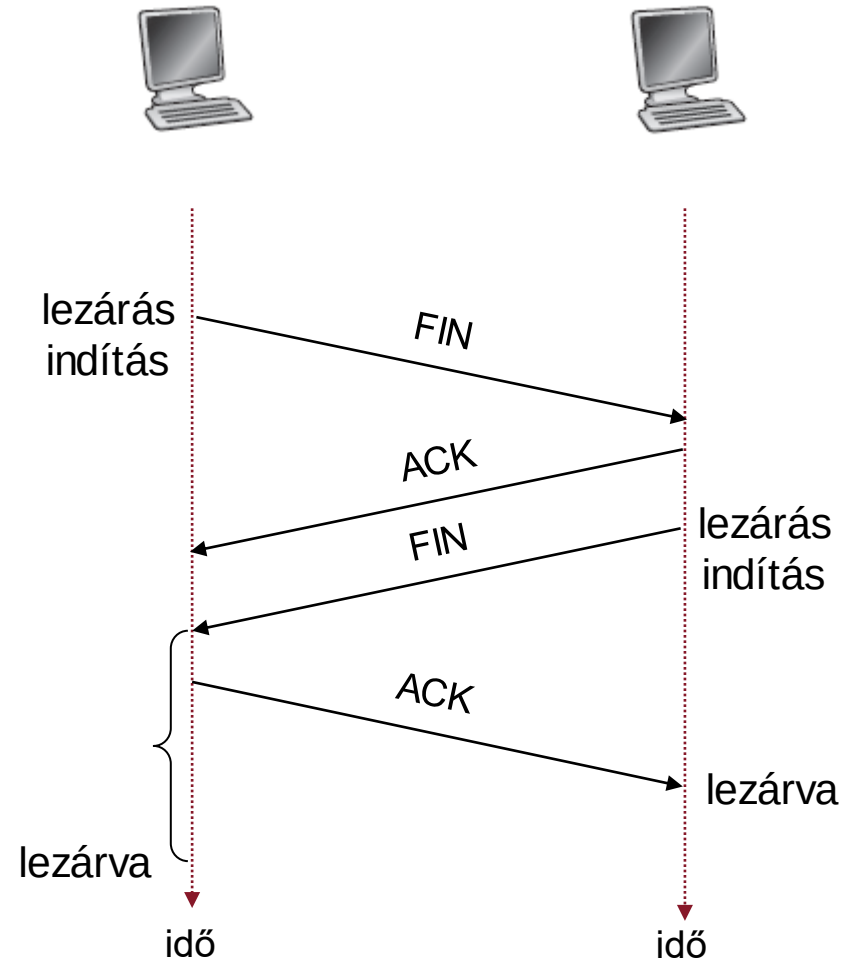
KAPCSOLAT LEZÁRÁSA

- Kapcsolatlezárás: négyutas kézfogás (four-way-handshake)
 - Tipikusan a kliens kezdeményezi a socketjén keresztül `clientsock.close()`

- A kliens indítja a lezárást és **FIN** típusú vezérlő szegmenst küld a szervernek
- A szerver válaszol egy **ACK** szegmenssel
- A szerver **FIN** típusú vezérlő szegmenst küld a kliensnek és indítja a lezárást



4. A kliens megkapja FIN típusú vezérlő szegmenst és időzített várakozás állapotba lép
5. A kliens válaszol egy ACK szegmenssel
6. A szerver megkapja az ACK-ot és lezártnak tekinti a kapcsolatot
7. A kliensnél lejár az időzítő és lezártnak tekinti a kapcsolatot





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

TCP torlódáskezelés, QUIC
2023. március 24.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu

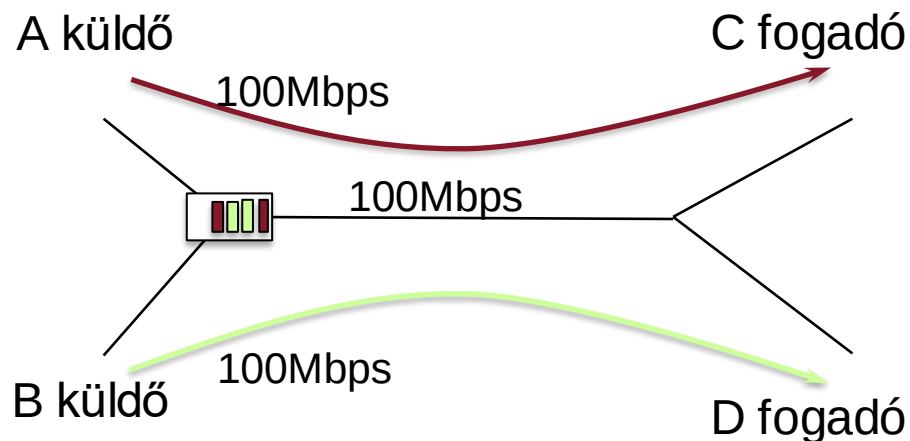


1. Torlódáskezelés a TCP-ben
2. TCP átbocsátás és igazságosság
3. További TCP funkciók
4. A QUIC protokoll

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

A TÚLTERHELÉS PROBLÉMÁJA

- A probléma okai:
 - Minden küldő a lehető legnagyobb sávszélességgel küldi az adatokat
 - A küldők összes forgalma meghaladja egy link átocsátó kapacitását
- **Végtelen kimeneti puffer esetén**
 - A csomagok várakozási ideje végtelen nagyra nőhet
 - A lejárt időzítők miatti újraküldések csak súlyosbítanak
- **Véges kimeneti puffer esetén**
 - A várakozási idő korlátos (de nagy lehet)
 - Csomagvesztés lép fel
 - Újraküldés veszteség miatt is
- A csomagok **torlódása (congestion)**
 - Jó lenne elkerülni
 - A küldő oldal tud érte tenni



- A TCP-ben kihasználhatók a megbízható szállításra használt mechanizmusok
- A **végpont-végpont** forgalom információi
 - Nem a hálózati eszközök jelzik a torlódást
 - Késleltetési információk a nyugták alapján
 - Csomagvesztési (szegmensvesztési) információk a nyugták hiánya vagy késése alapján
 - Nem tudni, hogy pontosan hol van torlódás (akár több helyen is lehet)
- Gyorsan kell cselekedni, ezért egyszerű mechanizmus kell
- **Sebességillesztési szolgáltatás**
 - Nem a fogadóhoz, hanem **a hálózat átbocsátóképességéhez illeszkedik**

A TORLÓDÁSKEZELÉS RÉSZLETEI

- A következő nyugtáig átküldhető adatok mennyiségét korlátozó ablak: **torlódási ablak, CongWin**
- A küldő betartja az alábbi szabályt:

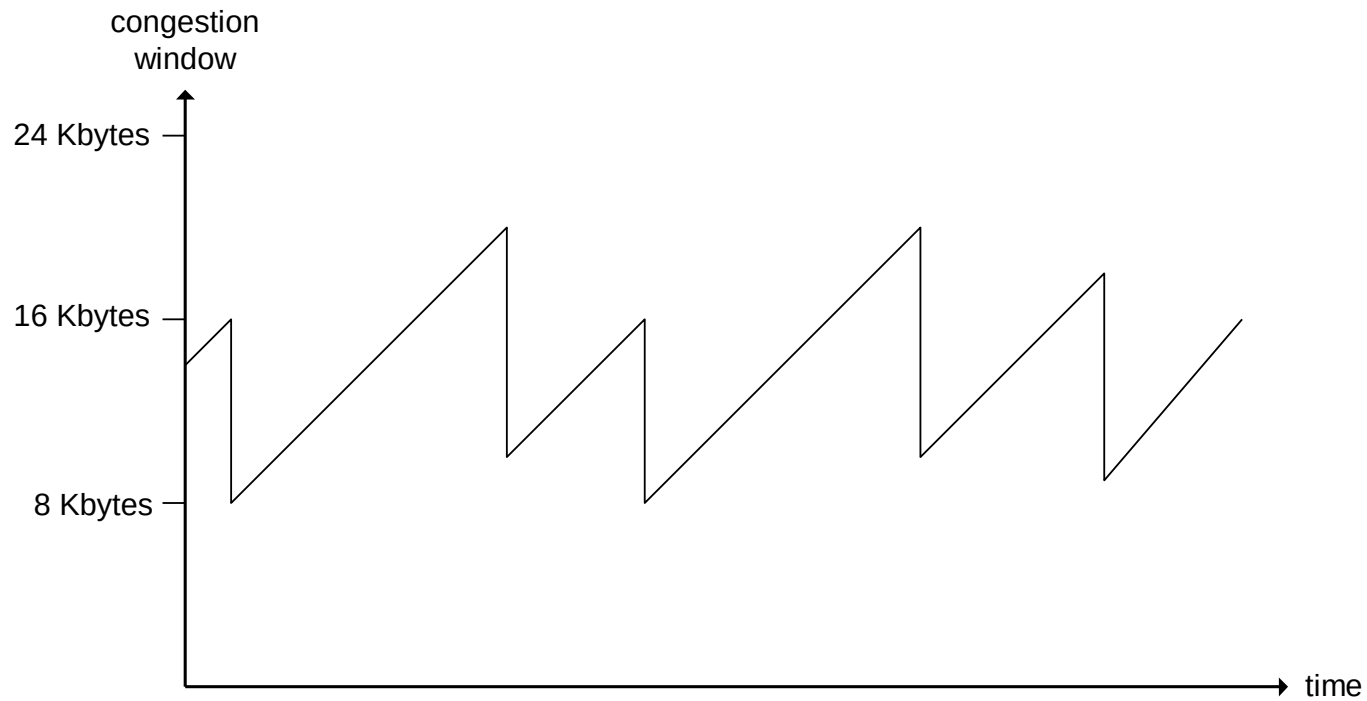
$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWin}$$
- **Átlagos sávszélesség (megközelítőleg)**

$$\text{CongWin} / \text{RTT}$$
- Megközelítés:
 - Ha minden rendben – növeljük az ablakot
 - Ha baj (csomagvesztés) van – csökkentjük az ablakot
- A torlódási ablak dinamikusan követi a hálózat állapotát

- Minden rendben = nem duplikált nyugta
- Torlódás érzékelése a küldőnél – (feltételezett) csomagvesztés
 - Időtűllépés nyugta várása közben
 - Három duplikált nyugta érkezése
- Az ablakot szabályozó mechanizmusok
 - Lassú indulás (Slow Start, SS)
 - Torlódás-elkerülés (Congestion Avoidance, CA) – AIMD
 - Időtűllépés esetén nagyon konzervatív

- Additív növelés (increase)
 - Lineáris növelés: egy MSS-sel növeljük az ablakot minden RTT-ben
- Multiplikatív csökkentés (decrease)
 - Exponenciális csökkentés: a felére csökkentjük az ablakot ha elveszett egy csomag
- Fűrészfogszerű viselkedés
- A cél a maximális, még átvihető sávszélességgel küldeni
- Torlódás esetén erőteljes beavatkozás

NÖVELJÜK az ablakot, DE ezzel egyre több adatot is küldünk -> TORLÓDÁS alakulhat ki!
CSÖKKENTJÜK az ablakot, MIVEL túlterheltük a hálózatot -> SOK TORLÓDÁS volt!



- A kapcsolat kezdetén

$$\text{CongWin} = 1 \text{ MSS}$$

- A tényleges átbocsátóképességnél jóval kisebb lehet

- Például

- MSS: 500 bájt
- RTT: 20 ms
- Torlódási ablak: 500 bájt
- Kezdeti sávszélesség: 200kbps

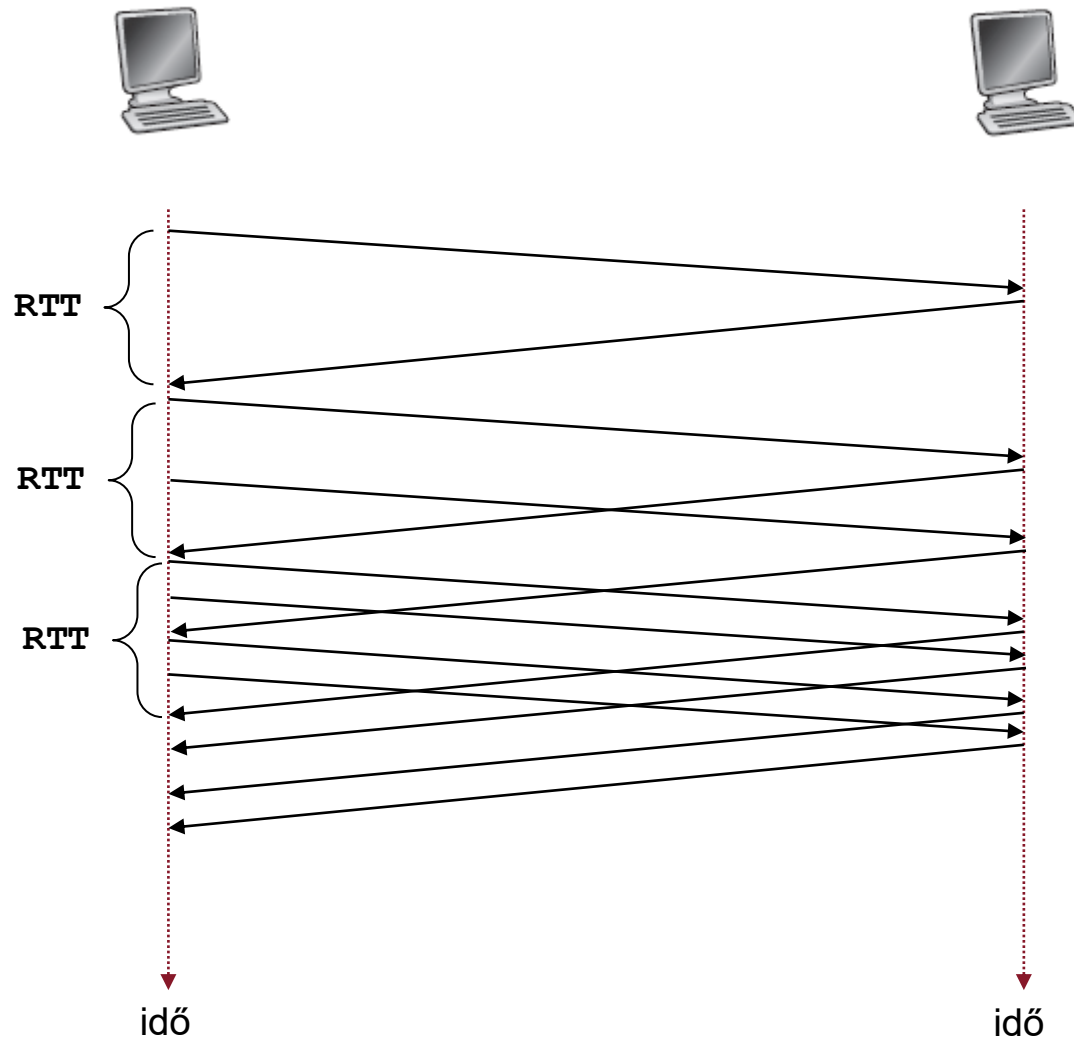
- Gyorsan fel kellene futtatni

- Exponenciális növelés
- Amíg nem lép fel csomagvesztés

- Minden nyugtaérkezéskor egy MSS-sel növeljük az ablakot

- Minden RTT alatt megkétszereződik (megközelítőleg)

LASSÚ INDULÁS – ILLUSZTRÁCIÓ

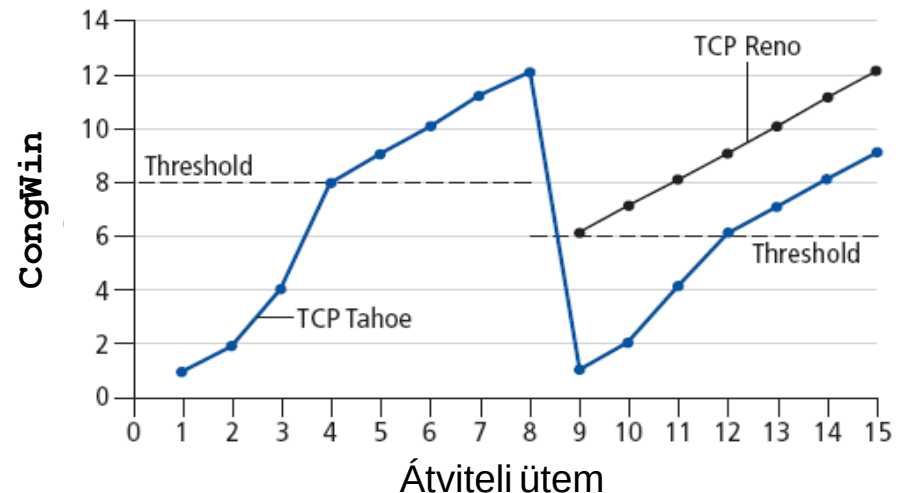


- Három duplikált nyugta esetén
 - A hálózaton még átküldhetők csomagok
 - Fast Retransmit újraküldés
 - Felezve az ablakot folytathatjuk additív növekedéssel (CA)
- Időtúllépés után
 - Nagyobb a baj, nem jönnek nyugták
 - Kezdjük előlről, lassú indítással (SS)
- TCP változatok
 - Tahoe – nem tesz különbséget, mindig SS-sel folytatja
 - Reno – hatékonyabb a torlódáskezelés
 - További változatok (NewReno, SACK, FACK, Vegas, CUBIC, stb.) – tovább csiszolnak bizonyos részeken, pl. más függvényeket használnak
 - Nem csak a torlódáskezelésben különbözhetnek, extra opciókat is használnak

CSOMAGVESZTÉS UTÁNI SS

- Mikor kellene az exponenciális növekedésből lineáris növekedésre átállni?
 - Az első vesztes helyett egy korábban beállított küszöbnél
 - Amikor elérjük a legutolsó vesztesssel végződő ablak felét

- Megvalósítás
 - **Threshold** változó
 - Csomagvesztés után az ablak felére állítjuk be



- Ha **CongWin** kisebb mint **Threshold** akkor **Slow Start** fázis, exponenciális ablaknövelés -> **GYORS** megtaláljuk a határt!
- Ha **CongWin** nagyobb mint **Threshold** akkor **Congestion Avoidance** fázis, lineáris ablaknövelés -> **ÓVATOSAN** tapogatózunk!
- Miután három duplikált nyugta jön, a **Threshold** és a **CongWin** változót is **CongWin/2** -re állítjuk -> dupla ACK enyhe jelzés elég felezni!
- Ha időtúllépés (timeout) lép fel, a **Threshold** változót **CongWin/2** -re, a **CongWin** változót **1 MSS** -re állítjuk
|
-> timeout **súlyos** jelzés -> nem kockáztatunk!

1. Torlódáskezelés a TCP-ben
2. TCP átbocsátás és igazságosság
3. További TCP funkciók
4. A QUIC protokoll

- Mekkora átvitel (átbocsátás) adódik ebből a működésből?
 - Figyelman kívül hagyva az SS állapotot
- Tegyük fel, hogy W ablakméretnél lép fel csomagvesztés
 - W méretnél az átvitel W/RTT
 - A felezés után $W/(2*RTT)$ –től lineárisan nő
 - Az átlag $0.75W/RTT$
- Ez nagyon leegyszerűsített modell volt
 - Valójában a hálózat állapotától függ a W és az RTT is
 - A TCP mohón, de szabályozottan fogyasztja a link-kapacitásokat

1. Torlódáskezelés a TCP-ben
2. TCP átbocsátás és igazságosság
3. További TCP funkciók
4. A QUIC protokoll

- Egy elveszett szegmens miatt duplikált nyugták jöhetnek
 - Csak azt adják meg, hogy mi az első hiányzó bájtk
 - A sorrenden kívüli, de **megkapott szegmenseket** nem kellene újraküldeni
 - Nagy RTT esetén nem kapjuk meg időben a kumulatív nyugtát
- **SACK** – a fogadó nyugtában küldi vissza a sorrenden kívüli bájtok tartományait -> Selective ACK, fogadó pontosan megmondja, mely szegmensek érkeztek meg! 120 hiányzik, de 140-160-ig megvan!
 - Korlátozott mennyiségben
 - A fejléc opcióit felhasználva
 - A kapcsolat felépítésekör egyeztetnek a felek a használatról
- Az újraküldést önmagában is gyorsíthatja



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Hálózati réteg
2023. március 27.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



1. Portok besorolása
2. A hálózati réteg funkciói
3. IPv4
4. IPv6

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

PORTOK BESOROLÁSA

- Milyen portot használhat egy alkalmazás?
- **Elterjedt hálózati protokollok**
 - Minden hálózatot használó elem használja őket
 - „Szabványos” portszámok kellene
- **Alkalmazások és operációs rendszer szolgáltatások**
 - Lehetnek eltérések az egyes rendszerek között
 - Szolgáltatáshoz rendelt portszámok
- **Felhasználói programok ideiglenes portjai**
 - Az előző csoportokba nem tartozó kommunikáció
 - Tetszőleges, vagy az operációs rendszer által hozzárendelt port

Az Internet Assigned Numbers Authority (IANA) regisztrálja:

System (Well-known) portok: 0 – 1023

20 & 21: File Transfer Protocol (FTP)
 22: Secure Shell (SSH)
 23: Telnet
 25: Simple Mail Transfer Protocol (SMTP)
 53: Domain Name System (DNS)
 80: Hypertext Transfer Protocol (HTTP)
 110: Post Office Protocol (POP3)
 119: Network News Transfer Protocol (NNTP)
 143: Internet Message Access Protocol (IMAP)
 161: Simple Network Management Protocol (SNMP)
 443: HTTP Secure (HTTPS)

User (Registered) portok: 1024 – 49151

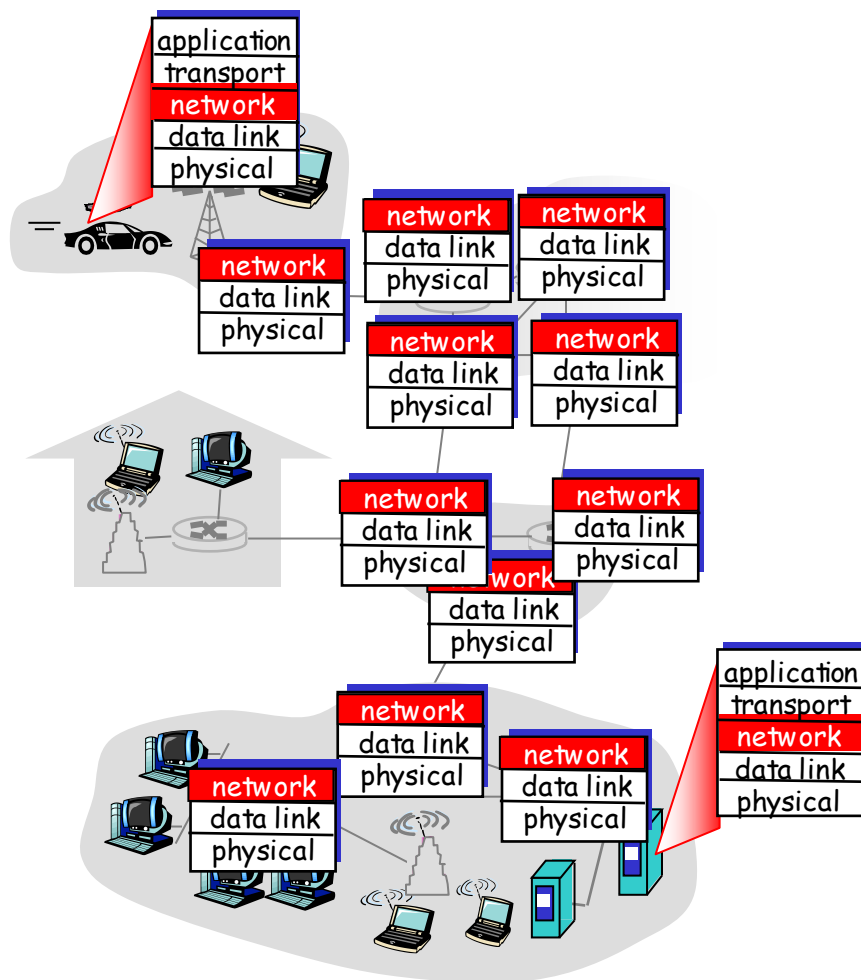
Dynamic (Private, Unregistered) portok: 49152 – 65535

A TCP/IP protocol stack rétegei:

- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, SMTP, HTTP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, útvonalválasztó protokollok
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - PPP, Ethernet
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti összeköttetéseken

1. Portok besorolása
2. A hálózati réteg funkciói
3. IPv4
4. IPv6

- **Cél: transzport szegmens eljuttatása a küldő hoszttól a fogadó hosztnak**
- A hálózati réteg és a transzportréteg átviteli szolgáltatásainak alapvető különbsége:
 - Transzport réteg: két – távoli hoszton futó - processz között
 - Hálózati réteg: két hoszt között (a közbülső routerek közreműködésével)
- **A küldő oldalon a szegmensek datagramokba csomagolása**
- A fogadó oldalon a megérkezett szegmens kicsomagolása és átadása a transzport rétegnek
- Hálózati réteg protokolljai minden hosztban és routerben
- **A router minden rajta áthaladó IP datagram fejlécét feldolgozza**

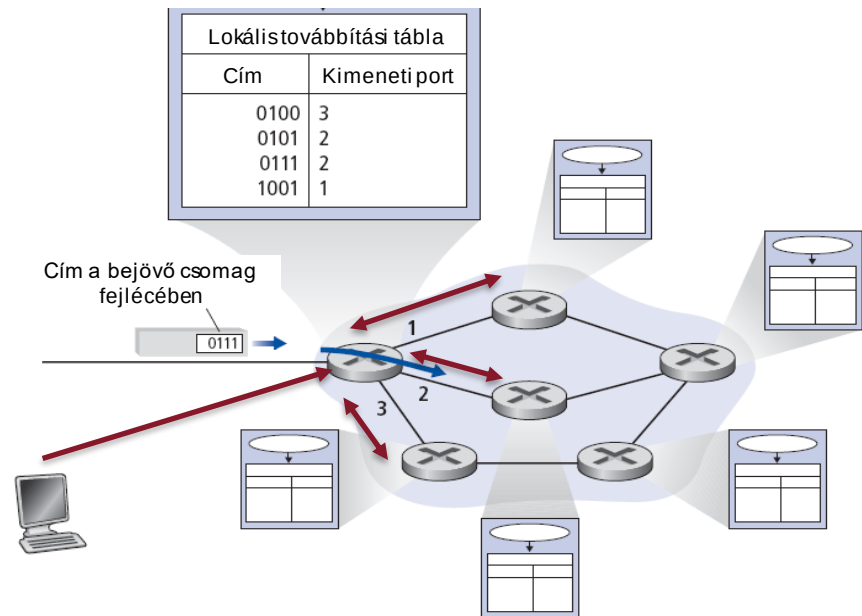


- **Címzés**
 - Végpontok **azonosítása**
 - Több végpont (al)hálózatba szervezése
- **Csomagtovábbítás (forwarding)**
 - Csomag **mozgatása** a router bemenetéről a router megfelelő kimenetére
 - Melyik a megfelelő?
- **Útvonalválasztás (routing)**
 - a csomag **útjának meghatározása** a forrástól a nyelőig
 - útvonalválasztó (**routing**) algoritmusok
- Analógia: utazás autóval
 - Címzés: szálloda címe
 - Routing: utazás megtervezése a kiindulástól a végcélig
 - Forwarding: áthaladás egy autópálya csomóponton



ÚTVONALVÁLASZTÁS ÉS TOVÁBBÍTÁS EGYÜTTMŰKÖDÉSE

- A routerben a továbbítás lokális döntés alapján történik
- A router lokális döntésének alapja
 - Amit előre kiszámolt utak alapján megkap
 - Amit más routerektől kapott adatok alapján kiszámol



- Az Interneten alkalmazott hálózati protolloknál **NINCS**
 - Csak csomag (datagram) küldés
 - Nem épül fel út, nem értesítjük a közbeeső routereket
 - Egy alkalmazáshoz tartozó adatok egymástól független csomagokban
 - Az út nem is biztos, hogy ugyanaz
 - Nincs garancia az átvitel minőségére
 - Nincs állapotinformáció a végpontokról
- Más architektúrákban fontos (volt)
 - Virtuális áramkörök
 - ATM, Frame Relay, X.25
- Hasznos lehetne a QoS elvárások betartásához
 - Az úton érintett routerekben erőforrást foglalunk
 - Resource reSerVation Protocol
 - MultiProtocol Label Switching (MPLS) – címkekapcsolás

1. Portok besorolása
2. A hálózati réteg funkciói
3. IPv4
4. IPv6

IPV4 DATAGRAM

• IP protokoll
verziószám

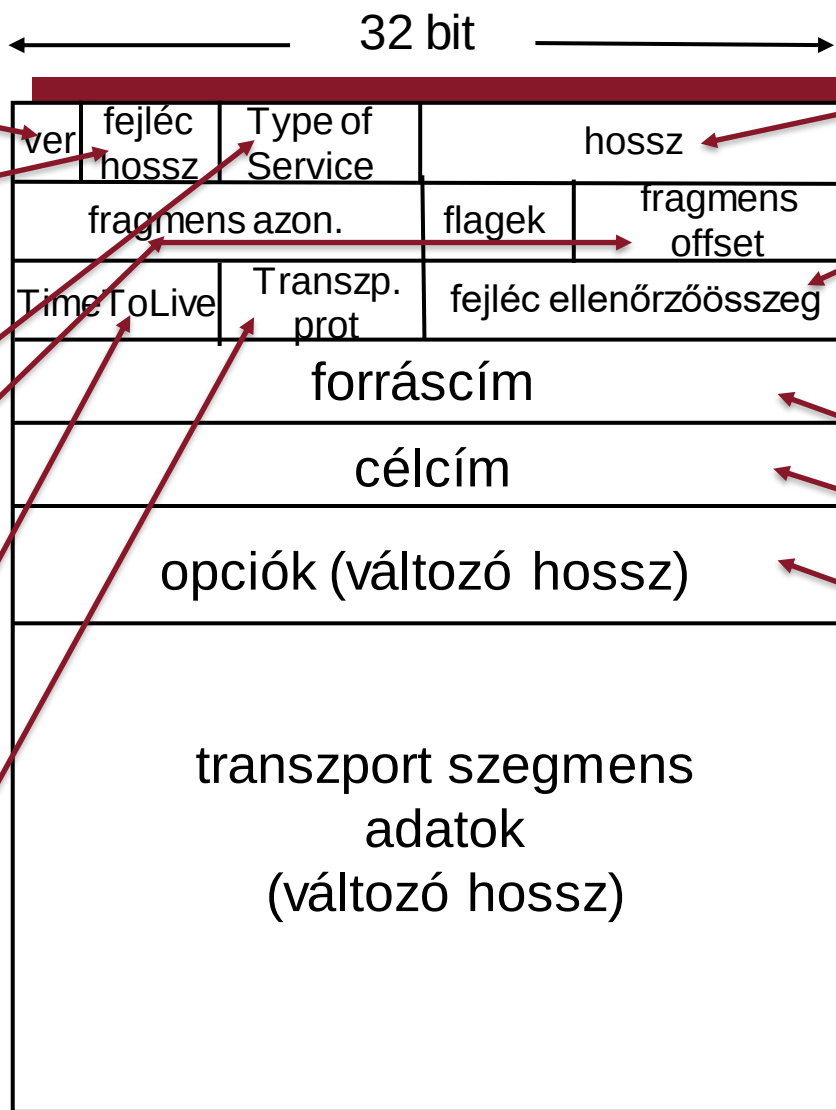
• Fejléc hossza
bájtban

• Információ a
csomag kezelési
besorolásáról
(QoS-hez)

• Információk a
darabolásról
(fragmentációról)

• Hátralévő ugrások
(hopok) maximális
száma – minden
router eggyel
csökkent

• Az adatokat küldő
protokoll



• Teljes hossz
bájtban

• Ellenőrző összeg
csak a fejlécre

• Küldő címe

• Fogadó címe

• Opciók, például:

–Időbélyeg

–Érintendő

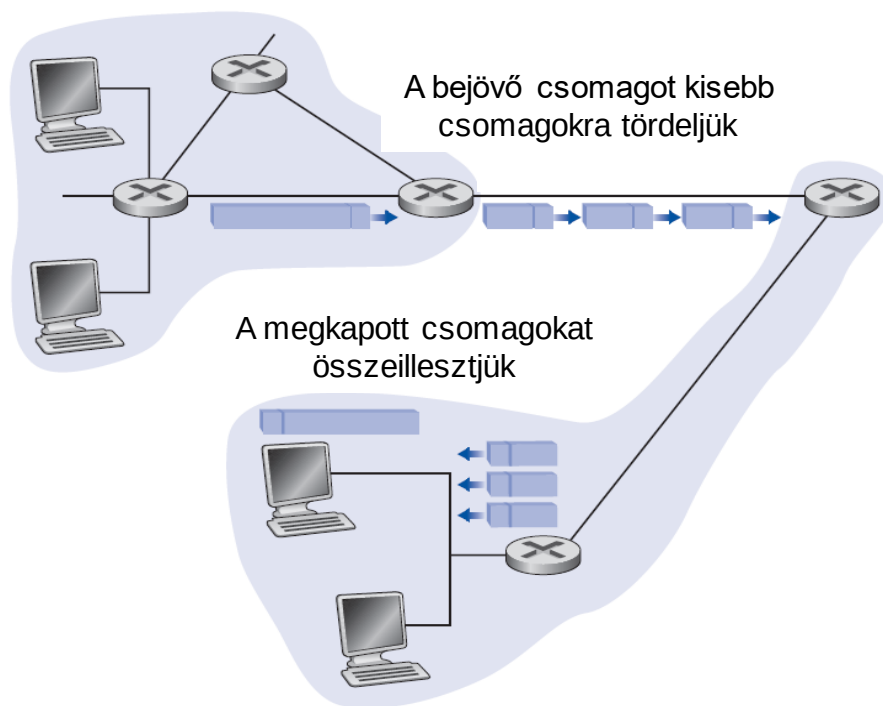
ruterek listája

Ki küldi hova?
Mi van benne?
Meddig él?

DARABOLÁS ÉS ÖSSZERAKÁS

- Az IP alatti rétegben korlátozott a keretek mérete
- MTU: Maximum Transmission Unit
- Különböző linkeken különböző lehet
- Az IP csomagok darabolására lehet szükség
 - A routerek darabolnak
 - Összeillesztés csak a fogadónál
 - Információk a csomagok fejlécében (8 bájtos blokkban számolva)
- Darabok, töredékek, fragmensek

-> CSAK IPv4-ben van darabolás! (Routernél!)



Küldő -> router -> fogadó
Küldendő csomag: 4000 bájt
De a link **MTU-ja** csak 1500 bájt

-> DARABOLNI kell!

DARABOLÁS ILLUSZTRÁCIÓ

- Példa

- 3980 bájtos TCP szegmens
- A következő link MTU-ja 1500 bájt

- IP fejléchoossz

- 20 bájt

- Fragmensekben lévő adat maximális hossza

- 1480 bájt

- Darabolás

- 1480+1480+1020

	length	ID	fragflag	offset	
	=4000	=13	=0	=0	



	length	ID	fragflag	offset	
	=1500	=13	=1	=0	

	length	ID	fragflag	offset	
	=1500	=13	=1	=185	

	length	ID	fragflag	offset	
	=1040	=13	=0	=370	

- Jó, mert
 - A küldő alkalmazásnak nem kell törődnie az útvonal egyes linkjeinek jellemzőivel
 - Illeszkedik a rétegezett szemlélethez
- Nem jó, mert
 - Terheli a routert – késlelteti a csomagot
 - Egy elvesző fragmens miatt egy teljes szegmenst újra kellhet küldeni (TCP)
- **Célszerű elkerülni** MANPSÁG MÁR NEM JELLEMZŐ!!!
- Megoldás: az útvonal legkisebb MTU-ját kellene használni egyből (Path MTU discovery)
 - DF (Don't-Fragment-Bit) flag beállítása a fejlécben
 - Ha emiatt el kell dobni, arról visszajelzést kap a küldő
 - Kisebb MTU-val újrapróbáljuk
 - Addig amíg megfelelő MTU-t nem találunk

- IPv4

- 32 bites cím

255.255.255.255 -> max ÉRTÉK
256 -> 2^8

- Könnyebb olvashatóság kedvéért 4 darab oktetre (nyolc bites részre) bontva

223.1.1.1 = $\underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$

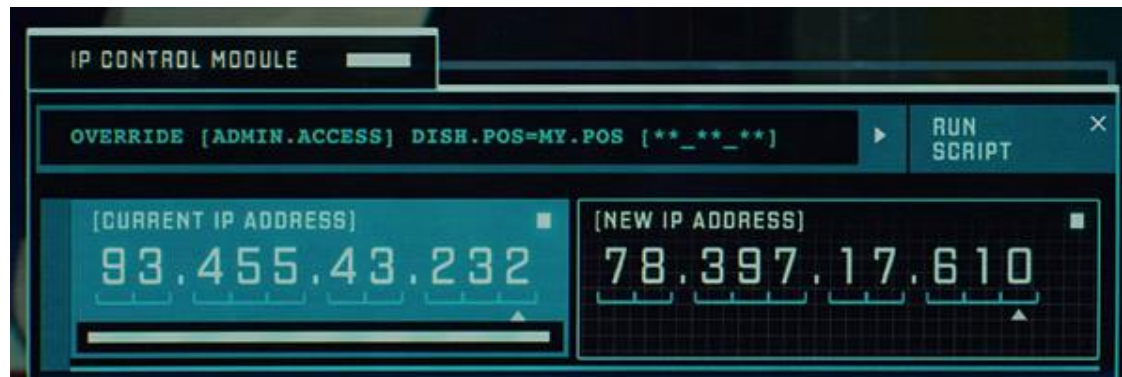
- Iron man 3



BINÁRIS-DECIMÁLIS ÁTALAKÍTÁS

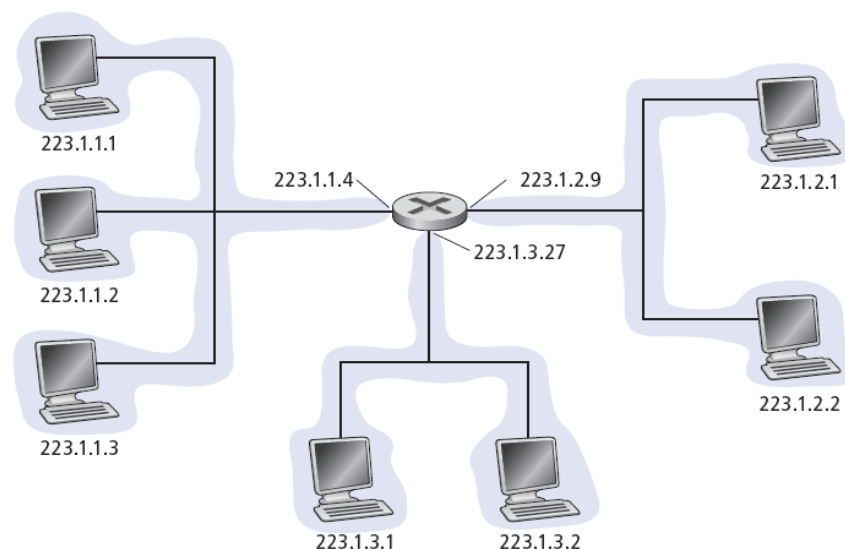
$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$$

- $11011111_B = 1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 223$
- Maximális érték:
 $11111111_B = 100000000_B - 1 = 2^8 - 1 = 255$



- Egy hoszt, vagy egy router egyik **interfészét** azonosítja
- Interfész a rendszer és a link között
 - Általában egy hálózati kártya (NIC) valósítja meg a hosztban
 - A routereken portokhoz kapcsolódnak, de lehetnek „virtuálisak” is
 - Általában egy routernek több interfésze is van
 - Egy interfész – egy IPv4 cím

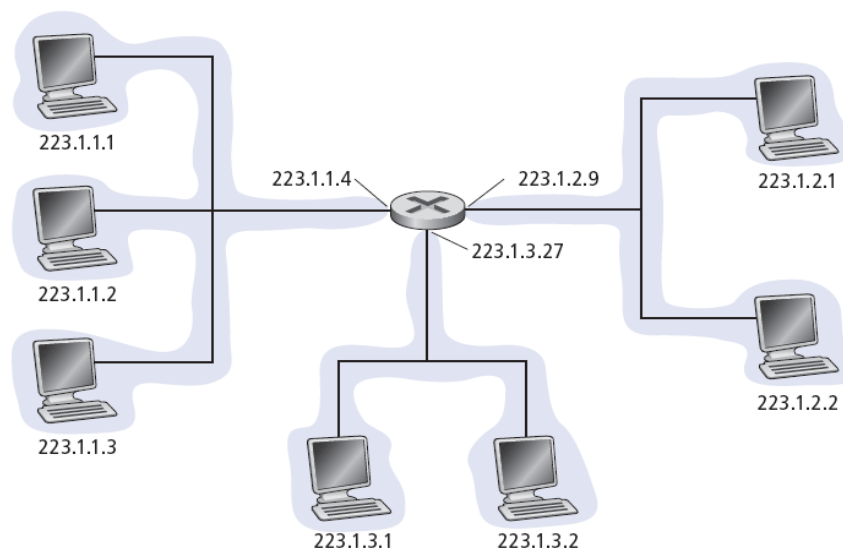
!!!!



IP CÍMZÉS - HÁLÓZATOK

- Az IP cím két részből áll
 - **netid**: a felső bitek azonosítják a hálózatot
 - **hostid**: az alsó bitek azonosítják az interfészt a hálózaton belül
- A hálózatok értelmezése
 - Azon interfészek halmaza, amiknél a netid azonos
 - Az egy hálózatban lévő elemek
 - Szomszédosak
 - Úgy küldhetnek egymásnak, hogy nem kell keresztülmenni egy routeren sem
- Hálózati maszk
 - A hálózati részt maszkolja a címből
 - A felső bitjei egyesek a többiek nullások

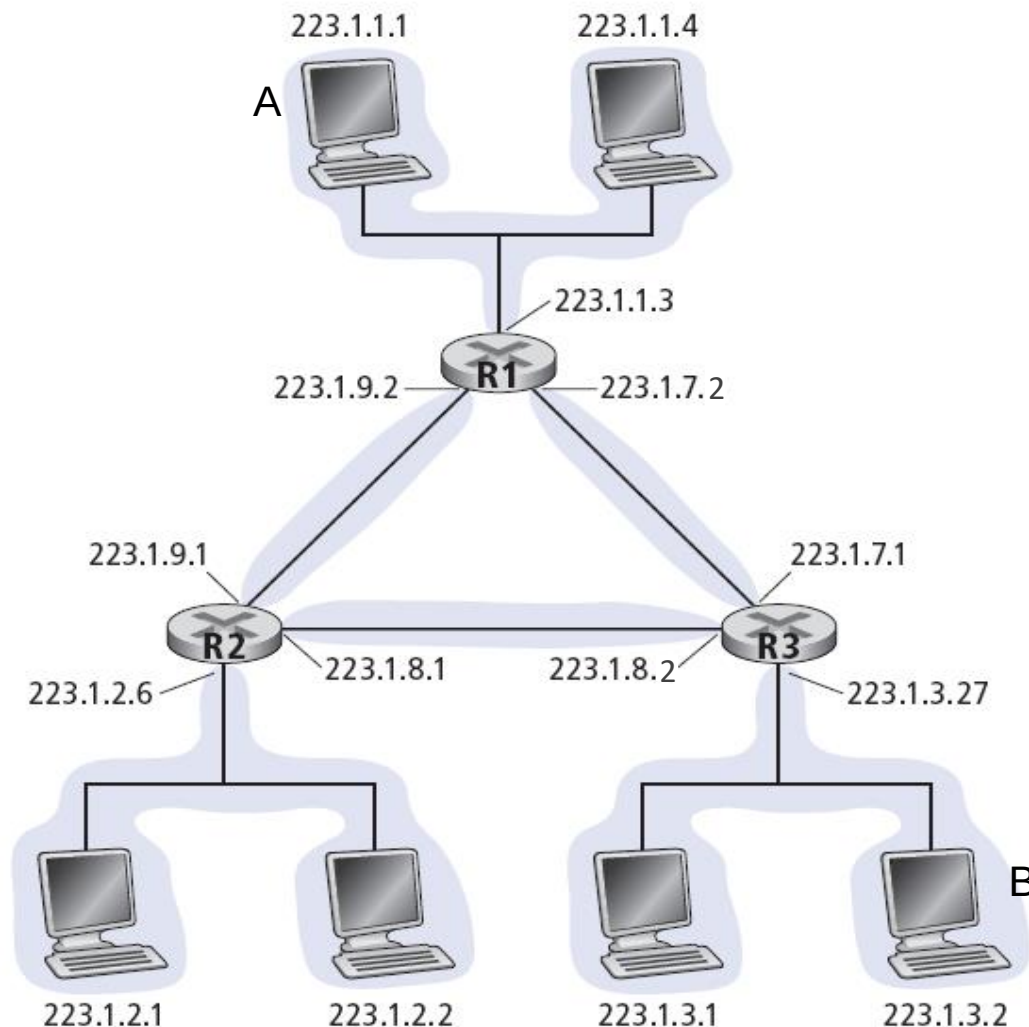
- Három hálózat egy routerrel összekötve
 - Az első három oktet azonosít



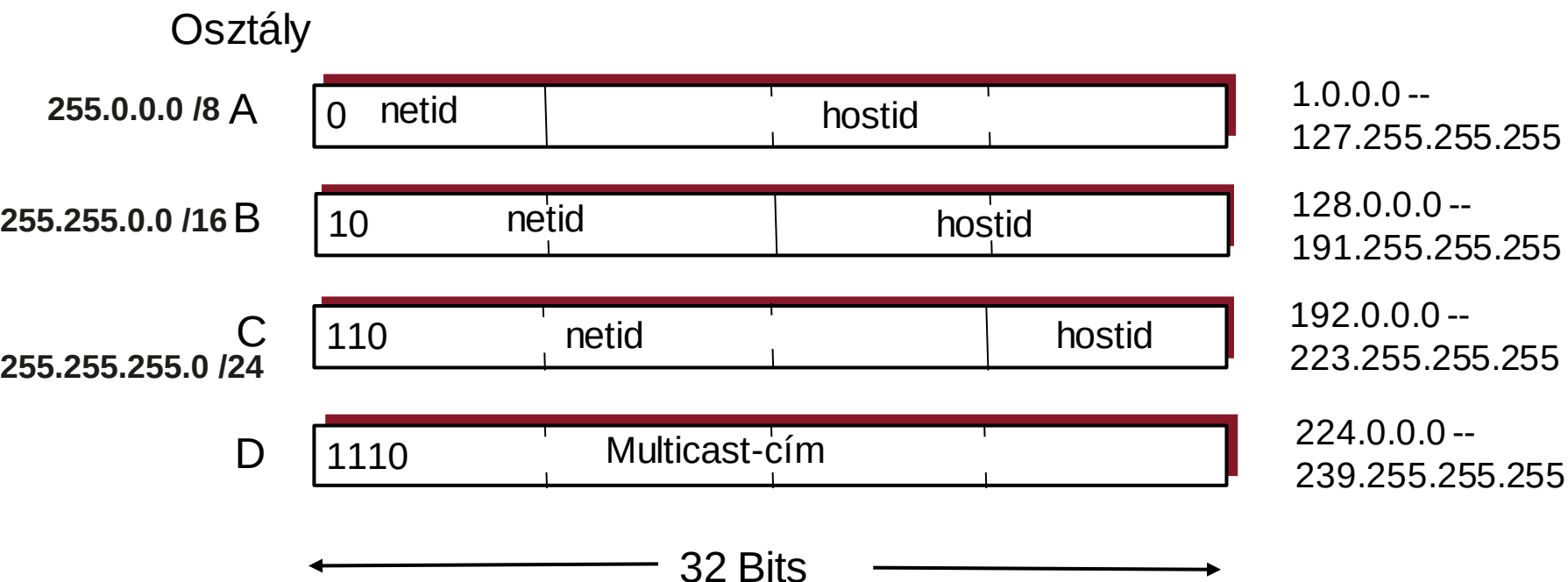
- Hálózati maszk
 - 24 bites – 24 darab egyes
 - 255.255.255.0

CÍMZÉS ILLUSZTRÁCIÓ

- Hány hálózat van a példában?
 - 6
- Legalább milyen hosszú maszkot kell használnunk?
 - 24
- Mi kerül a címmezőibe annak a csomagnak, amit A-ból B-be küldünk?
 - Forrás: 223.1.1.1
 - Cél: 223.1.3.2

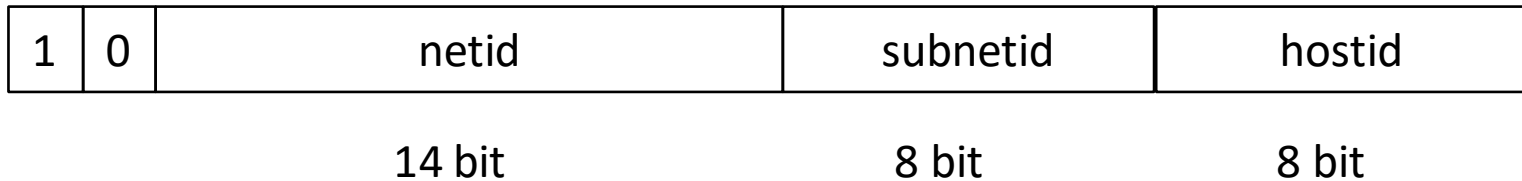


- Eredetileg az IPv4-ben osztályokra osztották a címeket
- Az osztály határozta meg a maszk hosszát
- **Osztályalapú címzés (classful addressing)**



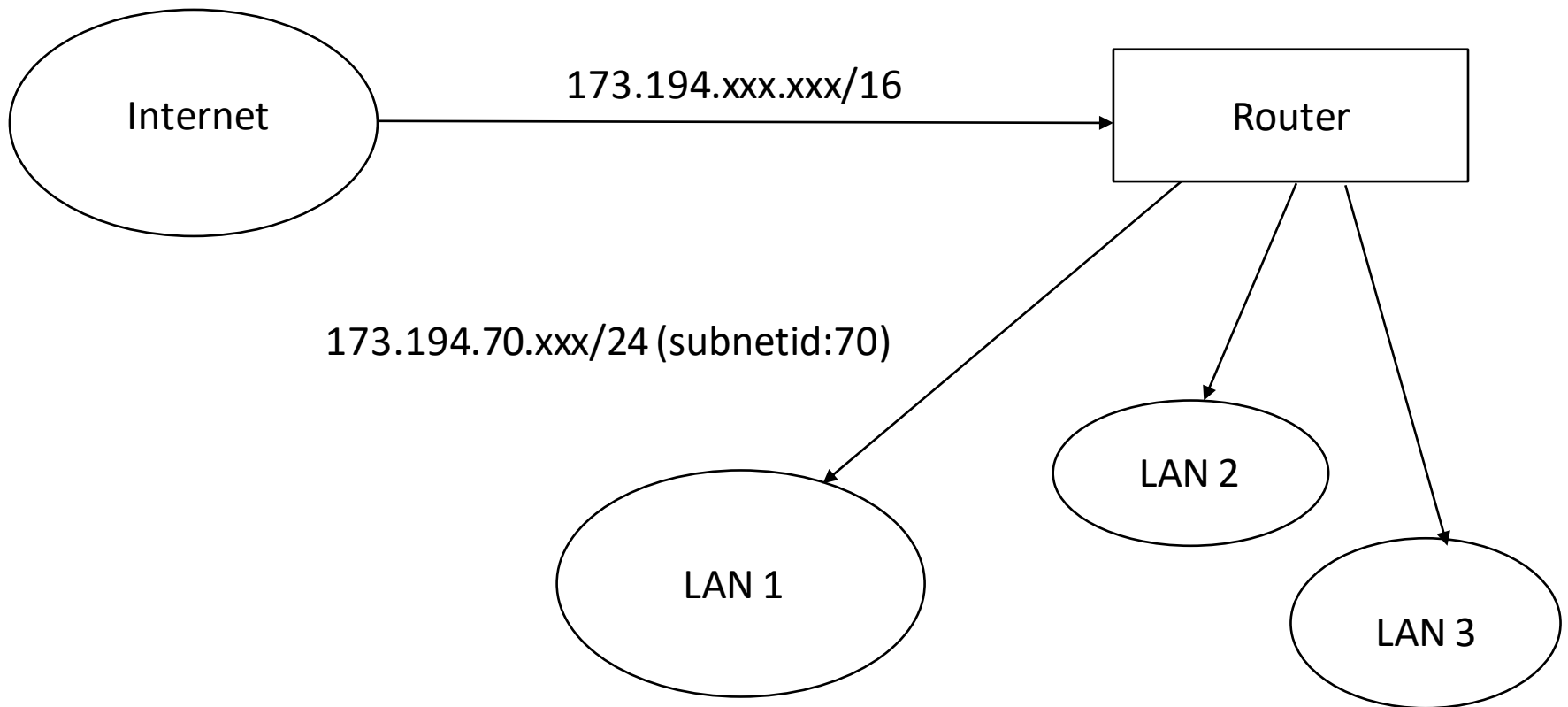
ALHÁLÓZATOKRA BONTÁS

- Az A és B osztályú hálózatok irreális méretűek
- Egy-egy hálózatot további **alhálózatokra (subnet)** lehet bontani subnetid-eket bevezetve



- Ez csak példa. A felosztás és az alhálózati azonosítók a hálózatot kezelő szervezet lokális döntései

- Egy hálózat felosztása kívülről nem kell, hogy látható legyen



- A felosztás után meghatározható az alhálózati maszk (subnet mask)

- Megadja, hogy hány bit azonosítja az alhálózatot
- A rendszer az IP címmel együtt tárolja

16 Bit	8 Bit	8 Bit
1111111111111111	11111111	00000000

Alhálózati maszk: 0xfffff00=255.255.255.0 vagy /24

- A maszk segítségével egy interfésznél eldönthető, hogy egy másik interfész hozzá képest hol van
 - Ugyanabban az alhálózatban (szomszédos)
 - Ugyanabban az (osztályalapú) hálózatban, de másik alhálózatban
 - Másik hálózatban

- Saját cím: 173.194.39.104
- Alhálózati maszk: 255.255.255.0
- PC-A címe: 173.194.39.96, PC-B címe: 173.194.55.96
- Hozzánk képest hol van PC-A és PC-B?
 - $173.194.39.104 \text{ \& } 255.255.255.0 = 173.194.39.0$
 - $173.194.39.96 \text{ \& } 255.255.255.0 = 173.194.39.0$
 - azonos, ugyanaz az alhálózat
 - $173.194.55.96 \text{ \& } 255.255.255.0 = 173.194.55.0$
 - különböző, másik alhálózat
- Osztályalapú hálózati cím: 173.194.0.0
- Alhálózat azonosító: 39

- Az (al)hálózaton belül a hostid-vel azonosítunk
- Nem megengedett azonosítók
 - Csupa nulla bit
 - Ez maga a **hálózati cím (netid)**
 - Erre nem küldhető csomag
 - Csupa egyes bit
 - **Szórási (broadcast) cím**
 - Az ide küldött csomagot minden szomszéd megkapja
- Például
 - 173.194.70.0/24 hálózati cím
 - 173.194.70.255/24 szórási cím
 - 192.168.1.40/29 ?
 - 192.168.1.47/29 ?
 - 192.168.1.255/29 ?

Hálózati címnél: utolsó oktettje MINDIG PÁROS!!
PI: 70, 40

Szórási címnél: utolsó oktettje MINDIG PÁRATLAN!!!
PI: 255, 47

- Milyen hálózatban van a www.bme.hu?
- IP cím: 152.66.115.203 =
10011000.01000010.01110011.11010011
- Hálózati maszk: 255.255.0.0
11111111.11111111.00000000.00000000
- B osztály
- Hálózati cím: 152.66.0.0
- Szórási cím: 152.66.255.255
- Első és utolsó interfész azonosító: .0.1 és .255.254
- Hány interfész használható a hálózatban:
 $256 * 256 - 2 = 65\ 534$
- Vajon tényleg így van?

EZEK! nem használhatóak normál internetes forgalomra!
OTTHON/CÉGNÉL! HASZNÁLATOS!

Address Block	Present Use	Reference
0.0.0.0/8	"This" Network	[RFC1700, page 4]
10.0.0.0/8	Private-Use Networks	[RFC1918]
14.0.0.0/8	Public-Data Networks	[RFC1700, page 181]
24.0.0.0/8	Cable Television Networks	--
39.0.0.0/8	Reserved but subject to allocation	[RFC1797]
127.0.0.0/8	Loopback	[RFC1700, page 5]
128.0.0.0/16	Reserved but subject to allocation	--
169.254.0.0/16	Link Local	--
172.16.0.0/12	Private-Use Networks	[RFC1918]
191.255.0.0/16	Reserved but subject to allocation	--
192.0.0.0/24	Reserved but subject to allocation	--
192.0.2.0/24	Test-Net	
192.88.99.0/24	6to4 Relay Anycast	[RFC3068]
192.168.0.0/16	Private-Use Networks	[RFC1918]
198.18.0.0/15	Network Interconnect Device Benchmark Testing	[RFC2544]
223.255.255.0/24	Reserved but subject to allocation	--
224.0.0.0/4	Multicast	[RFC3171]
240.0.0.0/4	Reserved for Future Use	[RFC1700, page 4]

1. Portok besorolása
2. A hálózati réteg funkciói
3. IPv4
4. IPv6

- Eredeti motiváció (90-es évek): a **32 bites címzési tér hamarosan kimerül**
 - Valóban kimerült
 - Egy kicsit el lehet még odázni a végét, de csak átmenetileg
 - Hogyan kap majd a rengeteg (sokszor mobil) végpont IP címet?
- További motivációk
 - Egyszerűbb fejléc a **gyorsabb feldolgozás** érdekében
 - **Szolgáltatásminőség** (QoS) korrektebb támogatása
- IPv6 fejléc
 - Fix (40 bájt) hosszúságú !!!!!
 - Kiterjeszthető további mezőkkel

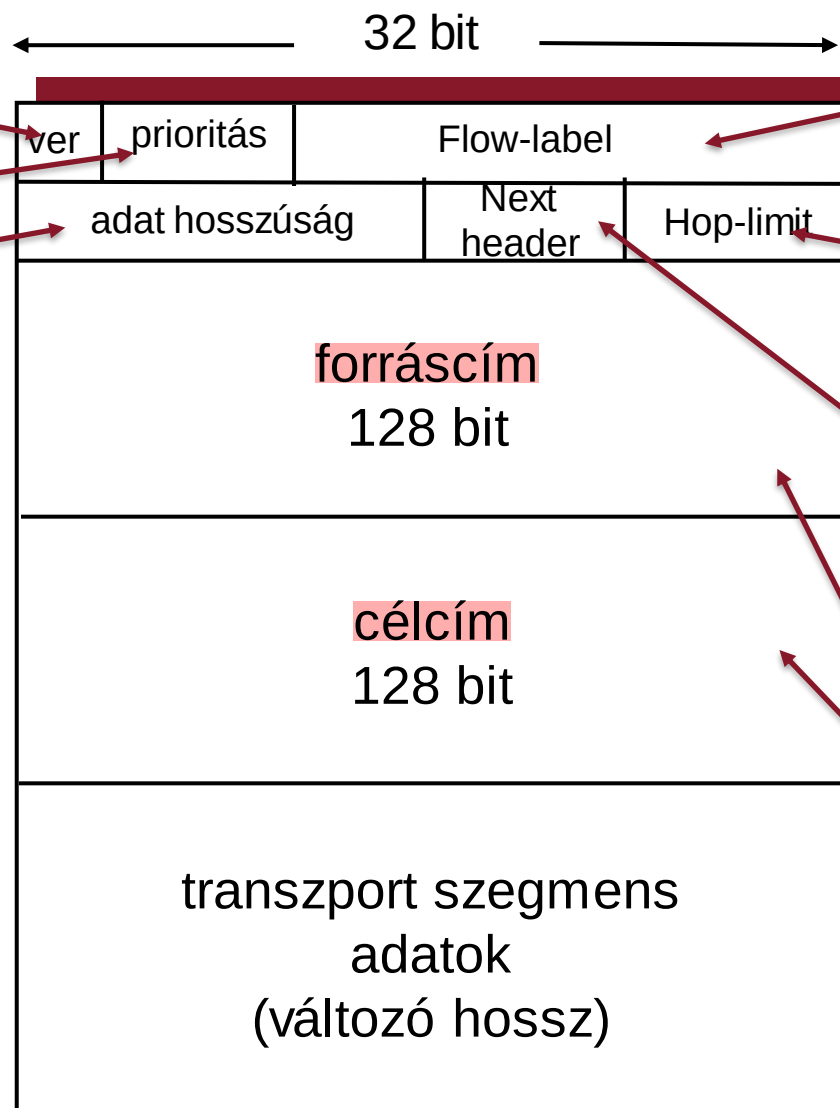
IPV6 DATAGRAM

• IP protokoll verziószám

- A csomag prioritása
- Hány bájtnyi adatot visz át

• Alapesetben

- Nincs ellenőrzőösszeg
- Nincs fejléchossz
- Nincsenek opciók
- Nincs darabolás



• Összetartozó csomagok azonosítása

• Hátralévő ugrások (hopok) maximális száma

• Az adatokat küldő protokoll, vagy hivatkozás további fejléc mezőkre

• Küldő címe

• Fogadó címe

- 128 bit 16 bites (hextet) részekben
- Hexadecimális formában
2001:0738:0000:407f:113b:0000:0000:cadd
- A bevezető nullák a hextetben elhagyhatók
2001:738:0:407f:113b:0:0:cadd
- Egy helyen elhagyható egy csak nullákat tartalmazó sorozat (célszerűen a leghosszabb)
2001:738:0:407f:113b::cadd
 - Több elhagyása többértelműsége vezet
- Így sem lesz könnyű megjegyezni egy ilyen
 - DNS: AAAA

- A koncepció azonos az IPv4-nél látottal
 - Hálózati cím, alhálózati cím, interfész azonosító
- Hálózati maszk hossza alapértelmezetten 64
 - Így is nagyon sok hálózat és nagyon sok interfész van
2021:3:8:15:15:acad::c1ca / 64
- Alhálózatok képzése is ugyanúgy megy
- Nagy különbség
 - Egy interfésznek **több** globális IPv6 címe lehet
 - Van lokális címe is a hálózaton belüli forgalmazáshoz
- **Link-local cím**
 - Csak az adott (al)hálózatban kell egyedi legyen
 - FE80::/8 tartományból
- Különböző vezérlési célokra egyéb speciális címtartományok
 - Pl. egy ilyet használunk szórás cím helyett

!!!!



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

IP alhálózatok

2023. március 31.

Mészáros András

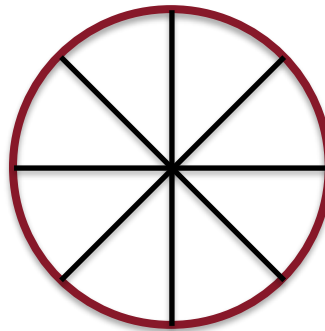
BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



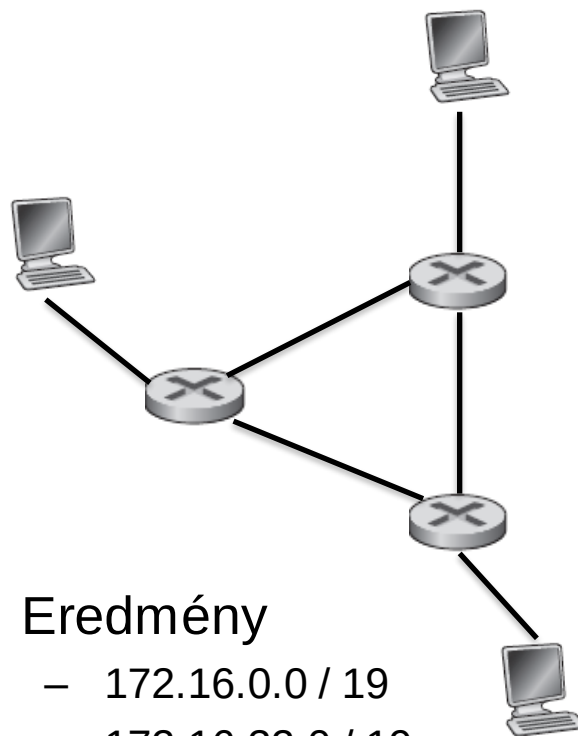
1. Alhálózatok tervezése
2. Változó hosszúságú maszkok

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat.

- A szervezetekhez rendelt osztályalapú hálózatok
 - Túl nagyok, túl sok interfész
 - Nem illeszkednek a szervezet felépítéséhez, hálózati céljaihoz
- Kisebb alhálózatokra osztják fel a hálózatot
- Első megközelítés
 - Ismert a szükséges alhálózatok száma
 - Azonos méretű darabokra osztjuk a címteret
 - Betartva az IPv4 címezés szabályait
- Azonos hosszúságú maszkok



- Címezzük meg az ábrán látható 6 alhálózatot
- Rendelkezésre álló hálózat:
172.16.0.0 / 16
- Összesen $2^{32-16} = 65536$ darab cím
- Egyenletes felosztás
 - Hányfelé?
 - Hatfelé: 10922 cím alhálózatonként, de jó lesz így?
 - Csak 2 hatvány jöhet szóba darabszámra, mert a maszkkal csak azt lehet kezelni
- Osszuk nyolc részre
 - Három bitet használunk fel az interfész részből



- Eredmény
 - 172.16.0.0 / 19
 - 172.16.32.0 / 19
 - 172.16.64.0 / 19
 - 172.16.96.0 / 19
 - 172.16.128.0 / 19
 - 172.16.160.0 / 19

- Hány interfész tehető egy alhálózatba?
 $65\,536 / 8 = 8192$
 - Szükség van ennyire?
 - Mit tehetünk, ha újabb routerekkel bővül a hálózatunk?
 - Használatlan alhálózatok
 - Újrafelosztás: kerüljük el ha lehet
 - Továbbosztás
- Hány használatlan alhálózat maradt?
 - Kettő, de ezek is használhatók bővítéskor
- A szükséges darabszámok felől közelítve az egyenletes felosztáshoz
 - Csak akkora legyen egy alhálózat, amekkora kell
 - Eredmény
 - $172.16.0.0 / 30 : 172.16.0.1 - 172.16.0.2$
 - $172.16.0.4 / 30 : 172.16.0.5 - 172.16.0.6$
 - ...
 - $172.16.0.20 / 30 : 172.16.0.21 - 172.16.0.22$
 - Így hány használatlan hálózat marad? ($65\,536 / 4 - 6 = 16378$)

- Minimális számú alhálózat esete

1010 1100.0001 0000.0000 0 0000.0000 0000

- Rövid maszk /19
- Sok interfész
- Egy-egy alhálózat könnyen bővíthető
- Új alhálózattal nehezebben bővíthető

- Minimális méretű alhálózatok esete

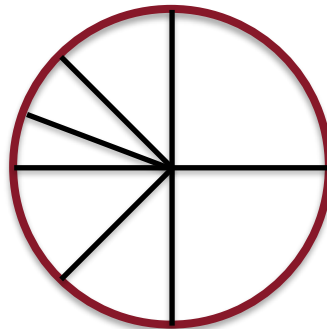
1010 1100.0001 0000.0000 0000.0000 0000

- Hosszú maszk /30
- Kevés interfész
- Egy-egy alhálózat nehezen vagy nem is bővíthető
- A sok használatlan alhálózat bővítéskor bevethető
 - Csak kis méretű új alhálózatokkal

- Láttuk, hogy egyik véglet sem az igazi
- Tartsuk szem előtt a bővíthetőséget mindkét értelemben
 - Túl kicsi és túl nagy sem jó
 - Legyen a maszk a középúton, pl. 24 bites
- Alhálózatokban lévő interfészek száma
 - Akár elég nagy eltérés is lehet az igények között
 - Közvetlen router-router kapcsolathoz elég kettő
 - Egy nagyobb LAN-nál akár több száz kellhet
 - Legyen a maszk a középúton, pl. 24 bites
- Nagyon rossz kihasználtságú az eredmény
- Ennél hatékonyabb megoldás kellene
 - Muszáj az összes alhálózatnak ugyanakkorának lennie?

1. Alhálózatok tervezése
2. Változó hosszúságú maszkok

- Új megközelítés
 - Ne ragaszkodjunk az egyenletes felosztáshoz, legyen egyenetlen
 - Az egyes alhálózatok nem azonos méretűek
- A címzés alapszabályai továbbra is megmaradnak
 - 2 hatványa lesz az alhálózatban a címek darabszáma
- Egy-egy alhálózat mérete illeszkedjen az adott darabszámhoz
 - Nyilván ne legyen kevesebb
 - Sokkal több sem
- Különböző hosszúságúak lesznek a maszkok
 - Variable Length Subnet Mask (VLSM) séma



- Készüljünk a LAN-ok bővülésére
- Rendelkezésre álló hálózat:

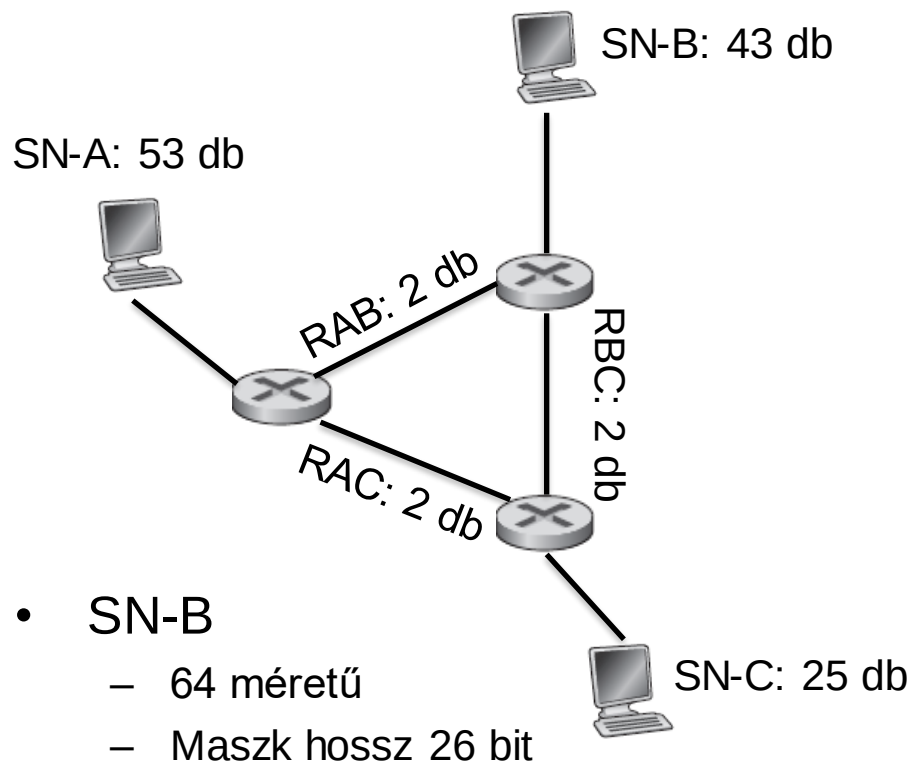
172.16.0.0 / 16

- Egyenetlen felosztás
 - Menjünk sorban
 - Kezdjük a legnagyobb alhálózattól
 - Addig osszuk, amíg kell

- SN-A
 - 53 darab – 64 méretű legyen
 - A hostid rész hossza $\log_2 64 = 6$ bit
 - Maszk hossza $32 - 6 = 26$ bit

1010 1100.0001 0000.0000 0000.0000 0000

- 172.16.0.0/26 , 255.255.255.192
- Szórási cím: 172.16.0.63
- Interfészek: .0.1 - .0.62
- 1 router port és 61 hoszt



- SN-B
 - 64 méretű
 - Maszk hossz 26 bit

1010 1100.0001 0000.0000 0000.0100 0000

- 172.16.0.64/26, 255.255.255.192
- Szórási cím: 172.16.0.127
- Interfészek: .0.65 - .0.126
- 1 router port és 61 hoszt

• SN-C

- 25 darab – 32 méretű legyen
- A hostid rész hossza $\log_2 32 = 5$ bit
- Maszk hossza $32 - 5 = 27$ bit

1010 1100.0001 0000.0000 0000.1000 0000

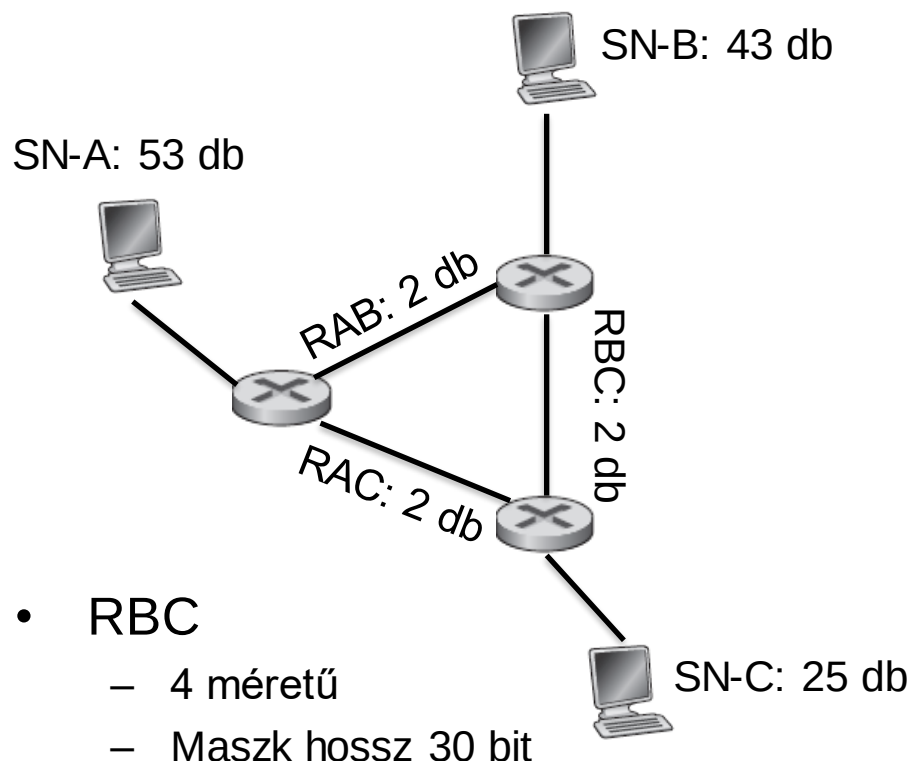
- 172.16.0.128/27, 255.255.255.224
- Szórási cím: 172.16.0.159
- Interfészek: .0.129 - .0.158
- 1 router port és 29 hoszt

• RAB

- 2 darab – 4 méretű legyen
- A hostid rész hossza $\log_2 4 = 2$ bit
- Maszk hossza $32 - 2 = 30$ bit

1010 1100.0001 0000.0000 0000.1010 0000

- 172.16.0.160/30, 255.255.255.252
- Szórási cím: 172.16.0.163
- Interfészek: .0.161 - .0.162
- 2 router port



• RBC

- 4 méretű
- Maszk hossz 30 bit

1010 1100.0001 0000.0000 0000.1010 0100

- 172.16.0.164/30, 255.255.255.252
- Szórási cím: 172.16.0.167
- Interfészek: .0.165 - .0.166
- 2 router port

• RAC

- 4 méretű
- Maszk hossz 30 bit

1010 1100.0001 0000.0000 0000.1010 1000

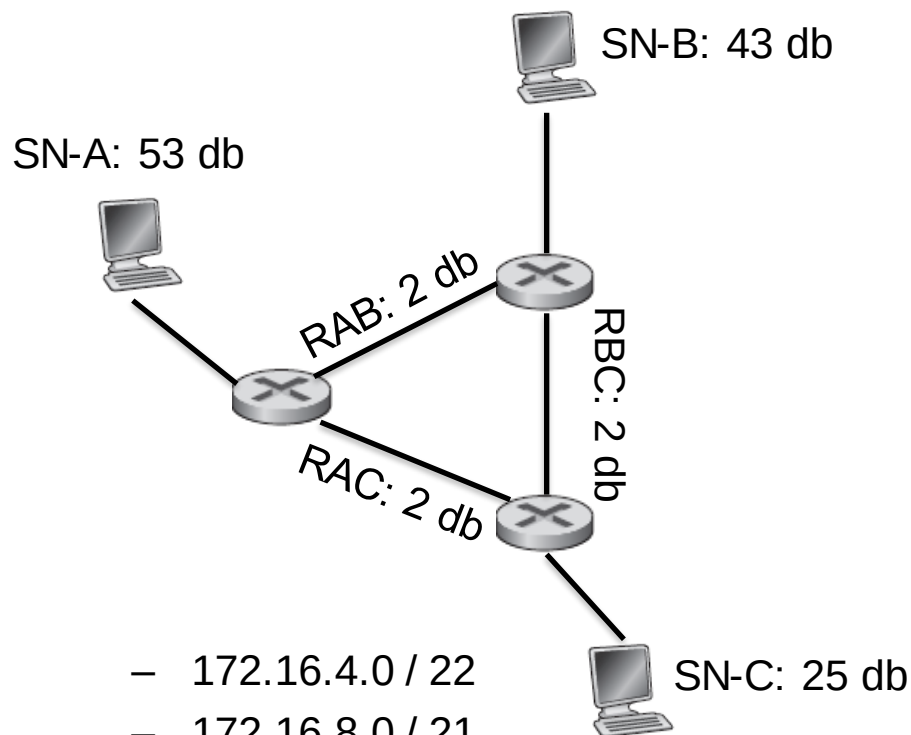
- 172.16.0.168/30, 255.255.255.252
- Szórási cím: 172.16.0.171
- Interfészek: .0.169 - .0.170
- 2 router port

• Tudjuk bővíteni a LAN-okat?

- Igen, korlátozottan

• Milyen tartományok maradtak szabadon?

- 172.16.0.172 / 30
- 172.16.0.176 / 28
- 172.16.0.192 / 26
- 172.16.1.0 / 24
- 172.16.2.0 / 23



- 172.16.4.0 / 22
- 172.16.8.0 / 21
- 172.16.16.0 / 20
- 172.16.32.0 / 19
- 172.16.64.0 / 18
- 172.16.128.0 / 17

- Célszerű a legnagyobb alhálózattól kezdve haladni a felosztható tartományon
- A címek darabszáma kettő hatványa
- Hálózati és szórási cím csökkenti az interfészek számát
- A következő alhálózat tartománya közvetlenül követheti
 - **Trükk:** 256-nál kisebb címtartománynál nem kell bitszinten nézegetni, hogy hol kezdődik a következő szabad alhálózat, elég a hálózati címhez hozzáadni a tartomány méretét
 - Pl: SN-C: 172.16.0.128 / 27 $\longrightarrow$ RAB: 172.16.0.(128+32), maszk: /30
- A darabszám meghatározza a maszk hosszát
 - **Trükk:** 24-nél hosszabb maszknál a tartomány méretéből könnyű megkapni a maszk utolsó oktetjét, csak ki kell vonni 256-ból
 - Pl: SN-C: 172.16.0.128 / 27 $\longrightarrow$ maszk: 255.255.255.(256-32)
- A trükkök kiterjeszthetők nagyobb tartományokra is (csak nem az utolsó oktetben kell alkalmazni őket)

- Négy LAN
 - LAN1: 30 darab
 - LAN2: 31 darab
 - LAN3: 16 darab
 - LAN4: 600 darab
- Felosztható tartomány: 172.16.16.0 / 20
- Darabszámok, hálózatok, maszkok
 - LAN4: 1024 db, 172.16.16.0/22, 255.255.252.0
 - LAN2: 64 db, 172.16.20.0/26, 255.255.255.192
 - LAN1: 32 db, 172.16.20.64/27, 255.255.255.224
 - LAN3: 32 db, 172.16.20.96/27, 255.255.255.224

- IPv6 esetén nem annyira szorít a hatékonyság
- Az óriási címtartományok miatt nem baj, ha pazarolunk
- A /64-es hálózatot darabolhatjuk akár egyenletesen, az alhálózatok számának megfelelően
 - Kevés, de nagy alhálózat lesz
 - További alhálózatokat ezekből tudunk kihasítani
 - Végül egyfajta VLSM lesz
- Kicsit nehezebb követni, mert hexadecimális számokkal dolgozunk
- Például:
 - Osszuk fel négyfelé a 2001:DB8:172:16::/64 hálózatot
 - Eredmény:
 - 2001:DB8:172:16:0000::/66
 - 2001:DB8:172:16:4000::/66
 - 2001:DB8:172:16:8000::/66
 - 2001:DB8:172:16:C000::/66

- Az előbbi példára könnyebben átlátható megoldás lehet ez is:

2001:DB8:172:16:0::/80

2001:DB8:172:16:1::/80

2001:DB8:172:16:2::/80

2001:DB8:172:16:3::/80

|

> Általában ezt növeljük!
Legegyszerűbb!



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

IP címek kiosztása, címfordítás
2023. április 14.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



1. CIDR
2. Cím kiosztás, DHCP
3. Címfordítás
4. Datagram továbbítás

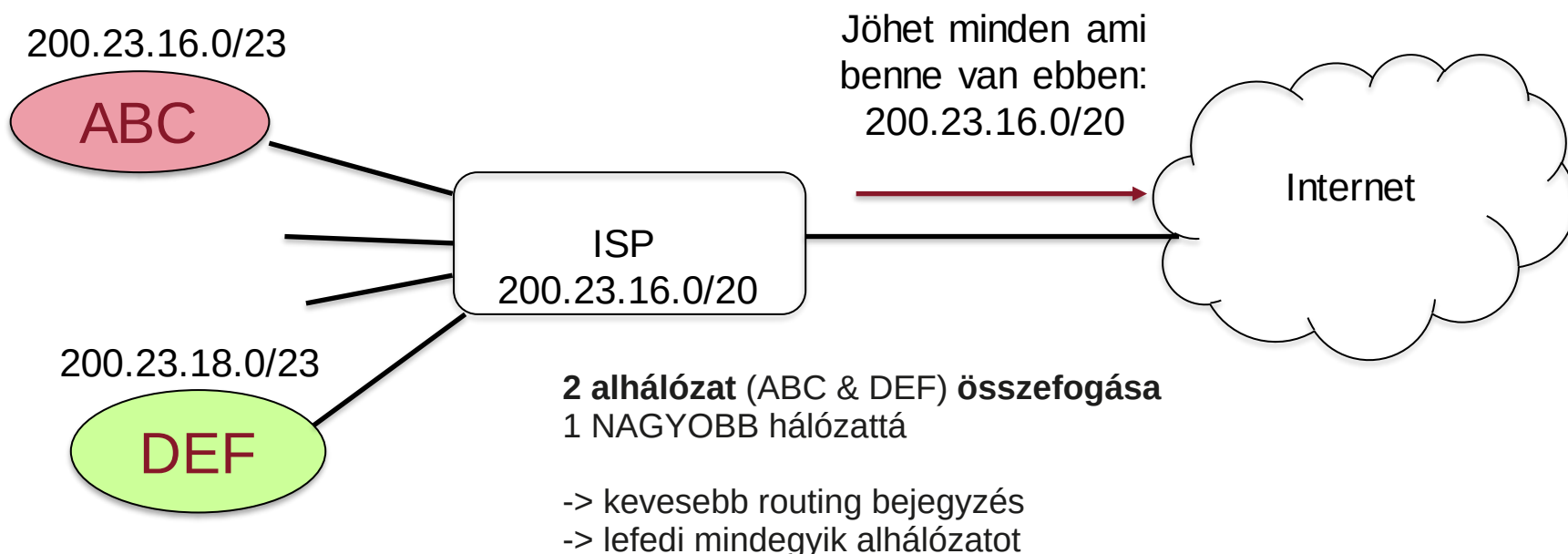
A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat

- IPv4 címtartományok (hálózatok) kiosztása
 - Központi szolgáltató szervezetek döntenek (pl. IANA)
 - Osztályozási szabályokat figyelembe véve
 - Nehezen követhető, az egymást követő hálózatok tulajdonosainak semmi közük egymáshoz
- Problémák
 - **Címek pazarlása** Legfőbb indok a leváltásra!
 - Óriási, kihasználatlan tartományok egyes cégeknek
 - Irányítási (routing) **táblák növekedése**
 - Nem lehet összefogni címeket
 - Minden hálózatot külön kell beletenni
 - A táblák frissítésének gyakorisága
 - Sok független hálózat
 - Sok változás
 - Sok frissítés

Classful helyett -> Classless

- **CIDR: Classless InterDomain Routing**
- Alapötlet
 - Az alhálózatokra osztás után a cím mellé a maszkot is kell tárolni
 - Nincs explicit határ a netid és a subnetid között
- Hálózatok kiosztása
 - Az IANA által kezelt összefüggő tartományok régió alapú szétosztása
 - Regional Internet Registries (RIR)
 - APNIC (Asia Pacific Network Information Centre) – Ázsia/Csendes Óceán
 - ARIN (American Registry for Internet Numbers) – Észak-Amerika, Afrika déli része
 - LACNIC (Regional Latin-American and Caribbean IP Address Registry) – Latin-Amerika és a Karib-térség
 - RIPE NCC (Réseaux IP Européens) - Európa, Közel-kelet, Középázsia és Afrika északi része

- A regionális szervezetek összefüggő tartományokat tudnak adni az ISP-knek
- Az ISP-k **összefüggő tartományokat** tudnak adni a felhasználóiknak
- **Írányítási táblákban összevonhatunk bejegyzéseket**
 - Az alhálózatra bontás fordítottja

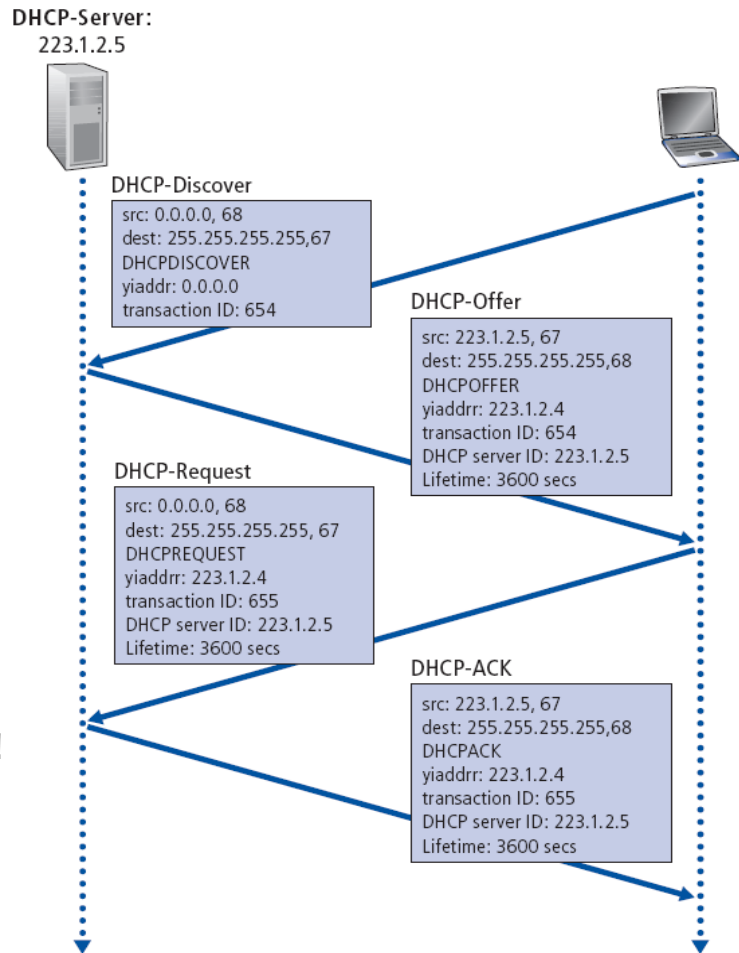


1. CIDR
2. Címkiosztás, DHCP
3. Címfordítás
4. Datagram továbbítás

- Kiindulás: megvannak az alhálózatok
- Következő lépés címek beállítása az interfészeken
- Manuális konfiguráció
 - IP cím
 - Maszk
- Automatikus konfiguráció
 - IPv4: DHCP - Dynamic Host Configuration Protocol
 - IPv6: a router képes automatikus címkonfigurációra (SLAAC – StateLess Address AutoConfiguration), vagy használhatja a DHCPv6-ot

- Routersen tipikusan ezt használjuk
- Szervereken tipikusan ezt használjuk
- Többféle hibát ejthetünk, például...
 - Hozszt interfészen beállított cím és maszk
 - 202.21.256.71, 255.255.250.0
 - Egy router két interfészen beállított címek és maszkok:
 - 203.55.22.57, 255.255.255.192
 - 203.55.22.24, 255.255.255.240
 - Két szomszédos hosztnál beállított címek és maszkok:
 - 204.88.54.15, 255.255.255.192
 - 204.88.54.72, 255.255.255.192
 - Router interfészen beállított cím és maszk
 - 205.66.87.48, 255.255.255.240
 - Router interfészen beállított cím és maszk
 - 206.44.76.95, 255.255.255.224

- **Szerver-kliens alapú protokoll** (alk. réteg)
- 1. A hálózathoz csatlakoztatott hoszt címet kér: **DHCP-Discover** üzenetet küld **Van itt valaki DHCP szerver?**
 - A teljes szórási címre (255.255.255.255)
 - A 67-es portra
- 2. A DHCP szerver **DHCP-Offer** üzenettel válaszol és küld egy címet **Igen, ajánlok neked egy IP címet!**
 - A teljes szórási címre
 - A 68-as portra
 - Több szerver is lehet
- 3. A hoszt elkéri a kapott címet **DHCP-Request** üzenetet küldve **OK, kérem az IP címet!**
- 4. A DHCP szerver kiadja a címet a **DHCP-Ack** üzenettel **Rendben, tied az IP cím!**



- A szerver nyilvántartja a kiadható és kiadott címeket
 - Különböző módokat használhat (pl. permanens, ideiglenes)
 - Meghosszabbíthatja a kölcsönzést (lease) Kiosztott IP cím bizonyos ideig él!
 - Újra kiadhatja a címet, miután lejár a kölcsönzés, vagy a kliens visszaadja
- További paraméterek
 - Alapértelmezett átjáró
 - DNS szerver

1. CIDR
2. Címkiosztás, DHCP
3. Címfordítás
4. Datagram továbbítás

- Motivációk

- Kifelé ugyanaz a cím látsszon, akkor is, ha belül változtatunk
- Több hoszt lehessen ugyanazon a címen keresztül elérhető
 - Kevés az IPv4 cím -> elfogy az IPv4 címek
 - Privát címek a belső hálózatban -> más cím az internet felé
- Egy hoszt belső (valódi) címe ne látsszon kifelé

- Okok

- Csak egy címünk van, de több hosztunk
- A hosztok címeit automatikusan osztottuk ki
- ISP váltás miatt ne kelljen belül mindent átcímezni
- Biztonság (minimális szint)

- **NAT - Network Address Translation**

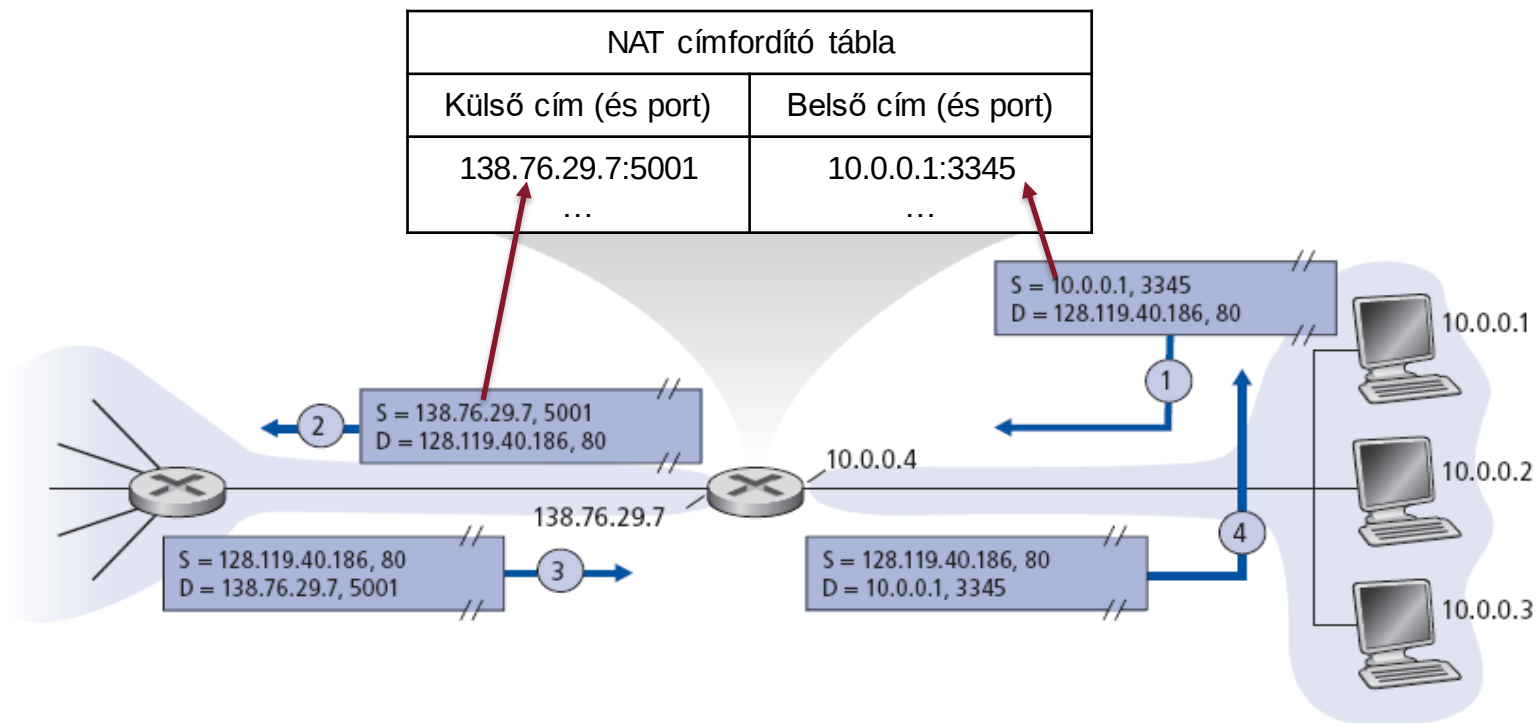
- A belső cím csak belül egyértelmű
- Az Internethez hozzákapcsoló router **ki- és visszacseréli** a belső címet egy globálisan egyértelmű címre
- Szükség esetén a router a szállítási rétegbeli azonosítókön is változtat
- A router egy **címfordító (transzlációs) táblát** használ

- A címfordítás beállítása

- **Statikusan – kézzel hozzárendeljük** `ip nat inside source static 192.168.1.1 200.1.1.5`
 - Szükség esetén a portokat is átírjuk
- **Dinamikusan – a router rendeli hozzá egy poolból**
 - Szükség esetén a portokat is átírja NAT vs PAT

```
ip nat pool NATPOOL 200.1.1.1 200.1.1.10 netmask 255.255.255.0
access-list 1 permit 192.168.1.0 0.0.0.255
ip nat inside source list 1 pool MYPOOL
```

NAT – TÖBB HOSZT EGY CÍM MÖGÖTT



- A LAN-t elhagyva **azonos forráscím** az összes csomagban (a példában 138.76.29.7)
- Megkülönböztetés a portszám alapján

- A LAN-ban a **valódi címeket** használjuk
- A példában ez a 10.0.0.0/24 privát tartomány

- **Megvalósítás a routerben**

- **Külső és belső oldal azonosítása**
- Az Internet felé kimenő datagramokban a küldő cím lecserélése
- **Dinamikus esetben az aktuális címfordítás részleteinek megjegyzése**
 - **Belső IP cím és port**
 - **Külső IP cím és port**
- Az Internet felől jövő datagramokban a célcím lecserélése

- **Megjegyzések**

- A 16 bites port azonosítók miatt sok hoszt lehet egy cím mögött
- Nem feltétlenül kell a portot is fordítani

- **Ellentmondások**

- A router a hálózati rétegben dolgozik, nem kellene a portokhoz nyúlnia
- **Sérül a végponttól-végpontig (end-to-end) küldött datagram elve**
 - Figyelembe kell venni egy alkalmazás fejlesztésekor is

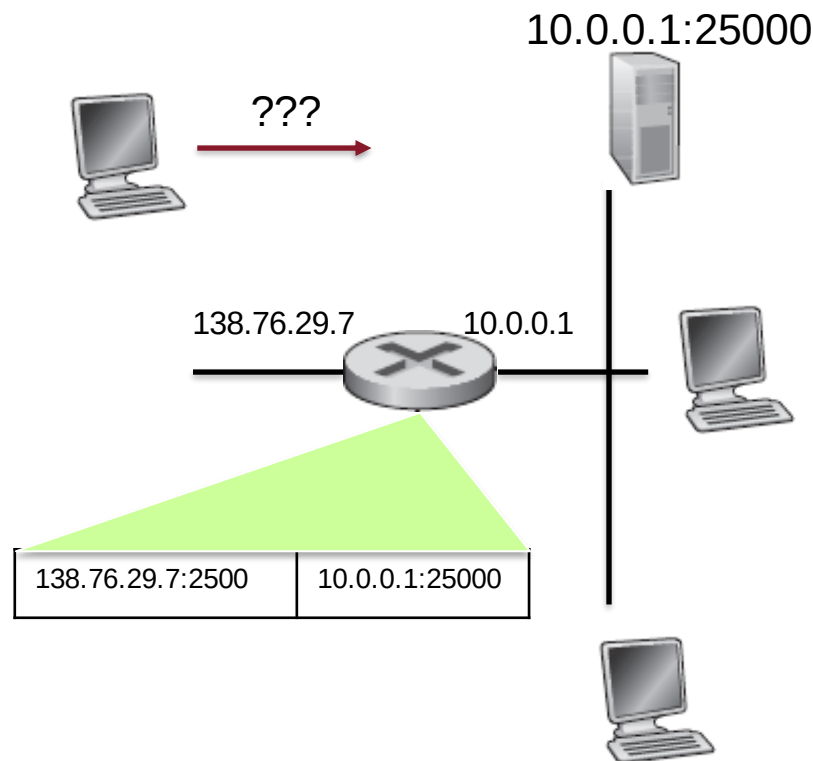
- **NAT traversal** Internet -> 200.1.1.5 -> Router (NAT) -> ???
Melyik belső gépnek szól? -> nem tudja!

• Probléma

- A kliens a 10.0.0.1 (valódi) címmel rendelkező szerverhez akar kapcsolódni
- Ez a cím privát cím, amit tipikusan nem lehet elérni az Internet felől (csak lokálisan alkalmazzuk)
- A kifelé látszó egyetlen cím a 138.76.29.7

• 1. megoldás: **statikus NAT**

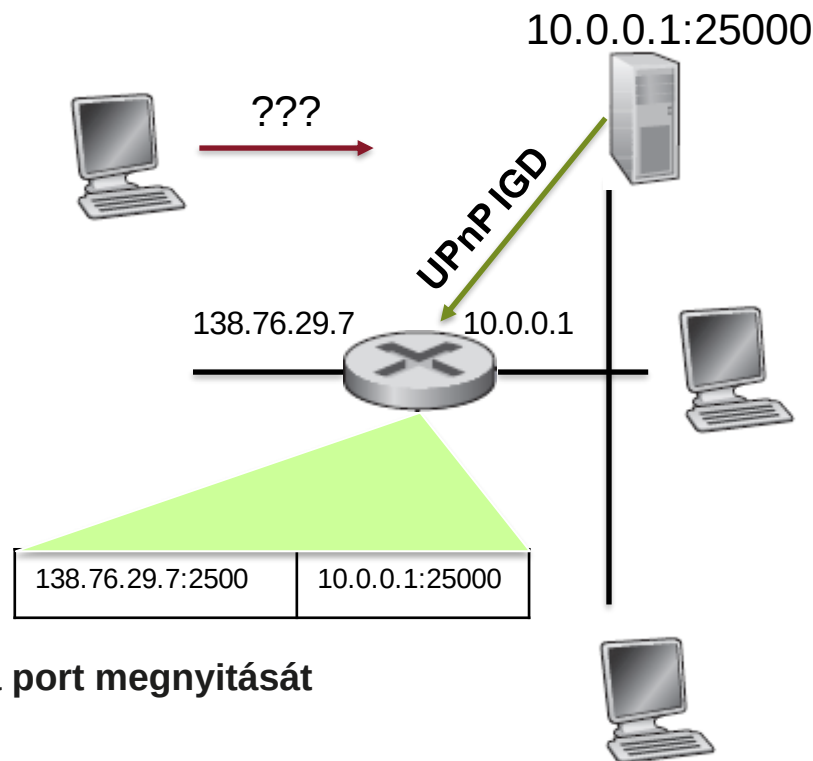
- A szerver valódi port-cím párját egy kívülről látható port-cím párra fordítjuk
- Például a 138.76.29.7: 2500-ra érkező datagramok a 10.0.0.1:25000 felé mennek tovább



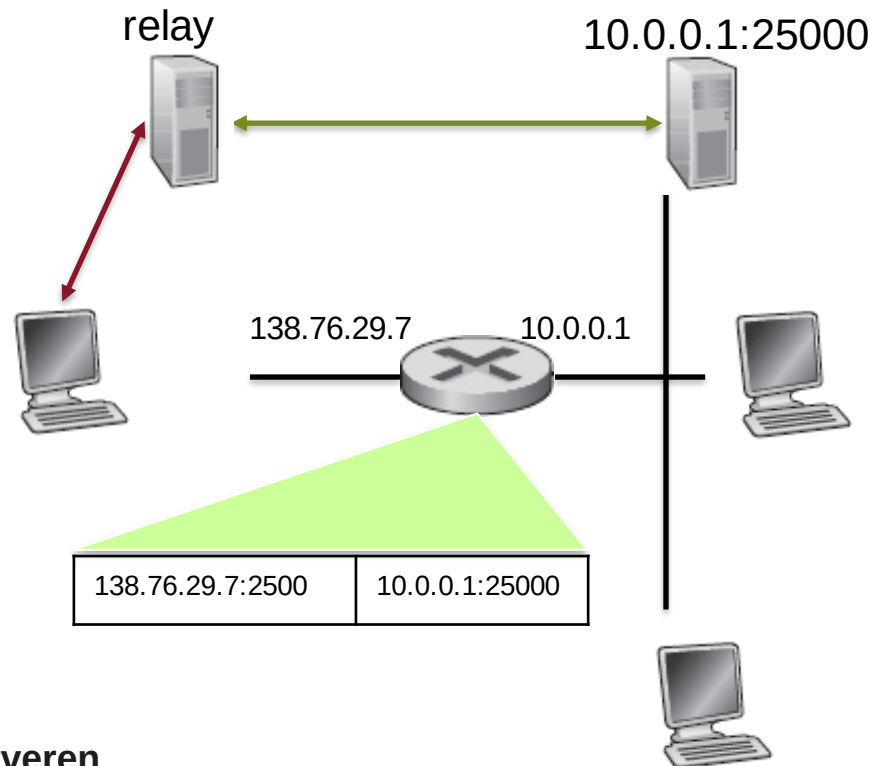
-> Port forwarding
pl: otthoni router

- 2. megoldás: statikus NAT – automatikus konfigurációval
 - A címfordító táblát beállíthatná a szerver is
 - Az Universal Plug and Play (UPnP) Internet Gateway Device (IGD) Protocol a NAT mögötti hosztoknak lehetővé teszi
 - A külső cím megismerését
 - A fordítási tábla megismerését
 - A fordítási tábla módosítását

-> alkalmazás automatikán kéri a routertől a port megnyitását
pl: online játékok, torrent



- 3. megoldás: köztes szerver használata – **relay**
 - A NAT mögött lévő szerver kapcsolódik a köztes szerverhez (relay)
 - A kliens a köztes szerveret szólítja meg
 - A relay ismeri a valódi szerver nyilvános címét és portját
 - A relay továbbítja a kliens csomagjait a valódi szerver felé

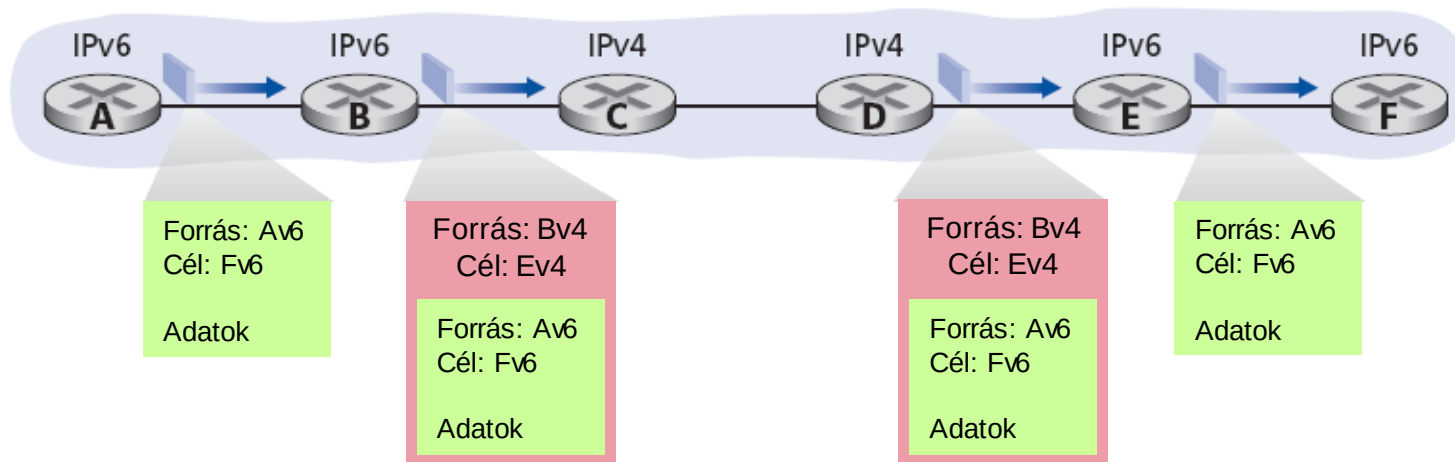


-> relay = közvetítő szerver
Minden adat átmegy egy harmadik szerveren
pl: ez lehet DHCP relay példája is!

- Large Scale NAT (LSN) megoldások
 - NAT az ISP hálózaton belül is
 - Így már két helyen is átalakítják az eredeti címet
- A NAT csak ideiglenesen megoldás az IPv4 kimerülésre, végső megoldás az IPv6
- Két külön világ, ami jelenleg párhuzamosan működik
- Sok routerben és hosztban mindkettő elérhető (dual stack)
- Hogyan lehet átjárni közöttük?
 - NAT64: IPv6 cím fordítása IPv4 címre (és vissza)
 - Alagutazás (tunneling): átbújtatás az IPv4 hálózatrészen



- Megvalósítás

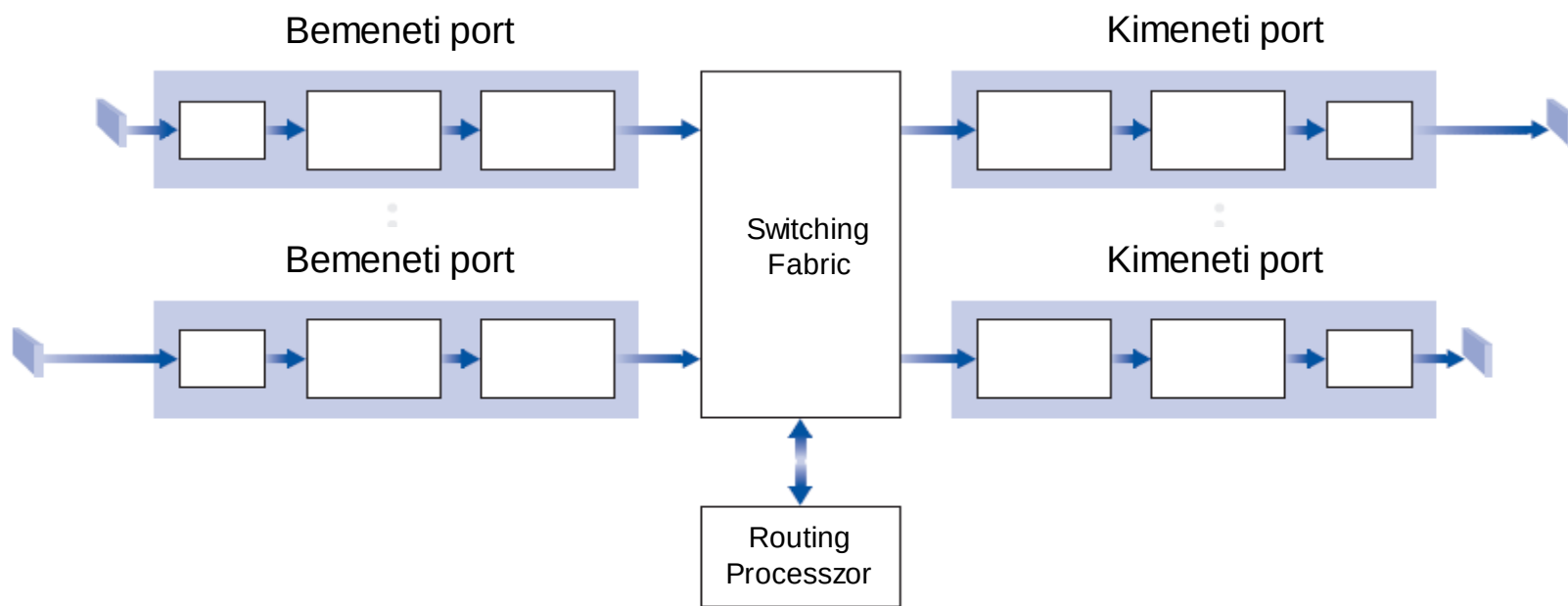


- Az IPv6 datagramot beletesszük egy IPv4 datagramba
- Átküldjük az alagúton
- A túloldalon kivesszük az IPv4-ből és továbbítjuk IPv6 datagramként

1. CIDR
2. Címkiosztás, DHCP
3. Címfordítás
4. Datagram továbbítás

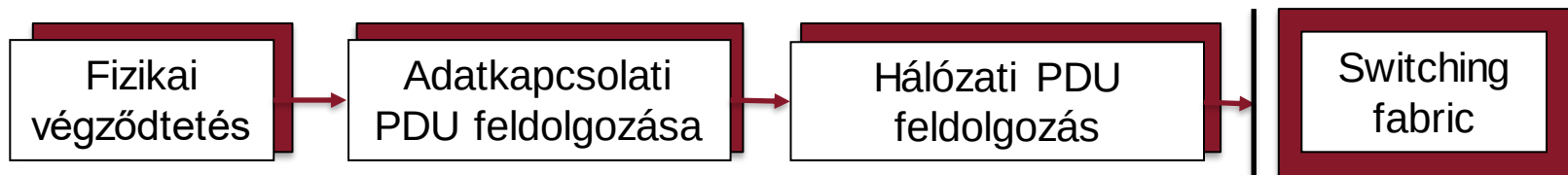
- **Két fő funkció**

- Útmeghatározó protokollok és algoritmusok futtatása
- Az egyik bemeneten beérkező csomag továbbítása egy kimenetre

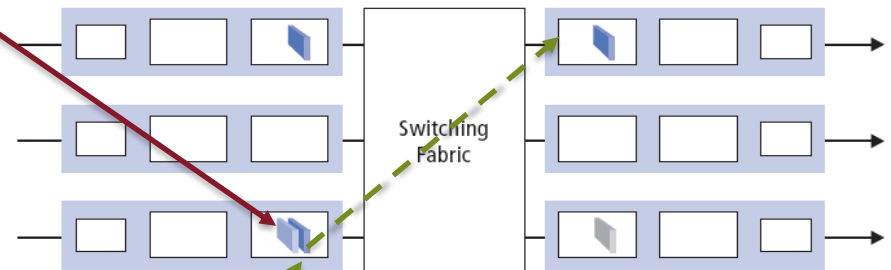
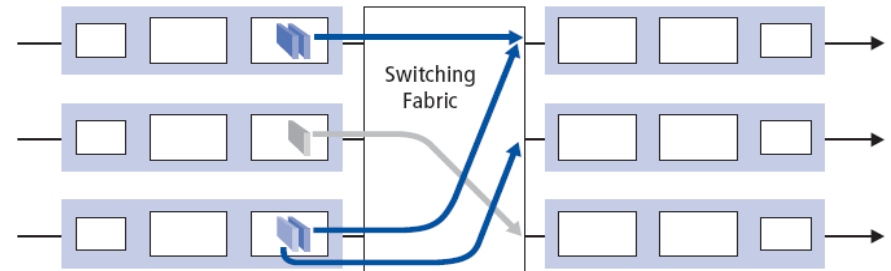


- Hogyan kerül a routerhez a datagram?
 - A hoszt a cím és a maszk alapján eldönti, hogy a cél vele egy hálózaton van-e
 - Ha igen, szomszédos hosztnak küld
 - Ha nem, akkor egy átjárón át az Internet felé
 - Alapértelmezett átjáró – **default gateway**
 - A hosztnál meg kell adni, hogy lehessen indítani a datagramot ebbe az irányba
 - Az első router hoszt felé eső interfésze
 - Egy hálózatban van a hoszttal
- A router a célcím alapján küldi tovább
 - Közvetlenül továbbítható egy hozzákapcsolt hálózatba
 - Távoli hálózat esetén az irányítási tábla bejegyzései alapján

- Fizikai réteg
 - Vezeték végződtes
 - Bitszintű vétel
- Adatkapcsolati réteg (pl. Ethernet)
 - Szomszédos hálózatelemtől kapott PDU
- Hálózati réteg
 - datagram fejléc feldolgozás, NAT
 - Továbbítási irány meghatározás az irányítási (routing) tábla alapján
 - Sorba állítás
- Sebesség cél
 - A feldolgozás sebessége legalább az adatok érkezési sebessége legyen
- Sorbanállás, ha a kapcsológép (switching fabric) nem elég gyors

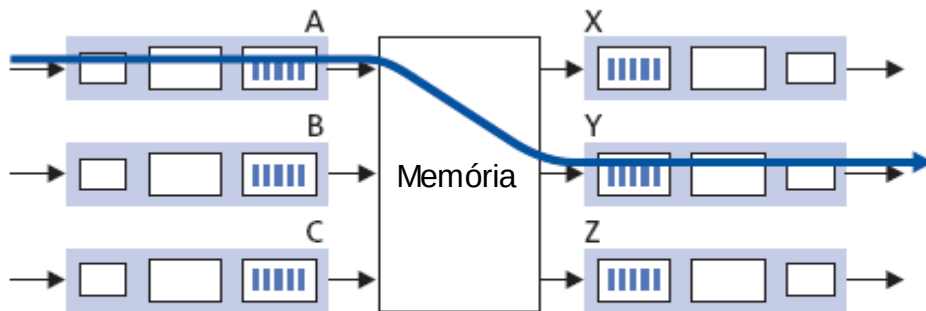


- Sorbanállás, pufferekés, ha a kapcsológép nem elég gyors az összes bemenethez képest
 - Késleltetés
 - Csomagvesztés
- Head Of Line (HOL) blokkolás
 - Lehetne továbbítani a datagramot, de feltartja egy másik
 - Ez a másik a kimenet foglaltsága miatt a kapcsológépre vár

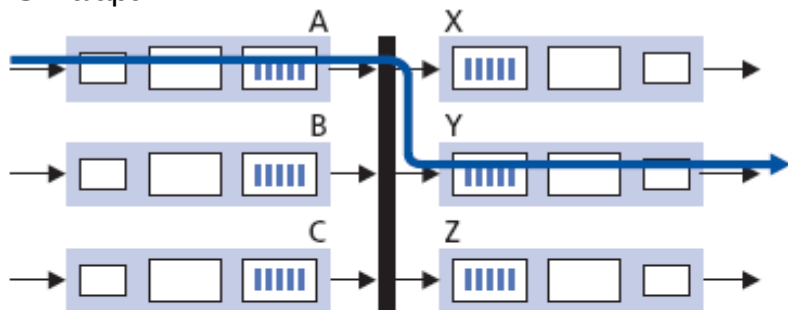


KAPCSOLÓGÉP ELRENDEZÉSEK

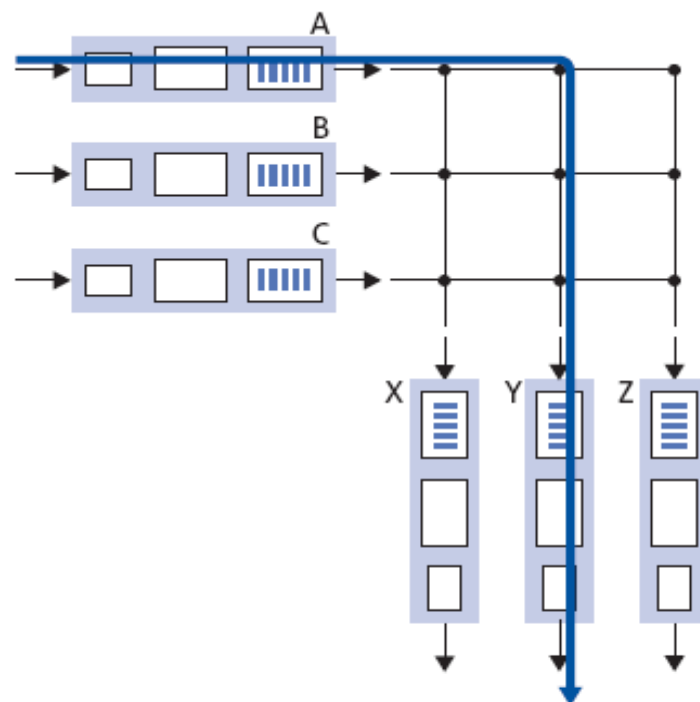
Memória-alapú



Sín-alapú



Keresztkapcsoló

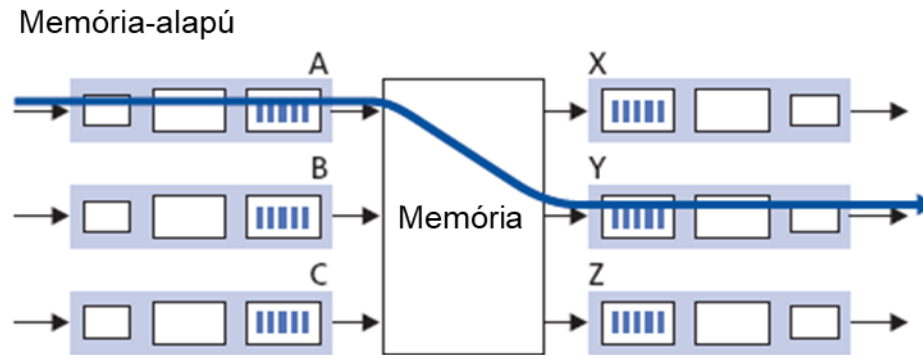


Bemeneti
port

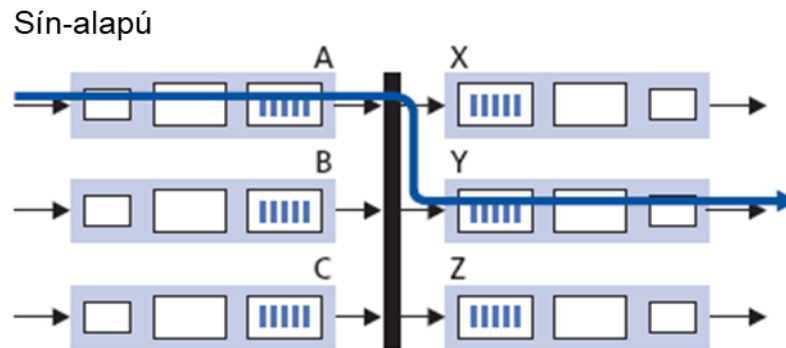


Kimeneti
port

- Első generációs routerekben
- Hagyományos számítógépek a CPU felügyelete alatti kapcsolással
- Csomagok beírása a memóriába
- A kapcsolási sebességet a memóriaelérés sávszélessége határozza meg
 - datagramonként két memóriaelérés

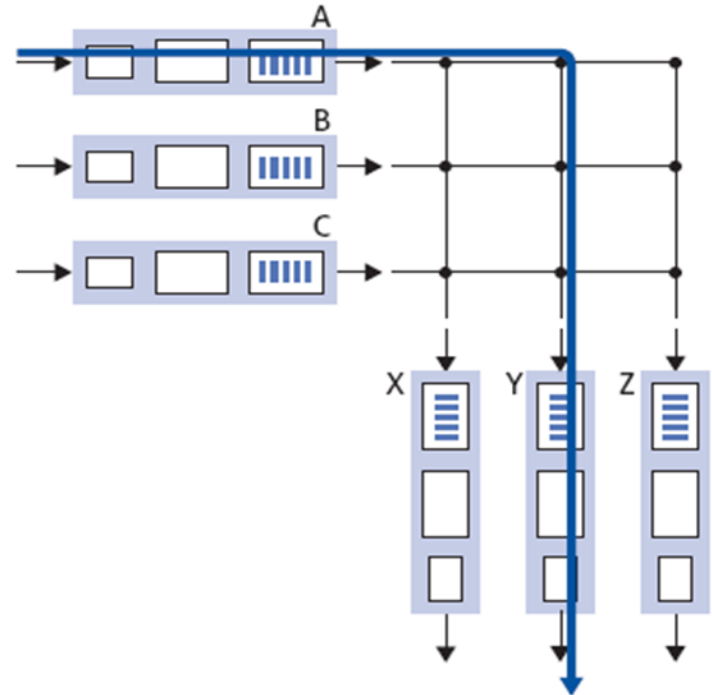


- A bemeneti port memóriájából a kimeneti port memóriájába egy közös sínen (bus) jut el a datagram
- A kapcsolási sebességet a sín sávszélessége korlátozza
- A teljesen osztott erőforrás szűkös lehet

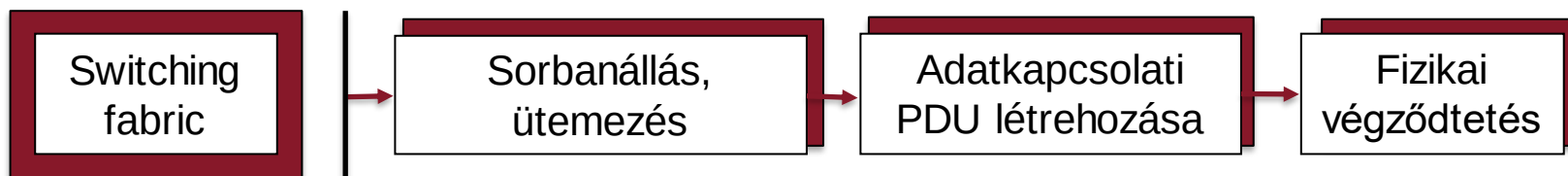


KAPCSOLÁS KERESZTKAPCSOLÓ HÁLÓZATON

- Crossbar kapcsolás
- A cél átlépni a sínek sávszélességéből adódó korlátokat
- Minden bemenet közvetlenül köthető minden kimenetre
 - Banyan hálózatok
 - Eredetileg többprocesszoros rendszerek processzorainak összekapcsolására
- Gyorsítás
 - a datagramok fix méretű cellákra tördelése
 - a cellák kapcsolása a kapcsolóban
- Akár több száz Gbps



- Sorbanállás, pufferekés, ha a kimenő link nem elég gyors
 - Késleltetés
 - Csomagvesztés
- Ütemezés
 - A következőként elküldendő datagram kiválasztása
- Adatkapcsolati réteg (pl. Ethernet)
 - Szomszédos hálózatelemnek küldendő PDU
- Fizikai réteg
 - Vezeték végződés
 - Bitszintű adás





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Útvonalválasztás
2023. április 21.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



1. Irányítási tábla
2. ICMP
3. Statikus irányítás
4. Dinamikus irányítás

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat

- Címzés, továbbítás, útvonalválasztás
- Irányítási, továbbítási szabályok listája
 - Lokálisan érvényes
 - Célszerű globális ismeret alapján (is) kitölteni
- A szabályok bejegyzések
 - (A)hálózatokra vonatkoznak
 - A CIDR elveit figyelembe véve, a maszkot is tartalmazzák
 - A következő lépést mutatják
 - Kimenő interfészt megadva
 - Hálózatot megadva – rekurzívan („Arra van, mint a ...”)
 - Időről időre változhatnak
 - Átfedhetnek

Cél	Irány
10.9.42.64 / 26	GigabitEthernet 0/3
192.168.3.0 / 24	GigabitEthernet 0/1
10.9.32.0 / 19	192.168.3.1

TOVÁBBÍTÁS A TÁBLA ALAPJÁN

- **Több bejegyzésre is illeszkedő cím**
 - Gyakori
 - **A leghosszabban illeszkedőt használjuk – Longest Prefix Matching**

Átfedésben lévő alhálózatok esetén!
Leghosszabban passzoló szabály!
 - Prefix – a maszkolással adódó hálózati cím
 - Prefix hossz – maszk hossza
- **A CIDR miatt van lehetőség címek összefogására**
 - Nagyobb tartományok egy bejegyzésként
 - Kivételek külön bejegyzésként
- **Nulla hosszú maszk**
 - **Mindenre illeszkedő tartomány**
 - Egy bit sem számít a címből
 - **0.0.0.0 / 0**

-> BÁRHONNAN jött mehet ARRÁ!
Minden csomagot továbbít!
 - **Alapértelmezett út, default route**
 - Bármilyen másik illeszkedés hosszabb prefixet jelent

- Példa irányítási tábla

Cél	Irány
10.9.42.64 / 26	GigabitEthernet 0/3
192.168.3.0 / 24	GigabitEthernet 0/1
10.9.32.0 / 19	192.168.3.1

- Továbbítandó datagram célcíme

Subnet (/26 vagy /19) számít!!!
Melyik a pontosabb

- 192.168.3.78

A 24 bites maszk miatt illeszkedik a 2. bejegyzésre – Gi0/1

- 10.9.55.74

0000 1010.0000 1001.0011 0111.0100 1100

a 3. bejegyzésre illeszkedik – Gi0/1

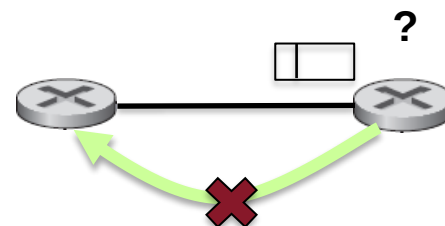
- 10.9.42.74

0000 1010.0000 1001.0010 1010.0100 1100

az 1. bejegyzésre illeszkedik
a 3. bejegyzésre illeszkedik

PROBLÉMA A TOVÁBBÍTÁSNÁL

- **Nincs illeszkedő bejegyzés**
 - Küldjük vissza?
 - **Dobjuk el!**
 - Jelezzünk vissza
- **A visszaküldés nagyon nem jó ötlet**
 - Lokális és egyszerű döntés a továbbítás
 - Nem függ attól, hogy merről jött a datagram
 - Hurkot alakítanánk ki
 - Az előző router újra csak nekünk küldené
- **Hurok** -> megoldás az STP (Spanning Tree Protocol), amely feszítőfát képez a hálózatból!
 - **Kerülendő**
 - Gondos konfigurációval elkerülhető
 - Feleslegesen keringő datagramok, felesleges terhelés
 - A lokális és egyszerű döntések miatt a tábla alapján nem ismerhető fel
 - Kezelése a datagram TTL mezőjének figyelésével



1. Irányítási tábla
2. ICMP
3. Statikus irányítás
4. Dinamikus irányítás

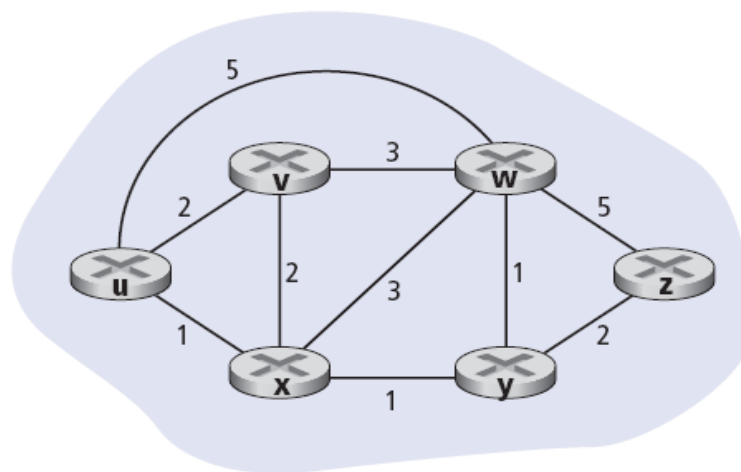
- **Hogyan jelezzünk vissza?**
 - A hálózati réteghez tartozó vezérlési protokollal
 - **Internet Control Message Protocol**
- **A hosztok és routerek használják hálózati információk terjesztésére**
 - **Hibajelzések**: elérhetetlen hálózat, hoszt, port, protokoll
 - **Echo-kérés és -válasz**: a ping ezen alapul
- IP datagramokba csomagolt üzenetek
- ICMP üzenet: típus, kód, és az üzenetet kiváltó IP datagram fejléce és első 8 adatbájta

Type	Code	Leírás
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

- Feladat
 - Összegyűjteni a cél felé útba eső routerek listáját
 - Közben információt gyűjteni a routerek időbeli távolságáról (RTT mérése)
- Megoldás
 - A traceroute program egy UDP csomagot küld a cél címére, de a **TTL-mezőt 1-re** állítja
 - Az első routerben a TTL mező 0-ra csökken
 - el kell dobni a csomagot
 - hibaüzenetet kell visszaküldeni: **ICMP Time-Exceeded**
 - Az adatok feldolgozása után ugyanezt kell folytatni a **TTL mezőt 2-re** állítva, és így tovább...
 - Végül egy ICMP Port-Unreachable üzenetet kell kapnunk a célhoszttól
 - Olyan UDP portot használ, ami valószínűleg nincs nyitva
- Problémák
 - Már volt róla szó: dinamikus útvonalak, becsapós RTT-k
 - Az ICMP üzenetben lévő cím a routeren a bejövő interfész címe

1. Irányítási tábla
2. ICMP
3. Statikus irányítás
4. Dinamikus irányítás

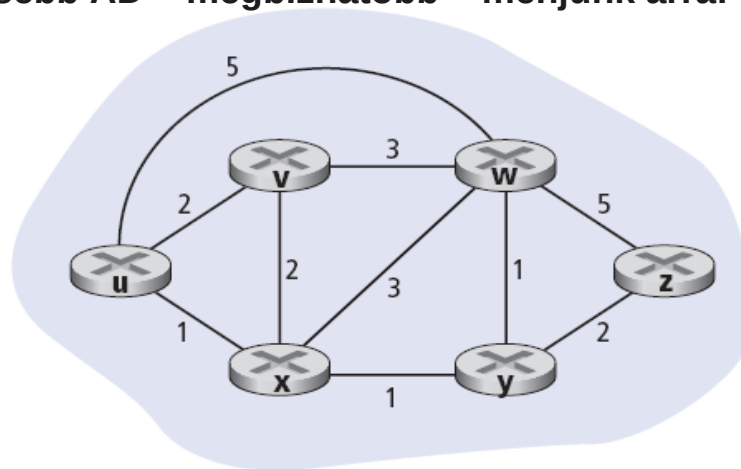
- Tipikus interpretáció
- Egyszerűsített modell
- Gráf: $G = (N, E)$
 - $N =$ **csomópontok** (routerek)
halmaza =
 $\{ u, v, w, x, y, z \}$
 - $E =$ **élek** (linkek, routereket összekötő alhálózatok) halmaza =
 $\{ (u,v), (u,x), (u,w), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$
- Megjegyzések:
 - Irányított élek lehetnek
 - Két csomópontot több él is összeköthet, de most feltesszük, hogy nincs ilyen
 - A gráfokkal más rétegeket is leírhatunk



- $c(a,b)$ = az a és b csomópontokat összekötő él (link) költsége, metrikája
 - Például $c(w,z) = 5$
- A költség itt az útvonalválasztást befolyásolja, például:
 - Minden él költsége 1
 - A link sávszélességétől függ
 - A link aktuális terhelésétől függ
- Jellemzően additív mennyiség
 - Az $(a_1, a_2, a_3, \dots, a_p)$ út teljes költsége = $c(a_1, a_2) + c(a_2, a_3) + \dots + c(a_{p-1}, a_p)$
- Routing – legkisebb költségű út kiválasztása

|
> AD (Administrative Distance)
-> Több routing protokoll is mond egy útvonalat ugyanarra a célra. Melyik a jobb?

Kisebb AD = megbízhatóbb = menjünk arra!



AD értékek:
Static route = 0
OSPF = 110
RIP = 120

- Alapvető kérdés: hány végponthoz kell eljuttatni egy adott datagramot, hány cél van?
 - Egy – unicast
 - Több – multicast
 - Összes – broadcast
 - Több alternatíva – anycast

-> Legfőképpen IPv4-nél használjuk

-> Legfőképpen IPv6-nál használjuk. Legközelebbi interfész
- Egyetlen útvonal van (pl. unicast esetben)?
 - Mindig ugyanarra küldünk az adott cél felé
 - Nem egyenletesen osztjuk szét a forgalmat az erőforrásokon
 - Ha vannak alternatívák, pl. azonos költséggel, használjuk őket

- Globális vagy elosztott információra támaszkodik?

- Globális információ

- Minden router ismeri minden részletét a gráfnak (csomópontok, élek, költségek)
 - Pl. kézi tervezés
 - Pl. link-állapot alapú (OSPF) protokollok

- Elosztott információ

- Minden egyes router csak a közvetlen környezetét ismeri
 - A szomszédos routerek kicserélik az ismereteiket és az alapján számolgatnak
 - Pl. távolság-vektor alapú (RIP) protokollok

- Statikus vagy dinamikus?

- Statikus

- A hálózat állandóságát feltételezi
 - A routerekben ritkán frissítik a bejegyzéseket
 - Elég egyszer kiszámolni
 - Kézi konfiguráció

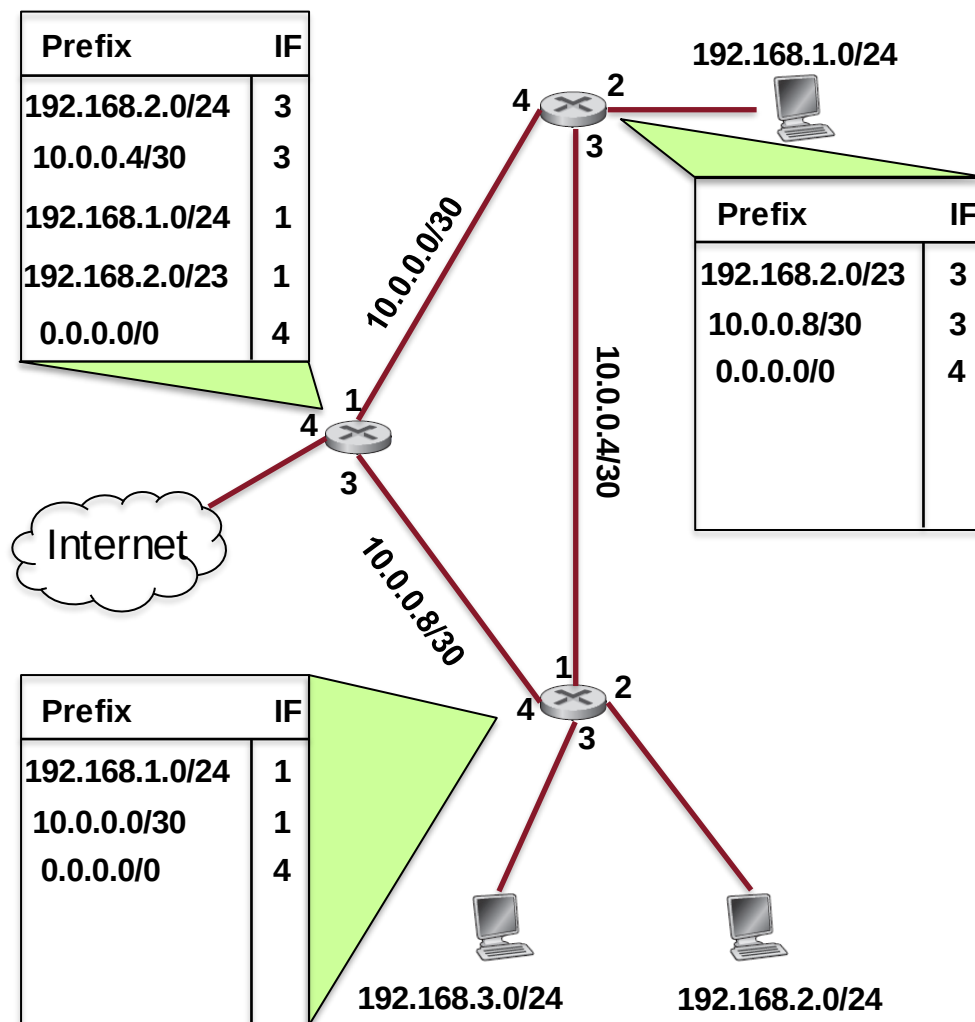
- Dinamikus

- A hálózat változásait követi
 - A routerek rendszeresen frissítik a bejegyzéseket
 - Rendszeresen újra kell futtatni az algoritmusokat
 - Automatizált konfiguráció

- Mikor melyik jobb?

- A teljes hálózat ismeretében számítjuk ki
- Kézzel bevitt bejegyzések
 - Statikus út – static route
- Hátrányok
 - Rugalmatlan
 - Sok bejegyzés esetén nehéz elvégezni és karbantartani
- Előnyök
 - Nincs routerközi kommunikáció
 - Nem terheli a hálózatot
 - Biztonságosabb
 - Nem terheli a routert
 - Nem függ a többi routertől
- Jellegzetes alkalmazás
 - Kisebb hálózatrészekben
 - A dinamikus megoldás kiegészítéseként
 - Egyetlen linkkel bekötött hálózatrészek esetén
 - Alapértelmezett esetekre
- Mikor hat?
 - Ha a Longest Prefix Matching után választani kell, akkor a statikus utat választja
 - Több statikus út is megadható egy hálózathoz (prefixhez)
 - Különböző költség
 - Tartalék útvonal

- Példa
 - Három router teljesen összekötve
 - Egyetlen kijárat az Internet felé
 - A címtervezésen már túl vagyunk
 - Nem számítunk bővítésre
- Statikus bejegyzések
 - Statikus út
 - Tartalék út (AD érték)
 - Alapértelmezett út

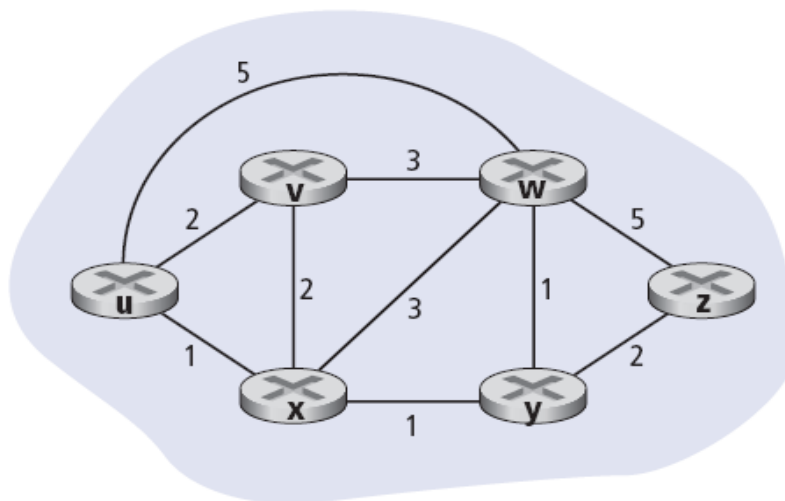


1. Irányítási tábla
2. ICMP
3. Statikus irányítás
4. Dinamikus irányítás

- **A teljes hálózati gráfot ismerjük**
 - Minden router információkat küld a többieknek a hozzá csatlakozó alhálózatok (linkek és LANok) állapotáról
 - Mindenütt ugyanaz a kép – nagyon fontos
 - **Linkállapot (link-state) alapú**
- Optimalizált, vagy heurisztikus döntés
- **Leggyakoribb megoldás: Dijkstra algoritmus**
 - A forrás csomópontból a többi csomópont felé menő legrövidebb utak meghatározására
- A megoldás alapötlete
 - **Iteratívan bővítjük a már megoldott csomópontok halmazát**
 - A halmazzal szomszédosak közül mindig legközelebbit vesszük hozzá a halmazhoz
- Jelölések
 - Az eddigi jelöléseket kiegészítve
 - A $c(a,b)$ költséget ∞ -re állítjuk, ha nincs él a és b között
 - N' : a már megoldott csomópontok halmaza
 - $D(v)$: az eddig ismert legrövidebb távolság a forrás és v között az N' halmazon át, kezdetben ∞ is lehet
 - $p(v)$: a forrásból v -be tartó eddig ismert legrövidebb úton a v -t megelőző csomópont

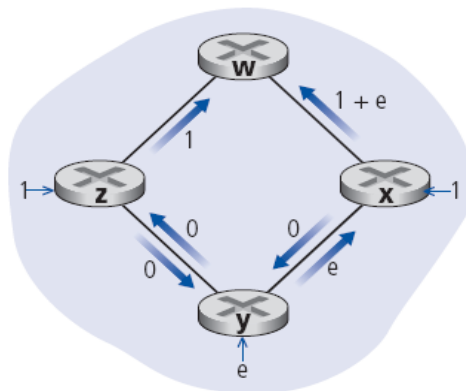
A DIJKSTRA ALGORITMUS ILLUSZTRÁCIÓJA

Iteráció	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	<i>u</i>	2, <i>u</i>	5, <i>u</i>	1, <i>u</i>	∞ , -	∞ , -
1	<i>ux</i>	2, <i>u</i>	4, <i>x</i>	1, <i>u</i>	2, <i>x</i>	∞ , -
2	<i>uxy</i>	2, <i>u</i>	3, <i>y</i>	1, <i>u</i>	2, <i>x</i>	4, <i>y</i>
3	<i>uxyv</i>	2, <i>u</i>	3, <i>y</i>	1, <i>u</i>	2, <i>x</i>	4, <i>y</i>
4	<i>uxyvw</i>	2, <i>u</i>	3, <i>y</i>	1, <i>u</i>	2, <i>x</i>	4, <i>y</i>
5	<i>uxyvwz</i>	2, <i>u</i>	3, <i>y</i>	1, <i>u</i>	2, <i>x</i>	4, <i>y</i>

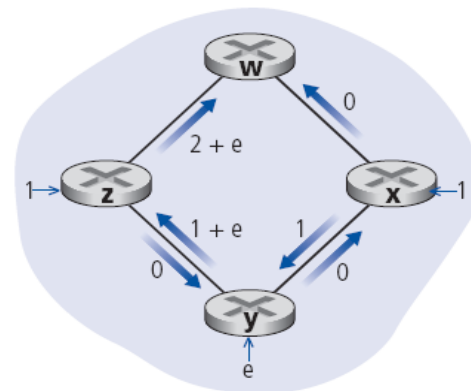


- Az algoritmus futási ideje
 - **Számítási komplexitás** n csomópont esetén
 - Minden iterációban meg kell vizsgálni az N' halmazon kívüli elemeket
 - Összesen $n(n+1)/2$ lépés, $O(n^2)$ komplexitás
 - Gyorsítható az adatok trükkös tárolásával: $O(n \log n)$
- A csomópontok ugyanazt futtatják, csak más forrással
- Mikor kell újra futtatni?
 - Minden költségváltozást le kell követni **újraszámítással**
 - Nem mindig van szükség az összes út (a teljes fa) újraszámítására

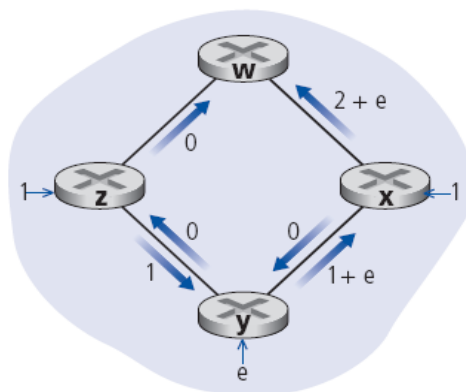
- Bizonyos esetekben gyakori újraszámítás kellhet
 - Az útvonalak ide-oda váltogatnak
 - Például terheléstől függő metrika esetén
- További zavarok okai
 - Az egyes csomópontok függetlenül (nem szinkronizálva) számolnak
 - A linkállapot információs csomag nem mindenkihez jut el
 - Egy link két irányában eltérő metrika



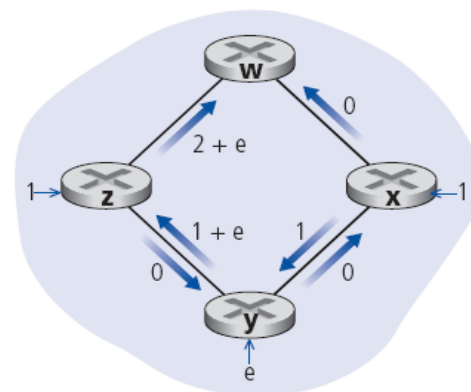
1. Kezdeti routing



2. x és y átáll a w-be z-n keresztül vezető útvonalakra



3. x, y és z átáll a w-be x-n keresztül vezető útvonalakra

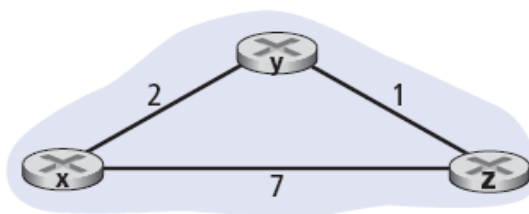
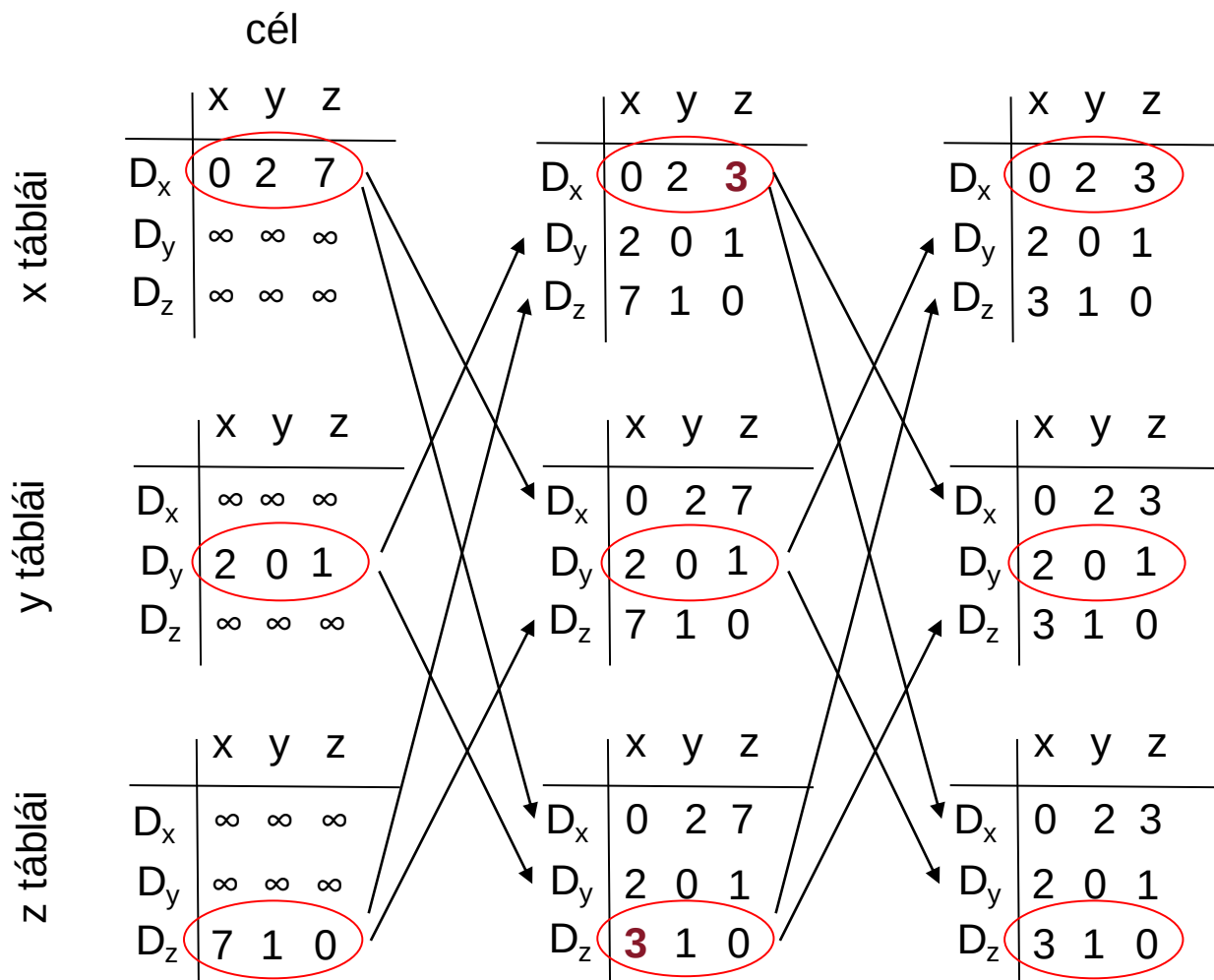


4. x, y és z átáll a w-be z-n keresztül vezető útvonalakra

- A hálózati gráfnak csak egy részét ismerik a csomópontok
 - Minden router csak a szomszédjainak küld információt arról, hogy tőle milyen messze vannak az alhálózatok
 - Sehol nincs teljes hálózati kép
 - **Távolságvektor (distance-vector) alapú**
- Optimalizált döntés, de elég hiányos információ alapján
- Megoldás: **Bellmann-Ford algoritmus**
 - A forrás csomópontból a többi csomópont felé menő legrövidebb utak meghatározására
- Jelölések
 - Az eddigi jelöléseket kiegészítve
 - A $d_a(b)$ távolság az a -ban ismert legkisebb távolság b -hez
- A megoldás alapötlete
 - Rendszeresen frissítjük a távolságokat (B-F egyenlőség)
$$d_a(b) = \min_m \{c(a, m) + d_m(b)\}$$
 - A minimum képzésben a -nak az összes m szomszédját figyelembe vesszük
- A megoldás részletei
 - Egy csomópont tárolja a saját távolságvektorát
$$\mathbf{D}_a = [d_a(b) : b \in N]$$
 - Feldolgozza a szomszédoktól kapott távolságvektorokat
$$\mathbf{D}_m = [d_m(b) : b \in N]$$

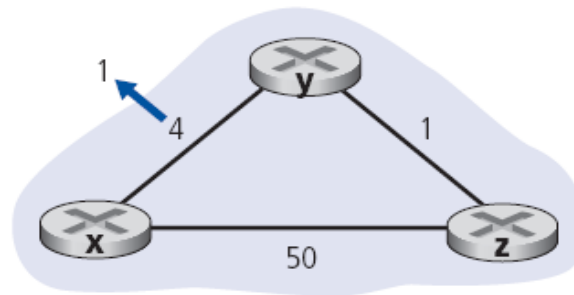
- Egy csomópont szétküldi a szomszédainak a távolságvektorát
 - Bekapcsolás után (csak a hozzákapcsolt LAN-okat tartalmazza)
 - Ha változik a vektor
- Egy csomópont frissíti a távolságvektorát
 - Amikor egy szomszédjától új vektort kap
 - Ha változnak a hozzákapcsolt LAN-ok költségei
 - Javíthatók a távolságok a B-F egyenlőség alapján
- A csomópont eltárolja minimális távolsághoz tartozó irányt
- Aszinkron számítás
- Iteratív számítás
- Elosztott, aggregált információk alapján
- Ideális esetben konvergál a minimális költségű irányokhoz

A TÁVOLSÁGVEKTOR ALAPÚ ALGORITMUS ILLUSZTRÁCIÓJA



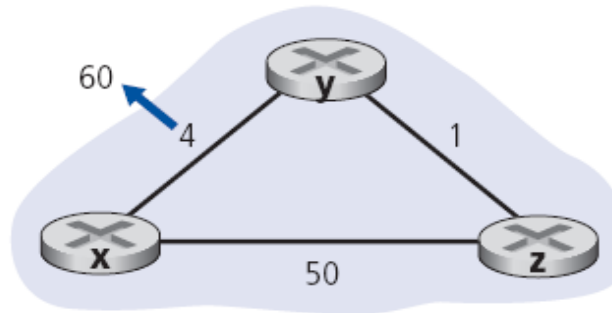
Iterációk

- A csomópont érzékeli, hogy egy hozzákapcsolt link költsége csökkent
- Újraszámolja a távolság vektorát
- Ha volt változás, szétküldi a szomszédoknak
 - A szomszédoknál is csak csökkenhetnek a távolságok (egyetlen iteráció alatt beáll az új érték)
 - Az összes csomópontnál csak csökkenhetnek a távolságok



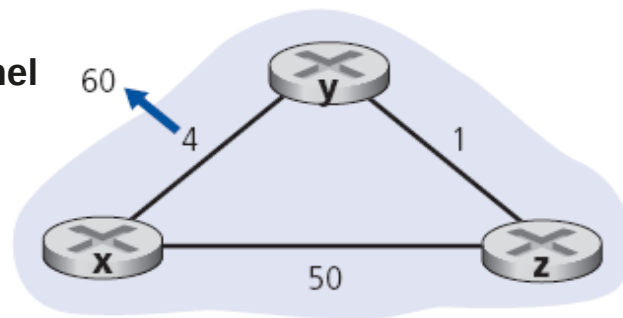
- A jó hír gyorsan terjed

- A csomópont érzékeli, hogy egy hozzákapcsolt link költsége nőtt
- Újraszámolja a távolság vektorát
- Ha volt változás, szétküldi a szomszédoknak
 - A szomszédoknál a korábbi információk alapján lehetnek rövidebbnek tűnő utak másfelé, és ezt hirdetik is
 - Sok iteráció után derül ki, hogy tévedés volt, és addig **hurkok** is lehetnek



- A rossz hír lassan terjed
 - **Végtelenig számolás** (count-to-infinity)

- Az volt a baj, hogy
 - A z azt hirdette y felé, hogy tud egy 5 hosszú utat x felé
 - Azután y hirdette z felé, hogy ő egy 6 hosszút tud
 - És így tovább
- Ne hirdessük a célt abba az irányba, amerre irányítjuk a forgalmat
 - **Split horizon** elv -> nem hirdet vissza!
- Még jobb, ha kifejezetten jelezzük is ezt
 - Végtelen értéket hirdetünk
 - **Poisoned reverse** -> visszahirdet végtelennel
- Nagyobb hálózatokban ez sem mindig elég



- Üzenetek száma
 - LS: n darab csomópont, E darab él: sok, $O(nE)$ üzenet
 - DV: nagyobb üzenetek, de csak a szomszédok felé
- Konvergálás sebessége
 - **LS: gyors**
 - Linkállapotok szétküldésének ideje
 - Oszcillálhat
 - **DV: változó**
 - Ideiglenes hurkok kialakulhatnak
 - Végtelenig számolás problémája
- Robusztusság (hibatűrés): Mit eredményez, ha meghibásodik egy router?
 - LS
 - A csomópont hamis adatokat küld a kapcsolódó linkekről
 - Az útvonalak nem biztos, hogy optimálisak maradnak
 - DV
 - A router hibás távolságvektort küld (az egész útról rossz lesz az információ)
 - Az információk elterjednek az egész hálózatban, mindenki hamis távolságokat számol és hirdet
 - Katasztrofális hatású lehet, különösen, ha kis távolságokat hirdet



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Routing protokollok
2023. április 24.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



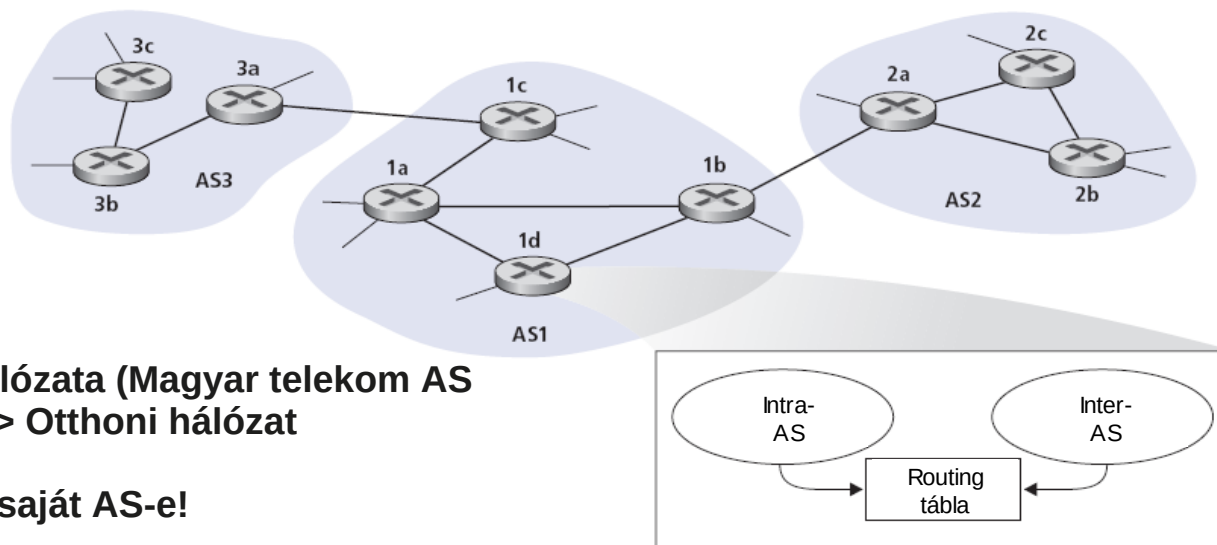
1. Hierarchikus routing
2. RIP
3. OSPF
4. BGP
5. Broadcast, Multicast

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat

- Eddigi feltételezésünk az Internet magjáról
 - Minden router egyenlő
 - A hálózat lapos (flat), nincs hierarchia
 - Nem így van
- Skálázhatósági (nagyságrendi) probléma
 - Több százmillió hálózat
 - Egyenként tárolni a routing táblában lehetetlen
 - Az összevonva tárolást segíti a CIDR
 - Minden hálózatot hirdetni óriási terhelés lenne
- Adminisztrációs (hálózatigazgatási, üzemeltetési) probléma
 - Minden szervezetnek más lehet az elképzelése és a szabályzata
 - Mindenki lássa mindenkiét?

- A routereket csoportokba, ún. **autonóm rendszerekbe (autonomous system, AS)** szervezzük
 - Legtöbbször az üzemeltető szervezeteknek megfelelően
 - Gyakran egy szervezet – egy AS
 - Néha egy szervezet – több AS (pl. nagyobb ISP-k)
- **Gateway-Router**
 - Az AS olyan routere, amely össze van kötve egy másik AS routerével
- Az egy AS-ben lévő routerek egy routing protokollt futtatnak
 - **Interior Gateway Protocol (IGP) – “Intra-AS” protokoll**
 - A különböző AS-ekben akár eltérő IGP futhat
- Az AS-ek között is szükség van routingra
 - **Exterior Gateway Protocol (EGP) – „Inter-AS” protokoll**

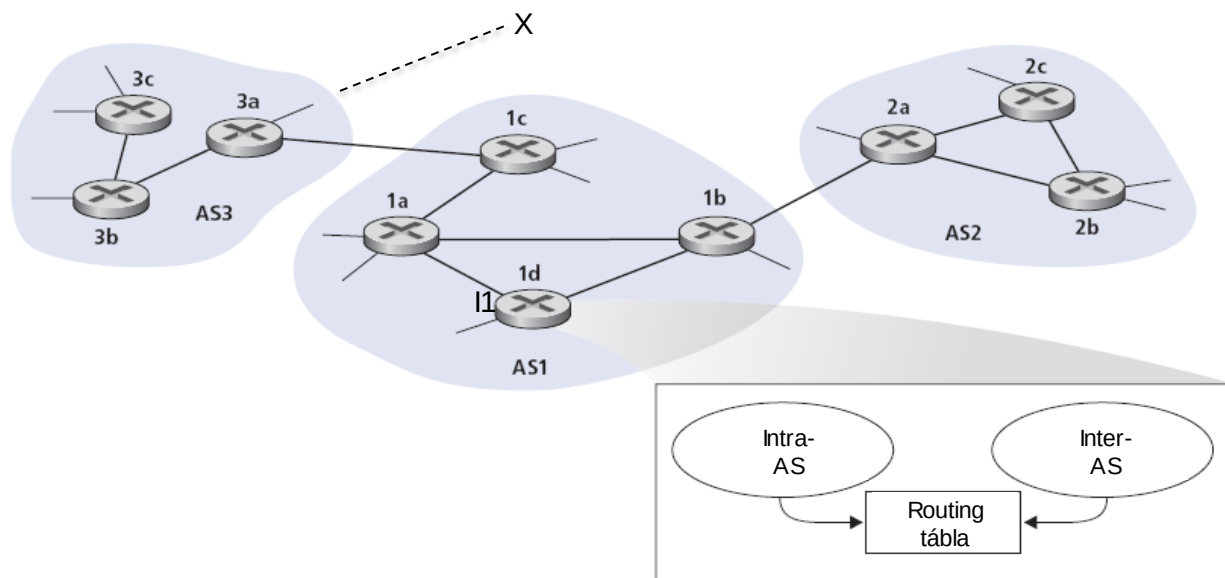
- A routing táblák kitöltése
 - Az adott AS-en belüli célok: Intra-AS routing alapján
 - Az adott AS-en kívüli célok: Inter-AS routing alapján
- Inter-AS routing feladata
 - Külső cím felé menő csomagot melyik gateway felé küldje egy AS1-beli router?
 - Fel kell térképezni az AS-ek hálózatát
 - Az információkat el kell juttatni AS-1 routereihez



Internet -> ISP hálózata (Magyar telekom AS)
-> Router (NAT) -> Otthoni hálózat

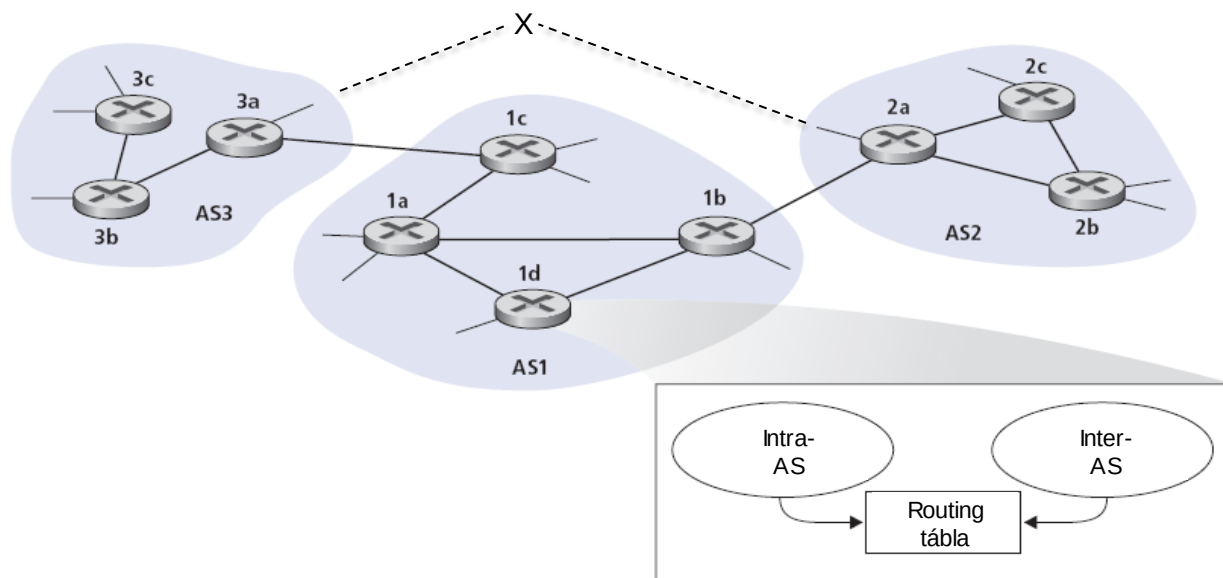
PI: BME-nek van saját AS-e!

- Az AS1 megtanulja Inter-AS routinggal, hogy az X hálózat AS3-felé érhető el
 - A megfelelő gateway az 1c
 - AS2 felé nem
- Az információt elterjeszti AS1 routerei között
 - Az Intra-AS routing segítségével
- Az 1d router bejegyzi a routing táblájába
 - Az 1c router az I1 interfész felé van, tehát az X is az I1 interfész felé van



INTER-AS ALTERNATÍV UTAK

- Az AS1 megtanulja Inter-AS routinggal, hogy az X hálózat már AS2-felé is elérhető
- AS1 routereiben csak egy irányt kell bejegyezni
 - Az Intra-AS routingból ismert költségeket is figyelembe kell venni
- Forró krumplici (hot potato) döntés – legolcsóbb kijutás az AS-ből
 - Az 1b közelebb van 1d-hez, menjen arra a csomag X felé
 - Gyakori, de nem az egyetlen megoldás

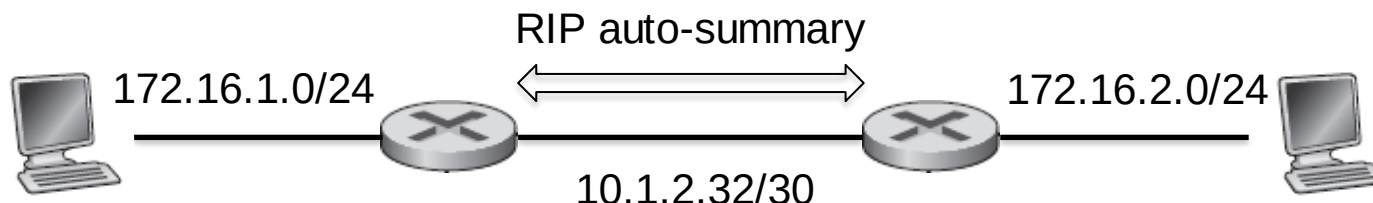


- **Intra-AS – IGP**
 - **RIP** – Routing Information Protocol
 - **OSPF** – Open Shortest Path First
 - **EIGRP** – Extended Interior Gateway Routing Protocol
 - Cisco fejlesztés, de már RFC-ben is elérhető
 - **IS-IS** – Intermediate System to Intermediate System
 - Inkább ISO-OSI, mint TCP-IP
 - Elsősorban ISP-knél alkalmazzák
- **Inter-AS – EGP**
 - **BGP** – Border Gateway Protocol

1. Hierarchikus routing
2. RIP
3. OSPF
4. BGP
5. Broadcast, Multicast

- Távolságvektor alapú routing
- Verziók
 - v1 – RFC 1058, v2 – RFC 2453, lefelé kompatibilis
 - IPv6 támogatás: RIPng – RFC 2080
- Költség (metrika)
 - Hopszám
 - Hozzákapcsolt hálózat költsége 1
 - Maximális érték 15 – végtelen érték 16
- Üzenetek – RIP hirdetések (advertisements)
 - A router távolságvektorokat küld a szomszédoknak
 - 30 másodpercenként, vagy közvetlen kérésre
 - UDP felett
- Szomszéd elérhetősége
 - 180 másodpercnyi csend után feltételezi
 - Poisoned reverse megoldással elterjeszti a rossz hírt

- **Osztályalapú hálózatkezelés a RIPv1-ben**
- **Osztályalapú konfiguráció**
 - A protokollal hirdető hálózatokat meg kell adni
 - **A RIP-nél csak maszk nélkül adhatók meg hálózatok (osztályalapú címkezelés)**
 - RIPv2-nél nem feltétlenül baj RIPv2 MÁR classless!
- **Automatikus összegzés (auto-summary)**
 - **Alapértelmezetten osztályalapon összevont címtartományokat hirdet**
 - Baj lehet, ha egyébként osztály nélküli címkiosztást használunk



1. Hierarchikus routing
2. RIP
3. OSPF
4. BGP
5. Broadcast, Multicast

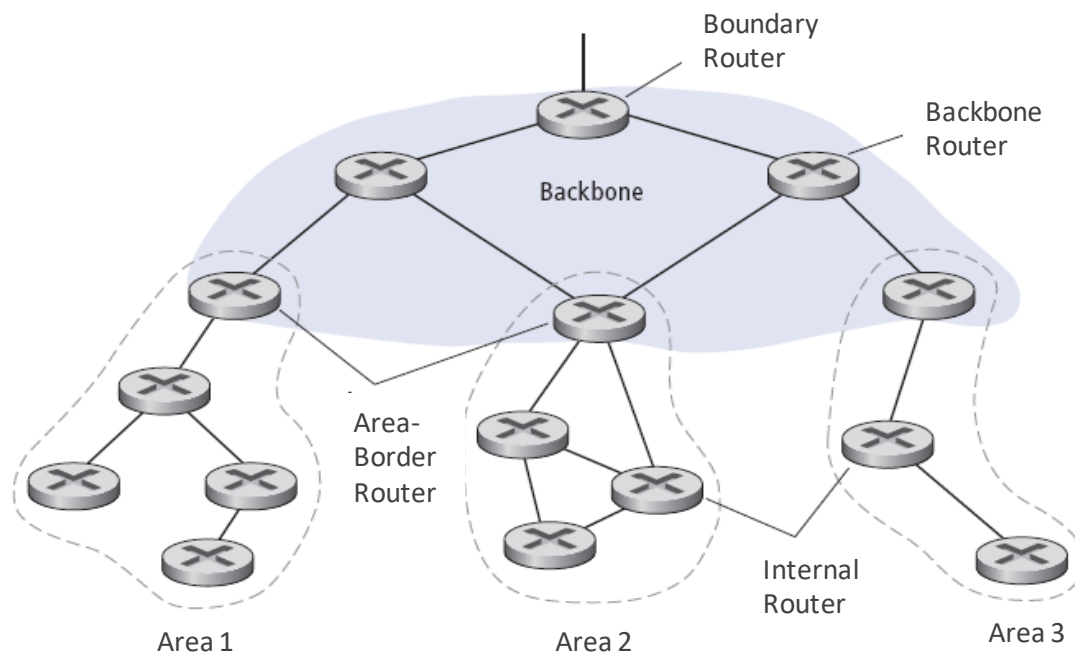
- Verziók
 - OSPFv2 – RFC 2328
 - IPv6 támogatás: OSPFv3 – RFC 5340
- „Open” – szabadon felhasználható
- „Shortest Path First” – legrövidebb út
- Támogatja az osztályfüggetlen címezést (CIDR)
 - Konfigurációkor a hálózatok hozzáadása maszkkal
 - Hirdetett információkban is
- Nincs osztályalapú összevonás (auto-summary)

- **Linkállapot alapú routing**
 - Minden OSPF router a hozzákapcsolt linkekről gyűjt információt
 - **Információk hirdetése minden OSPF routernek (Link State Advertisement, LSA)**
 - Periodikusan
 - **Változás esetén**
- Az OSPF LSA-k közvetlenül az IP datagramba vannak csomagolva
 - Egy megfelelő (multicast) IP címre küldve jut el a többiekhez
- Az **LSA-k alapján minden OSPF router megismeri a routereket és linkeket**
 - A hálózat térképe (Topology Map) előáll a routerekben
 - Minden routerben ugyanaz
- Útvonalszámítás a Dijkstra algoritmussal
 - A routing táblába csak az első hop kerül be
 - Változás esetén (nem feltétlenül teljes) újraszámolás

- Biztonság: az OSPF üzenetekre előírható autentikáció
- Sávszélesség alapú, vagy kézzel állítható költség
- Egy célhálózat felé több azonos költségű alternatív útvonalat is képes előállítani és kezelni
 - Terhelésmegosztás
- A multicasting beépített támogatása
 - Multicast OSPF (MOSPF) ugyanazt a hálózattérképet használja fel
- Hierarchikus hálózatkezelés nagyobb AS-ek esetén

• Kétrétegű hierarchia

- Pontosan egy **gerinchálózati körzet (backbone area)**, azonosítója: 0 area 0
- Több, a **gerincről lelógó további körzet (local area)**



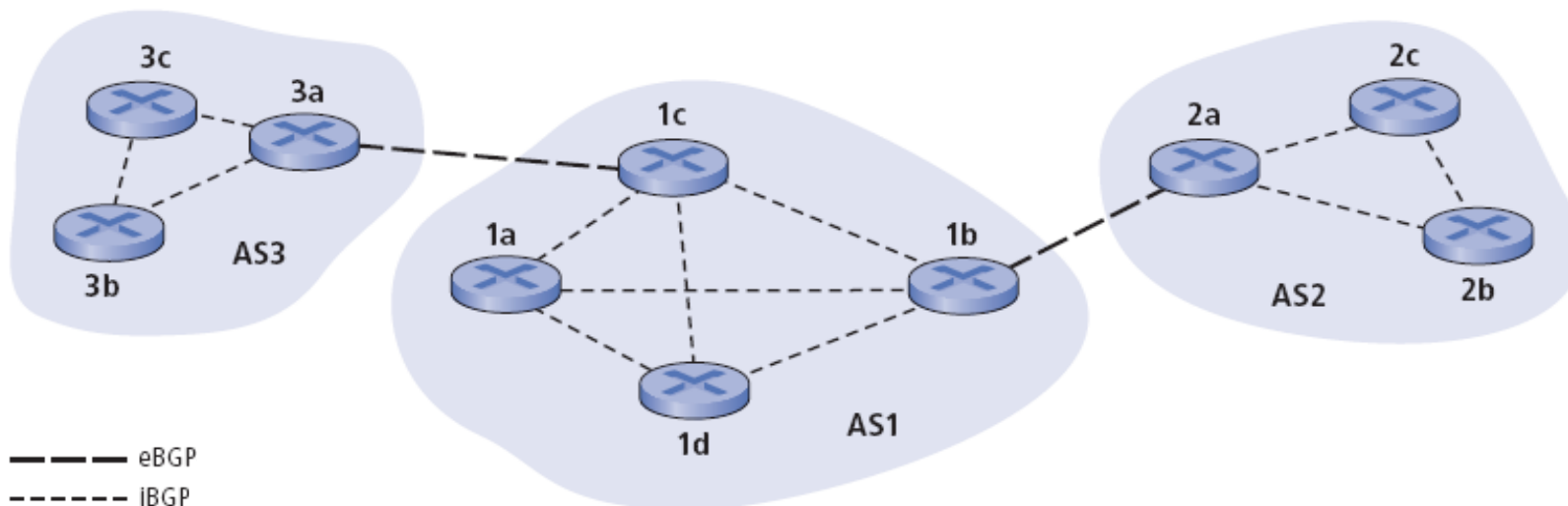
- LSA hirdetés korlátozása
 - Az LSA-kat csak az areán belül hirdetjük
 - Minden router ismeri a saját areájának a topológiáját
- Area-Border-Router (ABR)
 - A saját lokális areájában számított távolság-információkat összefogja
 - Az összefogott információkat hirdeti a gerinc-area felé
 - A gerinc-areából kapott összefogott információkat hirdeti a lokális areájában
 - Így végül minden router értesül a más areákban lévő hálózatokról is
- Routing
 - Areán belüli cél – intra-area
 - Másik areában lévő cél – inter-area
 - A gerincen keresztül
- Backbone-Router
 - A gerincben működteti az OSPF-t
- Boundary-Router
 - Kapcsolat más AS-ek felé – gateway

1. Hierarchikus routing
2. RIP
3. OSPF
4. BGP
5. Broadcast, Multicast

- 4. verzió: BGP-4 – RFC 4271 EGP - Inter-AS
 - Az Internet de-facto szabványa az Inter-AS routingra
- Lehetővé teszi, hogy
 - Megismerjük a szomszédos AS-ekben lévő hálózatok elérhetőségi információit
 - Ezeket az információkat keresztüljuttassuk a saját AS-en
 - Útvonalakat határozzunk meg egy távoli hálózat felé
 - Az elérhetőségtől függően
 - Az alkalmazott **politikának (policy)** megfelelően
 - Saját hálózataink elérhetőségét más AS-ek felé hirdessük
 - Prefixek: hálózat, vagy összefogott hálózatok

- BGP-t futtató routerekből párokat képzünk (**BGP-Peers**)
 - A párok információkat cserélnek egymás között
 - Kommunikáció TCP felett megvalósított BGP-viszony(lat)okkal (**BGP session**)
 - Azonos AS-ben lévő routerek esetén belső BGP viszony: **internal BGP session (iBGP)**
 - Eltérő AS-ben lévő routerek esetén külső BGP viszony: **external BGP session (eBGP)**
- A BGP viszonyok nem feltétlenül egyetlen fizikai linknek felelnek meg
- Egy prefix hirdetésével az adott AS „megígéri”, hogy az abba tartozó célcímű datagramokat továbbítja
- A másik AS-ből kapott prefixeket aggregálva is tovább lehet hirdetni

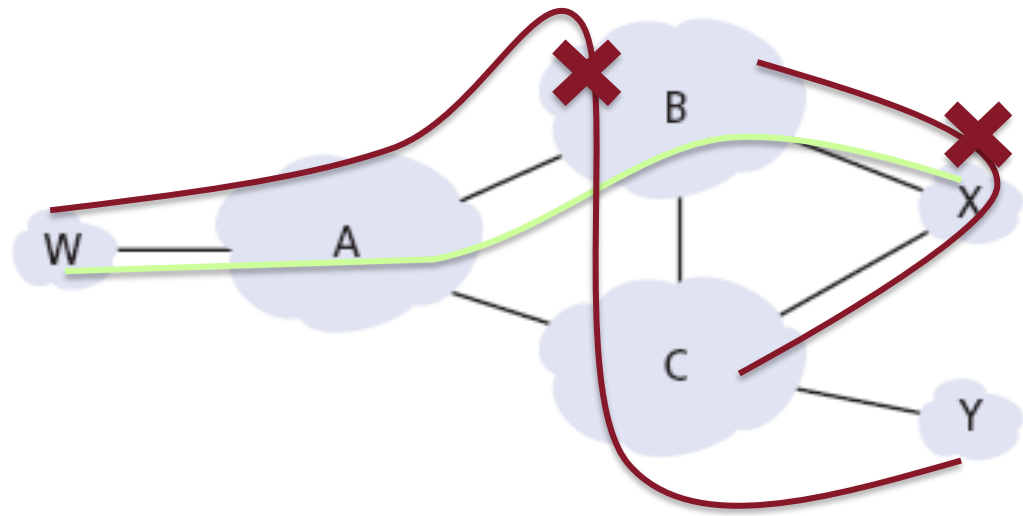
- AS3 hirdeti az elérhető prefixeket a 3a és 1c közötti eBGP viszonyon keresztül
- Az 1c iBGP viszonyokat használ, hogy elterjessze ezeket AS1-ben
- Az 1b továbbhirdeti a prefixeket AS2 felé az 1b és 2a közötti eBGP viszonyon keresztül



- A router egy prefix hirdetésekor útvonal-attribútumokat is küld
 - A BGP terminológiában
prefix + attribútumok = route
- Fontos attribútumok
 - AS-PATH
 - Azon AS-ek listája, amiken a prefix hirdetése keresztülment
 - Minden továbbhirdetéskor kiegészül
 - NEXT-HOP
 - Két AS akár több gateway páron keresztül is össze lehet kötve
 - Egyértelművé kell tenni, hogy melyik irányból jött a hirdetés
 - Ez alapján lehet meghatározni a kilépési linket és az irányt
- A kapott hirdetések kezelése a beállított politikától függ

- Útvektor alapú
 - Nem ismert a teljes topológia
 - Ismert a cél felé vezető AS-ek listája
- Csak a politikának megfelelő hirdetésekkel kell foglalkozni
- Több irányból is behirdetett prefixek esetén választani kell
 - Legrövidebb AS-PATH
 - Legközelebbi NEXT-HOP
 - Egyéb paraméterek

- A,B,C szolgáltatói hálózatok (provider network)
- X,W,Y a szolgáltatók klienseinek hálózatai
 - Az X két szolgáltatóhoz is be van kötve (dual homed)
- X nem szeretne B-ből C-be menő forgalmat átvinni
 - A C felől jövő hirdetések nem küldi tovább
- B sem szeretne Y-ból W felé küldött forgalmat továbbítani
 - Nem hirdeti az A-tól kapott AW utat C felé
 - X felé viszont hirdeti tovább BAW útként



- BGP vagy IP hijacking
 - Hibás hirdetések miatt magára húzza a forgalmat egy szervezet
 - Rossz konfiguráció katasztrofális hatással
- Például
 - 1997 április 25
 - "AS 7007 incident"
 - A MAI RIP információkat küldött szét BGP-vel
 - Fekete lyukká vált (black-hole)
 - 2006 január 22
 - A Con-Edison az Internet forgalom nagy részét elterelte
 - 2008 február 24
 - Pakisztán blokkolni akarta a bejövő youtube forgalmat
 - Az egész youtube-ot elérhetetlenné tette egy órára
 - 2017 augusztus 27
 - A Google hibája miatt sok szerver elérhetetlenné vált Japánban
 - BGP route leak – RFC 7908

- Politika

- Inter-AS

Egy szervezet kontrollálni szeretné, hogy milyen forgalom és hogyan mehet át az AS-en

- Intra-AS

Saját forgalom, saját adminisztráció, nincs szükség politikai döntésekre

- Nagyságrend

- A hierarchikus routing

csökkenti a táblabejegyzéseket és a routing hirdetési forgalmat

- Inter-AS – Kritikus

- Intra-AS – Nem annyira kritikus

- Teljesítmény

- Inter-AS

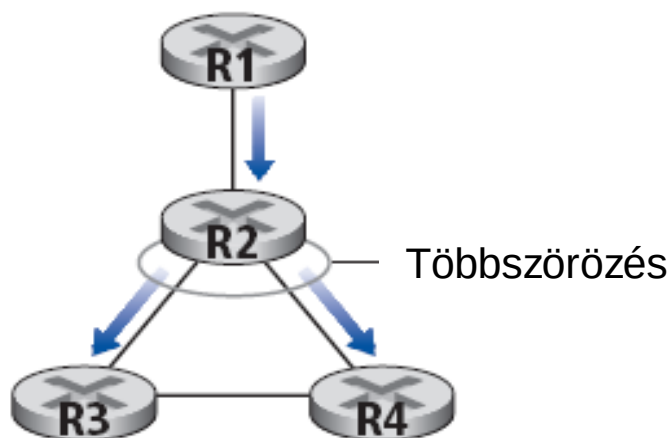
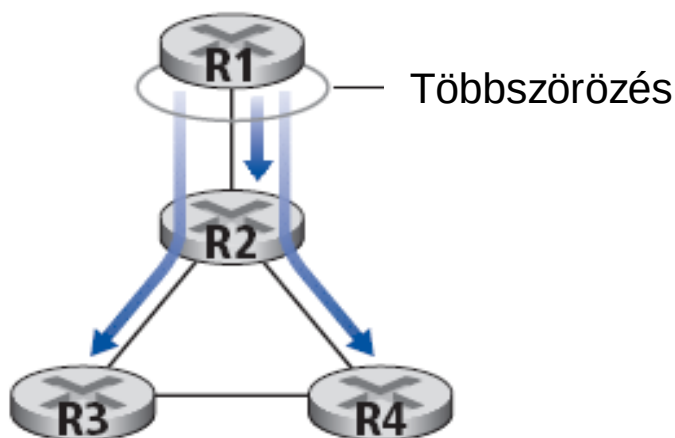
A politika fontosabb a teljesítménynél

- Intra-AS

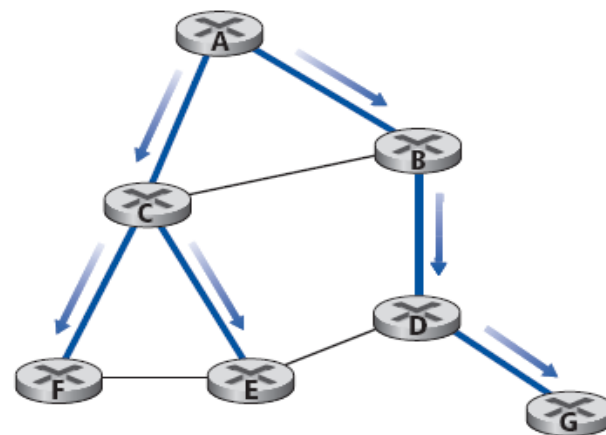
Koncentrálnak a teljesítményre

1. Hierarchikus routing
2. RIP
3. OSPF
4. BGP
5. Broadcast, Multicast

- Ugyanazt a datagramot kell eljuttatni az összes hoszthoz
- Egy LAN-on belül – data link rétegben kell megoldani
- Több hálózat esetén – a routerben szükséges a duplikálás
- Hol többszörözzünk?
 - A forráshoz közel – nem hatékony, és kérdés, hogy kik a célok
 - A közbeeső routereknél – az összesnél?



- Alapmegoldás: **elárasztás (flooding)**
 - Minden router minden irányba másolatot küld
 - Nyilván felesleges terhelést jelent
 - Vihar, broadcast storm alakulna ki – a többszörözött datagramot is többszörözzük...
- Korlátozott elárasztás
 - Csak a még nem látott datagramokat duplikálja és küldi tovább a router
 - Azonosított datagramok, vagy
 - Azonosított irányok
 - Csak a forrás felől jövőt fogadjuk el
Reverse Path Forwarding, RPF
- **Feszítőfa (spanning tree) alkalmazása**
 - Csak kiválasztott linkeken mehet a forgalom



- Egy datagramot több végponthoz kell eljuttatni
 - Például média-folyamok, videokonferencia
- IP esetén speciális címre küldjük
 - IPv4 : 224.0.0.0 / 4 tartomány (D osztály)
 - IPv6 : FF00::/8 tartomány
 - A cím egy csoportot jelöl ki, aminek tagjai fogadják az „adást”
- A csoporthoz csatlakozás és leiratkozás
 - **Internet Group Management Protocol (IGMP)**
 - A routernek tudnia kell, hogy van-e olyan hoszt akit rajta keresztül kell elérni, és tagja a csoportnak
- Elérés **feszítőfa** segítségével
- Útvonalak meghatározásához külön routing protokollok kellhetnek, vagy az unicast irányítást használjuk fel (**Protocol Independent Multicast, PIM**)



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Az adatkapcsolati réteg
2023. április 28.

Mészáros András

BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



A TCP/IP protocol stack rétegei:

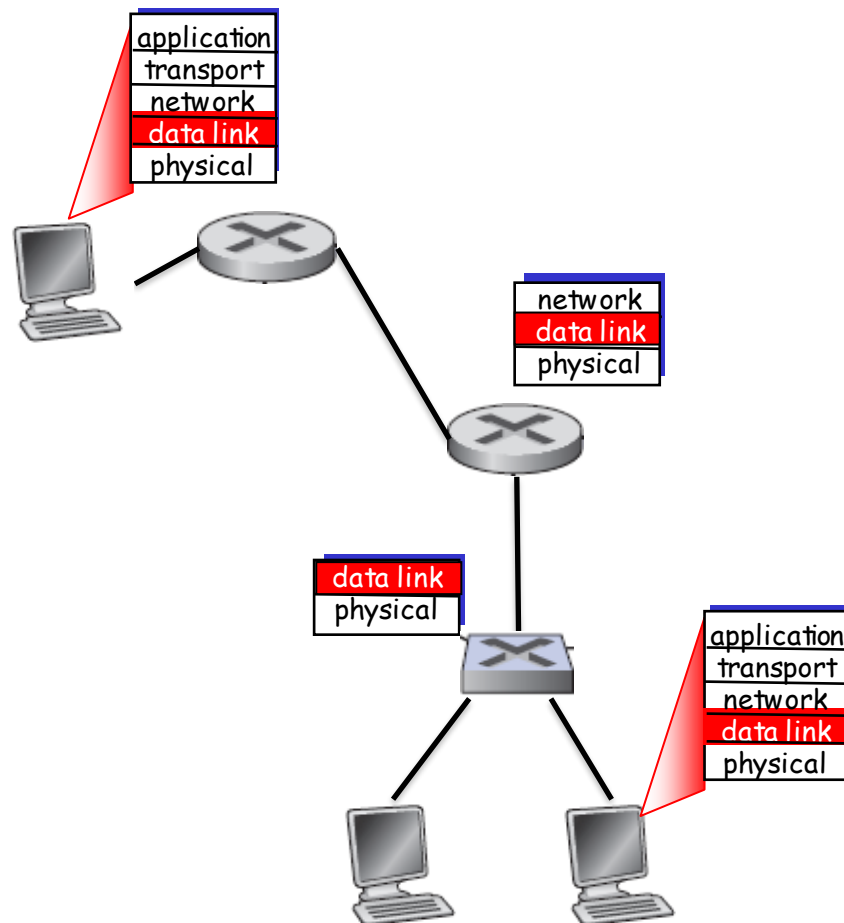
- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, SMTP, HTTP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, útvonalválasztó protokollok
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - PPP, Ethernet
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti összeköttetéseken

1. Az adatkapcsolati réteg szolgáltatásai
2. Címzés az L2 rétegben
3. Lokális hálózatok (LAN-ok)
4. Ethernet

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat

ADATKAPCSOLATI RÉTEG – L2

- **Cél: hálózati datagram átvitele két szomszédos, hálózati funkciót megvalósító elem között**
- A szomszédos elemek között egy **link** (adatkapcsolat) van
 - Vezetékes link
 - Vezetéknélküli link
 - **LAN**
- A link egyik végén a datagramok **keretekbe csomagolása**
- A link másik végén a megérkezett datagram kicsomagolása és átadása a hálózati rétegnek
- Az egyes linkeken eltérő protokoll lehet, eltérő szolgáltatásjellemzőkkel
 - Például **IEEE802.11, GigabitEthernet**
 - Eltérő sebesség
 - Eltérő biztonság



- Keretezés
 - Fej- és farokrész (**Header és Trailer**) illesztése az adathoz
- Hozzáférés az átviteli médiához
 - Mi legyen, ha több szomszéd egyszerre akar adni ugyanazon a linken?
- **Címzés fizikai cím alapján** L2, Switchek nem tudnak logikai címeket kezelni!
 - A cél-szomszéd azonosításához
 - **Nem a logikai (IP) címről van szó**
- Keretek megbízható átvitele
 - Pufferelés a végpontokon
 - **Bithibadetekció** (esetleges újraküldetés)
 - Bithibakorrekción
- Küldési irányok kezelése egy linken
 - **Szimplex**
 - **Félduplex**
 - **Duplex** (full-duplex)

- Logical Link Control (**LLC**) alréteg
 - Az adat átvitelét szervezi
 - Multiplexáló funkció – a felsőbb rétegből érkező PDU-kra
 - Opcionális funkciók (nyugtázás, folyamszabályzás)

- Media Access Control (**MAC**) alréteg
 - Hibadetekció/hibajavítás
 - Közeghozzáférés
 - Címzés
 - Keretezés

- **Hibadetekció**

- **Redundáns bitek** a trailer-ben

- Nemcsak az adatra (datagram), hanem a fejrészre is vonatkozhat
 - Nem minden hiba észlelhető, mert nem csak egy bit lehet hibás
 - Például: paritásbit, transzport réteg ellenőrző összeg

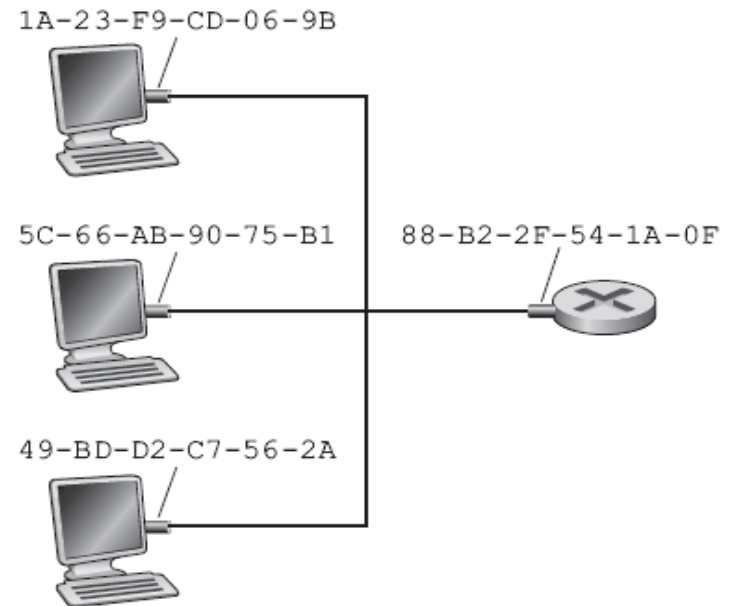
- **Hibajavítás**

- Nem csak észleljük, hogy hiba volt, hanem azt is tudjuk hol
 - Például: kétdimenziós paritás, ciklikus redundancia – CRC

1. Az adatkapcsolati réteg szolgáltatásai
2. Címzés az L2 rétegben
3. Lokális hálózatok (LAN-ok)
4. Ethernet

- **Logikai cím – „egyedi” azonosítás az Interneten**
 - A hálózati rétegben használt cím (IPv4, IPv6)
 - Cél a datagram továbbítása távoli hálózatba
 - Az alhálózat elhelyezkedése kikövetkeztethető
- **Fizikai cím – egyedi azonosítás az alhálózatban**
 - Az adatkapcsolati rétegben használt cím (MAC cím)
 - Cél: egy keret továbbítása egy szomszédos elemhez, aminek van logikai címe (végpont, router interfész)
 - Nem fontos az alhálózat elhelyezkedése, mert csak azon belül használjuk
 - **48 bites azonosító** (az IEEE 802-es protokolloknál)
 - A hálózati adapter (kártya, port) ROM-jába **égetve**
 - Esetleg szoftverrel beállítható a használt érték
 - Olvasható formátum – hexaszámokkal leírt bájtok
 - Például: 01-23-45-67-89-ab vagy 01:23:45:67:89:ab

- Az adatkapcsolati réteget megvalósító technológiákban
 - Ethernet (IEEE 802.3)
 - WiFi (IEEE 802.11)
 - Bluetooth
 - Korábban
 - Token Ring (IEEE 802.5)
 - FDDI (IEEE 802.4 token bus)
 - ATM
- Az (al)hálózaton, LAN-on belül egyedi cím kell legyen
 - Szórási (broadcast) cím az L2-ben: FF-FF-FF-FF-FF-FF
 - Többesadás (multicast) címek, pl: 01-00-05-xx-xx-xx

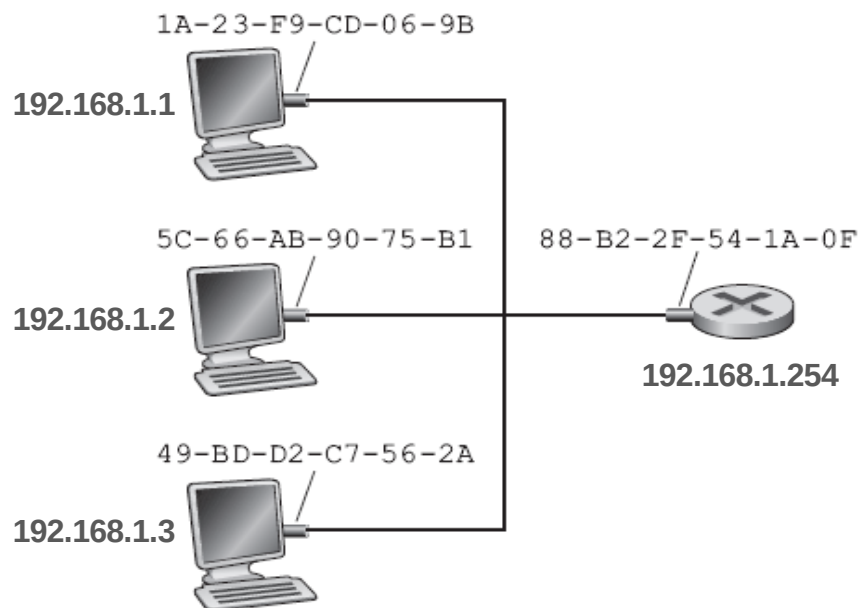


- A kiosztást az IEEE felügyeli
 - Az egyes gyártók a teljes címtér egy részét vásárolják meg
 - Az első három bájt: Organizationally Unique Identifier – **OUI**
- **MAC cím**
 - **Nincs hierarchia** – hordozható

Nem hierarchikus miért?
Csak LAN-ban használt, nem globális azonosító!

 - Egy hálózati kártya átvihető egy másik LAN-ba
- **IP cím**
 - **Hierarchikus szervezés** – nem hordozható
 - Egy végpont nem tehető át (egyszerűen) egy másik hálózatba az IP címével együtt, szükség van annak átírására
- **Analógia – név és lakcím**
 - Név + születési idő (Kovács István, 2000. január 1.) – MAC cím
 - Lakcím – IP cím

- Honnan tudjuk meg, hogy melyik **fizikai cím tartozik egy adott logikai címhez?**
 - Melyik fizikai címre menjen a 192.168.1.2 célú datagramot tartalmazó keret?
- Address Resolution Protocol (ARP)**
- A hálózati elemek tárolják a feloldott címeket
 - Logikai cím – fizikai cím összerendelések**
 - Adott idő (pl. 20 perc) után törlődik egy bejegyzés
 - Kézzel is törölhető



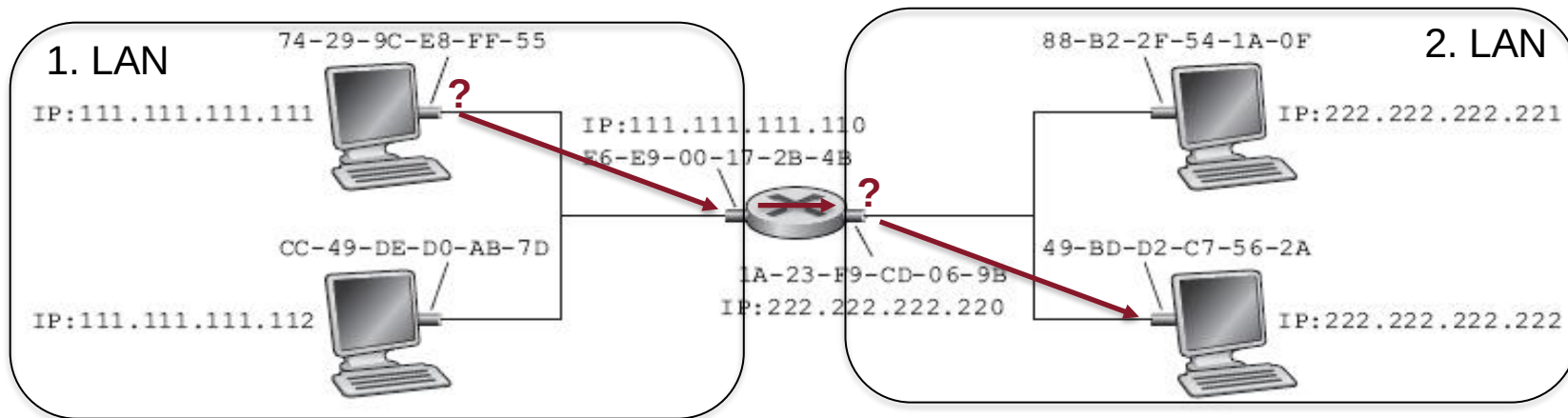
ARP, L2-ben működik. A hálózati rétegről érkező IP címet fordítja fizikai címre.

Ki tudja a 192.168.1.1 MAC címét? (Broadcast üzenet)
A gép válaszol: ÉN!! Címem 00:1A:2B:3C:4D:5E (unicast)

AZ ARP MŰKÖDÉSE

1. Az A hoszt a B hosztnak küldene egy datagramot
 - B MAC címe nincs még benne az A hoszt arp táblájában
2. Az A elküld egy ARP-Query keretet a B IP címével
 - A broadcast MAC címre
 - A LAN összes hosztja és interfésze megkapja a keretet
3. B felismeri a saját IP címét és válaszol egy ARP-Reply kerettel
 - A válasz keretet A-nak címzi
 - A válasz tartalmazza B MAC címét
4. Az A felveszi a B IP címét feloldó bejegyzést az arp táblájába
5. Az A el tudja küldeni a datagramot B-nek
 - A most már ismert MAC címet használva a kerethez
- Az ARP automatikusan indul
 - Nincs szükség kézi konfigurációra
- IPv6
 - Neighbor Discovery Protocol – NDP
 - Hasonló mechanizmus
 - ICMPv6 üzenetek a többi hoszt megszólítására

- A datagramot az alapértelmezett átjárón (default gateway) át kell küldenünk
 - Router interfész címének feloldása (az 1. LAN-ban) – ARP !!!!
 - Keret átküldése a routernek
 - Datagram kicsomagolása a keretből
- Továbbítás a routing tábla alapján
 - Célcím feloldása a megfelelő (a 2.) LAN-ban - ARP !!!!
 - Datagram becsomagolása
 - Keret átküldése a hosztnak



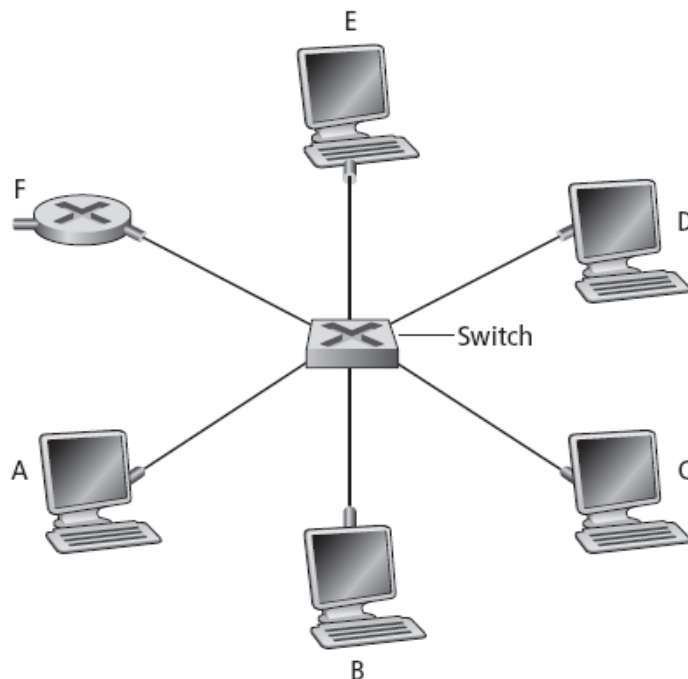
1. Az adatkapcsolati réteg szolgáltatásai
2. Címzés az L2 rétegben
3. Lokális hálózatok (LAN-ok)
4. Ethernet

- **Szomszédos hálózatelemek**
 - Logikailag szomszédos elemek halmaza
 - Vezetéknélküli esetben fizikailag is szomszédok
- **Ismétlő (repeater) funkció**
 - Két kábel fizikai összekötése
 - Ma már inkább több kábelt összekötő eszközzel – hub
 - Minden keret megjelenik minden kábelen – logikailag egy közeg
- **Ütközés (collision) – a közegre (médiumra) kerülő keretek egymást zavarják**
- **Kapcsoló (switch) funkció**
 - Kezeli az adatkapcsolati rétegbeli PDU-kat (kereteket)
 - Szétdarabolja a LAN-t különálló közegekre
 - Csökkenti az ütközések valószínűségét

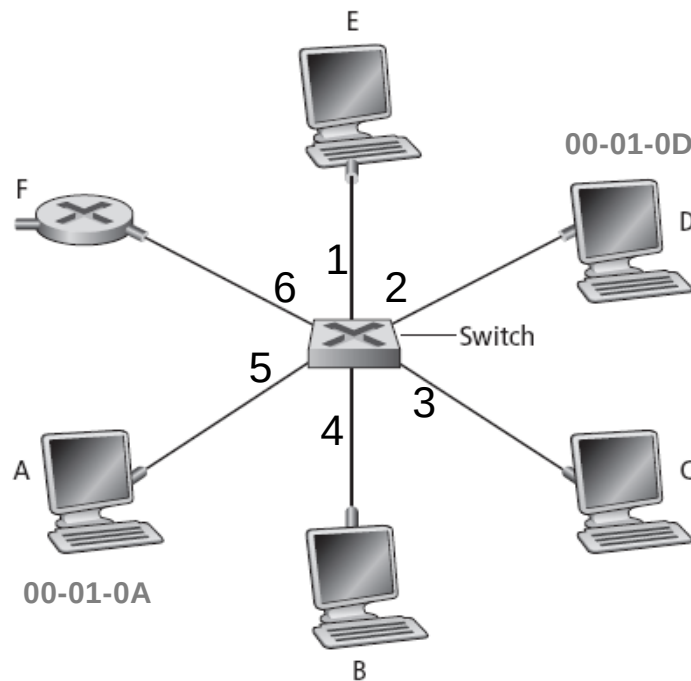
- Keretek feldolgozása
 - A kereteket nem neki, csak rajta keresztül küldik
 - A kapott keretet pufferelem
 - A MAC cím alapján eldönti, hogy melyik irányba továbbítsa
 - Ezzel csökken az ütközés esélye
- A továbbítás kicsit hasonlít a routingra, de
 - Csak a LAN-on belül megy
 - Sokkal egyszerűbb
 - Sokkal gyorsabb
- Transzparens funkció
 - A hosztok nem tudják, hogy vannak switchek a LAN-ban !!!!!
- Automatikus tanulás – MAC learning
 - Nem kell külön konfigurálni
- Hatékonyság
 - Akár több keret is átmehet a hálózaton egy időben, ütközés nélkül

MAC CÍMEK TANULÁSA

- A switch egy táblázatban tartja nyilván a MAC cím – port párokat
- Tanulás a switchhez érkező keretek forráscímei alapján
- Ha nincs a táblában a keresett cím, akkor minden irányba kiküldi !!!!!
- Kivéve arra, amerről jött
- Adott idő után elfelejti !!!!!
- Vagy kézzel törölhető

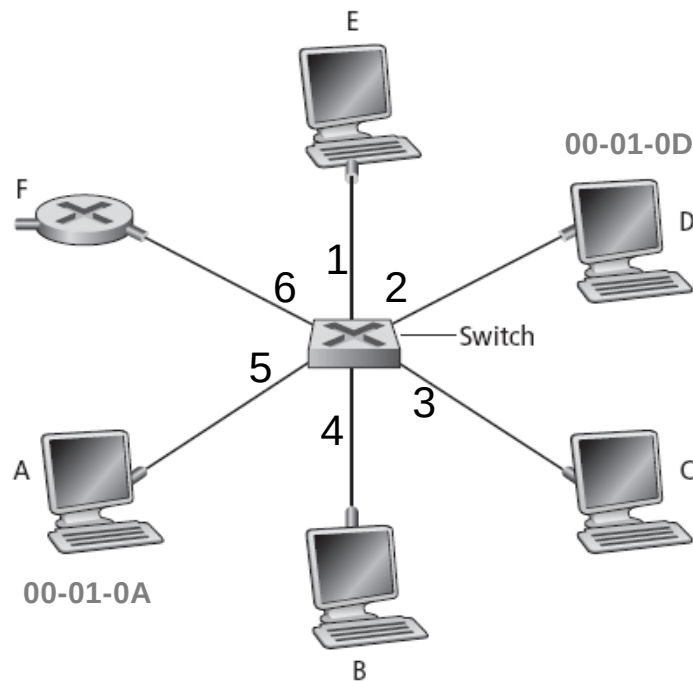


- IP datagramot tartalmazó keret küldése A-ból D-be
 - A cél MAC címe már ismert A-ban
- **Lépések**
 1. Keret küldése A-ból
 2. Keret érkezik az 5-ös porton
 3. Címtábla frissítése A MAC címével
 4. Keret szétküldése az 1,2,3,4 és 6 portokon
 5. Keret fogadása D-ben



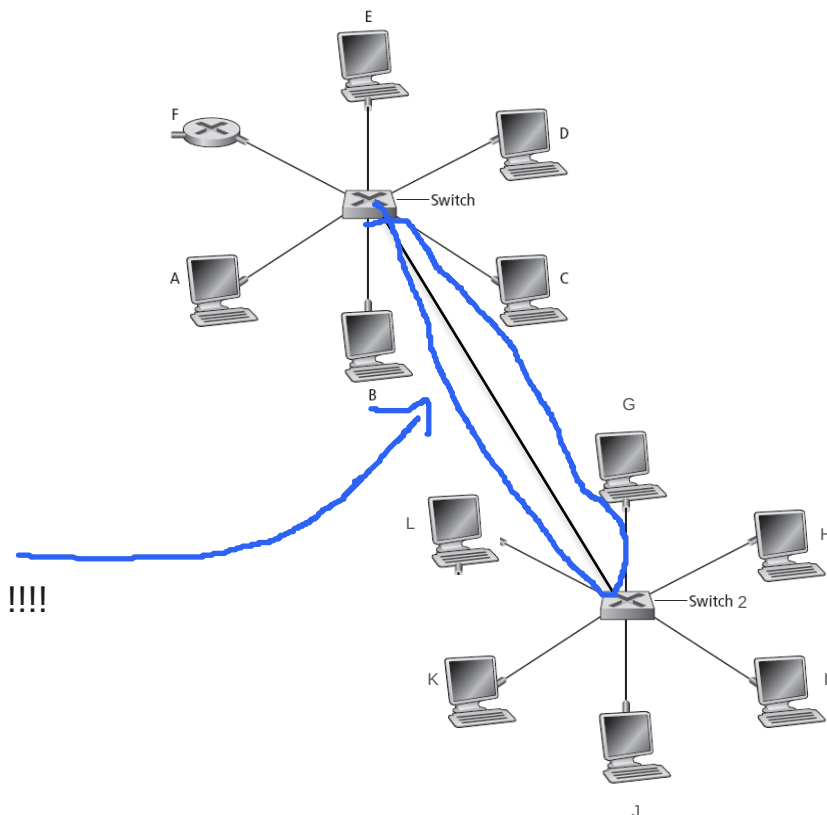
MAC cím	Port
00-01-0A	5

- D válaszol A-nak
 1. Keret küldése D-ből
 2. Keret érkezik a 2-es porton
 3. Címtábla frissítése D MAC címével
 4. Keret kiküldése az 5-ös porton
 5. Keret fogadása A-ban



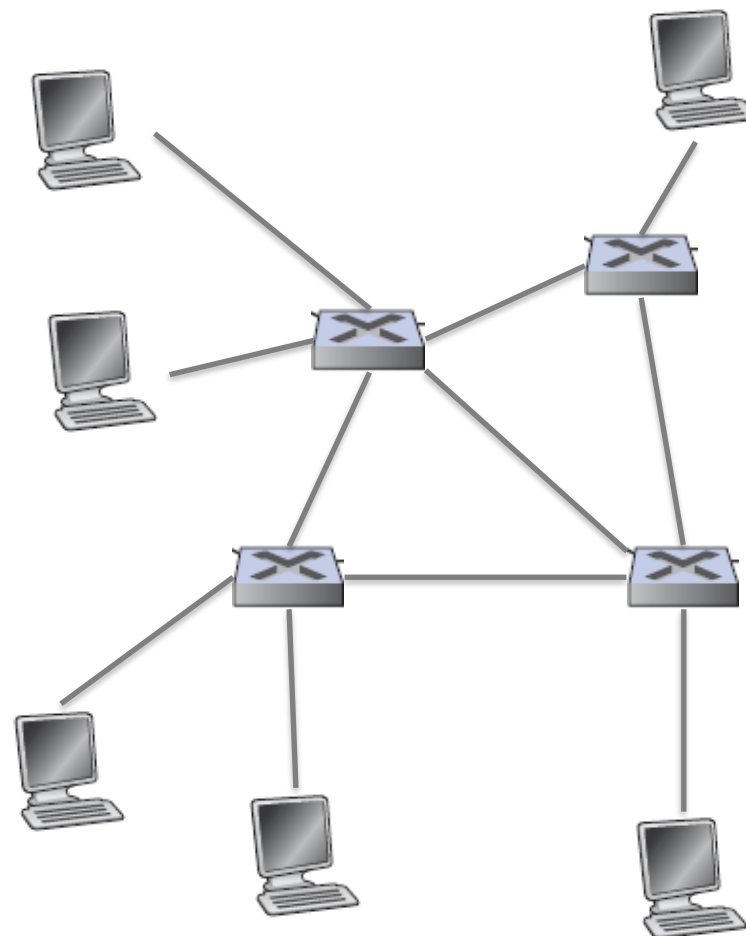
MAC cím	Port
00-01-0A	5
00-01-0D	2

- **Portok a switchben**
 - Nagyszámú
 - Véges
- A LAN méretének növelése
 - Nagyobb switch – több port (nem skálázható)
 - Több switch
- **Switch bemenetén nem csak egy hoszttól jöhetnek keretek**
 - A MAC táblában több cím is lehet ugyanahhoz a porthoz rendelve

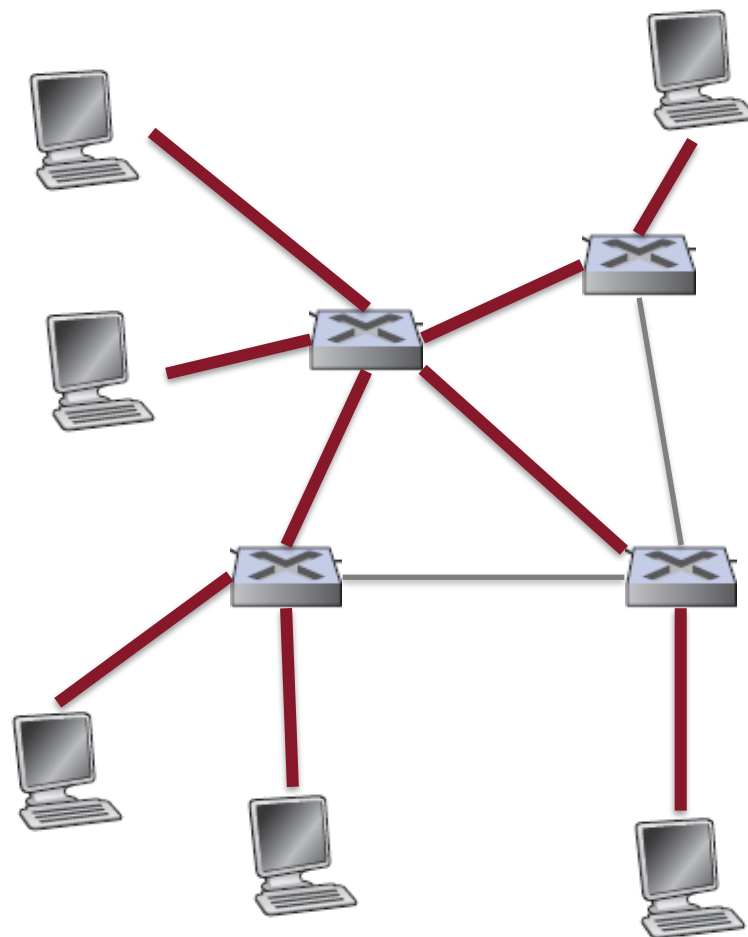


REDUNDANCIA A LAN-BAN

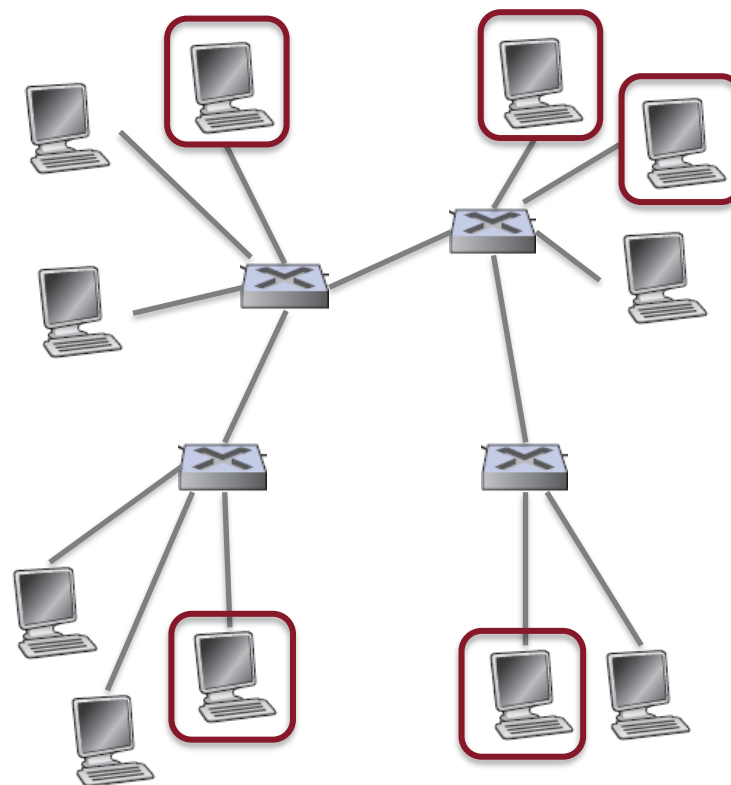
- **Nagyobb LAN-okban sérülékenységet jelenthet, ha minden csak egyszeresen van összekötve**
 - Fa szerkezetű gráf
 - Egy kábel hibája szétvághatja a hálózatot
- Redundancia
 - Többszörösen összekötött gráf
 - Hiba esetén is legyen alternatíva
- Hibamentes esetben
 - A szerkezetben lévő körök miatt továbbítási hurkok alakulhatnak ki
 - A switch a tanulás során felesleges irányokban is kiküldhet kereteket
 - Többszörös többszörözés...



- **A megoldás**
 - Csak erre kiválasztott linkeken menjen forgalom
 - Feszítőfa képző protokoll – Spanning Tree Protocol (STP)
- Hiba esetén
 - Az eredetileg nem használt linkekkel egészítik ki a fát
 - Továbbra is hurokmentes
- Új elem csatlakozása esetén változik a kép
 - Új switch
 - Új link



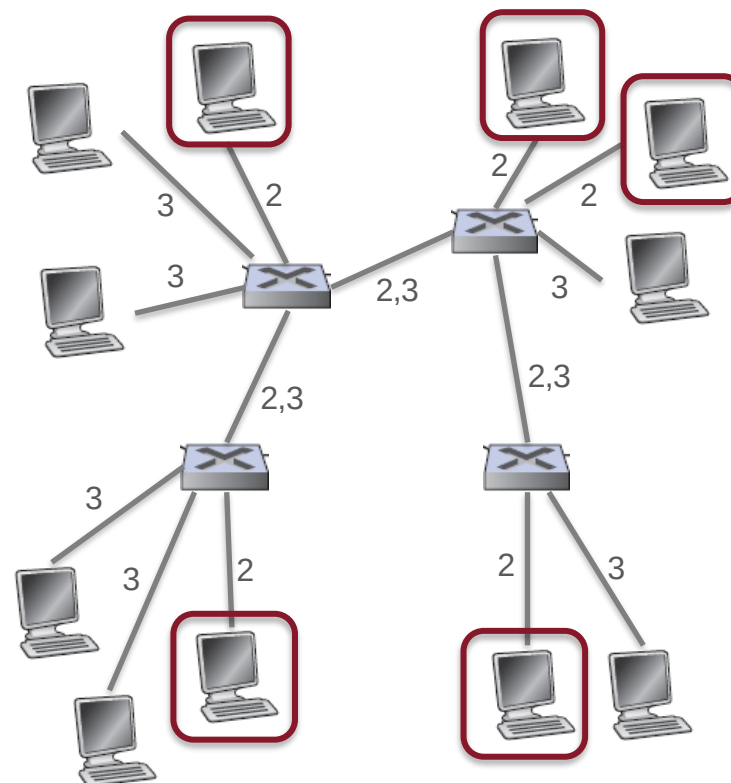
- Méret vagy felhasználási cél miatt **több LAN** kialakítása
 - Fizikailag elkülönítve – túlzott kábel- és berendezés-igény
 - Logikailag elkülönítve – azonos L2 hálózaton
- **VLAN – virtuális LAN**
 - Azonosítani kell, hogy melyik forgalom melyik VLAN-ba tartozik
 - Ha többféle is átmegy egy linken: a keret fejléc speciális mezőjébe írt azonosítóval – **jelölés (tag) alapú**
 - A switchek kezelik, a hosztok nem feltétlenül tudnak róla



- **VLAN-tag** – IEEE 802.1q
 - 12 bites fejléc mező
 - VLAN 1 – alapértelmezett (default)
 - Normál VLAN-ok tartománya 1-1005
 - Kiterjesztett (extended) tartomány 1006 - 4094
- Alkalmazásokhoz is köthető VLAN
 - Például voice VLAN
- **Menedzsment VLAN** kijelölése
 - A switchek konfigurálása csak ezen keresztül megengedett
- A switch funkciókat VLAN-onként kell ellátni
 - Külön MAC cím tábla
 - (Lehet) külön feszítőfa
- **Előnyök**
 - Olcsó
 - Egyszerű konfiguráció és rugalmas kialakítás
 - A különválasztott forgalmak miatt nyújt némi adatbiztonságot
 - Kisebb méretű LAN-ok, kevesebb broadcast forgalom

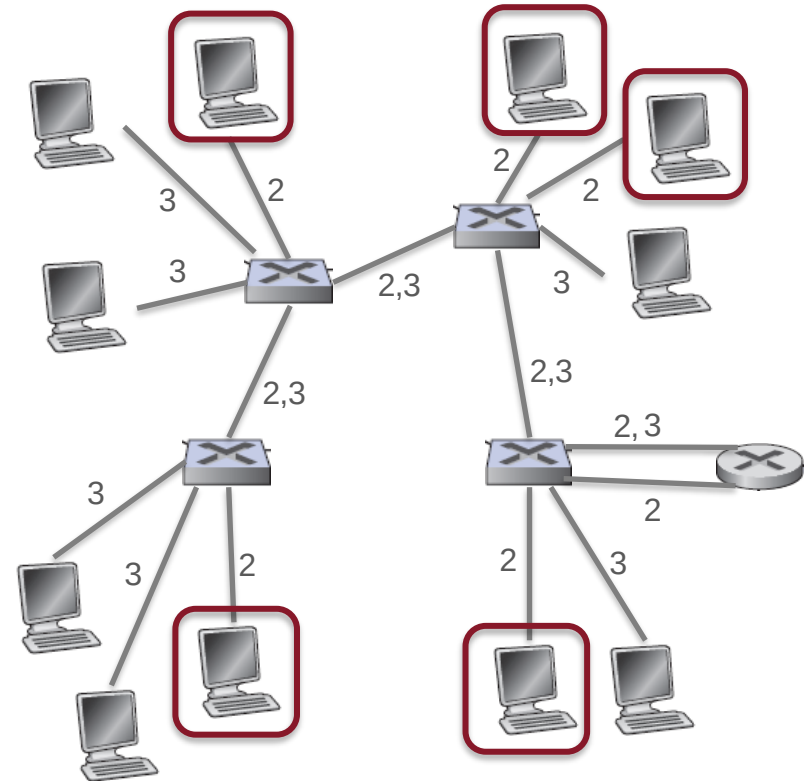
VIRTUÁLIS LAN-OK BEÁLLÍTÁSA

- A switch-ek portjait VLAN-okhoz rendelhetjük
 - Elérési (**access**) portok
 - Tipikusan egy hoszt felé
 - Egy porthoz egy VLAN
 - Nem kell jelölgetni a kereteket
 - Trönk (**trunk**) portok
 - Switchek között
 - Egy porton több VLAN forgalma megy
 - Jelölgetni kell
 - Jelöletlen keretek – natív VLAN
- A nem használt VLAN-ok letiltása növeli a biztonságot



VIRTUÁLIS LANOK ÖSSZEKÖTÉSE

- Egy VLAN egy IP alhálózatot jelent
- Az összekapcsoláshoz routerre van szükség !!!!
- Csatlakozás a routerhez
 - Külön interfészeken
 - Könnyebb konfigurálni
 - Költségesebb
 - Egy trunk porton keresztül
 - Router on a stick
 - Hatékonyabb
 - Külön alinterfészeket kell konfigurálni a router portján

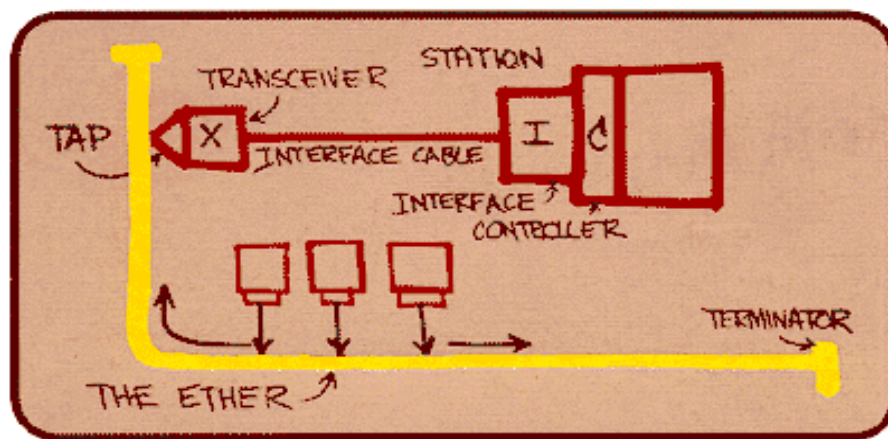


1. Az adatkapcsolati réteg szolgáltatásai
2. Címzés az L2 rétegben
3. Lokális hálózatok (LAN-ok)
4. Ethernet

- Több mint 40 éve kezdték fejleszteni
- A piacot szinte kizárólagosan uraló LAN technológia
- Olcsó – párezer forintos hálózati kártyák
- Világszerte használt technológia – magas fokú kompatibilitás
- Véletlen hozzáférés a médiához (később lesz még róla szó)
- Az eredeti 10, és 100 Mbps sávszélesség mára 1,10, 40, 100 Gbps-ra nőtt
- Nem csak LAN technológia, hanem a hálózat magjában is ezt használják

L2 legfőbb protokolja!

Kereteket továbbít LAN-on belül!



- Előhang (preamble) és keretkezdés (SFD)
 - 7 bájt 10101010 tartalommal, és egy bájt 10101011 tartalommal
 - Az adó és a vevő szinkronba hozására
- Keret vége
 - Adott hosszúságú csendes szakasz
- MAC cím – 6 bájt
- Típus – 2 bájt
 - Milyen adatot tartalmaz
 - Például: 0800 – IPv4 datagram, 86DD – IPv6 datagram, 0806 – ARP
- Adat – 46-1500 bájt
- Ellenőrző bitek (Frame Checking Sequence) – FCS, 4 bájt
 - Bithiba felismerés CRC alapon
 - Hiba esetén a keret eldobása

Preamble	Cél MAC címe	Forrás MAC címe	Típus	Adat	FCS
----------	--------------	-----------------	-------	------	-----

- **Kapcsolatmentes adatátvitel szomszédos elemek között (egy LAN-ban)**
- **Nem megbízható átvitel**
 - Bithiba esetén egyszerűen eldobja a keretet
 - Nincs ismételt átküldés
- **Ütközés esetén lehet ismétlés**
 - A közeghozzáférésről bővebben később
- **A fizikai rétegbeli szabványok is tartoznak hozzá**
 - Többféle sávszélesség
 - Többféle médium
 - Azonos MAC protokoll és keretformátum



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

HÁLÓZATOK ALAPJAI ÉS ÜZEMELTETÉSE

Közeghozzáférés, fizikai réteg
2023. május 8.

Mészáros András

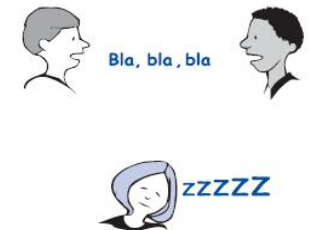
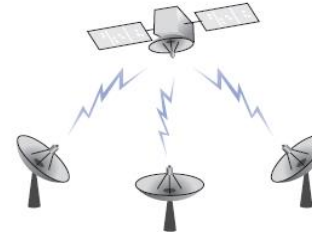
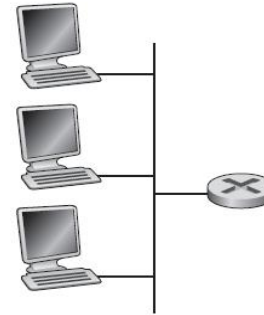
BME Hálózati Rendszerek és Szolgáltatások Tanszék
meszarosa@hit.bme.hu



1. Többszörös hozzáférés
2. Vezetéknélküli LAN (WLAN)
3. Fizikai réteg
4. Összefoglalás

A fóliák elkészítéséhez felhasználtuk Jim Kurose és Keith Ross „Számítógép hálózatok működése” című könyvéhez készült fóliákat

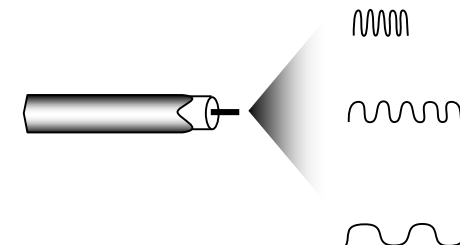
- **Kétpontos link**
 - Pont-pont összekötést adó közeg (médium)
 - Például két router között
- **Üzenetszórásos (broadcast) link**
 - Közös felhasználású közeg több ponthoz
 - Eredeti Ethernet
 - Kábelmodemes hozzáférés feltöltési (upstream) iránya
 - IEEE 802.11 WLAN



- Egyetlen **közös felhasználású** broadcast médium, (csatorna)
- Azonos időszakban több végpont is szeretne adatot küldeni
 - Egy végpont több különböző helyről származó információt kap meg
 - **ütközés** (collision)
 - Az összekeveredő jelek miatt használhatatlan
- Protokoll a többszörös hozzáférés lebonyolításához
 - Elosztott (az összes végponton futó) algoritmus a csatorna megosztásának vezérlésére
 - Meghatározza, hogy mikor küldhet egy adott csomópont
 - Az összehangoláshoz szükséges információ is a többszörös hozzáférésű csatornán megy (in-band)
- Medium Access Control - MAC

- Adott számú kisebb egységre felosztott csatorna
 - Minden végponthoz egyetlen adási egységet rendel
 - Minden érkező jelet venni kell
- Hatékonyság
 - Állandó terhelés esetén jó
 - Ingadozó, egyenetlen terhelés esetén kihasználatlan egységek vagy túlterhelés
 - Egységek végpontokhoz rendelése külön vezérlést igényel

- Például
 - TDMA (idő)
 - FDMA (frekvencia)
 - SDMA (tér)
 - CDMA (kód)
 - WDMA (hullámhossz)



- Egyszerre egy csomópont küldhet, de mindig más –
Taking-turns
 - A küldő meghatározása lekérdezéssel – **polling**
 - Kitüntetett mester csomópont kérdezi le az igényeket és osztja ki az adásjogot
 - A küldő meghatározása **token** segítségével
 - Nincs mester, az a csomópont adhat, akinél van a token
 - Elkerülhető a különböző küldőktől jövő jelek ütközése
- Hatékonyság
 - Egyenetlen terhelésnél is kihasznált
 - Nagy terhelésnél sincs ütközés

- A küldés időpontja véletlenszerű – **Random Access**

- A csatorna nincs fixen felosztva
- Küldéskor a teljes csatornához férnek hozzá a végpontok
- Ütközések léphetnek fel
- Fő kérdések

- **Hogyan érzékeljük az ütközést?**
- **Mi legyen ütközés esetén?**

Alap probléma:

Wifi routerre többen rá vagytok csatlakozva.
MIND ugyanazon a rádiófrekvencián akar küldeni.

-> Megoldás lehet:

- csatornaosztásos protokollok (TDM/FDM) - rosszabb
- véletlen hozzáférésű protokollok!

- **Hatékonyság**

- Alacsonyabb terhelés esetén jó, késleltetés szempontból is
- Nagy terhelésnél nagyobb esély az ütközésre

- **Ütközési tartomány – collision domain**

- Végpontok (interfészek) halmaza, akiknek a keretei ütközhetnek
- Egy switch-hez való duplex bekötés esetén gyakorlatilag csak egy interfész van benne – nincs is ütközés

- **Egyszerű ALOHA** Küldj amikor akarsz! Ha ütközés van? Várj és küldd újra!
 - Nincs szinkronizáció
 - Nagyszámú végpont esetén a throughput legfeljebb a kapacitás 0.18-ad része
- **Réselt ALOHA** Csak időrések elején küldhetsz!
 - **Időréseket** használnak a csomópontok
 - Nem lehet akármikor küldeni, csak az időrés elején
 - Kétszeresre növeli a throughputot
- Vivőérzékeléses megoldás: **CSMA** – Carrier Sense Multiple Access
 - Küldés előtt a csomópont ellenőrzi, hogy szabad-e a csatorna
 - Késleltetett elküldés Hallgass mielőtt küldesz! Már küld valaki? VÁRJ!
 - A jel terjedési késleltetése miatt lehetséges ütközés
 - A vezeték hossza, vagy a besugárzott tér nagysága befolyásolja a hatékonyságot

- **Ütközés-érzékeléses kiterjesztés a CSMA-hoz**

- **Küldés csak, ha üres a csatorna**

- **Ütközés felismerésekor abbahagyja** a küldést a csomópont

Küldés közben figyel.

Ha ütközést észlel -> megállítja

Vár véletlenszerűt -> újraküldi

PI: Ethernet

- **Érzékelés**

- Vezetékes esetben nem gond

- **Vezetéknélküli esetben nem elég** (terjedési csillapítás)

Wifinél: **CSMA/CA**

- **Ethernet**

> Nem tud ütközést detektálni

- A CSMA/CD-t használja

- Ütközés érzékelés esetén

- **A keret küldését abbahagyja és speciális jelet sugároz (jam signal) !!!!**

- Az ütközött keretet később újraküldi

- Exponenciális visszalépés

- Az újraküldés késleltetési idejét véletlenszerűen választja

- A véletlen választási tartományának mérete újabb ütközés esetén a kétszeresére nő

1. Többszörös hozzáférés
2. Vezetéknélküli LAN (WLAN)
3. Fizikai réteg
4. Összefoglalás

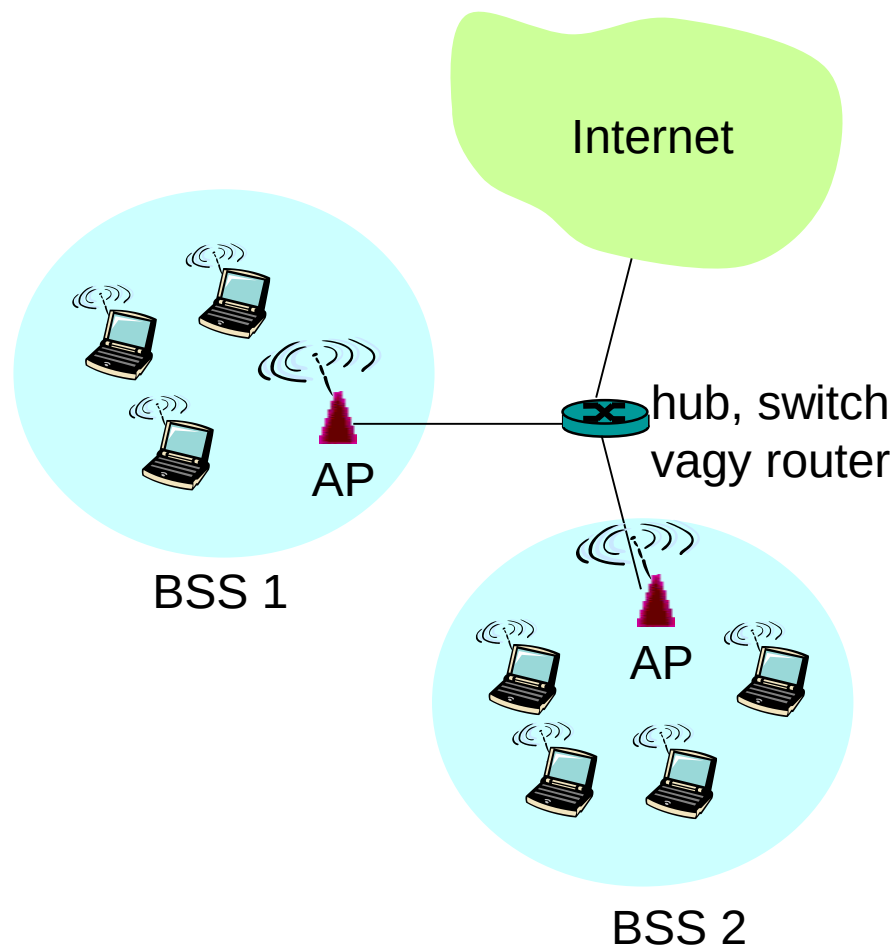
- **Vezetéknélküli hálózat – rádiójelek sugárzásával**
- **WiFi** (nem Wireless Fidelity)
- Jellemző összefüggés
nagyobb vivőfrekvencia –
nagyobb adatsebesség – kisebb
hatótávolság
- Elérhető hálózatfelépítési módok
 - **Infrastruktúra mód** – a végpont egy elérési ponton (**access point – AP**) keresztül kapcsolódik a hálózathoz
 - **Ad-Hoc mód** – egymáshoz kapcsolódó végpontok hálózata
- Számos változat

Változat	Vivőfrekvencia (GHz)	Max. sávszélesség (Mbps)
802.11	2.4	2
802.11a	5	54
802.11b	2.4	11
802.11g	2.4	54
802.11n	2.4/5 MIMO	600
802.11ac	5	3400*
802.11ad	60	8000*
802.11ax	2.4/5/6	9600*

Megszokott: Laptop -> AP -> router pl: **WiFi router**

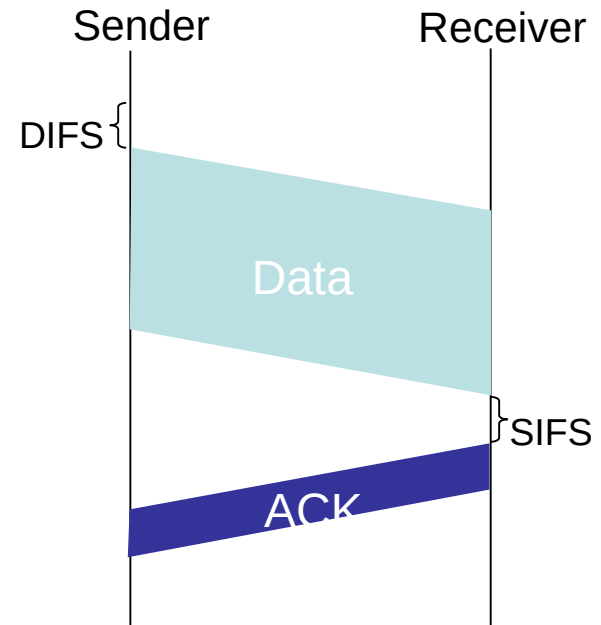
Laptop <-> Telefon pl: **AirDrop, WiFi direct**

- **AP – Bázisállomás**
- Azonosított cellák - Basic Service Set (BSS)
 - **SSID azonosító**
- A switch/router funkció gyakran egybe van építve az AP-val
- Egy csatlakozó végpont
 - „Társul” az AP-hez
 - Az AP-k által sugárzott jeleket figyeli
 - **Aktív/passzív pásztázás** (scanning)
 - Beacon (jelzőfény, irányadó jel) keret: SSID, MAC cím
 - Kiválasztja az AP-t
 - Esetleg autentikációra is szükség van
 - **Általában DHCP-val konfigurál magának IP címet**



- Az ütközés felismerése nehézkes
 - Rejtett terminál probléma
 - Jelgyengülés (elnyomás)
- **Ütközésselkerülés – Collision Avoidance**
- Küldés
 - Adott (DIFS) ideig üres csatorna esetén teljes keret átküldése
 - Foglalt csatorna esetén véletlen értékről visszaszámolás
 - Csak szabad csatorna esetén
 - Nyugta elmaradásakor intervallum kétszerezés
 - Distributed Inter-Frame Space

- Fogadás
 - A keret megérkezése után rövid (SIFS) idővel **nyugta** küldése
 - Short Inter-Frame Space



ÜTKÖZÉSELKERÜLÉS JAVÍTÁSA

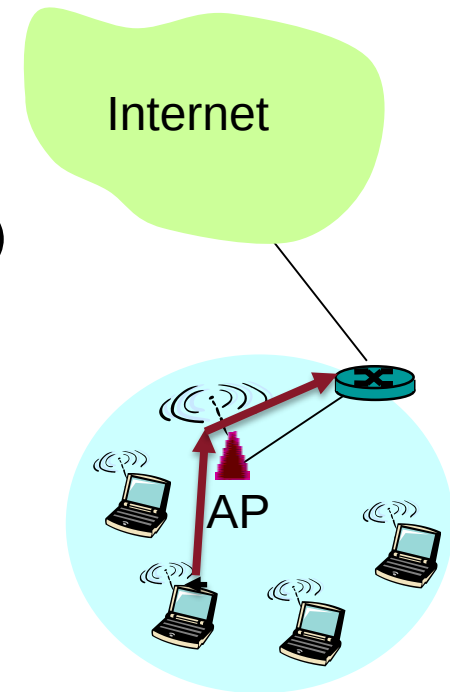
- A csatornáról nem mindig dönthető el, hogy szabad-e
 - Rejtett terminál – egy másik küldő jelét nem vesszük, de a célnál ütközés van



- Véletlen hozzáférés helyett csatorna-lefoglalás kis vezérlőkeretekkel
1. Küldés előtt **RTS** (Request-to-Send) keret küldése az AP-nak
 - Ezek ütközhetnek, de rövidek
 2. Az AP **CTS** (Clear-to-Send) keretet szór szét
 3. Az igénylő küld, a többiek csendben várnak
 4. Az AP-től jövő nyugta után lehet újra igényelni

802.11 KERET ÉS CÍMZÉS

- A küldő és a cél fizikai címe mellett kell még egy cím
 - A keret végső célja a router interfésze, vagy egy másik hoszt a LAN-ban (3. cím)
 - A küldés közvetlen célja az AP (1. cím)
 - Küldő címe (2. cím)
 - Ad-Hoc módban további cím is kellhet (4. cím)
- Csatorna-lefoglalás időtartama
- Nyugtázás sorszámozással
- Vezérlési információk
 - Például: típus
 - Vezérlés: RTS, CTS, ACK
 - Menedzsment: Társulás, Beacon



Vezérlés	Időtartam	1. cím	2. cím	3. cím	4. cím	Sorszám	Adat	CRC
----------	-----------	--------	--------	--------	--------	---------	------	-----

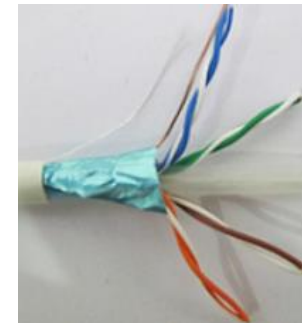
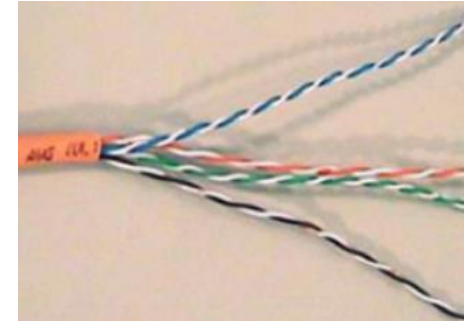
- Wireless Personal-Area Networks (WPAN)
 - Néhányszor 10 méteres hatótávolság
 - Ad-Hoc elrendezés
 - Master-slave kommunikáció
 - IEEE 802.15 – Bluetooth
- WiMAX
 - Szélessávú mikrohullámú elérési link
 - Pont-pont megoldás bázisállomások között – kábel helyett mikrohullám
 - Akár néhányszor 10 kilométer
 - Sebesség akár 1 Gbps
 - Pont-multipont elrendezés – végpontok bekötése a cellában
 - IEEE 802.16
- Műholdas rendszerek
- Nyilvános (celluláris) mobilhálózatok

1. Többszörös hozzáférés
2. Vezetéknélküli LAN (WLAN)
3. Fizikai réteg
4. Összefoglalás

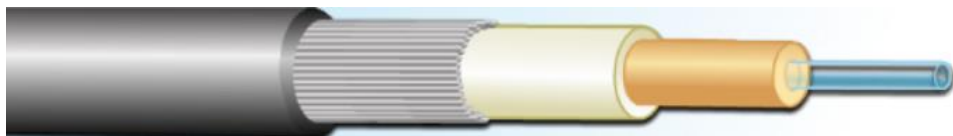
- Végződtető vagy kapcsoló hardverek és fizikai közegek
- Feladatok
 - A keret tartalmát kódolja
 - A biteknek megfelelő fizikai (elektronikus, optikai, rádiós) jeleket előállítja
 - A jeleket átküldi a fizikai közegen
 - Veszi a fizikai jeleket
 - Dekódolja (értelmezi) a jelet és a bitekből visszaállítja a keretet
- Szabványosítás
 - A hardverre vonatkozik
 - Több szervezet is dolgozik rajta, pl: IEEE, ISO, ITU-T
- Vonali kódolás – encoding
 - A keretben lévő üzenet elemeinek kódolása bitekre, például Morse, Manchester kódolás (régí Ethernet megoldás)
- Jelzés – signalling
 - Egy-egy jel fizikai reprezentációja és annak elküldése/fogadása

- Időzítés és feszültségérték a jelzés alapja
- Zavaró hatások és kiküszöbölésük
 - Elektromágneses interferencia (EMI) és rádiófrekvenciás interferencia (RFI)
 - Külső hatás
 - Földelés és árnyékolás
 - Áthallás (Crosstalk)
 - A szomszédos vezetékek egymásrahatása
 - Összezsavarás
- Jellemző megoldások
 - Árnyékolatlan csavart érpár – Unshielded Twisted Pair, UTP
 - Árnyékolt csavart érpár – Shielded Twisted Pair, STP
 - Koaxiális kábel

- UTP
 - Kategóriákba sorolják
 - Cat5 – Ethernet 100BASE-TX szabványhoz
 - Cat5e – Ethernet 1000BASE-TX szabványhoz
- STP
 - Drágább, de jobb
- Coax
 - Antennák csatlakoztatása
 - Kábeles Internet/kábelTV bekötés
 - Optikával kombinálva: Hybrid Fiber-Coax HFC



- **Fényjelek átvitele üvegszálon** Egyszerűbben **optikai kábel** :O
- **Jellemzők** **Monomódusú vs Multimódusú**
 - **Nagy sáv szélesség**
 - **Nagyobb távolságok**
 - Jelerősítés, újragenerálás szükséges lehet
 - Sérülékenyebb
 - Nehezebb telepíteni
- **Alkalmazás**
 - **WAN és MAN hálózatokban elsősorban ilyen technológia van**
 - Hozzáférési hálózatokban – FTTx, pl.: **Fiber To The Home**
 - Akár LAN-okban is, pl. 100BASE-FX



- Típusok

- Monomódusú

- Egyetlen fénynyaláb megy a szálban
- Nagyobb sáv szélesség
- Vékonyabb (a mag < 10 mikron), drágább kábel és drága ledék

- Multimódusú

- Több nyaláb megy a szálban
- Kisebb sáv szélesség
- Vastagabb (a mag 50-60 mikron), olcsóbb kábel és olcsó ledék

- Hullámhosszosztás

- Sötét szál – nem használja ki a teljes sáv szélességet

- **WDM** Több adatot küldünk ugyanazon a kábelben egyszerre! Pl: Kossuth, Petőfi és Bartók rádió egy kábelben, de más hullámhosszon

- Különböző hullámhosszú fényjelek más-más kapcsolat jelét viszik
- Több tíz Gbps egy hullámhosszon
- Több tíz hullámhossz egy kábelben
- Speciális (és drága) kapcsolók kellenek az átkapcsoláshoz,
- Hullámhossz konverzió...

1. Többszörös hozzáférés
2. Vezetéknélküli LAN (WLAN)
3. Fizikai réteg
4. Összefoglalás

A TCP/IP protocol stack rétegei:

- **alkalmazási (application)**
 - a hálózati alkalmazásokat támogatja
 - FTP, IMAP, SMTP, HTTP, DNS, DHCP
- **szállítási (transport)**
 - Adatátvitel processztől processzig
 - TCP, UDP, QUIC, socketek*
- **hálózati (network)**
 - adatok (csomagok) mozgatása a forrás és nyelő hosztok között
 - IP, ICMP, RIP, OSPF, BGP
- **adatkapcsolati (link)**
 - adatok (csomagok) továbbítása a szomszédos hálózatelemek között
 - Ethernet, WiFi
- **fizikai (physical)**
 - bitek továbbítása a szomszédos csomópontok közti az összeköttetéseken

- Alkalmazási réteg
 - Szolgáltatás hálózati alkalmazásoknak
 - Szöveges kérés-válasz üzenetek
- Szállítási réteg
 - Kapcsolat távoli processzek között
 - Socketek: adatfolyamok vagy datagramok átvitele
 - Nyalábolás és nyalábbontás
 - TCP vagy UDP
 - UDP: egyszerű, megbízhatatlan (médiafolyamok, rövid tranzakciók)
 - TCP: megbízható, extra funkciók (forg. szab., torlódáskezelés)
 - QUIC: UDP felett megbízható átvitel - beépített titkosítással

- Hálózati réteg
 - Kapcsolat távoli hosztok közt
 - Címzés
 - IPv4, IPv6
 - egyedi azonosítás
 - alhálózatok kialakítása (VLSM)
 - címkiosztás (DHCP) és címfordítás (NAT)
 - Útvonalválasztás
 - routing tábla (lokális szabályok)
 - routing protokollok – link-állapot és távolságvektor (RIP, OSPF, BGP)

- Adatkapcsolati réteg
 - Kapcsolat logikailag szomszédos eszközök között
 - Címzés
 - fizikai címek (vs logikai), címfordítás (ARP)
 - LAN-ok működése
 - címtanulás (MAC learning)
 - redundancia a LAN-okban (STP)
 - VLAN-ok
 - közeghozzáférés
 - Ethernet, WiFi
- Fizikai réteg
 - Fizikai közeg és azon haladó jelek
 - Rézkábel, optika, vezeték nélküli közeg



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

